Internet of Things

Systems, Technologies, and Business Value

Internet of Things

Systems, Technologies, and Business Value

FIRST EDITION

Dr. Liang Li

A Textbook for the IoT Education Course

Internet of Things: Systems, Technologies, and Business Value, First Edition

Copyright © 2027 by Dr. Liang Li. All rights reserved.

Published by Dr. Liang Li.

No part of this publication may be reproduced, distributed, stored in a retrieval system, or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, scanning, or otherwise, without the prior written permission of the copyright holder, except for brief quotations embodied in critical reviews and certain other noncommercial uses permitted by copyright law. Requests for permission to reproduce material from this work should be directed to the author.

Limit of liability and disclaimer of warranty. The author has made every effort to ensure that the information in this book is accurate at the time of writing. IoT products, cloud services, prices, and regulations change quickly, and the author makes no representations or warranties with respect to the accuracy or completeness of the contents and specifically disclaims any implied warranties of merchantability or fitness for a particular purpose. The worked examples, calculations, and lab guides are for education; readers should verify specifications with vendors and follow local electrical, radio, and safety codes before building or deploying any system. The author shall not be liable for any loss or damage arising from the use of this book.

Trademarks. Product and company names mentioned in this book, including Amazon Web Services, AWS IoT Core, Alexa, Google Home, Bluetooth, Wi-Fi, Arm, Espressif ESP32, Nordic Semiconductor, Raspberry Pi, and NxEco, are trademarks or registered trademarks of their respective owners. They are used for identification and teaching purposes only. Their mention does not imply endorsement by the author, and the use of a product as a case study does not imply that its maker sponsored or reviewed this book.

Figures and data. Unless otherwise noted, figures and tables were created by the author. Data shown from the MySmartDrop portal were taken from the author's own account. The IoT Education Course demo system values are simulated.

Web addresses. Web addresses were checked in October 2026. Addresses may change after publication.

Cataloging data (prepared by the author)
Names: Li, Liang, author.
Title: Internet of things : systems, technologies, and business value / Dr. Liang Li.
Description: First edition. Includes bibliographical references and index.
Subjects: Internet of things—Textbooks. Embedded computer systems. Wireless sensor networks. Cloud computing. Business—Data processing.

ISBN (print): to be assigned
ISBN (ebook): to be assigned

First edition, 2027

10 9 8 7 6 5 4 3 2 1

Contents

List of Figures ix

List of Tables xi

Preface xiv

How to Use This Book xvi

1 Foundations of IoT Systems: Architecture, Systems Thinking, and AIoT 1

1.1 What is the Internet of Things? 2
1.2 IoT in daily life and industry 3
1.3 Core components of an IoT system 3
1.4 The four-layer IoT architecture 4
1.5 How an IoT system works: sense, connect, process, interact 6
1.6 Thinking in systems 7
1.7 Looking inside a real IoT device 9
1.8 Security and privacy: a first look 10
1.9 From smart to intelligent: the Artificial Intelligence of Things 11
1.10 IoT application case: NxEco smart irrigation 12
Lab 1: IoT device teardown 16

2 Inside the IoT Device: Sensors, Actuators, and Microcontrollers 19

2.1 The three core components of an IoT device 20
2.2 Sensors 20
2.3 From physical signal to digital data 22
2.4 Sensor characteristics 24
2.5 Actuators 26
2.6 The microcontroller: the brain of the device 27
2.7 Arm and RISC architectures 30
2.8 The sense, process, act control loop 31
2.9 Putting it together: a smart irrigation controller 34
2.10 The global IoT hardware market 34
2.11 IoT application case: how NxEco senses the garden 35
Lab 2: Build and justify an IoT control loop 40

3 IoT Access and Networking: From the OSI Model to Short-Range Wireless 44

3.1 IoT access and IoT communication 45
3.2 Data communications networks 45
3.3 The OSI reference model 45
3.4 The physical layer 47
3.5 The data link layer 50
3.6 The network and transport layers 51
3.7 Wi-Fi and local area networks 52
3.8 Why IoT networking is different 53
3.9 Short-range access technologies 53
3.10 A protocol selection framework 55
3.11 Network topologies 57

3.12 IoT network security....58
3.13 IoT application case: connecting NxEco to the network....58
Lab 3: Wireless site survey....62

4 IoT Edge and Cloud Computing....65
4.1 The three-tier IoT architecture....66
4.2 The end-to-end pipeline....67
4.3 Why constrained devices need a gateway....68
4.4 What the edge does: four objectives....69
4.5 Northbound communication and edge runtimes....70
4.6 MQTT: the messaging protocol of IoT....71
4.7 Bidirectional control and digital twins....73
4.8 Security and identity between edge and cloud....74
4.9 Cloud computing fundamentals....74
4.10 The IoT cloud service: four core modules....76
4.11 IoT cloud providers....79
4.12 IoT application case: NxEco's edge and cloud split....79
Lab 4: MQTT publish/subscribe....84

5 IoT Firmware and Software: From the Microcontroller to the Cloud....87
5.1 The standard workflow of an IoT system....88
5.2 The four-layer firmware and software architecture....88
5.3 What makes IoT software different....89
5.4 What is firmware?....89
5.5 The layered firmware architecture....90
5.6 The firmware program flow....92
5.7 Ten key characteristics of IoT firmware....94
5.8 Building firmware: tools and procedure....94
5.9 Reliability, testing, and over-the-air updates....96
5.10 Edge and gateway software....98
5.11 The data-centric edge and cloud software stack....99
5.12 IoT application case: NxEco firmware and software....99
Lab 5: Build, break, and harden....105

6 Securing IoT Systems: Trust, Privacy, Ethics, and Sustainability....108
6.1 Why IoT security is hard....109
6.2 Threats across the IoT stack....109
6.3 Defense in depth and zero trust....111
6.4 Cryptography essentials....113
6.5 Mutual TLS and least privilege....114
6.6 Assessing risk....116
6.7 Privacy....117
6.8 Security regulation and standards....118
6.9 Ethical and organizational impacts....119
6.10 Environmental impacts and sustainability....120
6.11 IoT application case: trust and sustainability in NxEco....121
Lab 6: IoT security and privacy review....125

7 IoT System Design Example: Building and Testing an End-to-End Demo....127
7.1 System overview: five tiers, one pipeline....128

7.2 Tier 1: perception-layer hardware....129
7.3 Firmware and the deep-sleep duty cycle....130
7.4 Tier 2: the proximity network....133
7.5 Tier 3: the edge gateway....135
7.6 Northbound security and provisioning....136
7.7 Tier 4: the AWS cloud back end....137
7.8 Tier 5: the presentation layer....139
7.9 Putting it together: journeys and latency....140
7.10 System bring-up in four phases....142
7.11 Testing an IoT system....143
7.12 From demo to real deployment....144
7.13 IoT application case: NxEco compared with the demo system....144
Lab 7A: Stress-test the demo system, Part A (Week 6)....148
Lab 7B: Verify, break, and redesign, Part B (Week 7)....151

8 The Artificial Intelligence of Things (AIoT)....153
8.1 IoT plus AI equals AIoT....154
8.2 From smart to intelligent....154
8.3 AIoT system integration....155
8.4 Where AI lives....155
8.5 The TinyML workflow....157
8.6 Why cloud AI still dominates....158
8.7 Cloud AI and IoT platforms....159
8.8 Core AI technologies in IoT....160
8.9 Challenges of implementing AIoT....161
8.10 Applying AIoT to the demo system....161
8.11 IoT application case: making NxEco intelligent....162
Lab 8: Design an AIoT enhancement....165

9 IoT Business Models, Business Cases, and Roadmaps....167
9.1 IoT business domains....168
9.2 IoT business models....168
9.3 Value across the IoT technology stack....169
9.4 Strategic obstacles....171
9.5 Trends shaping IoT business....171
9.6 Case 1: Xiaomi, the smart-home ecosystem....172
9.7 Case 2: Enel X, the grid orchestrator....173
9.8 Building an IoT business case....175
9.9 The implementation roadmap....177
9.10 Communicating the solution....179
9.11 IoT application case: the NxEco business model....180
Lab 9: Business proposal and final presentation....184

A Answers to Review Questions....187

B Formula Sheet and Demo System Reference....195
B.1 Formula sheet....195
B.2 IoT Education Course demo system reference card....196
B.3 Bill of materials (three sensor nodes and one gateway)....197

C Glossary..... 198

Index..... 206

List of Figures

Chapter 1 Foundations of IoT Systems

1.1 The four-layer IoT architecture....5
1.2 A systems view of a smart irrigation controller, showing elements, interconnections, boundary, environment, and purpose....8
1.3 Traditional IoT follows a linear sense, transmit, store, analyze sequence and relies on people to act....11
1.4 The NxEco smart irrigation system mapped to the four-layer IoT architecture....13

Chapter 2 Inside the IoT Device

2.1 Sampling and quantization....23
2.2 Inside a microcontroller....28
2.3 The six-step IoT control loop....32
2.4 Hysteresis....33

Chapter 3 IoT Access and Networking

3.1 The OSI reference model with typical IoT protocols....46
3.2 Three basic digital modulation techniques carrying the bit pattern 1 0 1 1 0....48
3.3 Typical range and data rate of IoT access technologies....56
3.4 Star and mesh topologies....57

Chapter 4 IoT Edge and Cloud Computing

4.1 The three-tier device, edge, and cloud architecture....66
4.2 Daily data volume for 100 sensors with and without edge filtering (logarithmic scale)....70
4.3 MQTT publish/subscribe....72
4.4 Ingestion and routing in AWS IoT Core....76

Chapter 5 IoT Firmware and Software

5.1 Layered firmware architecture....91
5.2 Firmware program flow: initialization, main loop, interrupts, and sleep, protected by a watchdog....93
5.3 A/B over-the-air update with automatic rollback....97

Chapter 6 Securing IoT Systems

6.1 Defense in depth for IoT. Each layer assumes the one outside it may fail....112
6.2 The mutual TLS handshake between an edge gateway and AWS IoT Core (simplified)....114
6.3 A likelihood and impact risk matrix....117

Chapter 7 IoT System Design Example

7.1 The five-tier IoT Education Course demo system....128
7.2 Current profile of one sensor node over two cycles (logarithmic scale)....131
7.3 Estimated battery life versus sleep interval for the four-phase profile of Worked Example 7.2 with a 2,000 mAh cell....132
7.4 Downlink sequence for a fan command, with acknowledgment....141
7.5 Cumulative uplink latency: best and worst case per hop against the 120 ms target....142

Chapter 8 The Artificial Intelligence of Things (AIoT)

8.1 Where AI can run in an IoT system, from microwatt microcontrollers to kilowatt cloud clusters....155

8.2 The TinyML workflow....157

Chapter 9 IoT Business Models, Business Cases, and Roadmaps

9.1 Indicative gross margins across the IoT technology stack....170

9.2 Cumulative cash flow and payback for the cold-chain business case....177

9.3 A phased IoT implementation roadmap with stage gates....178

List of Tables

Front matter

P.1 Chapter map xvi
P.2 Suggested reading paths xvii

Chapter 1 Foundations of IoT Systems

1.1 Major categories of IoT applications 3
1.2 Hardware and software at each layer of the four-layer architecture 5
1.3 Where the NxEco case appears in this book 13

Chapter 2 Inside the IoT Device

2.1 Major hardware at each layer 20
2.2 Common sensor types 21
2.3 Sensor characteristics 24
2.4 Common IoT actuators 26
2.5 Key functions of an MCU in an IoT device 28
2.6 MCU versus CPU 29
2.7 Popular MCU families 30
2.8 Why RISC suits IoT 31
2.9 The control loop in a smart irrigation controller 34
2.10 Indicative IoT market size (estimates vary by source) 34
2.11 NxEco's sources of sensing information 35
2.12 Five-day forecast shown on the MySmartDrop Home page 36
2.13 Test record template for Lab 2 41

Chapter 3 IoT Access and Networking

3.1 The seven OSI layers 46
3.2 Physical-layer hardware for wireless communication 48
3.3 Selected Wi-Fi generations 52
3.4 Comparison of IoT access technologies 56
3.5 Why NxEco uses Wi-Fi 59
3.6 Survey record template for Lab 3 63

Chapter 4 IoT Edge and Cloud Computing

4.1 The three stages of the edge-to-cloud pipeline 67
4.2 WPAN versus WLAN 67
4.3 Hardware architecture of an IoT edge node 68
4.4 The four edge objectives 69
4.5 MQTT quality of service levels 72
4.6 Cloud service models 75
4.7 Key protocols between devices, cloud, and clients 78
4.8 IoT services of the three largest cloud providers (check current documentation) 79
4.9 Placement of NxEco functions 80
4.10 MQTT lab test log 85

Chapter 5 IoT Firmware and Software

5.1 Eight characteristics of IoT software....89
5.2 Firmware, operating systems, and applications....90
5.3 FreeRTOS tasks in the demo sensor node....92
5.4 Ten key characteristics of IoT firmware....94
5.5 Edge and gateway software environments....98
5.6 The data-centric IoT software stack....99
5.7 NxEco app functions mapped to IoT layers....100
5.8 Software health indicators of the NxEco app....101
5.9 Lab 5 test record template....106

Chapter 6 Securing IoT Systems

6.1 IoT threats by layer....109
6.2 STRIDE threats in IoT....111
6.3 Layered security in the IoT Education Course demo system....115
6.4 Selected privacy laws relevant to IoT....118

Chapter 7 IoT System Design Example

7.1 Sensor nodes in the demo system....129
7.2 End-device hardware....129
7.3 ESP32 versus nRF52840 for a battery sensor node....130
7.4 Pin map of sensor_node_01 (design reference)....130
7.5 Local wireless protocol options for the demo....133
7.6 BLE advertising versus connected GATT....133
7.7 GATT database of the demo node (design reference)....134
7.8 Gateway hardware and software....135
7.9 Rule actions in the demo cloud....138
7.10 Push and pull paths from cloud to app....138
7.11 Uplink latency budget of the demo design....141
7.12 Four-phase system bring-up....142
7.13 Defect log fields....143
7.14 The demo system and NxEco compared....145
7.15 Team test log template....150

Chapter 8 The Artificial Intelligence of Things (AIoT)

8.1 Smart versus intelligent IoT....154
8.2 Comparing AI locations....157
8.3 Selected cloud AI and IoT platforms (verify current offerings)....159
8.4 Core AI technologies in IoT....160
8.5 AIoT enhancements for the demo system....161
8.6 AIoT enhancements for NxEco....162

Chapter 9 IoT Business Models, Business Cases, and Roadmaps

9.1 IoT business models....168
9.2 Indicative margins across the IoT stack (vary widely by company)....169
9.3 Xiaomi's "Human × Car × Home" strategy....172
9.4 Xiaomi and Enel X compared....174
9.5 Implementation roadmap phases....178

9.6 Watering minutes by zone from the MySmartDrop Water Record....181
9.7 Executive panel feedback form....185

Appendix B Formula Sheet and Demo System Reference

B.1 Formula sheet....195
B.2 Demo system reference card....196
B.3 Demo bill of materials (indicative prices)....197

Preface

The Internet of Things (IoT) has moved out of the laboratory and into almost every industry. Farms schedule irrigation from soil-moisture readings, hospitals monitor patients after discharge, utilities balance national grids with millions of connected batteries and chargers, and factories detect a failing bearing weeks before it stops a production line. By the end of 2025, analysts estimated that roughly 21 billion IoT devices were connected worldwide, with close to 40 billion expected by 2030 (Sinha, 2025). Behind each of those devices is the same chain of engineering decisions: what to sense, how to process it, how to move it, where to store it, how to secure it, and how to turn it into value.

This textbook was written for the **IoT Education Course**, an undergraduate course that treats IoT as both a technology and a business tool. It follows one guiding idea: *an IoT product is a system, and every part of the system is a trade-off*. A sensor node that runs for years on a coin cell cannot also stream video. A gateway that keeps a factory running during an internet outage costs more than one that does not. A business model that sells hardware at cost only works if services pay for it later. Students who learn to see and explain these trade-offs can design, evaluate, and justify IoT solutions in any industry.

A running example and a test bench

Two systems appear throughout the book. The first is the **NxEco smart irrigation system**, a commercial weather-based irrigation controller that contains every layer of an IoT system in one small box: weather-based sensing, a microcontroller, valve outputs, a Wi-Fi module, a cloud service, web portals, and a phone app. It is introduced as an IoT application case in Section 1.10, and each later chapter examines the part of the NxEco system that the chapter teaches: weather data collection in the sensing chapter, Wi-Fi in the networking chapter, the weather engine in the cloud chapter, and so on. The second is the **IoT Education Course demo system**, a browser-based simulation of a five-tier pipeline: ESP32 sensor nodes, Bluetooth Low Energy (BLE), a Raspberry Pi edge gateway, AWS IoT Core, and a mobile app. Students can open the demo at https://liangli99.github.io/IoT-Demo/dashboard.html with any desktop browser, without hardware or a cloud account. The values it shows are simulated, but they follow the topics, rules, and latency budget described in Chapter 7. Students should treat it the way an engineer treats any test bench: run it, measure it, and check whether what it reports is consistent.

A note on currency

IoT changes quickly. Several services that appeared in IoT textbooks only a few years ago have since been retired, including Google Cloud IoT Core (retired in 2023) and AWS IoT Analytics (support ended in December 2025). Market figures in this book are estimates from

the cited sources and should be read as orders of magnitude rather than exact values. Students are encouraged to check vendor documentation before recommending any specific product or service in their assignments.

Acknowledgment of sources

Each chapter ends with a reference list in APA 7th edition format. The lists include the required readings, videos, and podcasts from the course blueprint, so that students can move directly from the textbook to the original sources.

Dr. Liang Li

How to Use This Book

This book can be read from beginning to end as the text for the eight-week IoT Education Course, or used selectively as a reference. This section explains how it is organized, what the recurring features are for, and which conventions it follows.

How this book is organized

The nine chapters follow the eight weeks of the course and build one layer at a time, from the physical device to the business case.

Table P.1 Chapter map

Chapter	Title	Course week
1	Foundations of IoT Systems: Architecture, Systems Thinking, and AIoT	Week 1
2	Inside the IoT Device: Sensors, Actuators, and Microcontrollers	Week 2
3	IoT Access and Networking: From the OSI Model to Short-Range Wireless	Week 3
4	IoT Edge and Cloud Computing	Week 4
5	IoT Firmware and Software: From the Microcontroller to the Cloud	Week 5
6	Securing IoT Systems: Trust, Privacy, Ethics, and Sustainability	Weeks 3 to 6
7	IoT System Design Example: Building and Testing an End-to-End Demo	Weeks 6 and 7
8	The Artificial Intelligence of Things (AIoT)	Week 7
9	IoT Business Models, Business Cases, and Roadmaps	Weeks 7 and 8

Chapter 6 gathers the security, privacy, ethical, and environmental material that the course introduces in several weeks into one place, so that students have a single reference for the security, ethical, organizational, and environmental impacts of IoT. Chapter 7 is the integration chapter. It walks through the class demo system (Li, 2026) tier by tier and doubles as the test manual for the Week 6 and Week 7 collaborations.

Features of each chapter

- **Learning objectives** open each chapter and state what students will be able to do after studying it.
- **Where this fits** connects the chapter to the weekly collaboration, assignment, or Socratic Metric (SM) discussion.
- **Key Concept** boxes highlight the ideas that later chapters depend on.

- **Worked Examples** show the calculations that engineers use to make design decisions: ADC resolution, signal strength, bandwidth, battery life, latency budgets, model size, and payback period. Every number in every worked example can be reproduced with a calculator.
- **Global Perspective** boxes show how the same technology is regulated, priced, or deployed differently around the world.
- **Review Questions** at the end of each chapter mirror the format of the course midterm: multiple-choice questions, short-answer questions, and one or two applied problems. Answers appear in Appendix A.
- **Lab Guides** give step-by-step instructions for each hands-on collaboration: the Wokwi simulator, the HiveMQ public MQTT broker, and the IoT Education Course demo system.

Suggested reading paths

Table P.2 Suggested reading paths

Reader	Suggested chapters	Focus
Full course (eight weeks)	1 to 9, in order, with the lab at the end of each chapter	The complete system, from device to business case
Technical emphasis	1, 2, 3, 4, 5, 7, then 6 and 8	Devices, networks, edge and cloud, firmware, and testing
Management and business emphasis	1, 4, 6, 8, 9, with Sections 1.10, 4.12, and 9.11 on NxEco	Architecture, risk, intelligence, and value
Self-study with the demo system	1, 7, then the chapter for each tier you want to study	Hands-on exploration with the browser demo

Conventions used in this book

- **Key terms** appear in **bold** where they are first explained and are defined again in the glossary (Appendix C). The index lists the page of each key term.
- `Monospace type` is used for code, MQTT topics, file names, identifiers, and values typed into a tool, for example `iot/telemetry/+`.
- *Italic type* marks the names of buttons, menus, and screens, for example *Connect to WiFi*, and the titles of publications.
- Numbers use SI units, with U.S. customary units where a product is specified that way. Currency is shown as US$. Percentages are written as “percent” in running text and as % in tables.
- Cross-references name the section, figure, or table, for example Section 4.10, Figure 7.3, or Table 9.6. Figures and tables are numbered by chapter.

- Citations follow APA 7th edition. Each chapter has its own reference list.

For instructors

Answers to all review questions are in Appendix A, and Appendix B contains a formula sheet and a reference card for the demo system that can be given to students during assessments. Each lab guide states which course collaboration or assignment it supports, the tools required, and the deliverables, so that the labs can be assigned without additional handouts.

For students

Read the learning objectives before each chapter and return to them afterwards: if you can do what each objective says, you have understood the chapter. Work every worked example with a calculator before reading the solution, and run the demo system while you read Chapter 7. The review questions follow the format of the course midterm, so they are the best preparation for it.

CHAPTER 1

Foundations of IoT Systems: Architecture, Systems Thinking, and AIoT

Learning Objectives

After studying this chapter, students will be able to:

1. Define the Internet of Things and trace its evolution from machine-to-machine communication to the Artificial Intelligence of Things.
2. Identify the core components of an IoT system: sensors, actuators, microcontrollers, firmware, wireless access, gateways, cloud services, and user interfaces.
3. Explain the four-layer IoT architecture of perception, network, middleware, and application, and place the components of a real product in the correct layer.
4. Apply the five systems concepts of elements, interconnections, purpose, boundary, and interface to analyze an IoT product.
5. Recognize the main security and privacy risks of connected devices.
6. Describe how artificial intelligence moves IoT systems from reactive to proactive operation.

Where this fits: Week 1. This chapter supports Collaboration 1 and the GAP Discussion *IoT Device Teardown: Mapping a Real Product to the IoT Architecture*. The four-layer model introduced here is the vocabulary for every later chapter and lab.

Opening case: water, data, and a dry summer

In many wine-growing regions, from La Rioja in Spain to the Barossa Valley in Australia and the Western Cape in South Africa, summers are getting hotter and drier. Water is expensive, and in drought years it is rationed. A grower who irrigates on a fixed timer wastes water on cool, cloudy days and under-waters the vines during heat waves.

A smart irrigation controller changes that pattern. A soil-moisture probe in each block of vines reports how much water is actually in the root zone. The controller compares the reading with a set point, opens a valve only when the soil is dry, and closes it when the target is reached. The controller also downloads the local weather forecast. If rain is likely

tomorrow, it skips tonight's cycle. Every reading and every valve action is logged to a cloud service, so the grower can check the vineyard from a phone and compare water use across seasons.

Nothing in this system is exotic. It contains a sensor, a small computer, a valve, a radio, a server, and an app. What makes it valuable is that all of these parts work together toward one purpose. This chapter introduces the vocabulary and the models that engineers and managers use to describe such systems.

1.1 What is the Internet of Things?

The **Internet of Things (IoT)** is a network of physical objects ("things") that contain sensors, actuators, processing, and communication capabilities, and that exchange data with other devices and systems over a network, usually the internet, with little or no human involvement (Gillis & Yasar, 2025). The key word is *physical*. A web server or a laptop is connected, but it is not usually called an IoT device. A thermostat, a parcel tracker, a heart-rate strap, a water pump controller, and a traffic signal are IoT devices because they measure or change something in the physical world.

The term was coined in 1999 by **Kevin Ashton**, then working at Procter & Gamble, in a presentation about using radio-frequency identification (RFID) tags to track products through the supply chain. Ashton's argument was that computers knew only what people typed into them, and that the next step was for computers to gather data about things directly, without human data entry.

The ideas behind IoT are older than the name. Industrial plants have used sensors and programmable logic controllers since the 1970s, and **machine-to-machine (M2M)** communication over telephone and cellular links was common in the 1990s for vending machines, utility meters, and alarm systems. What changed in the 2000s and 2010s was cost and scale:

- **Cheap sensors.** Micro-electro-mechanical systems (MEMS) put accelerometers, gyroscopes, and pressure sensors on silicon chips that cost cents.
- **Cheap computation.** A 32-bit microcontroller with a built-in radio now costs a few US dollars.
- **Ubiquitous connectivity.** Wi-Fi, Bluetooth Low Energy, cellular IoT, and low-power wide-area networks made it possible to connect a device almost anywhere.
- **Cloud computing.** Instead of building a data center, a company can rent storage and processing by the hour and scale it to millions of devices.
- **Smartphones.** A phone became the default user interface for almost every consumer device.

The result is scale. IoT Analytics estimated 18.5 billion connected IoT devices at the end of 2024, about 21 billion at the end of 2025, and roughly 39 billion by 2030 (Sinha, 2025). Wi-Fi, Bluetooth, and cellular technologies together account for most of these connections.

> **Key Concept: IoT, IIoT, and AIoT.** *IoT* is the general term for connected physical devices. The *Industrial Internet of Things (IIoT)* applies IoT to manufacturing, energy, transportation, and other industrial settings, where reliability and safety requirements are much stricter. The *Artificial Intelligence of Things (AIoT)* combines IoT with artificial intelligence so that devices and systems can learn, predict, and act on their own (Section 1.8 and Chapter 8).

1.2 IoT in daily life and industry

IoT devices are usually grouped by who uses them and what they do. Table 1.1 summarizes the main categories with global examples.

Table 1.1 Major categories of IoT applications

Category	Typical devices	Example value
Consumer and smart home	Smart speakers, thermostats, locks, lights, robot vacuums, wearables	Convenience, energy savings, security
Healthcare (Internet of Medical Things)	Continuous glucose monitors, heart monitors, smart inhalers, hospital asset tags	Remote patient monitoring, fewer readmissions
Agriculture	Soil-moisture probes, weather stations, livestock collars, irrigation controllers	Water and fertilizer savings, higher yields
Industrial (IIoT)	Vibration sensors, machine controllers, robots, energy meters	Predictive maintenance, less downtime
Transportation and logistics	Fleet trackers, cold-chain loggers, connected cars, RFID pallets	Route optimization, product quality, safety
Smart cities and utilities	Smart meters, streetlights, parking sensors, air-quality stations	Lower operating cost, better public services
Energy	EV chargers, home batteries, solar inverters, grid sensors	Balancing renewable supply with demand

Several of these categories appear later in the book as case studies. Chapter 9, for example, compares Xiaomi, which built one of the world's largest consumer IoT ecosystems, with Enel X, which uses connected batteries, chargers, and building systems to create virtual power plants.

1.3 Core components of an IoT system

Every IoT system, from a fitness tracker to a wind farm, is built from the same family of components. Figure 1.1 later shows how they fit into layers. First, each component deserves a short definition.

Sensors detect a physical quantity, such as temperature, light, motion, pressure, humidity, or soil moisture, and convert it into an electrical signal. Sensors are the system's "eyes and ears."

Actuators do the opposite. They take an electrical command and turn it into physical action: a valve opens, a motor turns, a relay switches a heater, a speaker sounds an alarm. Actuators are the system's "hands and muscles."

Microcontrollers (MCUs) are small computers on a single chip that read the sensors, run the device's logic, drive the actuators, and manage communication. Chapter 2 explains why IoT devices use MCUs rather than the processors found in laptops.

Firmware is the low-level software stored in the MCU's non-volatile memory. Without firmware, the microcontroller is a silent piece of silicon. Chapter 5 is devoted to firmware.

Wireless access is the "first mile" link that moves data off the device: Bluetooth Low Energy, Zigbee, Wi-Fi, LoRaWAN, or cellular IoT. Chapter 3 compares these technologies.

Gateways connect devices that cannot reach the internet on their own. A gateway might translate Zigbee into Wi-Fi, buffer data during an outage, or filter noisy readings before they leave the building. In many homes, the gateway is simply the Wi-Fi router.

Cloud services (also called IoT servers or IoT platforms) receive data from thousands or millions of devices, store it, analyze it, and send commands back. Chapter 4 covers cloud platforms such as AWS IoT Core and Azure IoT Hub.

User interfaces are the apps, web dashboards, voice assistants, and automated reports through which people see the data and control the devices.

One more ingredient is invisible but essential: **power**. A device powered from the mains can run a fast processor and a Wi-Fi radio all day. A device running on a coin cell must sleep almost all of the time. Power is often the constraint that drives every other design decision.

1.4 The four-layer IoT architecture

An **architecture** is a model that divides a complex system into parts with clear responsibilities. Several IoT architectures exist, with three, four, five, or even seven layers. This book uses the **four-layer architecture** as its main model because it is simple enough to apply to any product and detailed enough to separate the important design decisions. Figure 1.1 shows the four layers.

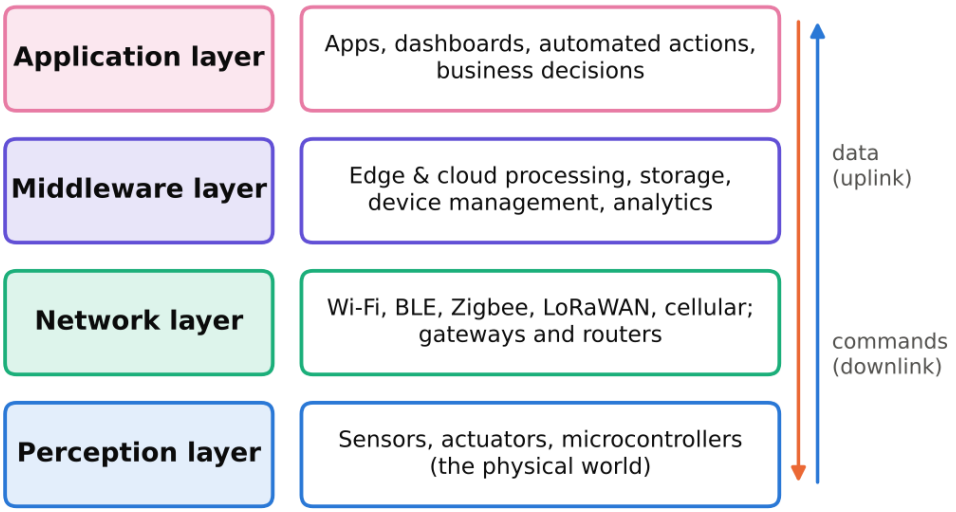


Figure 1.1 The four-layer IoT architecture. Data flows upward from the physical world to applications; commands flow downward to actuators.

1. Perception layer (the sensing layer). This layer touches the physical world. It contains sensors, actuators, and the microcontroller that reads and drives them. Its job is to turn physical phenomena into data and data into physical action.

2. Network layer (the transmission layer). This layer moves data between devices and the rest of the system. It includes radios, gateways, routers, and the protocols that carry data. Engineers choose network technologies by range, power budget, bandwidth, and cost.

3. Middleware layer (the processing layer). This layer receives, stores, and processes data. It includes edge computing on gateways and local servers, as well as cloud platforms that manage device identity, route messages, run rules, and store history.

4. Application layer (the value layer). This layer delivers value to people and organizations: dashboards, mobile apps, alerts, automated actions, and business decisions. A smart irrigation app that shows water use per field, or a maintenance system that opens a work order when a motor vibrates abnormally, belongs here.

Table 1.2 lists typical hardware and software at each layer.

Table 1.2 Hardware and software at each layer of the four-layer architecture

Layer	Main function	Typical hardware	Typical software and protocols
Perception	Sense and act	Sensors, actuators, MCUs, batteries	Firmware, drivers, RTOS
Network	Transmit	Radios, antennas,	BLE, Zigbee, Wi-Fi, LoRaWAN,

Layer	Main function	Typical hardware	Typical software and protocols
		gateways, routers	LTE-M, IP, MQTT
Middleware	Process and store	Edge servers, gateways, cloud data centers	Message brokers, databases, rules engines, analytics
Application	Deliver value	Smartphones, tablets, PCs, displays	Mobile apps, dashboards, enterprise systems

Key Concept: Uplink and downlink. Data moving from sensors toward applications travels on the **uplink** (also called northbound). Commands moving from applications toward actuators travel on the **downlink** (southbound). A complete IoT system needs both: the uplink to know what is happening, and the downlink to do something about it.

Other architectural models

The four-layer model is not the only one. Later chapters use two variants that describe the same systems from different angles:

- The **three-tier model** (device, edge, cloud) in Chapter 4 focuses on *where computation happens*.
- The **five-tier model** (perception, proximity network, edge gateway, cloud, presentation) in Chapter 7 splits the network layer into a short-range link and a gateway, because in the class demo system these are separate pieces of hardware with different protocols and security mechanisms.

All of these models describe the same flow of data from the physical world to people and back. Students should be comfortable translating between them.

1.5 How an IoT system works: sense, connect, process, interact

A practical way to describe IoT operation is a four-step framework:

1. **Sensing.** Devices measure the physical world. A soil probe reports 18 percent volumetric water content.
2. **Connectivity.** The reading travels over a wireless link and the internet to a server. The probe sends its reading by radio to the controller, which forwards it over Wi-Fi.
3. **Processing.** Software, on the device, at the edge, or in the cloud, interprets the data and decides what to do. The controller compares 18 percent with a set point of 22 percent and decides that the block needs water.
4. **Interface and action.** The decision is carried out and reported. The valve opens, and the grower's app shows "Block 3 irrigating, 25 minutes remaining."

Key characteristics of IoT systems

IoT systems share several characteristics that make them different from traditional information systems:

- **Interconnectivity.** Everything can, in principle, connect to everything else, which creates enormous opportunity and enormous attack surface.
- **Heterogeneity.** Devices from different vendors use different hardware, operating systems, and protocols, so interoperability is a constant challenge.
- **Dynamic state.** Devices sleep, wake, move, lose connectivity, and run out of battery. The system must tolerate change.
- **Enormous scale.** A single company may manage millions of devices. Anything done manually per device, such as installing a certificate or updating firmware, becomes impossible.
- **Resource constraints.** Many devices have kilobytes of memory, milliwatts of power, and slow, intermittent links.
- **Safety.** IoT devices can unlock doors, dose medicine, and stop machines. A software bug can cause physical harm.

Worked Example 1.1: How much data does an IoT fleet generate?

A smart irrigation company has 10,000 controllers in the field. Each controller sends one status message every 15 minutes. Each message is 50 bytes. How much data does the fleet send per day?

Messages per controller per day = 24 hours × 4 messages per hour = 96 messages.

Data per controller per day = 96 × 50 bytes = 4,800 bytes = 4.8 KB.

Data for the fleet per day = 10,000 × 4.8 KB = 48,000 KB = 48 MB.

Interpretation. Forty-eight megabytes per day is small for a cloud platform. The challenge is not the volume of data but the *number of connections*: 10,000 devices, each with its own identity, firmware version, and network conditions. If each controller also uploaded a 2 MB camera image once per day, the fleet would generate 20 GB per day, more than 400 times as much. Data volume depends far more on *what* is sent than on *how many* devices send it.

1.6 Thinking in systems

IoT products fail more often because their parts do not work together than because any single part is broken. A sensor may be accurate, but if the firmware samples it at the wrong time, the data is useless. A cloud service may be reliable, but if the gateway cannot reconnect after an outage, data is lost. **Systems thinking** is a way of analyzing a product as a whole. Five concepts are especially useful:

- **Elements** are the parts of the system: the sensor, the controller, the valve, the Wi-Fi module, the cloud service, and the app.
- **Interconnections** are the relationships and flows between elements: the soil-moisture reading that flows from the sensor to the controller, or the command that flows from the controller to the valve.
- **Purpose** (or function) is what the system is for. For a smart irrigation controller, the purpose is to deliver the right amount of water at the right time. Purpose is the

most important and least visible part of a system. It is revealed by behavior, not by a label.

- **Boundary** separates the system from everything else. Deciding where the boundary lies is a design choice. Is the weather forecast service part of the system or part of its environment?
- **Interface** is the point where two elements, or the system and its environment, meet and exchange information or energy. A sensor port, a radio protocol, an API, and a touchscreen are all interfaces.

Everything outside the boundary is the system's **environment**: the weather, the soil, the plants, the water supply, and the user. Figure 1.2 shows these concepts applied to a smart irrigation controller.

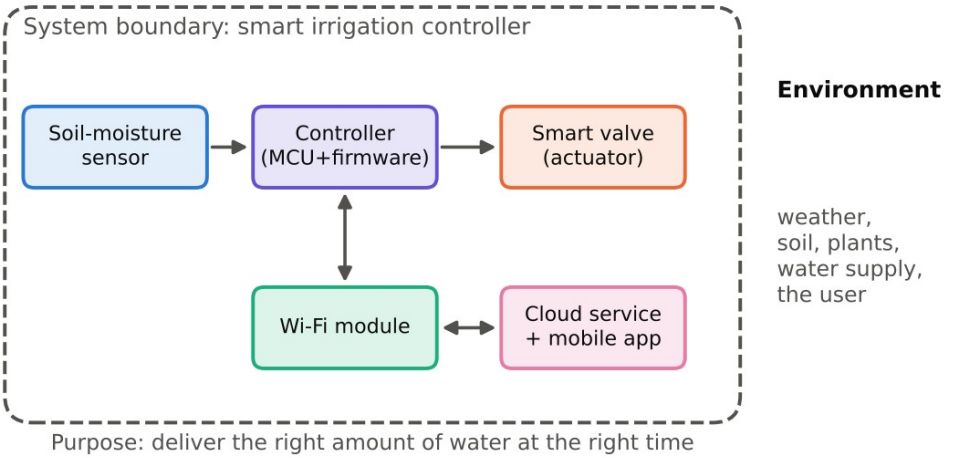


Figure 1.2 A systems view of a smart irrigation controller, showing elements, interconnections, boundary, environment, and purpose.

Feedback loops

The most important pattern in IoT systems is the **feedback loop**. The controller measures soil moisture, acts on the valve, and then measures again to see the effect of its action. This closed loop is what allows the system to run without people. Chapter 2 describes the six steps of the device-level control loop in detail, and Chapter 7 tests a closed loop in the class demo system.

Feedback loops can also behave badly. If the controller opens the valve at 21.9 percent moisture and closes it at 22.0 percent, it will switch on and off constantly as readings fluctuate. Engineers add **hysteresis**, a gap between the switch-on and switch-off thresholds, to prevent this (Chapter 2).

Emergent behavior

A system can show behavior that none of its parts shows alone. A single smart thermostat saves energy in one home. Ten thousand smart thermostats connected to a utility can reduce peak demand on the grid during a heat wave, which no single thermostat could do. This **emergent** property is the basis of the virtual power plant business model in Chapter 9. Emergence works in the other direction too: thousands of poorly secured cameras became a botnet that disrupted major websites in 2016 (Section 1.7).

1.7 Looking inside a real IoT device

A **teardown** is an analysis of a product that identifies its components and how they work together. Engineers perform physical teardowns with screwdrivers; students can perform a "paper teardown" from product documentation, photographs, and videos. The smart irrigation controller is a good subject because every layer of the architecture is visible.

Hardware. Opening a typical Wi-Fi irrigation controller reveals:

- A **power supply** that converts mains alternating current into low-voltage direct current for the electronics and, in many models, 24 V alternating current for the valve solenoids.
- A **microcontroller** that runs the schedule and reads the sensor inputs.
- A **display and buttons** for local control when the network is down.
- **Sensor ports** for rain and soil-moisture sensors.
- **Valve driver circuits** (relays or triacs) that switch each irrigation zone on and off. These are the actuator interfaces.
- A **Wi-Fi module and antenna** that connect the controller to the home or farm router.

Firmware. The controller's firmware reads the sensors, runs the watering schedule, switches the valves, keeps time, stores settings in non-volatile memory so they survive power cuts, and manages the Wi-Fi connection.

Network. The controller connects to the local Wi-Fi network, which acts as its gateway to the internet. Wi-Fi is reasonable here because the controller is mains powered and usually mounted near a building. A battery-powered soil probe in a distant field, by contrast, would need a lower-power, longer-range technology such as LoRaWAN (Chapter 3).

Cloud. The controller maintains a connection to the manufacturer's cloud service. The NxEco controller, this book's running application case (Section 1.10), runs its IoT server on rented infrastructure from a general-purpose cloud provider (Akamai, formerly Linode) rather than on a specialized IoT platform. The cloud stores schedules and history, pulls weather forecasts, and relays commands from the app.

Application. The phone app lets the user set schedules, start a zone manually, and see water use. The cloud may also run "smart" features automatically, such as skipping a cycle when rain is forecast.

Worked Example 1.2: Mapping a smart thermostat to the four layers

Place each component of a smart thermostat in the correct layer: (a) temperature and humidity sensor; (b) relay that switches the furnace; (c) Wi-Fi radio; (d) home router; (e) manufacturer's cloud service that stores schedules and learns the household's routine; (f) phone app; (g) monthly energy report emailed to the user.

Component	Layer	Reason
(a) Temperature and humidity sensor	Perception	Measures the physical environment
(b) Furnace relay	Perception	Actuator that changes the physical world
(c) Wi-Fi radio	Network	Transmits data off the device
(d) Home router	Network	Gateway between the home network and the internet
(e) Cloud service	Middleware	Stores, processes, and learns from data
(f) Phone app	Application	User interface for monitoring and control
(g) Energy report	Application	Delivers value (insight) to the user

Note that the thermostat itself spans two layers: its sensor and relay are in the perception layer, while its radio is in the network layer. A single physical product often spans several layers.

1.8 Security and privacy: a first look

Every connected device is a computer on a network, and every computer on a network can be attacked. IoT devices are attractive targets because they are numerous, often poorly protected, physically accessible, and rarely updated. In October 2016, the **Mirai** botnet infected hundreds of thousands of cameras and home routers that still used factory-default usernames and passwords. The attackers used these devices to flood the domain name provider Dyn with traffic, making major websites unreachable for users in North America and Europe for several hours.

Privacy risks are just as serious. A smart meter that reports electricity use every few seconds can reveal when a household wakes up, cooks, and goes on vacation. A fitness tracker reveals location and health. A smart speaker records voice. The data may be shared with partners, sold, stolen, or requested by authorities, often in ways that users do not expect.

The basic defenses are simple to state and hard to implement at scale:

- **Unique credentials** for every device, never shared default passwords.
- **Encryption** of data in transit and at rest.

- **Secure updates** delivered over the air, so that vulnerabilities can be fixed after sale.
- **Data minimization**: collect only what the purpose requires.
- **Transparency**: tell users what is collected and why.

Governments are now turning these practices into law. The United Kingdom banned universal default passwords on consumer connectable products in 2024, and the European Union's Cyber Resilience Act will require security by design and multi-year security updates for products with digital elements sold in the EU, with full obligations applying from December 2027. Chapter 6 covers IoT security, privacy, ethics, and sustainability in depth.

Global Perspective: The same device, different rules. A smart camera sold in the European Union must meet the General Data Protection Regulation (GDPR) when it processes images of people, and from 2027 the Cyber Resilience Act. The same camera sold in the United Kingdom must meet the Product Security and Telecommunications Infrastructure (PSTI) regime. In the United States, manufacturers can apply for the voluntary U.S. Cyber Trust Mark label. Companies that sell globally usually design to the strictest market's rules so that one product works everywhere.

1.9 From smart to intelligent: the Artificial Intelligence of Things

A traditional "smart" IoT device follows fixed rules written by an engineer: *if soil moisture is below 22 percent, open the valve*. The device is **reactive**. It responds to conditions after they occur, and anything more sophisticated, such as finding patterns in a season of data, is done later by people.

The **Artificial Intelligence of Things (AIoT)** combines IoT with artificial intelligence. If IoT is the nervous system that senses the world, AI is the brain that decides what to do about it (Kirvan et al., 2025). AIoT systems analyze data as it arrives, predict what will happen, and act before a problem occurs. They are **proactive**. Figure 1.3 contrasts the two approaches.

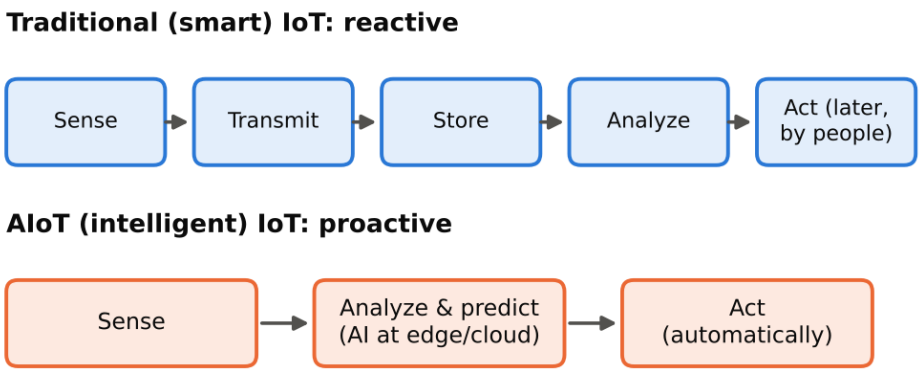


Figure 1.3 Traditional IoT follows a linear sense, transmit, store, analyze sequence and relies on people to act. AIoT shortens the loop to sense, analyze, act.

Two mechanisms make this possible:

- **Edge AI** runs trained models on the device or gateway itself. A camera on a factory line can inspect every part in milliseconds without sending video to the cloud. Edge AI reduces latency, saves bandwidth, and keeps sensitive data local.
- **Fleet learning** uses the cloud to train models on data from thousands or millions of devices. Each device sees only its own small world, but the fleet sees patterns that no single device could learn. The improved model is then pushed back to every device.

Two examples illustrate the range of AIoT:

- An **autonomous vehicle** is an AIoT system on wheels. Cameras, radar, and ultrasonic sensors feed onboard AI that detects lanes, vehicles, pedestrians, and traffic signs in real time. The cloud collects difficult driving situations from the whole fleet, retrains the models, and distributes updates over the air.
- A **small learning robot**, such as an educational robot or a robot vacuum, uses onboard sensors and modest processors to map a room, avoid obstacles, and improve its route over time.

For the smart irrigation controller, an AIoT upgrade might combine soil-moisture readings, weather forecasts, and a crop-water model to predict how much water each block will need over the next three days, then schedule irrigation for the hours when electricity and water are cheapest. That is the difference between reacting to dry soil and preventing it. Chapter 8 explains where AI can run in an IoT system and what it costs.

1.10 IoT application case: NxEco smart irrigation

Throughout this book, one commercial product serves as a running **IoT application case**: the **NxEco** smart irrigation system, designed by NxEco, Inc. of Newport Beach, California. It is the smart irrigation controller introduced in Section 1.7, and it is small enough to understand completely while containing every layer of a real IoT system. Each later chapter returns to NxEco and examines the part of the system that the chapter teaches (Table 1.3).

> **The NxEco system at a glance.** A wall-mounted controller (the HWN12-200, with 12 zones expandable to 36) switches 24 V AC irrigation valves. It connects over 2.4 GHz Wi-Fi or Ethernet to the NxEco cloud, which collects local weather and evapotranspiration (ET) data and adjusts every zone's watering time each day. Owners and landscapers control it from the NxEco mobile app, the MySmartDrop web portal, the SNAP professional portal, or a voice assistant. The product is certified under the U.S. EPA WaterSense program, and NxEco states savings of up to 40 to 50 percent of landscape water (NxEco, n.d.-a, n.d.-b).

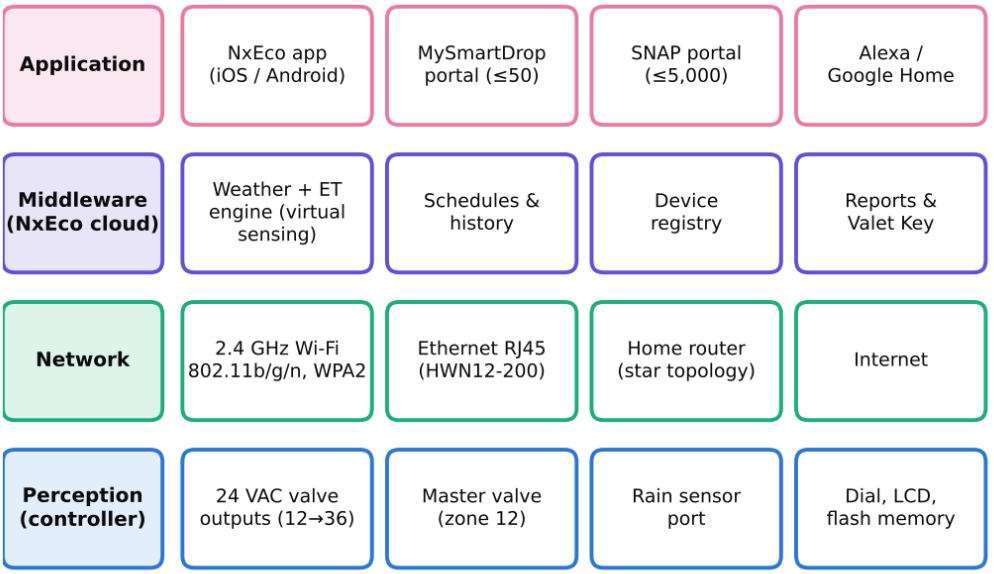


Figure 1.4 The NxEco smart irrigation system mapped to the four-layer IoT architecture.

Table 1.3 Where the NxEco case appears in this book

Chapter	NxEco technology examined	Key idea
2 Sensing and hardware	Weather and ET collection as "virtual sensing"; rain sensor; 24 V AC valve outputs	Weather data can replace some physical sensors, at a cost in accuracy
3 Networking	2.4 GHz Wi-Fi, WPA2, Ethernet, button-based provisioning	Why a mains-powered controller can use Wi-Fi directly
4 Edge and cloud	Local timer survivability; cloud weather engine; three portals	Splitting work between the device and the cloud
5 Firmware and software	Schedules, registration, firmware versions, app releases	Software life cycle of a consumer IoT product
6 Security and sustainability	Valet Key delegation, credential hygiene, water savings	Least privilege and environmental value
7 System design	NxEco compared with the five-tier demo system	When a gateway is needed and when it is not
8 AIoT	From rule-based ET adjustment to learning irrigation	How AI could make the product proactive
9 Business	No-subscription model, rebates, professional portal	Monetizing a consumer IoT product

Students can see the product in operation through NxEco's public website, product videos, and manuals, and many will recognize the pattern in other smart irrigation brands sold worldwide.

Chapter summary

- The Internet of Things connects physical objects that sense, act, process, and communicate. The term dates from 1999, but the explosion in scale came from cheap sensors, cheap microcontrollers, ubiquitous wireless connectivity, cloud computing, and smartphones.
- Every IoT system contains sensors, actuators, microcontrollers, firmware, wireless access, gateways, cloud services, and user interfaces, all shaped by the device's power source.
- The four-layer architecture divides IoT systems into perception, network, middleware, and application layers. Data flows up (uplink) and commands flow down (downlink).
- Systems thinking analyzes a product through its elements, interconnections, purpose, boundary, and interface. Feedback loops let IoT systems run without people; emergent behavior appears only when many parts work together.
- Connected devices create security and privacy risks, from botnets built on default passwords to inferences about daily routines. Regulation is turning secure design into a legal requirement.
- AIoT adds learning and prediction, moving IoT from reactive to proactive operation through edge AI and fleet learning.

Key terms

actuator; AIoT; application layer; architecture; boundary; downlink; edge AI; element; emergent behavior; environment; feedback loop; firmware; fleet learning; gateway; hysteresis; IIoT; interconnection; interface; Internet of Things; M2M; microcontroller (MCU); middleware layer; network layer; perception layer; purpose; sensor; systems thinking; teardown; uplink.

Review questions

Multiple choice

1. Who coined the term "Internet of Things" in 1999, in the context of supply chain management?
    a. Kevin Ashton
    b. Tim Berners-Lee
    c. Vint Cerf
    d. Mark Weiser
2. In the four-layer architecture, which layer chooses protocols based on range, power budget, and bandwidth?

a. Perception
b. Middleware
c. Network
d. Application

3. A smart irrigation controller opens a valve when soil is dry. The valve is an example of a(n):
   a. sensor
   b. gateway
   c. middleware service
   d. actuator
4. In systems thinking, everything outside the boundary of a system is called the system's:
   a. element
   b. purpose
   c. interconnection
   d. environment
5. Which statement best describes the uplink?
   a. Commands flowing from an app to an actuator
   b. Data flowing from sensors toward applications
   c. Firmware flowing from the cloud to a device
   d. Power flowing from a battery to a radio
6. Which characteristic of IoT systems makes manual, per-device configuration impractical?
   a. Heterogeneity
   b. Enormous scale
   c. Safety
   d. Dynamic state
7. The Mirai botnet of 2016 spread mainly by exploiting:
   a. weak encryption algorithms
   b. factory-default usernames and passwords
   c. flaws in the MQTT protocol
   d. physical tampering with devices
8. In the AIoT analogy, if IoT is the nervous system, AI is:
   a. the muscles
   b. the skin
   c. the brain
   d. the power supply

9. Training a model on data from thousands of devices and pushing the improved model back to each device is called:
    a. edge filtering
    b. fleet learning
    c. protocol translation
    d. data minimization
10. Which layer delivers value through dashboards, alerts, and business decisions?
    a. Perception b. Network c. Middleware d. Application

Short answer

11. Explain the difference between a sensor and an actuator, and give one example of each from a smart home.
12. Why is the purpose of a system described as its most important but least visible element? Use an IoT example.
13. Identify one privacy risk created by a smart electricity meter and one practice that reduces it.
14. Explain how hysteresis prevents a feedback loop from switching on and off too often.

Applied problems

15. A logistics company fits 2,000 refrigerated trucks with a temperature logger that sends a 64-byte message every 30 seconds. (a) How many bytes does one truck send per day? (b) How much data does the fleet send per day, in megabytes? (c) The company considers adding a 300 KB door-camera image every time a door opens, about 40 times per day per truck. By what factor does the daily data volume grow?
16. Choose a fitness tracker. Map at least six of its components to the four layers of the IoT architecture, and identify the system's purpose, boundary, and one interface.
17. **NxEco case.** Using Figure 1.4, place each of the following NxEco elements in a layer of the four-layer architecture and identify one interface between layers: valve output, Wi-Fi radio, weather engine, MySmartDrop portal, rain sensor.

Lab 1: IoT device teardown

Collaboration 1 and GAP Discussion. Teams of two or three students analyze one device together in the live session. Each student then posts individually in the GAP Discussion. The block diagram is the Collaboration 1 component; the written post and peer replies form the GAP Discussion.

Objective. Map a real IoT product to the four-layer architecture and analyze its engineering and business choices.

Materials. Any IoT product that a team member uses at home, at work, or on campus (for example, a smart thermostat, fitness tracker, smart lock, irrigation controller, or industrial sensor); the manufacturer's website and manuals; teardown photos or videos; a diagramming tool or paper.

Procedure.

1. **Choose the device** and collect its documentation: product page, user manual, regulatory filings (radio certification documents often include internal photos), and any published teardown.
2. **List the elements.** Identify the sensors, actuators, microcontroller, radio, power source, gateway (if any), cloud service, and app.
3. **Draw the block diagram.** Draw the device across the four layers: perception (sensors, actuators, MCU), network (wireless technology and hub or gateway), middleware (edge or cloud processing and storage), and application (app, dashboard, or automated action). Use arrows to show the uplink and downlink. A photo of a hand-drawn diagram is acceptable.
4. **Trace the data flow.** Follow one reading from the sensor to the user, naming each element and interface it passes through.
5. **Analyze the wireless choice.** Explain why the manufacturer likely chose its wireless technology, considering range, power, bandwidth, and cost.
6. **Identify one security or privacy risk** specific to the device, and propose one practical mitigation.
7. **Describe the business value** of the device and one AIoT enhancement that would make it proactive rather than reactive.
8. **Write the primary post** (at least 200 words) covering steps 4 to 7, and attach the diagram.
9. **Reply to at least two classmates** (50 or more words each). Propose an alternative design choice for each classmate's device, such as different connectivity, edge versus cloud processing, or a different power source, and explain the engineering trade-off.

Deliverables. Block diagram, primary post, two peer replies, APA 7th edition citations for all outside sources.

Self-check. Does the diagram show all four layers? Does every arrow have a direction? Is the wireless choice explained with at least two of the four factors? Is the risk specific to this device rather than generic?

References and further reading

Crockett, E. (2023, November 10). *The big list of IoT devices and IoT products*. Datamation. https://www.datamation.com/mobile/85-top-iot-devices/

Flynn, S. (2022, April 18). *A look at the top 5 IoT device types*. TechTarget. https://www.techtarget.com/iotagenda/tip/A-look-at-the-top-5-IoT-device-types

Gillis, A. S., & Yasar, K. (2025, July 21). *What is IoT (Internet of Things)?* TechTarget. https://www.techtarget.com/iotagenda/definition/Internet-of-Things-IoT

IoT For All. (n.d.). *IoT For All podcast* [Audio podcast]. https://www.iotforall.com/podcast

Kirvan, P., Yasar, K., & Gillis, A. S. (2025, August 1). *What is the artificial intelligence of things (AIoT)?* TechTarget. https://www.techtarget.com/iotagenda/definition/Artificial-Intelligence-of-Things-AIoT

Li, L. (2026). *IoT Education Course demo system* [Interactive web application]. GitHub Pages. https://liangli99.github.io/IoT-Demo/dashboard.html

Motivated Tech Space. (2025). *Internet of Things architecture for beginners* [Video]. YouTube. https://www.youtube.com/watch?v=9OEIVtYw5MY

NxEco. (2016). *Smart irrigation controller* [Video]. YouTube. https://www.youtube.com/watch?v=QYmRW3LWXDA

NxEco. (n.d.-a). *NxEco smart watering* [Website]. https://www.nxeco.com

NxEco. (n.d.-b). *Smart controller specification* (Ver. 10.1) [PDF]. https://www.nxeco.com/Public/public/file/manual/Nxeco_Smart_Control_Spec_Ver10.1.pdf

NxEco. (n.d.-f). *Features* [Web page]. https://www.nxeco.com/features.html

POND IoT. (2025). *What is AIoT? The power of AI and IoT combined* [Video]. YouTube. https://www.youtube.com/watch?v=wXVBHTfArrs

Rodriguez, A. (2026, February 23). *33 Internet of Things examples you should know*. Built In. https://builtin.com/internet-things/iot-examples

Simplilearn. (2021). *IoT | Internet of Things | What is IoT? | How IoT works?* [Video]. YouTube. https://www.youtube.com/watch?v=6mBO2vqLv38

Sinha, S. (2025, October 28). *Number of connected IoT devices growing 14% to 21.1 billion globally in 2025*. IoT Analytics. https://iot-analytics.com/number-connected-iot-devices/

Techsters Haven. (2025). *The evolution of IoT* [Video]. YouTube. https://www.youtube.com/watch?v=H9XJ83t0O2U

CHAPTER 2

Inside the IoT Device: Sensors, Actuators, and Microcontrollers

Learning Objectives

After studying this chapter, students will be able to:

1. Describe the three core components of every IoT edge device: sensor, controller, and actuator.
2. Explain how a physical signal becomes digital data through transduction, conditioning, sampling, quantization, and encoding, and calculate ADC resolution.
3. Compare sensors using sensitivity, accuracy, precision, range, linearity, and response time.
4. Classify actuators and explain their constraints of power, latency, and wear.
5. Explain the architecture of a microcontroller and justify why IoT devices use MCUs rather than CPUs.
6. Explain why Arm and RISC architectures dominate low-power devices.
7. Walk through the six-step sense, process, act control loop and design hysteresis for a real application.

Where this fits: Week 2. This chapter supports Collaboration 2 *Build and Justify an IoT Control Loop* and Assignment 1 *Control-Loop Lab Report and MCU vs. CPU Research*. Lab 2 at the end of the chapter is the step-by-step guide for the Wokwi build.

Opening case: a car full of sensors

A modern electric vehicle is one of the most sensor-dense products that consumers can buy. Its cameras recognize lanes, traffic signs, and pedestrians. Radar measures the distance and speed of the vehicle ahead for adaptive cruise control. Ultrasonic sensors in the bumpers guide automated parking. Inside the battery pack, dozens of temperature sensors watch every group of cells so that the battery management system can slow charging before a cell overheats. Accelerometers detect a collision and trigger the airbags in milliseconds.

Each of these functions follows the same pattern. A sensor detects something in the physical world. A small processor decides what it means. An actuator, such as a brake, a steering motor, a cooling pump, or an airbag inflator, does something about it. This chapter opens the box and looks at those three parts.

2.1 The three core components of an IoT device

A single IoT device usually spans two functional areas: the **perception layer**, where it senses and acts on the physical world, and a **pre-processing** area, where it digitizes data and runs local logic. Inside, almost every device contains three essential components (Figure 2.3 in Section 2.8 shows how they connect in a loop):

- **Sensor (physical input).** Detects an external condition such as heat, light, or pressure and converts it into an electrical signal.
- **Controller (processing).** The local "brain," usually a microcontroller unit (MCU), interprets the data, runs edge logic, and decides on a response.
- **Actuator (physical output).** Converts an electrical command into physical action: movement, heat, light, or sound.

Data flows from the sensor to the controller; commands flow from the controller to the actuator. Table 2.1 extends this view to the major hardware found at each layer of the four-layer architecture from Chapter 1.

Table 2.1 Major hardware at each layer

Layer	Major hardware
Perception (sensing)	Sensors (data input), actuators (physical output), microcontrollers
Network (transmission)	Radio modules, antennas, gateways, routers
Middleware (processing)	Edge servers, gateways with high-performance microprocessors, cloud servers
Application (value)	Smartphones, tablets, PCs, wall displays

2.2 Sensors

A **sensor** is a device that detects and responds to an input from the physical environment, such as temperature, motion, light, pressure, or humidity, and sends the resulting data to a system for processing or decision-making (AVSystem, 2024). The part of the sensor that actually responds to the physical quantity is the **transducer** or sensing element: a thermistor whose resistance changes with temperature, a photodiode whose current changes with light, or a tiny silicon mass in an accelerometer that moves when the device accelerates.

The role of sensors in IoT

Sensors are the bridge between physical reality and digital insight. In an IoT architecture they enable five capabilities:

1. **Data collection.** Capturing raw data from the physical environment.

2. **Signal conversion.** Turning an analog quantity into digital data (Section 2.3).
3. **Transmission.** Sending data, through the MCU, over IoT protocols such as MQTT, CoAP, or HTTP.
4. **Integration.** Working together with actuators, gateways, and cloud platforms.
5. **Control and feedback.** Enabling automation, for example turning on a fan when the temperature exceeds a threshold.

Types of sensors

Table 2.2 lists common sensor types, the physical principle each one uses, and a familiar example.

Table 2.2 Common sensor types

Sensor type	Operating principle	Example
Temperature	Resistance, voltage, or current changes with temperature (thermistor, RTD, thermocouple, silicon bandgap)	Digital thermometer, battery pack monitor
Light (photoelectric)	Light intensity changes electrical properties (photodiode, phototransistor, light-dependent resistor)	Automatic screen brightness in phones
Pressure	Pressure deforms a material, changing resistance or capacitance (piezoresistive, capacitive)	Tire-pressure monitor, barometer
Motion and acceleration	Inertia moves a micro-structure; MEMS or piezoelectric sensing	Accelerometer in phones and wearables
Proximity	Detects presence through infrared, ultrasonic, or capacitive fields	Parking sensors, touchless taps
Humidity	Moisture changes capacitance or resistance of a polymer film	Weather stations, HVAC
Gas and chemical	Chemical reaction or optical absorption changes an electrical property	CO_2 and carbon monoxide detectors
Magnetic	Hall effect or magnetoresistance responds to a magnetic field	Door and window sensors, motor position
Soil moisture	Capacitance or resistance of soil changes with water content	Smart irrigation probes

The accelerometer in a smartphone is a good example of how much engineering hides inside a tiny part. It contains a microscopic silicon mass suspended on springs between fixed plates. When the phone tilts or accelerates, the mass moves slightly, the capacitance between the plates changes, and on-chip electronics convert that change into a number. The phone uses those numbers to rotate the screen, count steps, and detect a fall (engineerguy, 2013).

Analog, digital, and smart sensors

Sensors deliver their output in one of three ways:

- An **analog sensor** produces a continuously varying voltage or current. The MCU must digitize it with its analog-to-digital converter. A thermistor in a voltage divider and the TMP36 temperature sensor are analog.
- A **digital sensor** produces a digital signal directly, often a simple on/off output (a reed switch or a passive infrared motion detector) or a timed pulse train (the DHT22 temperature and humidity sensor).
- A **smart sensor** contains its own ADC, calibration data, and sometimes a small processor, and it reports engineering values over a digital bus such as **I²C** or **SPI**. The Bosch BME280 (temperature, humidity, and pressure) and the MPU6050 motion sensor used in the class demo system are smart sensors.

Smart sensors simplify design and improve accuracy because calibration happens in the factory, but they cost more and still consume power while they convert.

2.3 From physical signal to digital data

A microcontroller can only process numbers, so every analog measurement passes through a conversion pipeline (Figure 2.1):

1. **Physical stimulus detection.** The transducer responds to the environment. For example, a thermistor's resistance falls as temperature rises.
2. **Signal conditioning.** The raw signal is amplified, filtered to remove noise, and shifted into the voltage range the ADC accepts.
3. **Sampling (discretization in time).** The continuous signal is measured at regular intervals. The **sampling rate** is the number of samples per second.
4. **Quantization (discretization in amplitude).** Each sampled voltage is rounded to the nearest of a fixed set of levels.
5. **Encoding.** Each level is represented as a binary number that the MCU can store, process, and transmit.

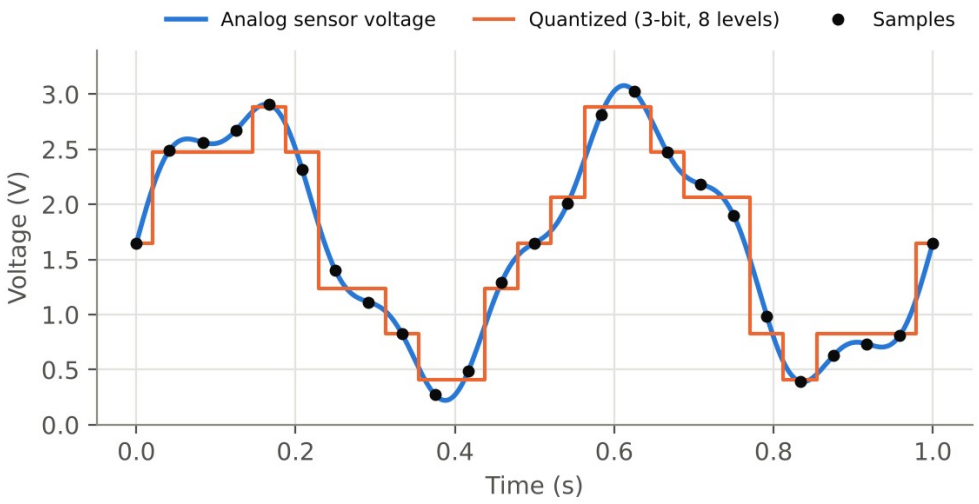


Figure 2.1 Sampling and quantization. The blue curve is the analog signal; black dots are samples; the orange staircase is what a 3-bit converter records. Real converters use 10 to 16 bits, so the steps are far smaller.

Sampling rate and the Nyquist criterion

How often must a signal be sampled? The **Nyquist criterion** states that the sampling rate must be more than twice the highest frequency in the signal, or information is lost and false patterns (aliasing) appear. Room temperature changes over minutes, so one sample every few seconds is plenty. Machine vibration may contain components at several kilohertz, so a vibration sensor may sample 10,000 or more times per second. Sampling faster than necessary wastes energy, memory, and bandwidth, which matters greatly in battery-powered devices.

Resolution of an analog-to-digital converter

An **analog-to-digital converter (ADC)** with n bits divides its input range into 2^n levels. The smallest voltage change it can detect, its **resolution** or one least significant bit (LSB), is:

$$LSB = V_{ref} \div 2^n$$

where V_{ref} is the reference voltage (the full-scale input).

Worked Example 2.1: ADC resolution

An ESP32 has a 12-bit ADC with a 3.3 V reference. An Arduino Uno has a 10-bit ADC with a 5 V reference. What is the resolution of each?

ESP32: number of levels = 2^{12} = 4,096. LSB = 3.3 V ÷ 4,096 = 0.000806 V ≈ **0.81 mV**.

Arduino Uno: number of levels = 2^{10} = 1,024. LSB = 5 V ÷ 1,024 = 0.00488 V ≈ **4.88 mV**.

Interpretation. The ESP32 can, in principle, resolve voltage changes six times smaller. In practice, electrical noise and converter non-linearity reduce the useful resolution, which is one reason engineers often choose a smart sensor with a dedicated, calibrated converter.

Worked Example 2.2: Converting an ADC reading to temperature

A TMP36 analog temperature sensor outputs 500 mV at 0 °C and adds 10 mV per °C. It is connected to the ESP32's 12-bit ADC (0 to 4,095 counts over 0 to 3.3 V). The ADC reads 930. What is the temperature, and what temperature change does one ADC count represent?

Voltage = 930 × 3.3 V ÷ 4,095 = 0.7495 V = 749.5 mV.

Temperature = (749.5 mV − 500 mV) ÷ 10 mV/°C = **24.9 °C**.

One count = 0.81 mV, and 0.81 mV ÷ 10 mV/°C ≈ **0.08 °C** per count.

Interpretation. The converter's resolution (0.08 °C) is much finer than the TMP36's stated accuracy (about ±1 to ±2 °C). Resolution is not accuracy: a reading can change in tiny steps and still be wrong by a whole degree.

2.4 Sensor characteristics

When engineers compare sensors, they use a standard vocabulary (Table 2.3).

Table 2.3 Sensor characteristics

Characteristic	Meaning	Example question
Sensitivity	How small a change in the input the sensor can detect, or how much the output changes per unit of input	Can it detect a 0.1 °C change?
Accuracy	How close a measurement is to the true value	Is 25.0 °C really 25.0 °C?
Precision (repeatability)	How close repeated measurements of the same input are to each other	Do ten readings agree?
Range	The minimum and maximum input the sensor can measure	Does it work at −40 °C in a freezer?
Linearity	Whether the output is proportional to the input across the range	Is the error the same at 0 °C and 80 °C?
Response time	How quickly the output follows a change in the input	Does it react in 1 second or 1 minute?
Drift	Slow change in output over time with a constant input	Is it still accurate after two years?

The difference between **accuracy** and **precision** is often illustrated with a target. A precise but inaccurate sensor puts all its shots in a tight cluster away from the bull's-eye: it is consistent but wrong, usually because of an offset that **calibration** can remove. An accurate but imprecise sensor scatters its shots around the bull's-eye: it is right on average but noisy, and averaging several readings helps. A good sensor is both.

Worked Example 2.3: Choosing a sensor for a vaccine refrigerator

A clinic must keep vaccines between 2 °C and 8 °C and receive an alarm within one minute if the temperature leaves that band. Three candidate sensors are available:

Sensor	Range	Accuracy	Max sampling rate	Interface
A: DHT22	−40 to 80 °C	±0.5 °C	0.5 Hz (one reading every 2 s)	Single-wire digital
B: Bosch BME280	−40 to 85 °C	±0.5 °C at 25 °C, about ±1 °C over full range	Many readings per second	I^2C or SPI
C: Waterproof DS18B20 probe	−55 to 125 °C	±0.5 °C from −10 to 85 °C	About 1 reading per 0.75 s at 12-bit resolution	1-Wire digital

All three cover the range and can sample fast enough for a one-minute alarm. The deciding factor is the environment: a refrigerator is cold and humid, and the sensing point should be inside a vial-like buffer so that brief door openings do not cause false alarms. The **waterproof DS18B20 probe** can be placed in a glycol bottle inside the refrigerator while the electronics stay outside, and it is accurate across the 2 to 8 °C band. Sensor B would be a good second sensor to log humidity and pressure, but it is less accurate at low temperatures. The justification uses three characteristics: range, accuracy within the operating band, and suitability for the environment.

Emerging sensors

Sensor technology continues to advance. Several categories are moving from research into products:

- **Biosensors** detect biological signals such as glucose, proteins, DNA, or pathogens. Continuous glucose monitors worn on the arm are the best-known example.
- **Flexible and wearable sensors** are printed on thin films or fabrics that bend with the body.
- **Self-powered sensors** harvest energy from light, heat, vibration, or radio waves, so they never need a battery change.
- **Neuromorphic sensors**, such as event cameras, process data at the sensor using brain-inspired spiking networks and report only changes, which saves power and bandwidth.
- **DNA-based and molecular sensors** use synthetic DNA strands that bind to a specific chemical or toxin.
- **Quantum sensors** exploit quantum effects to measure magnetic fields, gravity, or time with extreme precision.
- **Smart optical sensors** combine an image sensor with on-chip AI to detect objects without sending images off the device.

2.5 Actuators

If sensors are the "eyes and ears" of a system, **actuators** are its "hands and muscles." An actuator receives a command signal from a controller and converts electrical energy into physical action. This conversion of one form of energy into another is called **transduction** (RealPars, 2020).

Types of actuators

Table 2.4 summarizes the actuators most often found in IoT devices (Oliynyk, 2025).

Table 2.4 Common IoT actuators

Actuator	Motion or effect	How it works	IoT examples
Relay (electromechanical)	On/off switching	A small coil current pulls a contact closed, switching a separate high-power circuit	Switching heaters, pumps, lights
Solid-state relay (SSR)	On/off switching	Semiconductor switch with optical isolation; no moving parts	Compressors, heaters that switch often
Solenoid	Linear motion	Energized coil pulls a metal plunger	Electronic door bolts, valves
DC motor	Continuous rotation	Current in a magnetic field produces torque; speed set by PWM	Fans, pumps, small robots
Servo motor	Precise angle	Motor with built-in position feedback moves to a commanded angle (for example 45°)	Dampers, camera pan, robot joints
Stepper motor	Discrete steps	Moves a fixed angle per pulse, so position is known without feedback	3D printers, lab equipment, smart blinds
Smart valve	Flow control	Electromechanical valve controlled by a solenoid or motor	Irrigation zones, leak prevention
Heater, LED, buzzer	Heat, light, sound	Resistive, semiconductor, or piezoelectric conversion	Status indicators, alarms

Operational constraints of actuators

Sensors have become tiny and frugal thanks to MEMS technology, but actuators often remain the largest and most power-hungry part of a device. Three constraints shape every design:

- **Power consumption.** Moving a mass or heating an element requires much more current than sensing. A small relay coil draws about 70 to 90 mA, a DC fan a few hundred milliamps, and a solenoid lock several amps for a fraction of a second. Actuators often need a separate power supply.

- **Latency.** There is a physical delay between the command and the completed movement. A relay closes in about 10 ms, a servo may take half a second to rotate, and a large irrigation valve may take several seconds to open fully.
- **Wear and tear.** Moving parts degrade through friction, heat, and electrical arcing. A mechanical relay may be rated for around 100,000 operations at full load. A control loop that switches the relay every few seconds can wear it out in weeks, which is why hysteresis (Section 2.8) matters.

Driving an actuator safely

A microcontroller's input/output pins operate at 3.3 V or 5 V and can safely supply only a few milliamps to a few tens of milliamps. They cannot power a motor or a heater directly. Instead, the MCU sends a low-power signal to an **intermediate circuit** or **driver**:

- A **transistor or MOSFET** acts as an electronic switch that lets a small pin current control a large load current.
- A **relay module** includes the transistor, the relay, and a **flyback diode** that absorbs the voltage spike produced when a coil is switched off.
- An **opto-isolator** passes the signal through light so that the high-power side is electrically separated from the delicate MCU.
- A **motor driver** chip (an H-bridge) lets the MCU control a motor's speed and direction with pulse-width modulation (PWM).

Worked Example 2.4: Can the MCU drive the relay coil directly?

A 5 V relay coil has a resistance of about 70 Ω. An ESP32 GPIO pin is recommended for no more than about 20 mA continuous (the absolute maximum is about 40 mA). Can the pin drive the coil directly?

Coil current = V ÷ R = 5 V ÷ 70 Ω ≈ **71 mA**.

The coil needs about 3.5 times the recommended pin current and also needs 5 V, while the ESP32 pin supplies only 3.3 V. The answer is **no**. The design needs a transistor or MOSFET driver (or a ready-made relay module) powered from the 5 V supply, plus a flyback diode across the coil. This is the "relay or driver circuit" that Collaboration 2 asks students to identify.

2.6 The microcontroller: the brain of the device

A **microcontroller unit (MCU)** is a computer on a single chip. Unlike the central processing unit (CPU) in a laptop, which relies on separate chips for memory and input/output, an MCU integrates the processor, memory, and peripherals on one integrated circuit (Hashemi-Pour & Lutkevich, 2024). In an IoT device the MCU is the most critical hardware component: it reads the sensors, processes data locally, controls the actuators, and manages the radio. Without an MCU, a sensor is just a component whose resistance changes; with one, it becomes an intelligent node in a global network.

Core components of an MCU

Figure 2.2 shows the main blocks inside a typical IoT microcontroller.

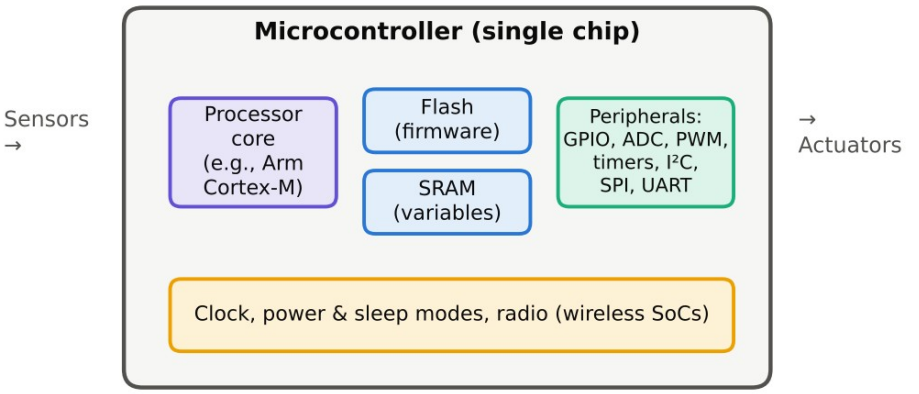


Figure 2.2 Inside a microcontroller. Processor, memory, and peripherals share one chip; wireless system-on-chip devices add a radio.

1. **Processor core.** Executes program instructions and performs arithmetic and logic. MCU cores are optimized for efficiency and predictable real-time response rather than raw speed. Common cores include the Arm Cortex-M family, RISC-V cores, and Tensilica Xtensa cores.
2. **Memory.**
   - **Flash (or ROM)** is non-volatile memory that stores the firmware, so the program survives when power is removed.
   - **RAM (SRAM)** is volatile memory for variables, buffers, and the stack during execution. It is small, typically kilobytes to a few hundred kilobytes.
3. **Input/output (I/O) peripherals.** Built-in hardware blocks that interface directly with the outside world: general-purpose input/output (**GPIO**) pins, the **ADC**, **PWM** outputs and timers, and serial buses (**I^2C, SPI, UART**). Wireless MCUs, often called systems-on-chip (SoCs), also include a Bluetooth, Wi-Fi, or IEEE 802.15.4 radio.

Supporting blocks manage clocks, power, and **sleep modes**. Sleep modes are what allow a battery-powered device to last for months: the MCU switches off most of its circuits and wakes on a timer or an external event (Chapter 7 calculates the effect).

Key functions of an MCU in IoT

Table 2.5 Key functions of an MCU in an IoT device

Function	Description
Data collection	Reads sensors through the ADC, GPIO, I^2C, or SPI

Function	Description
Data processing	Filters noise, computes averages, checks thresholds, runs small machine-learning models
Communication	Sends and receives data over Wi-Fi, Bluetooth, Zigbee, LoRa, or cellular modules
Control	Drives actuators such as fans, valves, and motors
Power management	Enters and exits ultra-low-power sleep states to extend battery life

MCU versus CPU

The simplest difference between an MCU and a CPU is what each contains on the chip and what it is designed to do (Table 2.6).

Table 2.6 MCU versus CPU

Feature	MCU (microcontroller)	CPU (microprocessor)
Memory	Integrated on chip (KB to a few MB)	External RAM and storage chips (GB)
Peripherals	Integrated (GPIO, ADC, timers, serial buses, often a radio)	External controller chips
Clock speed	Typically 16 to 240 MHz	Typically 1.5 to 5 GHz
Power	Microwatts asleep, milliwatts active	Watts to hundreds of watts
Operating system	None (bare metal) or a real-time OS such as FreeRTOS or Zephyr	Full OS such as Linux, Windows, or macOS
Real-time behavior	Predictable, deterministic timing	Less predictable because of caches and multitasking
Unit cost	Cents to a few dollars	Tens to hundreds of dollars
Best for	Dedicated embedded control (IoT end devices)	General-purpose computing (PCs, servers, gateways)

IoT end devices use MCUs because of **the three constraints: power, size, and cost**. A sensor node must run for months on a battery, fit inside a small enclosure, and cost little enough to deploy by the thousands. A CPU fails all three tests. Gateways and edge servers, by contrast, need to run Linux, handle many connections, encrypt traffic, and sometimes run AI models, so they use microprocessors such as the quad-core Arm Cortex-A72 in a Raspberry Pi 4 or the Arm-based modules in an NVIDIA Jetson.

Popular MCU families

Table 2.7 compares several MCU families that appear in student projects and commercial products. Specifications vary by exact part number; values shown are for the representative parts named.

Table 2.7 Popular MCU families

MCU (representative part)	Core and clock	Flash / RAM	Wireless	Typical use
Espressif ESP32 (ESP32-WROOM-32)	Dual-core Xtensa LX6, up to 240 MHz	4 MB flash (module) / 520 KB SRAM	Wi-Fi and Bluetooth	Smart home, prototypes, wearables
STMicroelectronics STM32 (STM32F103C8T6)	Arm Cortex-M3, 72 MHz	64 KB / 20 KB	None	Industrial control, medical devices
Microchip ATmega328P (Arduino Uno)	8-bit AVR, up to 20 MHz	32 KB / 2 KB	None	Education, simple controllers
Nordic nRF52840	Arm Cortex-M4F, 64 MHz	1 MB / 256 KB	Bluetooth LE, Thread, Zigbee	Beacons, wireless sensors, wearables
Texas Instruments MSP430 (MSP430FR2433)	16-bit, 16 MHz	15.5 KB FRAM / 4 KB	None	Ultra-low-power remote sensors

2.7 Arm and RISC architectures

Arm: an architecture, not a chip factory

Arm is a company headquartered in Cambridge, United Kingdom. It does not manufacture chips. Instead, it designs the **instruction set architecture (ISA)**, the vocabulary of instructions a processor understands, and processor cores that implement it, and it licenses these designs to chipmakers. STMicroelectronics (Europe), NXP (Europe), Nordic Semiconductor (Norway), Renesas (Japan), Qualcomm and Microchip (United States), and many Chinese and Indian vendors build Arm-based chips. Arm cores power almost every smartphone (for example Qualcomm Snapdragon and Samsung Exynos application processors) and a very large share of microcontrollers through the **Cortex-M** family.

Arm has become the industry and academic standard for several reasons:

- **A fixed instruction set.** Software written for one Cortex-M chip runs on another with minimal changes, which reduces fragmentation. Code for an STM32 can often be ported to an NXP or Microchip part.
- **A mature ecosystem.** More than 30 years of compilers (GCC, Arm Compiler, Keil, IAR), debuggers, and real-time operating systems are highly optimized for Arm.
- **Standard software layers.** The Cortex Microcontroller Software Interface Standard (**CMSIS**) gives developers a consistent way to access the core and peripherals.

- **Security. TrustZone** for Cortex-M provides hardware isolation between secure and non-secure code, which protects keys and critical functions.

RISC: simplicity as a design philosophy

RISC stands for **Reduced Instruction Set Computer**. A RISC processor uses a small set of simple instructions, most of which execute in a single clock cycle. The alternative, **CISC** (Complex Instruction Set Computer), used by Intel and AMD desktop processors, includes many complex, variable-length instructions. Arm, RISC-V, and AVR are RISC designs. RISC dominates IoT for three reasons (Table 2.8).

Table 2.8 Why RISC suits IoT

RISC feature	Impact on IoT
Power efficiency	Simpler logic requires fewer transistors and less current, extending battery life
Low heat	Fewer transistors switching means little heat in small, sealed enclosures
Predictability	Instructions take a fixed number of cycles, which supports deterministic real-time operating systems

Global Perspective: RISC-V, an open instruction set. RISC-V is a royalty-free, open-standard RISC instruction set that anyone can implement without paying license fees. It has grown quickly in IoT, especially in China, India, and Europe, where governments and companies want independence from licensed architectures. Espressif's ESP32-C3 and ESP32-C6 chips, for example, use RISC-V cores. For engineers, the practical lesson is that the choice of architecture now carries supply-chain and geopolitical considerations as well as technical ones.

2.8 The sense, process, act control loop

In an IoT system, hardware components interact in a continuous sequence that responds to environmental changes without human intervention. This **control loop** has six steps (Figure 2.3).

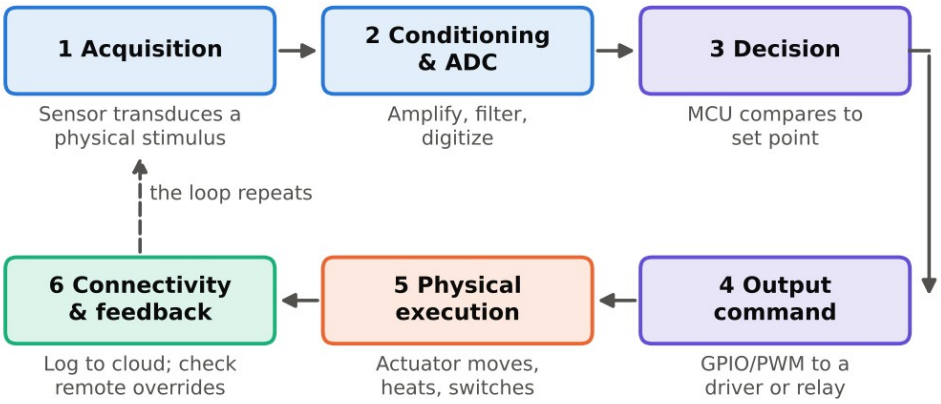


Figure 2.3 The six-step IoT control loop.

Step 1: Data acquisition (the sensor). The sensor detects a physical stimulus and transduces it into an electrical signal, usually a varying voltage. *Example:* a thermistor's resistance falls as the room warms, changing the voltage at the ADC pin.

Step 2: Signal conditioning and conversion. The analog signal is amplified and filtered, and the MCU's ADC samples it and converts it to a binary number, for example a 12-bit value between 0 and 4,095. Smart sensors perform this step internally and send the result over I^2C or SPI.

Step 3: Logic and decision making (the MCU). The firmware compares the value with a **set point** ("Is the temperature above 30 °C?") and calculates the necessary response.

Step 4: Output command. The MCU sends a low-power digital or PWM signal to a driver circuit, which isolates and protects the MCU from the load.

Step 5: Physical execution (the actuator). The actuator receives the boosted signal from the driver and performs the task. *Example:* a relay clicks shut to switch on a ventilation fan, or a servo rotates to open a window.

Step 6: Connectivity and feedback. While the physical action happens locally, the MCU uses its radio to report the event to a cloud dashboard and checks whether a remote user has sent an override command, such as "stop" or "change set point."

The loop then repeats. Because the sensor measures the result of the actuator's action, the loop is **closed**: it can correct itself.

Hysteresis: keeping the loop stable

A naïve control loop uses a single threshold: fan on above 26 °C, fan off below 26 °C. When the temperature hovers near 26 °C, noise makes the reading cross the threshold back and forth, and the fan switches on and off every few seconds. This **chattering** wears out relays, annoys users, and wastes energy.

Hysteresis solves the problem by using two thresholds: the actuator switches on at an upper threshold and off only when the value falls below a lower threshold. The gap between them is the **hysteresis band** or **dead band**. Figure 2.4 shows a fan controller with an ON threshold of 26 °C and an OFF threshold of 24 °C, the same values used by the class demo system's air-conditioning logic.

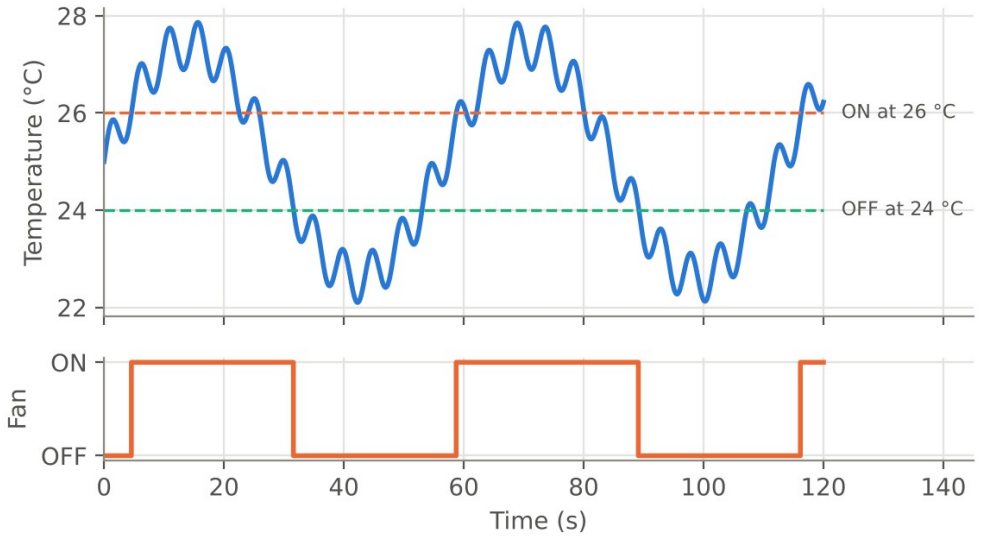


Figure 2.4 Hysteresis. The fan switches on at 26 °C and stays on until the temperature falls to 24 °C, so small fluctuations near either threshold do not cause rapid switching.

Worked Example 2.5: Designing hysteresis for a greenhouse fan

A greenhouse ventilation fan should keep the air below 30 °C. The temperature sensor has noise of about ±0.3 °C, and the relay is rated for 100,000 operations. Choose the thresholds.

1. **Set the ON threshold at the requirement:** 30.0 °C.
2. **Make the band wider than the noise.** A band narrower than the peak-to-peak noise (0.6 °C) would still chatter. A band of 2 °C is several times the noise and keeps the greenhouse within an acceptable range. OFF threshold = **28.0 °C**.
3. **Check relay life.** Suppose the band produces about 20 on/off cycles per day in summer. Relay life = 100,000 ÷ 20 = 5,000 days, about 13 years. With no hysteresis, chattering could cause several cycles per minute, more than 4,000 per day, and wear out the relay in less than a month.

The firmware logic is:

```
if (!fanOn && temp >= 30.0) fanOn = true;      // switch on at upper
threshold
else if (fanOn && temp <= 28.0) fanOn = false; // switch off at lower
threshold
```

The trade-off: a wider band means fewer switching cycles but larger temperature swings.

2.9 Putting it together: a smart irrigation controller

The smart irrigation controller from Chapter 1 maps neatly onto the six-step loop:

Table 2.9 The control loop in a smart irrigation controller

Step	Smart irrigation example
1 Acquisition	A capacitive soil-moisture probe's capacitance changes with water content
2 Conditioning and ADC	The probe's circuit converts capacitance to a voltage; the MCU samples it every 15 minutes
3 Decision	Firmware compares moisture with a set point (for example 22 percent) and checks the schedule and rain forecast
4 Output command	The MCU energizes a driver that switches 24 V AC to the zone's valve solenoid
5 Physical execution	The smart valve opens and water flows
6 Connectivity and feedback	The controller reports "Zone 3 on" to the cloud and checks the app for a manual "stop"

Notice that the decision in step 3 combines local data (moisture) with cloud data (the forecast). This combination of edge sensing and cloud intelligence is a recurring theme in Chapters 4 and 8.

2.10 The global IoT hardware market

IoT hardware is a global industry. Processor architectures designed in the United Kingdom are licensed to chipmakers in Europe, Asia, and the United States, manufactured in foundries mainly in Taiwan, South Korea, and China, assembled into modules across Asia, and sold in products worldwide. Market estimates vary widely depending on what is counted, but Table 2.10 gives an indication of scale from the Week 2 course presentation.

Table 2.10 Indicative IoT market size (estimates vary by source)

Market metric (2026 estimate)	Global	United States
Total IoT market	About US$1.05 to 1.35 trillion	About US$183 to 210 billion
IoT hardware only	About US$274 billion	About US$103 billion
Hardware share of total spending	About 25 to 30 percent	Higher share of regional spending

Market metric (2026 estimate)	Global	United States
Projected growth rate (CAGR)	About 14 percent (2026 to 2031)	About 8 percent (2025 to 2030)

Because products ship in millions of units, decisions made at the sensor and MCU level shape a product's cost, power use, and supply-chain risk. The global chip shortage of 2021 and 2022 showed how a single unavailable microcontroller could halt production of cars and appliances, and many companies now design boards that can accept a second-source part.

2.11 IoT application case: how NxEco senses the garden

A smart irrigation controller must know how much water the plants need. The most direct way is to measure soil moisture with a probe in every zone. NxEco takes a different approach: most of its "sensing" happens in the cloud, using weather information collected for the property's location. This approach is sometimes called **virtual sensing**, because a computed estimate stands in for a physical sensor.

Weather information collection

Table 2.11 NxEco's sources of sensing information (NxEco, n.d.-b, n.d.-c, n.d.-f)

Information source	What is collected	How NxEco uses it
Local weather service (via the NxEco cloud)	Daily forecast and real-time conditions for the owner's zip code: rain, temperature	Calculates each day's watering budget; skips or reduces watering after rain; stops watering near freezing
Monthly evapotranspiration (ET) data	Long-term average water loss from soil and plants for the location and month	Seasonal adjustment of run times
Plant information	Plant or grass type for each zone, drawing on NxEco's database and community tips	Per-zone fine tuning ("ET Fine Tune") on the HWN12-200
Rain sensor (optional, wired to the controller)	Whether it is raining on site	Interrupts watering locally, even without internet
Owner input	Manual "smart watering" percentage, rain delay of 1 to 72 hours	Temporary overrides for 24 hours or a set delay

Evapotranspiration (ET) is the amount of water that leaves the ground by evaporation from the soil plus transpiration from plants, usually expressed in millimeters or inches per day. Hot, sunny, dry, and windy days have high ET; cool, cloudy, humid days have low ET. Irrigation that replaces the water lost to ET, minus any rain, keeps plants healthy without waste.

Worked Example 2.6: Adjusting a zone's run time from weather data

NxEco does not publish its exact formula, but the principle can be shown with a simple ratio method. A lawn zone is programmed to run 20 minutes on a reference summer day with ET = 6.0 mm/day. Today the weather service reports ET = 3.6 mm/day and no rain.

Adjustment factor = today's ET ÷ reference ET = 3.6 ÷ 6.0 = 0.60, or **60 percent**.

Adjusted run time = 20 min × 0.60 = **12 minutes**.

If the forecast shows 5 mm of rain, which exceeds the day's ET, the cloud would skip watering altogether. If the temperature falls near freezing, watering stops to protect pipes and prevent icy paths.

Interpretation. Weather-based adjustment needs no sensor in the ground, which keeps installation simple. Its weakness is that it estimates rather than measures. A shaded zone, a sandy slope, or a broken pipe looks the same to the weather service as every other zone, which is why some competing products add soil-moisture probes or flow meters.

What the portal shows: weather data from a real installation

The weather inputs in Table 2.11 are visible in the **MySmartDrop** web portal. The Home page of a residential account in Mission Viejo, California, opened on October 22, 2026, displayed the five-day forecast that the cloud collects for the property's location (NxEco, 2026):

Table 2.12 Five-day forecast shown on the MySmartDrop Home page (NxEco, 2026)

Day	High / low (°F)	Conditions	Rain chance	Humidity
Thu Oct 22	68 / 59	Patchy rain nearby	76%	72%
Fri Oct 23	68 / 57	Patchy rain nearby	62%	73%
Sat Oct 24	71 / 57	Sunny	5%	67%
Sun Oct 25	75 / 60	Sunny	4%	63%
Mon Oct 26	78 / 63	Sunny	3%	56%

The table is a small lesson in virtual sensing. Temperature, humidity, and rain chance are exactly the variables that drive ET: as the week moves from cool, humid, and possibly wet to warm, dry, and sunny, ET rises, so the cloud should lower the watering budget on Thursday and Friday and raise it by Monday. The forecast also lets the system act **before** rain falls, which a wired rain sensor cannot do.

The same account's *Watering Plan* page for its main controller showed how these data reach the zones. **Program A** was in use; zone 1 ("Rose") was scheduled on **Monday and Friday**; **Smart Watering Status** was **ON**, meaning the weather adjustment is applied; and the per-controller **Finetune** slider was set to **98 percent**, a manual trim the owner can apply on top of the weather calculation. Four programs (A to D) can be stored, each with weekday or day-of-month scheduling and several start times.

Controller hardware

The perception-layer hardware is simple and robust:

- **Actuator outputs:** 12 zone outputs (8 on smaller models) that switch **24 V AC** irrigation solenoid valves, rated up to 8 VA, plus common terminals. Zone 12 can be assigned as a **master valve** or pump-start output, so the main supply opens only when a zone is watering.
- **Expansion:** the HWN24-2EXT module adds 24 zones through a 10-pin connector, for 36 zones in total.
- **Local interface:** an LCD, a rotary dial (Off, Auto Run, Manual Watering, Rain Delay, Schedule, Weather Adjustment, Settings), eight buttons, and two status LEDs.
- **Power:** a 24 V AC adapter (750 mA) supplies both the electronics and the valves.
- **Non-volatile memory:** programs, time, and settings are stored in flash memory and survive power cuts.
- **Environment:** a water-resistant enclosure for indoor (0 to 50 °C) or outdoor (–10 to 55 °C) use, 6.5 × 9.8 × 1.4 inches and 1.2 lb; an optional lockable outdoor cabinet (UCC100) is available.

NxEco does not publish the microcontroller or Wi-Fi chip used. Mapped to Section 2.8, the controller's control loop is: weather and ET data arrive (acquisition), the cloud and controller compute a run time (decision), the controller energizes a zone's 24 V AC output (output command), the valve opens (physical execution), and the run is logged to the cloud (feedback). Because there is no soil-moisture or flow sensor, the loop is **open** with respect to the soil: the system never measures whether the water actually reached the roots.

Chapter summary

- Every IoT edge device combines a **sensor** (input), a **controller** (processing), and an **actuator** (output).
- Analog signals become digital data through transduction, conditioning, sampling, quantization, and encoding. ADC resolution is $V_{ref}/2^n$; sampling must exceed twice the highest signal frequency.
- Sensors are compared by sensitivity, accuracy, precision, range, linearity, response time, and drift. Resolution is not accuracy.
- Actuators (relays, SSRs, solenoids, DC, servo, and stepper motors, and smart valves) are power-hungry, slow compared with electronics, and subject to wear. MCUs drive them through transistors, relay modules, opto-isolators, or motor drivers.
- An MCU integrates processor, memory, and peripherals on one chip. IoT end devices use MCUs because of power, size, and cost; gateways use microprocessors.
- Arm's licensing model and RISC's efficiency and predictability explain their dominance in low-power devices; RISC-V is an open alternative.

- The six-step control loop (acquisition, conditioning and ADC, decision, output command, physical execution, connectivity and feedback) runs continuously. Hysteresis keeps it stable.

Key terms

accuracy; actuator; analog-to-digital converter (ADC); Arm; chattering; CISC; CMSIS; control loop; driver circuit; firmware; flash memory; flyback diode; GPIO; hysteresis; I^2C; instruction set architecture (ISA); linearity; microcontroller unit (MCU); Nyquist criterion; opto-isolator; precision; PWM; quantization; range; relay; resolution; response time; RISC; RISC-V; sampling; sensitivity; sensor; servo motor; set point; smart sensor; solenoid; SPI; SRAM; stepper motor; system-on-chip (SoC); transducer; TrustZone; UART.

Review questions

Multiple choice

1. What is the primary role of a sensor in an IoT system?
    a. Convert electrical signals into motion
    b. Detect a physical change and convert it into an electrical signal or data
    c. Route packets across the internet
    d. Store data in the cloud
2. Which characteristic describes how close a measurement is to the true value?
    a. Precision
    b. Linearity
    c. Accuracy
    d. Response time
3. Why is a relay used between an MCU and a heater?
    a. It converts temperature into voltage
    b. It lets a low-power MCU signal switch a high-power load safely
    c. It increases the MCU's clock speed
    d. It stores firmware updates
4. What best distinguishes an MCU from a PC's CPU?
    a. An MCU always uses more power
    b. An MCU needs external RAM
    c. An MCU integrates processor, memory, and I/O peripherals on one chip
    d. An MCU is designed for general-purpose desktop computing
5. A 10-bit ADC has how many quantization levels?
    a. 10
    b. 100

c. 1,000
d. 1,024

6. A vibration signal contains frequencies up to 2 kHz. The minimum sampling rate that satisfies the Nyquist criterion is:
   a. more than 1 kHz
   b. more than 2 kHz
   c. more than 4 kHz
   d. more than 20 kHz
7. Which actuator moves to a commanded angle using built-in position feedback?
   a. Stepper motor
   b. Servo motor
   c. Solenoid
   d. Relay
8. Arm Holdings primarily:
   a. manufactures microcontrollers in its own factories
   b. designs instruction set architectures and cores and licenses them
   c. sells cloud IoT services
   d. produces sensors
9. Which RISC property is most important for a real-time operating system?
   a. Large instruction set
   b. Predictable instruction timing
   c. High clock speed
   d. Variable-length instructions
10. In the six-step control loop, comparing a reading with a set point occurs in step:
    a. 1 b. 2 c. 3 d. 5

Short answer

11. Explain the difference between resolution and accuracy, using a temperature sensor as an example.
12. List the three operational constraints of actuators and give an example of each.
13. Why do IoT end devices usually use MCUs while gateways use microprocessors? Refer to the three constraints.
14. What is chattering in a control loop, and how does hysteresis prevent it?

Applied problems

15. An STM32 has a 12-bit ADC with a 3.3 V reference. A pressure sensor outputs 0.5 V at 0 kPa and 4.5 V at 100 kPa, so it is connected through a voltage divider that halves its output. (a) What voltage reaches the ADC at 50 kPa? (b) What ADC count

corresponds to that voltage? (c) What pressure change does one ADC count represent?

16. A cold-storage room must stay below 5 °C. The compressor should not start more than 6 times per hour. Propose ON and OFF thresholds, justify the band, and describe one actuator constraint and how the design handles it.
17. **NxEco case.** A zone is programmed for 30 minutes at a reference ET of 7.0 mm/day. Today's ET is 4.2 mm/day and no rain is forecast. (a) What run time does the ratio method give? (b) Name one situation in which weather-based virtual sensing would water the zone incorrectly, and the physical sensor that would fix it.

Lab 2: Build and justify an IoT control loop

Collaboration 2 and Assignment 1, Part 1. Teams build a working sense, process, act loop in the free **Wokwi** online simulator, then write the control-loop lab report.

Scenario. A small business wants one task in its facility to run automatically. Choose a greenhouse, cold-storage room, parking garage, or smart office, and define the automated action and set point, for example: turn on a ventilation fan when temperature exceeds 30 °C.

Materials. A web browser; a free Wokwi account (https://wokwi.com); Wokwi documentation (https://docs.wokwi.com/).

Procedure.

1. **Create the project.** In Wokwi, choose *New Project* and select **ESP32** (or Arduino Uno). Wokwi opens a code editor (`sketch.ino`) and a diagram editor.
2. **Add the parts.** Click the **+** button and add a **DHT22** temperature and humidity sensor and an **LED** with a 220 Ω resistor (or a relay module or servo) as the actuator.
3. **Wire the circuit.** DHT22: VCC to 3V3, GND to GND, SDA (data) to GPIO 15. LED: anode through the resistor to GPIO 2, cathode to GND.
4. **Add the library.** Open the *Library Manager* and add **DHT sensor library** (Adafruit).
5. **Enter the firmware.** A minimal sketch with hysteresis and an invalid-reading check:

```
#include "DHT.h"
#define DHTPIN   15
#define DHTTYPE  DHT22
#define FAN_PIN  2               // LED stands in for the fan relay
const float ON_C  = 30.0;        // set point: switch on
const float OFF_C = 28.0;        // hysteresis: switch off
DHT dht(DHTPIN, DHTTYPE);
bool fanOn = false;

void setup() {
```

```
  Serial.begin(115200);
  dht.begin();
  pinMode(FAN_PIN, OUTPUT);
}

void loop() {
  float t = dht.readTemperature();          // Steps 1-2: acquire and
convert
  if (isnan(t)) {                           // reject invalid readings
    Serial.println("Sensor read error");
    delay(2000);
    return;
  }
  if (!fanOn && t >= ON_C)  fanOn = true;   // Step 3: decide
  else if (fanOn && t <= OFF_C) fanOn = false;
  digitalWrite(FAN_PIN, fanOn ? HIGH : LOW);// Steps 4-5: command and
execute
  Serial.printf("T=%.1f C  fan=%s\n", t, fanOn ? "ON" : "OFF"); // Step 6:
report
  delay(2000);                              // DHT22 maximum rate is 0.5 Hz
}
```

6. **Run and test.** Start the simulation. Click the DHT22 and drag its temperature slider above 30 °C, then down to 29 °C, then below 28 °C. Record the LED state and the Serial Monitor output at each step in a test table (Table 2.13).
7. **Share the evidence.** Save the project, copy the public link, and capture a screenshot of the actuator responding to the sensor change.
8. **Analyze the design.**
   - Walk through the loop using the six steps.
   - Justify the sensor choice using at least two characteristics (for example accuracy and response time).
   - Justify the MCU choice against one alternative from Table 2.7, and explain why an MCU, not a CPU, fits this device.
   - Identify one actuator constraint (power, latency, or wear) and how the design handles it (for example a relay module with a driver transistor and separate supply).
   - Build a mini bill of materials (BOM) with three to five real components, each with an approximate unit price and a supplier link.
9. **Suggest one improvement** (such as wider hysteresis, a lower-power MCU, or a more suitable sensor) and explain its trade-off.

Table 2.13 Test record template for Lab 2

Test	Sensor setting	Expected actuator state	Observed state	Serial output	Pass/Fail
1	25 °C	OFF			

Test	Sensor setting	Expected actuator state	Observed state	Serial output	Pass/Fail
2	31 °C	ON			
3	29 °C (falling)	ON (inside band)			
4	27 °C	OFF			
5	Sensor disconnected	OFF, error printed			

Assignment 1, Part 2 (MCU vs. CPU research). Compare at least three MCU models (for example ESP32-WROOM-32, STM32F103C8T6, ATmega328P, nRF52840, MSP430) and two microprocessors (for example an Intel Core or AMD Ryzen laptop processor, Apple M-series, NVIDIA Jetson Orin) in one table with the columns: Model, Manufacturer, Type, Core and clock, Flash/RAM, Built-in peripherals or wireless, Typical power, Operating system, Approximate unit price, Application example. Use datasheets or official product pages, and explain with the three constraints why IoT end devices use MCUs while gateways and edge servers use CPUs.

References and further reading

Arm. (n.d.). *What is RISC?* https://www.arm.com/glossary/risc

AVSystem. (2024, December 6). *Top 10 IoT sensor types: List of IoT sensors and IoT actuators*. https://avsystem.com/blog/iot/iot-sensors-iot-actuators

DigiKey. (2024, May 17). *Microprocessor vs. microcontroller: Unveiling distinctions*. Maker.io. https://www.digikey.com/en/maker/tutorials/2024/microprocessor-vs-microcontroller-unveiling-distinctions

engineerguy. (2013). *How a smartphone knows up from down* [Video]. YouTube. https://www.youtube.com/watch?v=KZVgKu6v808

FAME World Education Hub. (2025). *Data collect in IoT* [Video]. YouTube. https://www.youtube.com/watch?v=f1qXYWM8ALA

Hashemi-Pour, C., & Lutkevich, B. (2024, August 12). *What is a microcontroller?* TechTarget. https://www.techtarget.com/iotagenda/definition/microcontroller

NxEco. (n.d.-a). *NxEco smart watering* [Website]. https://www.nxeco.com

NxEco. (n.d.-b). *Smart controller specification* (Ver. 10.1) [PDF]. https://www.nxeco.com/Public/public/file/manual/Nxeco_Smart_Control_Spec_Ver10.1.pdf

NxEco. (n.d.-c). *NxEco smart weather-based irrigation controller: App manual* (Ver. 10.2) [PDF]. https://www.nxeco.com/Public/public/file/manual/NxEco_Smart_Manual_Ver10.2.pdf

NxEco. (n.d.-d). *Panel manual* (Ver. 10.1) [PDF]. https://www.nxeco.com/Public/public/file/manual/NeEco_Panel_Manual_Ver10.1.pdf

NxEco. (n.d.-f). *Features* [Web page]. https://www.nxeco.com/features.html

NxEco. (2026). *MySmartDrop smart watering portal* [Web application]. Retrieved October 22, 2026, from https://www.mysmartdrop.com

Oliynyk, K. (2025, December 1). *IoT actuators: Types, selection criteria & real-life examples*. Webbylab. https://webbylab.com/blog/actuators-in-iot-guide/

RealPars. (2020). *What is an actuator?* [Video]. YouTube. https://www.youtube.com/watch?v=LHn7O6PUaoY

RealPars. (2021). *What is a sensor? Different types of sensors, applications* [Video]. YouTube. https://www.youtube.com/watch?v=XI49uFm5HRE

STMicroelectronics. (2022). *Introduction to STM32: Back to the basics* [Video]. YouTube. https://www.youtube.com/watch?v=8DmJ7pnFE5M

Tech Tutor Hub. (2024). *IoT architecture: Device layer, communication layer, cloud layer, application layer* [Video]. YouTube. https://www.youtube.com/watch?v=pXu6qq3ATJM

TheUnlockr. (2024). *What is Arm?* [Video]. YouTube. https://www.youtube.com/watch?v=7s_rMhmfW3U

Waseem, U. (2026, September 8). *Microcontroller vs microprocessor: The real differences*. Wevolver. https://www.wevolver.com/article/microcontroller-vs-microprocessor-a-comprehensive-guide-to-their-differences-and-applications

White, E., & White, C. (Hosts). (n.d.). *Embedded* [Audio podcast]. https://embedded.fm/

Wokwi. (n.d.). *Wokwi documentation: Online electronics simulator*. https://docs.wokwi.com/

CHAPTER 3

IoT Access and Networking: From the OSI Model to Short-Range Wireless

Learning Objectives

After studying this chapter, students will be able to:

1. Distinguish IoT access (the "first mile") from end-to-end IoT communication.
2. Explain the seven layers of the OSI reference model and trace a sensor reading through them.
3. Describe modulation, physical-layer hardware, signal strength in dBm, and the role of the data link layer, and calculate transmission efficiency.
4. Compare IPv4, IPv6, and short WPAN addresses, and explain the role of TCP and UDP.
5. Explain how a Wi-Fi device joins a network and summarize the IEEE 802.11 generations.
6. Compare Bluetooth Low Energy, Zigbee, Z-Wave, Thread, RFID, NFC, and low-power wide-area networks.
7. Apply a four-constraint protocol selection framework and choose a star or mesh topology for a business deployment.
8. Explain the four layers of IoT network security.

Where this fits: Week 3. This chapter supports Collaboration 3 *Wireless Site Survey* (Collaboration Report 1) and SM Assignment 1 *How IoT Data Reaches the Network*. Every later lab, from MQTT to the cloud, builds on these networking foundations.

Opening case: fifty sensors and three floors

A growing medical clinic in Singapore wants to monitor temperature in its medicine refrigerators, detect water leaks under sinks, count room occupancy, and track wheelchairs and infusion pumps. The facilities manager estimates about 50 battery-powered sensors spread across three floors. The building already has Wi-Fi for staff and patients.

The obvious answer, "put everything on Wi-Fi," turns out to be a poor one. Wi-Fi radios drain small batteries in days or weeks, the patient network is crowded, and every new device on the staff network is a new security risk. The better answer depends on four questions that this chapter teaches students to ask: How far must each signal travel? How

much power does each device have? How much data does it send? And how much can each node cost? The answers lead to a choice of access technology and network topology that an engineer can justify to a client.

3.1 IoT access and IoT communication

IoT networking has two complementary parts:

- **IoT access** is the **first mile**: the physical and link-level mechanism by which a device enters the network. A BLE sensor advertising to a gateway, or a Wi-Fi thermostat associating with an access point, is performing access.
- **IoT communication** governs the end-to-end exchange, routing, and interpretation of data across the whole system: the IP addressing, transport protocols, and application messages that carry a reading from the sensor to a cloud server and a phone.

Access is about *how a device physically connects*. Communication is about *how data is routed and understood once it is connected*. Engineers often choose them separately: the same MQTT application message (Chapter 4) can travel over Wi-Fi, Ethernet, or cellular access.

3.2 Data communications networks

A **data communications network** collects data from terminal devices and transmits it to central computers (data ingestion), and performs the reverse process to distribute processed information back to the client nodes (bidirectional delivery). The internet is the largest such network, but the same principles apply to a factory floor network or a home network.

A helpful analogy is the postal system. A letter is written in a language both people understand, placed in an envelope with an address, collected by the local post office, sorted, carried by truck or plane between cities, delivered to a local office, and finally handed to the recipient. Each step has its own rules and does not need to know the details of the others. The truck driver does not read the letter, and the writer does not choose the truck route. Data networks are organized in the same layered way.

3.3 The OSI reference model

The **Open Systems Interconnection (OSI) reference model**, published by the International Organization for Standardization (ISO), divides network communication into seven layers (Cloudflare, n.d.). Each layer provides services to the layer above it and uses services from the layer below. Figure 3.1 shows the layers and typical IoT protocols at each one.

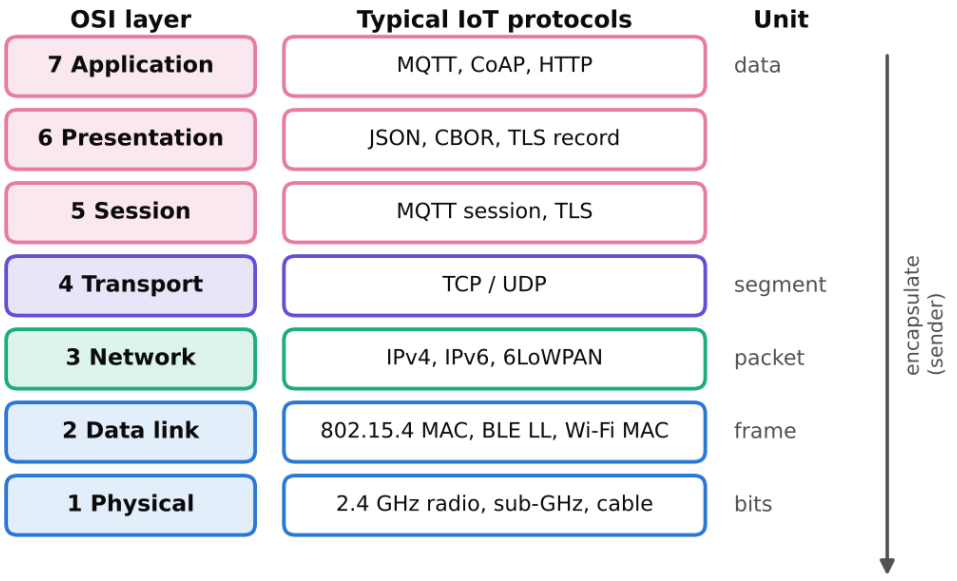


Figure 3.1 The OSI reference model with typical IoT protocols. On the sender, each layer wraps the data from the layer above (encapsulation); on the receiver, each layer removes its wrapper.

Table 3.1 The seven OSI layers

Layer	Name	Responsibility	IoT example
7	Application	Network services for applications	MQTT, CoAP, HTTP
6	Presentation	Data format, encoding, encryption	JSON, CBOR, TLS encryption
5	Session	Opening, maintaining, and closing sessions	MQTT session, TLS session
4	Transport	End-to-end delivery, segmentation, reliability	TCP, UDP
3	Network	Logical addressing and routing between networks	IPv4, IPv6, 6LoWPAN
2	Data link	Framing, physical (MAC) addressing, error detection on one link	Wi-Fi MAC, BLE link layer, IEEE 802.15.4 MAC
1	Physical	Transmission of raw bits as electrical, optical, or radio signals	2.4 GHz radio, sub-GHz radio, Ethernet cable

How messages move through the layers

On the sending device, the application creates a message. Each layer below adds its own header (and sometimes a trailer) containing the information it needs, a process called **encapsulation**. The unit of data has a different name at each layer: data at the application layer, a **segment** at the transport layer, a **packet** at the network layer, a **frame** at the data

link layer, and **bits** at the physical layer. On the receiving device, each layer reads and removes its header, passing the remainder upward (**decapsulation**).

Worked Example 3.1: Tracing a temperature reading through the layers

A Wi-Fi temperature sensor publishes the reading 24.8 °C to a cloud server.

1. **Application (7):** The firmware creates an MQTT PUBLISH message to the topic `factory/line1/temp` with the payload `{"t":24.8}`.
2. **Presentation (6):** The payload is encoded as JSON text; TLS encrypts it.
3. **Session (5):** The message belongs to an existing MQTT session over a TLS session with the server.
4. **Transport (4):** TCP places the bytes in a segment with source and destination port numbers (port 8883 for MQTT over TLS) and a sequence number.
5. **Network (3):** IP wraps the segment in a packet with the sensor's IP address and the server's IP address.
6. **Data link (2):** The Wi-Fi MAC wraps the packet in a frame addressed to the access point's MAC address, with a checksum.
7. **Physical (1):** The radio modulates the bits onto a 2.4 GHz carrier.

At each router along the way, only layers 1 to 3 are processed; the TCP segment and MQTT message pass through untouched until they reach the server. For a small battery-powered device, the lower layers matter most: the physical and data link layers determine how much energy each transmission costs, and every header added above them increases the number of bits the radio must send.

The internet itself is built on a simpler **TCP/IP model** with four layers: link (OSI 1 and 2), internet (3), transport (4), and application (5 to 7). The OSI model remains the standard teaching and troubleshooting vocabulary: engineers say "it's a layer 2 problem" to mean a link problem, such as a device that cannot associate with an access point.

3.4 The physical layer

The **physical layer** translates frames from the data link layer into electrical, optical, or radio signals, and back. It defines the hardware (cables, connectors, antennas), the signal characteristics (voltages, light pulses, radio frequencies), and the procedures for activating and deactivating a physical link.

Wired versus wireless

Wired links (copper Ethernet, coaxial cable, optical fiber, and industrial buses such as RS-485) are fast, reliable, and secure because the signal stays inside the cable. They are common for gateways, industrial controllers, and anything mains powered and fixed in place. Wireless links eliminate cabling cost and allow devices to move or be installed where cables cannot reach, at the cost of interference, limited range, and higher energy per bit for battery devices. Most IoT end devices are wireless; most gateways have a wired or cellular backhaul.

Modulation

A radio transmits information by changing a property of a high-frequency **carrier wave**. This is **modulation**. Three basic digital techniques change one property each (Figure 3.2):

- **Amplitude-shift keying (ASK)** changes the amplitude (strength) of the carrier.
- **Frequency-shift keying (FSK)** changes the frequency. Bluetooth Low Energy uses a variant called Gaussian FSK.
- **Phase-shift keying (PSK)** changes the phase (the timing of the wave's cycle). IEEE 802.15.4 (Zigbee, Thread) uses a form of PSK.

High-speed systems such as Wi-Fi and 5G combine amplitude and phase changes in **quadrature amplitude modulation (QAM)** to send many bits per symbol. More bits per symbol means higher speed but requires a stronger, cleaner signal, which is why Wi-Fi slows down as a device moves away from the access point.

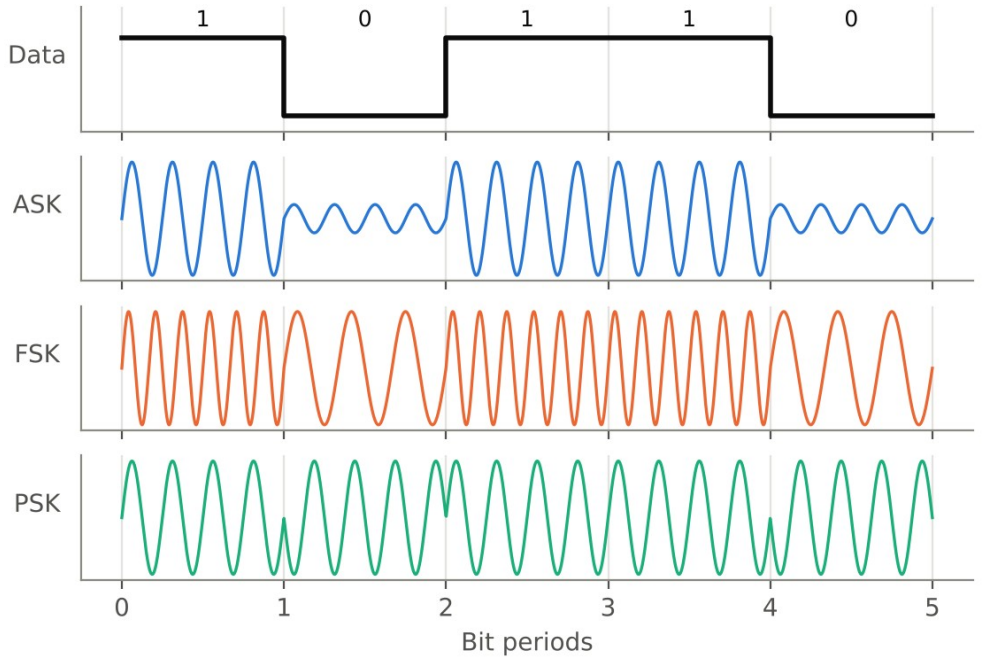


Figure 3.2 Three basic digital modulation techniques carrying the bit pattern 1 0 1 1 0.

Physical-layer hardware

The basic hardware of a wireless link includes (Table 3.2):

Table 3.2 Physical-layer hardware for wireless communication

Component	Function
Antenna	Radiates and receives radio waves; its size and placement strongly affect

Component	Function
	range
Transceiver	Combined transmitter and receiver that converts digital data into radio signals and back
Power amplifier (PA)	Boosts the transmitted signal for longer range, at the cost of power
Low-noise amplifier (LNA)	Amplifies weak received signals without adding much noise
Modem	Modulates and demodulates; bridges digital data and analog signals (for example a cellular or DSL modem)

In most IoT devices, the transceiver, modem, and often the MCU are integrated in one chip, with only the antenna and a few passive components outside it.

Frequency bands

IoT radios operate in two kinds of spectrum:

- **Unlicensed ISM bands** (industrial, scientific, and medical) can be used by anyone who follows power and duty-cycle rules. The **2.4 GHz** band is available worldwide and used by Wi-Fi, Bluetooth, Zigbee, and Thread, which makes it crowded. **Sub-GHz** bands, such as 868 MHz in Europe and India and 902 to 928 MHz in the Americas, travel farther and penetrate walls better; LoRaWAN, Sigfox, and Z-Wave use them. Because sub-GHz frequencies differ by region, a LoRaWAN product must be built for the right regional plan.
- **Licensed bands** are owned by mobile operators and used for cellular IoT (LTE-M, NB-IoT, 5G). They offer managed quality of service and wide coverage, with a subscription fee.

Signal strength in dBm

Received signal strength is usually expressed in **dBm**, decibels relative to one milliwatt:

$$P_{dBm} = 10 \times \log_{10}(P \div 1\,mW) \text{ and conversely } P = 10^{x}\,mW, \text{ where } x = P_{dBm} \div 10$$

Because the scale is logarithmic, every 3 dB is roughly a doubling or halving of power and every 10 dB is a factor of ten. Received signals are tiny, so values are negative: −30 dBm is an excellent signal next to a router, −67 dBm is a common minimum for reliable Wi-Fi voice and video, and −90 dBm is near the limit where many receivers lose the signal. The **received signal strength indicator (RSSI)** reported by phones and gateways uses this scale.

Worked Example 3.2: Converting dBm and estimating range

(a) A BLE gateway reports RSSI = −62 dBm for a sensor. What is the received power?

$P = 10^{x}$ mW with $x = -62 \div 10 = -6.2$, so $P = 10^{-6.2}$ mW $\approx 6.3 \times 10^{-7}$ mW, or about **0.63 nanowatts**. Radios work with astonishingly small amounts of power.

(b) Signal strength falls with distance. A simple **log-distance path-loss model** estimates

$$RSSI(d) = RSSI(d_0) - 10 \times n \times log_{10}(d \div d_0)$$

where d_0 is a reference distance (1 m), and n is the path-loss exponent (2 in free space, about 2.5 to 4 indoors). A sensor measures −45 dBm at 1 m. With n = 2.5, estimate RSSI at 10 m and 20 m.

At 10 m: $-45 - 10 \times 2.5 \times \log_{10}(10) = -45 - 25 =$ **−70 dBm**.

At 20 m: $-45 - 25 \times \log_{10}(20) = -45 - 25 \times 1.301 =$ **−77.5 dBm**.

Interpretation. Doubling the distance cost only 7.5 dB, but walls, metal shelving, and people add losses of several decibels each. This is why engineers survey a real site (Lab 3) instead of trusting a formula.

3.5 The data link layer

The **data link layer** manages communication between devices that share the same physical link or local network. It:

- **frames** data, wrapping each packet with a header and a trailer;
- **addresses** devices using **media access control (MAC) addresses**, unique hardware identifiers (48 bits for Wi-Fi, Ethernet, and Bluetooth; 64 bits or a short 16-bit address for IEEE 802.15.4);
- **detects errors** using a checksum such as a **cyclic redundancy check (CRC)**, so corrupted frames can be discarded and retransmitted;
- **controls access** to the shared medium so that devices do not talk over each other (Simplilearn, 2023).

Medium access and transmission efficiency

When many devices share a radio channel, they need rules for taking turns. Wi-Fi and IEEE 802.15.4 use **carrier-sense multiple access with collision avoidance (CSMA/CA)**: listen first, transmit only if the channel is quiet, and wait a random time if it is busy. Cellular networks and some industrial protocols use **time-division multiple access (TDMA)**, giving each device its own time slot.

Transmission efficiency is the ratio of useful data (the payload) to the total data sent:

$$\eta = payload\ bits \div (payload\ bits + overhead\ bits)$$

Several factors reduce efficiency: **frame overhead** (preambles, headers, and checksums), **collisions and retransmissions**, the **access mechanism** (time spent waiting for the

channel), **propagation and processing delays**, and **fragmentation** of large messages into many small frames. **Aggregation**, combining several small payloads into one frame, improves it.

Worked Example 3.3: Why small payloads are inefficient

(a) An Ethernet frame carries a 1,500-byte payload with 38 bytes of overhead (preamble, header, checksum, and inter-frame gap). Efficiency = 1,500 ÷ 1,538 = **97.5 percent**.

(b) A BLE sensor sends a 20-byte reading. The link-layer packet adds about 21 bytes of overhead (preamble, access address, headers, message integrity code, and CRC). Efficiency = 20 ÷ 41 = **48.8 percent**.

(c) A sensor sends a 12-byte reading inside a frame with 25 bytes of overhead. Efficiency = 12 ÷ 37 = **32.4 percent**.

Interpretation. IoT messages are tiny, so overhead dominates. Engineers improve efficiency by packing readings into compact binary formats (the class demo node sends a 12-byte packed structure, Chapter 7), by batching several readings into one transmission, and by using protocols with small headers.

3.6 The network and transport layers

IP addressing

The **Internet Protocol (IP)** operates at the network layer. It gives each device a logical address and routes **packets** between networks. Two versions are in use:

- **IPv4** uses 32-bit addresses (for example 192.168.1.100), giving $2^{32} \approx 4.3$ billion addresses. That is fewer than the number of connected devices in the world, so most networks share public addresses through **network address translation (NAT)**.
- **IPv6** uses 128-bit addresses (for example 2001:db8::1), giving $2^{128} \approx 3.4 \times 10^{38}$ addresses, enough for every device to have its own globally unique address.

Constrained wireless personal area networks (WPANs) such as IEEE 802.15.4 cannot afford 40-byte IPv6 headers in 127-byte frames. They use **short 16-bit addresses** inside the local network, and **6LoWPAN** compresses IPv6 headers so that IPv6 can run over low-power radios. Thread and Matter rely on this approach.

TCP and UDP

The transport layer links applications to the network.

- The **Transmission Control Protocol (TCP)** is connection-oriented and reliable. It breaks data into **segments**, numbers them, retransmits any that are lost (detected through acknowledgments, ACKs), and reassembles them in order at the destination. TCP software runs only at the sender and receiver, not in the routers between them. MQTT and HTTP run over TCP.

- The **User Datagram Protocol (UDP)** is connectionless and unreliable: it sends datagrams without acknowledgments or ordering. It has a smaller header and no connection setup, which suits very constrained devices and real-time streams. **CoAP**, a lightweight web protocol for constrained devices, runs over UDP and adds its own optional acknowledgments.

3.7 Wi-Fi and local area networks

A **local area network (LAN)** connects devices in one physical location, from a single home to an office tower or campus. A **wireless LAN (WLAN)** distributes connectivity by radio, usually through **access points (APs)**. A typical Wi-Fi system has three parts: end devices (phones, laptops, IoT devices), an access point or router that bridges the wireless and wired networks, and the wired backbone and internet connection behind it.

How a device joins a Wi-Fi network

Before it can send data, a Wi-Fi device follows a four-step procedure (The Default Guy, 2024):

1. **Scanning.** The device listens for **beacon** frames broadcast by access points (passive scanning) or sends **probe requests** (active scanning) to discover available networks.
2. **Authentication.** The device and access point verify each other. Modern networks use **WPA3**, whose Simultaneous Authentication of Equals (SAE) handshake resists offline password-guessing attacks.
3. **Association.** The access point admits the device, records it in its table, and both agree on parameters such as data rate and encryption keys.
4. **Data exchange.** Encrypted frames are sent and acknowledged.

Each step costs time and energy. A battery-powered Wi-Fi sensor that wakes, rejoins, sends one reading, and sleeps may spend more energy on joining than on its data.

IEEE 802.11 generations

Wi-Fi is defined by the IEEE 802.11 family of standards, marketed by the Wi-Fi Alliance under generation numbers (Table 3.3).

Table 3.3 Selected Wi-Fi generations

Generation	IEEE standard	Bands	Notable features
Wi-Fi 4	802.11n	2.4 and 5 GHz	MIMO; common in low-cost IoT modules
Wi-Fi 5	802.11ac	5 GHz	Wider channels, higher throughput
Wi-Fi 6 / 6E	802.11ax	2.4, 5 (and 6 GHz for 6E)	OFDMA for many simultaneous clients; Target Wake Time lets IoT devices schedule sleep

Generation	IEEE standard	Bands	Notable features
Wi-Fi 7	802.11be	2.4, 5, and 6 GHz	Multi-Link Operation uses several bands at once for speed and reliability
Wi-Fi HaLow	802.11ah	Sub-1 GHz	Long range (around 1 km) and low power, designed for IoT

3.8 Why IoT networking is different

Conventional computer networking was built for powerful, always-on, high-bandwidth devices. IoT devices are often highly constrained:

- tiny memory and processing power;
- milliwatt power budgets, sometimes with a goal of ten years on one coin cell;
- intermittent wireless connectivity;
- deployment in hostile places: underground, inside metal machines, outdoors, behind walls.

As a result, the protocol stack has been re-engineered from the ground up (Afaneh, 2020). IoT access technologies fall into three broad categories:

- **Short-range** (centimeters to about 100 m): NFC, RFID, Bluetooth Low Energy, Zigbee, Z-Wave, Thread.
- **Medium-range** (tens to hundreds of meters): Wi-Fi, Wi-Fi HaLow, mesh extensions of short-range technologies.
- **Long-range, low-power wide-area networks (LPWAN)** (kilometers): LoRaWAN, Sigfox, and the cellular IoT standards NB-IoT and LTE-M, with 5G RedCap for mid-range cellular devices.

3.9 Short-range access technologies

Bluetooth Low Energy

Classic Bluetooth was designed for continuous audio streaming. **Bluetooth Low Energy (BLE)**, introduced with Bluetooth 4.0, was designed for small, infrequent data from battery devices. BLE operates in the 2.4 GHz ISM band on 40 channels of 2 MHz each. Three channels (37, 38, and 39) are reserved for **advertising**, short broadcasts that announce a device and can carry small amounts of data; the other 37 carry data in **connections**, using **frequency hopping** to avoid interference. BLE devices achieve months or years of battery life because they spend almost all of their time asleep and transmit in bursts of a few milliseconds.

Two communication styles matter for IoT design (Chapter 7 uses both):

- **Advertising (broadcast).** The sensor periodically broadcasts its data; any nearby scanner can receive it. There is no connection and no acknowledgment. It is simple and very low power, like a radio broadcast.
- **Connections and GATT.** The sensor and a central device (a phone or gateway) form a connection and exchange data through the **Generic Attribute Profile (GATT)**, which organizes data into **services** and **characteristics** identified by UUIDs. For example, the standard Environmental Sensing service (0x181A) contains a Temperature characteristic (0x2A6E). Connections are reliable, bidirectional, and can be encrypted, like a phone call, but keeping the link open costs more energy.

Bluetooth 5 (2016) added a 2 Mbps mode for faster transfers, a long-range "Coded PHY" mode that trades speed for roughly four times the range, and extended advertising. **Bluetooth Mesh** enables many-to-many networks that extend range beyond the 10 to 100 m single-hop limit. **Bluetooth 6.0** (2024) added Channel Sounding for centimeter-level distance measurement, useful for digital car keys and item finders.

Zigbee and Z-Wave

Zigbee and **Z-Wave** were purpose-built for home and building automation. Zigbee runs on the IEEE 802.15.4 physical and MAC layers at 2.4 GHz with a data rate of 250 kbps. Z-Wave uses sub-GHz bands (for example 868 MHz in Europe and 908 MHz in North America), which penetrate walls better, and a classic Z-Wave network supports up to 232 devices.

Their defining feature is **self-healing mesh networking**. In a Zigbee network, a **coordinator** forms the network, mains-powered devices such as smart plugs and light switches act as **routers** that relay messages, and battery-powered sensors act as **end devices** that sleep most of the time and talk only to their parent router. If a router fails, messages find another path. A Zigbee mesh can cover an entire building with many hops (Connectivity Standards Alliance, 2024).

Thread and Matter

Thread is an IPv6-based mesh networking protocol that also uses IEEE 802.15.4 radios. Because every Thread device has an IPv6 address, devices can be reached end to end without a proprietary translation bridge. **Matter**, released in 2022 by the Connectivity Standards Alliance with backing from Apple, Google, Amazon, Samsung, and others, is a common application layer that runs over Thread, Wi-Fi, and Ethernet, so that a Matter-certified device works with any Matter ecosystem. Matter 1.5, released in late 2025, added cameras, soil sensors for irrigation, and energy tariff and EV-charging features (DFRobot, 2023).

RFID and NFC

Radio-frequency identification (RFID) tags can operate without batteries. In a **passive tag**, the reader's radio field energizes the tag just long enough for it to reply with its identifier. Ultra-high-frequency (UHF) RFID at 860 to 960 MHz can read tags at about 3 to

10 m, and is ubiquitous in supply chain logistics and retail inventory. Tagging millions of items with battery-powered devices would be economically impossible; passive RFID tags cost a few cents.

Near-field communication (NFC) is a 13.56 MHz technology limited to about 4 cm. The short range is a security feature: it is used for contactless payments, transit cards, access badges, and pairing devices with a tap (Eye on Tech, 2023).

Low-power wide-area networks

When devices are spread over kilometers, such as water meters across a city, soil probes across a farm, or trackers on shipping containers, short-range technologies cannot reach. **LoRaWAN** uses chirp spread-spectrum modulation in sub-GHz bands to reach several kilometers at data rates of a few hundred bits per second to tens of kilobits per second, with battery lives of five to ten years. **NB-IoT** and **LTE-M** are cellular standards that reuse mobile operators' licensed networks, offering wide coverage and managed service for a subscription fee. Chapter 4 discusses cellular backhaul and 5G in more detail.

3.10 A protocol selection framework

Which combination of access technology and application protocol is right for a deployment? The answer is determined by four intersecting constraints:

1. **Range.** How far must the signal travel, and through what obstacles?
2. **Power.** Is the device mains powered, rechargeable, or on a primary battery that must last years?
3. **Data rate (bandwidth).** Does it send a few bytes per hour or a video stream?
4. **Cost.** What can each node, the gateways, and any subscription cost over the product's life?

Secondary factors include latency, topology, device density, security features, ecosystem and interoperability, and regional regulations. A useful rule of thumb from the class demo design is: *range and speed cost power. Pick the slowest, shortest-range protocol that still meets the application's needs.* Figure 3.3 and Table 3.4 compare the main options.

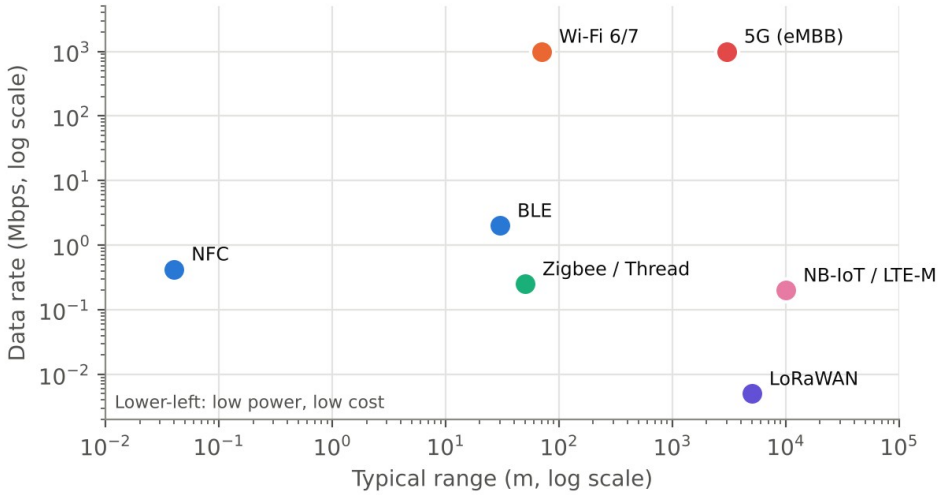


Figure 3.3 Typical range and data rate of IoT access technologies. Values are indicative; real performance depends on configuration and environment.

Table 3.4 Comparison of IoT access technologies

Technology	Standard	Band	Data rate	Typical range	Battery life	Topology
BLE 5	Bluetooth (IEEE 802.15.1 origin)	2.4 GHz	1 to 2 Mbps	10 to 100 m	Months to years	Star, mesh
Wi-Fi 4/5/6	IEEE 802.11	2.4, 5, 6 GHz	10s of Mbps to Gbps	30 to 100 m	Days to weeks (mains preferred)	Star (AP)
Zigbee 3.0	IEEE 802.15.4	2.4 GHz	250 kbps	10 to 100 m per hop	Years	Mesh
Thread	IEEE 802.15.4 + IPv6	2.4 GHz	250 kbps	10 to 100 m per hop	Years	Mesh
LoRaWAN	LoRa chirp spread spectrum	Sub-GHz	0.3 to 50 kbps	2 to 15 km	5 to 10 years	Star of stars
NB-IoT / LTE-M	3GPP cellular	Licensed	Tens of kbps to about 1 Mbps	Kilometers (operator coverage)	Years (with power-saving modes)	Star (cell tower)

Worked Example 3.4: Choosing access technology for 50 sensors in a three-floor office

Return to the clinic in the opening case: about 50 battery-powered sensors (refrigerator temperature, leak detection, occupancy) across three floors, each sending a few bytes every minute.

Constraint	BLE	Zigbee	Wi-Fi
Range	10 to 30 m indoors per hop; needs a gateway per floor or BLE Mesh	Mesh covers the building through mains-powered routers	AP coverage already exists
Power	Excellent	Excellent	Poor for coin-cell devices
Data rate	More than enough	More than enough	Far more than needed
Cost	Very low modules; a few gateways	Low modules; one coordinator plus routers	Low modules, but battery replacements and IT security overhead

Recommendation: Zigbee (or Thread) in a **mesh** topology. Mains-powered smart plugs or repeaters on each floor act as routers, so signals hop between floors and around walls, and the battery sensors sleep for years. A single gateway connects the mesh to the cloud over the clinic's wired network, on a separate network segment.

Trade-off accepted: a second radio network to install and maintain, and the need for some mains-powered routers. BLE with one gateway per floor would be a reasonable alternative, accepting more gateways in exchange for using phones as convenient commissioning tools. Wi-Fi is rejected because battery life would be measured in weeks.

3.11 Network topologies

How devices connect to each other and to the internet is not one-size-fits-all. The choice of **topology** has profound implications for reliability, coverage, power, and cost (Figure 3.4).

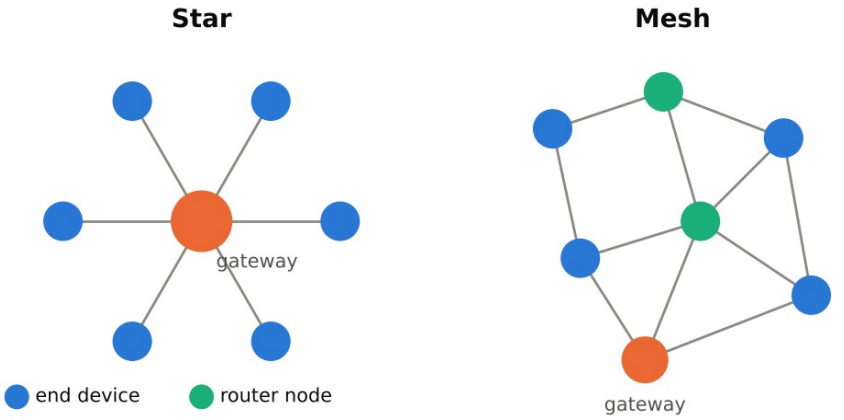


Figure 3.4 Star and mesh topologies. In a star, every device talks directly to the gateway. In a mesh, router nodes relay messages so that traffic can take several paths.

- **Star topology.** Every endpoint communicates directly with a central gateway, which handles all internet connectivity. It is the simplest model. Wi-Fi, LoRaWAN, Sigfox, and most cellular IoT use it. Its weaknesses are a **single point of failure** at the gateway and the range limit of each endpoint's own radio.
- **Mesh topology.** Every capable device acts as both an endpoint and a router, so messages hop through intermediate devices. Zigbee, Thread, and Bluetooth Mesh use it. Its strength is **self-healing**: the network automatically reroutes around failed nodes, which is critical in buildings with walls and interference. Its costs are complexity, latency that grows with hop count, and the need for always-on router nodes.
- **Tree and star-of-stars** variants combine the two. LoRaWAN, for example, is a "star of stars": end devices talk to any gateway in range, and gateways forward to a central network server.

3.12 IoT network security

Network security in IoT is structurally harder than in conventional IT because the threat surface is vast, devices are physically exposed, and update mechanisms are often absent (Kirvan et al., 2025). A layered approach addresses four areas:

1. **Device identity and authentication.** Each device needs a unique cryptographic identity, established at manufacturing, so that the network accepts only genuine devices. Shared default passwords must never be used.
2. **Transport security.** All data in transit is encrypted and integrity-protected, from the device to the gateway (for example BLE link-layer encryption with AES-128) and from the gateway to the cloud (TLS).
3. **Network segmentation.** IoT devices are isolated from enterprise systems on their own network segments or VLANs, with firewalls between zones. A compromised camera should never be able to reach the payroll server.
4. **Over-the-air (OTA) updates.** Devices must be patchable remotely. A device that cannot be updated remains permanently vulnerable to newly discovered flaws.

Chapter 6 develops these ideas into a complete security, privacy, and ethics framework.

3.13 IoT application case: connecting NxEco to the network

The NxEco controller reaches the internet through the customer's own home or office router, so it does not need a separate gateway.

- **Wi-Fi:** embedded IEEE 802.11b/g/n radio, **2.4 GHz only**, with **WPA2** security. The support pages warn that dual-band routers which advertise the 2.4 GHz and 5 GHz bands under one name can confuse setup, and recommend separating the bands if the controller cannot join (NxEco, n.d.-e).

- **Ethernet:** the HWN12-200 also has an RJ45 port for a wired connection, useful when the controller is in a garage or utility room far from the router.
- **Provisioning:** in the app, the owner chooses *Connect to WiFi*, holds the controller's Smart Wi-Fi button until the blue LED flashes, and taps *Apply*. A steady blue LED shows a Wi-Fi link (OSI layers 1 and 2); a green Internet LED shows that the controller can reach the NxEco cloud (layers 3 and above). During this step the app sends the home network's name and password to the controller; the app's 2019 release fixed a fault in reading the Wi-Fi network name, a reminder that provisioning is a common point of failure in consumer IoT (NxEco, Inc., 2020). These two LEDs are a practical example of layered troubleshooting: if blue is on but green is off, the problem is beyond the local network.

Applying the protocol selection framework from Section 3.10 explains the design:

Table 3.5 Why NxEco uses Wi-Fi

Constraint	NxEco situation	Consequence
Range	Controller is mounted on or inside the building, usually within range of the home router	Wi-Fi or Ethernet is sufficient; no long-range radio needed
Power	Mains powered through a 24 V AC adapter	The higher power draw of Wi-Fi is acceptable
Data rate	A few small messages per day (schedules, weather adjustments, status)	Any technology would do; Wi-Fi is far more than enough
Cost	Customers already own a Wi-Fi router	No gateway or cellular subscription to pay for

The choice of **2.4 GHz** rather than 5 GHz is also deliberate: 2.4 GHz signals travel farther and penetrate walls better, which matters when the controller is outside or in a garage, and 2.4 GHz radio modules are cheaper. The trade-off is a crowded band (Lab 3). The topology is a simple **star**: the controller is one more client of the router. Notice that the soil itself is never connected; a future product with wireless soil probes in each zone would face the battery-powered sensor problem of Worked Example 3.4 and would likely need BLE, Zigbee, or LoRa links back to the controller.

Chapter summary

- IoT **access** is the first-mile link into the network; IoT **communication** is end-to-end routing and interpretation.
- The **OSI model** has seven layers. Senders encapsulate data from the application layer down to bits; receivers decapsulate. Routers process only layers 1 to 3.

- The **physical layer** uses modulation (ASK, FSK, PSK, QAM) and hardware (antennas, transceivers, amplifiers, modems). Signal strength in **dBm** is logarithmic: 3 dB doubles power and 10 dB multiplies it by ten.
- The **data link layer** frames data, uses MAC addresses, detects errors, and controls access. Small IoT payloads make **transmission efficiency** low unless data is packed or batched.
- **IPv4** has about 4.3 billion addresses; **IPv6** has about 3.4×10^{38}. WPANs use 16-bit short addresses and 6LoWPAN. **TCP** is reliable and ordered; **UDP** is lightweight.
- Wi-Fi devices scan, authenticate (WPA3), associate, and exchange data. Wi-Fi 6 and 7 add features for dense networks; HaLow targets IoT.
- **BLE**, **Zigbee**, **Z-Wave**, **Thread/Matter**, **RFID**, **NFC**, and **LPWANs** each fit different range, power, data, and cost needs. Use the four-constraint framework to choose.
- **Star** topologies are simple but have a single point of failure; **mesh** topologies self-heal at the cost of complexity.
- IoT network security rests on device identity, transport encryption, segmentation, and OTA updates.

Key terms

6LoWPAN; access point; advertising (BLE); ASK; Bluetooth Low Energy (BLE); CoAP; CRC; CSMA/CA; data link layer; dBm; decapsulation; encapsulation; first mile; FSK; GATT; IPv4; IPv6; ISM band; LAN; LoRaWAN; LPWAN; MAC address; Matter; mesh topology; modulation; NB-IoT; NFC; OSI model; packet; physical layer; PSK; QAM; RFID; RSSI; segment; star topology; TCP; TDMA; Thread; transmission efficiency; UDP; WLAN; WPA3; Zigbee; Z-Wave.

Review questions

Multiple choice

1. How many layers does the OSI reference model have?
    a. 4
    b. 5
    c. 7
    d. 9
2. Which OSI layer routes packets using IP addresses?
    a. Physical
    b. Session
    c. Presentation
    d. Network

3. Which statement best describes TCP?
    a. A physical-layer standard for copper cables
    b. A transport-layer protocol that segments data and ensures reliable, ordered delivery
    c. A wireless modulation technique
    d. A cloud database service
4. Why can BLE devices run for years on a small battery?
    a. They use the 5 GHz band exclusively
    b. They spend most of their time in deep sleep and transmit briefly
    c. They transmit continuously at high power
    d. They draw power from the Wi-Fi router
5. A warehouse uses a star topology in which every sensor connects directly to one gateway. What is the main weakness?
    a. Messages must hop through many devices
    b. Sensors must act as routers
    c. The gateway is a single point of failure
    d. It cannot connect to the internet
6. A signal of −40 dBm is how many times stronger than a signal of −70 dBm?
    a. 3
    b. 30
    c. 100
    d. 1,000
7. Which modulation technique changes the frequency of the carrier?
    a. ASK
    b. FSK
    c. PSK
    d. TDMA
8. In the Wi-Fi joining procedure, which step uses WPA3's SAE handshake?
    a. Scanning
    b. Authentication
    c. Association
    d. Data exchange
9. Which technology is best suited for tracking pallets with tags that cost a few cents and need no battery?
    a. Wi-Fi
    b. UHF RFID
    c. LTE-M
    d. Zigbee

10. Which security layer isolates IoT devices from enterprise systems?
    a. Device identity b. Transport encryption c. Network segmentation d. OTA updates

Short answer

11. Using the OSI model, explain what happens to a temperature reading as it travels from a sensor to a cloud server. Which layers matter most for a small battery-powered device, and why?
12. Why can't most IoT end devices simply use the same networking as a laptop? Explain with power, memory, and intermittent connectivity.
13. Explain the difference between BLE advertising and a BLE GATT connection, and give one use for each.
14. Identify one security weakness specific to IoT networks and explain how one of the four security layers reduces it.

Applied problems

15. A gateway measures −50 dBm from a sensor at 1 m. Using the log-distance model with n = 3, estimate the RSSI at 8 m. If the receiver needs at least −90 dBm, what is the maximum estimated range?
16. A farm wants 120 soil-moisture probes spread over 300 hectares, each sending 10 bytes every 30 minutes, with a five-year battery life and no Wi-Fi in the fields. Use the four-constraint framework to recommend an access technology and topology, and state the trade-off.
17. **NxEco case.** Explain why the NxEco controller can use Wi-Fi directly while the battery sensor nodes in Chapter 7 use BLE and a gateway, referring to the four constraints of the protocol selection framework.

Lab 3: Wireless site survey

Collaboration 3 (Collaboration Report 1) and preparation for SM Assignment 1.

Objective. Measure a real radio environment, then act as the network engineer for a small business that wants to add about 50 battery-powered sensors to a building of the student's choice (an office, a clinic, a warehouse, or a restaurant).

Tools (free).

- *Windows:* open Command Prompt and run `netsh wlan show networks mode=bssid` to list nearby networks with channel, band, and signal (as a percentage); `netsh wlan show interfaces` shows the current connection's signal and rate.

- *macOS:* hold the Option key and click the Wi-Fi menu to see the current RSSI (dBm), channel, and band; *Wireless Diagnostics → Window → Scan* lists nearby networks.
- *Android:* a Wi-Fi analyzer app shows networks, channels, and dBm; **nRF Connect** (Nordic Semiconductor) lists nearby BLE devices with RSSI.
- *iOS:* Apple's AirPort Utility includes a Wi-Fi scanner that can be enabled in Settings.

Procedure.

1. **Survey the Wi-Fi environment.** Record every visible network's band (2.4, 5, or 6 GHz), channel, and signal strength. Count how many networks share each 2.4 GHz channel.
2. **Survey Bluetooth.** Record nearby BLE devices and their RSSI. Note which ones are recognizable (headphones, watches, beacons).
3. **Measure signal versus distance.** Measure the student's own network in at least three locations: near the router, one room away, and the farthest room. Record RSSI and the number and type of walls in between. Take screenshots.
4. **Summarize the survey** (Report part a). How crowded is the 2.4 GHz band? How does signal strength change with distance and walls? Compare the measurements with Worked Example 3.2.
5. **Recommend an access technology** (part b) for the business's 50 sensors (BLE, Zigbee, or Wi-Fi), using all four constraints of the protocol selection framework, and state the trade-off accepted.
6. **Choose a topology** (part c), star or mesh, and explain why it fits the building. Include a simple floor-plan diagram showing gateway and router placement.
7. **Format and submit** one Word or PDF file with screenshots, the diagram, and APA 7th edition citations.

Table 3.6 Survey record template for Lab 3

Location	Distance from router	Walls between	Band / channel	RSSI (dBm)	Notes
1 Near router					
2 One room away					
3 Farthest room					

Preparing for SM Assignment 1. The Socratic Metric discussion asks students to explain four questions aloud, as if to a friend: (1) the OSI journey of a temperature reading; (2) why IoT devices cannot use laptop networking; (3) BLE versus Zigbee versus Wi-Fi for 50 sensors in a three-floor office; and (4) one IoT-specific security weakness and the control that reduces it. Short-answer questions 11 to 14 and Worked Examples 3.1 and 3.4 are designed as practice. Practice explaining in your own words rather than reading a script.

References and further reading

Afaneh, M. (2020, April 21). *Wireless connectivity options for IoT applications: Technology comparison*. Bluetooth SIG. https://www.bluetooth.com/blog/wireless-connectivity-options-for-iot-applications-technology-comparison/

ByteByteGo. (2022). *What is OSI model?* [Video]. YouTube. https://www.youtube.com/watch?v=0y6FtKsg6J4

Cloudflare. (n.d.). *What is the OSI model?* Cloudflare Learning Center. https://www.cloudflare.com/learning/ddos/glossary/open-systems-interconnection-model-osi/

ComputerNetworkingNotes. (2024). *Basic concepts of computer networking* [Video]. YouTube. https://www.youtube.com/watch?v=1QHr2_DJ1A4

Connectivity Standards Alliance. (2024). *What is Zigbee?* [Video]. YouTube. https://www.youtube.com/watch?v=2p8G6JMvgUw

CyberSentriq. (2025). *IoT security: How to protect your smart devices* [Video]. YouTube. https://www.youtube.com/watch?v=GhGTzD7THXw

DFRobot. (2023, October 18). *Smart home protocols explained: Wi-Fi vs Bluetooth vs Zigbee vs Z-Wave vs Thread and Matter*. https://www.dfrobot.com/blog-13453.html

Dusun IoT. (2024). *The Zigbee network (star, tree, mesh) topologies explained* [Video]. YouTube. https://www.youtube.com/watch?v=QcYNeFtrzQk

Eye on Tech. (2023). *NFC vs. RFID: What's the difference?* [Video]. YouTube. https://www.youtube.com/watch?v=m0wXeSxQj9I

Kirvan, P., Yasar, K., & Shea, S. (2025, August 5). *What is IoT security?* TechTarget. https://www.techtarget.com/iotagenda/definition/IoT-security-Internet-of-Things-security

morerireless0. (2011). *Bluetooth basics: What is Bluetooth?* [Video]. YouTube. https://www.youtube.com/watch?v=e1p4c7Gf8d8

NxEco, Inc. (2020). *NxEco* (Version 2.2.4) [Mobile application software]. App Store. https://apps.apple.com/app/id972339992

NxEco. (n.d.-b). *Smart controller specification* (Ver. 10.1) [PDF]. https://www.nxeco.com/Public/public/file/manual/Nxeco_Smart_Control_Spec_Ver10.1.pdf

NxEco. (n.d.-c). *NxEco smart weather-based irrigation controller: App manual* (Ver. 10.2) [PDF]. https://www.nxeco.com/Public/public/file/manual/NxEco_Smart_Manual_Ver10.2.pdf

NxEco. (n.d.-e). *Support and frequently asked questions* [Web page]. https://www.nxeco.com/support.html

Simplilearn. (2023). *Data link layer in OSI model* [Video]. YouTube. https://www.youtube.com/watch?v=HNefQ1J4eFk

The Default Guy. (2024). *How WiFi actually works* [Video]. YouTube. https://www.youtube.com/watch?v=AFEhHGzidNI

CHAPTER 4

IoT Edge and Cloud Computing

Learning Objectives

After studying this chapter, students will be able to:

1. Describe the three-tier device, edge, and cloud architecture and the end-to-end pipeline of proximity network, backhaul, and presentation link.
2. Explain why constrained devices need gateways and describe edge-node hardware and software.
3. Apply the four edge objectives of data filtering, real-time analytics, protocol translation, and local survivability, and calculate bandwidth savings.
4. Explain the MQTT publish/subscribe model, topics and wildcards, quality of service, and retained messages.
5. Describe bidirectional control through device shadows, commands, and over-the-air updates.
6. Explain cloud service models and the four core modules of an IoT cloud service, using AWS IoT Core as the worked example.
7. Decide where processing should happen for a business scenario and identify the security and privacy implications.

Where this fits: Week 4. This chapter supports Collaboration 4 *MQTT Publish/Subscribe Lab* and Assignment 2 *MQTT Lab and Edge-or-Cloud Design Report*. Chapters 1 to 4 together form the content of the Week 5 midterm.

Opening case: fifty machines and a slow uplink

A plastics manufacturer in Monterrey, Mexico, runs 50 injection-molding machines. Each machine has vibration, temperature, and pressure sensors that produce readings every few milliseconds. The plant's internet connection is a shared business line that occasionally drops during storms. Management wants three things: an immediate stop if a machine's pressure spikes dangerously, a dashboard at the head office in another country, and a model that predicts bearing failures a week in advance.

These three goals pull in different directions. The emergency stop must happen in milliseconds and cannot depend on the internet. The dashboard must be reachable from anywhere. The predictive model needs months of data from every machine and powerful computers to train. No single location can do all three well. The solution is to split the work:

the machines' controllers handle the stop, an **edge gateway** in the plant filters data and keeps working during outages, and the **cloud** stores history, trains models, and serves the dashboard. This chapter explains how that split is designed.

4.1 The three-tier IoT architecture

The state-of-the-art topology for most IoT systems has three tiers (Figure 4.1):

1. **Device tier (perception).** Constrained end nodes with limited power and memory collect raw data and drive actuators.
2. **Edge tier.** A **gateway** acts as the bridge. It performs protocol translation (for example Zigbee to IP), processes data locally to filter out noise, and keeps critical functions running when the wide-area link fails.
3. **Cloud and application tier.** The back end (message broker, databases, analytics) and the presentation layer, where people interact with data through mobile apps and dashboards.

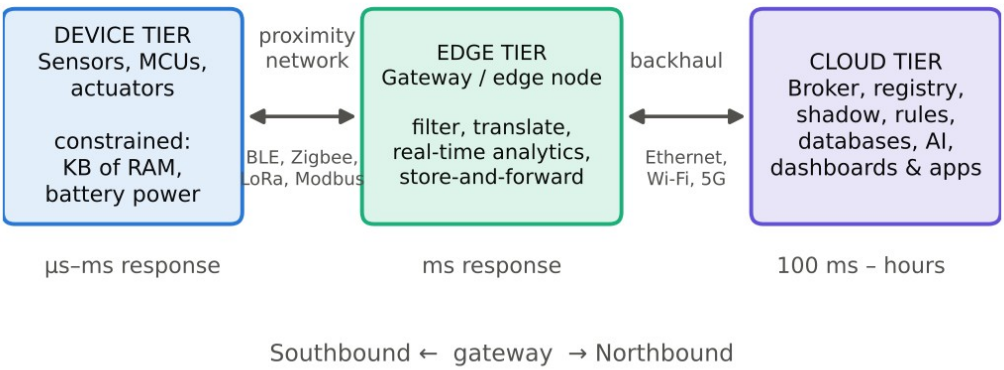


Figure 4.1 The three-tier device, edge, and cloud architecture. Response time grows and computing power increases from left to right.

The three-tier model answers a different question from the four-layer model in Chapter 1. The four-layer model asks *what function* each part performs; the three-tier model asks *where* each function runs.

IoT and 5G

Cellular networks offer an alternative path from device to cloud without a local gateway. In the **5G** architecture, any connecting device is called **user equipment (UE)**. Low-power IoT devices can use **5G RedCap** (reduced capability) terminals, a 5G class designed for wearables, industrial sensors, and video cameras that need more speed than NB-IoT but less

cost and power than a smartphone. The UE connects by radio to a base station called a **gNodeB (gNB)** in the **radio access network (RAN)**. The operator's core network then routes traffic to a **data network (DN)**, where the application server sits, often in a public cloud such as AWS, Azure, or Google Cloud. Some operators place computing inside their networks (**multi-access edge computing**, MEC) so that latency-sensitive applications run within a few milliseconds of the device.

4.2 The end-to-end pipeline

Data travels from sensor to phone in three stages:

Table 4.1 The three stages of the edge-to-cloud pipeline

Stage	Link	Also called	Typical technologies
A: Device to gateway	**Proximity network** (personal or local area network)	Short-range, field-area network, **southbound** communication	BLE, Zigbee, Thread, LoRa, Modbus/RS-485
B: Gateway to cloud	**Backhaul** (wide area network)	Network egress, uplink, **northbound** communication	Ethernet, fiber, Wi-Fi, LTE, 5G
C: Cloud to phone	**Presentation link**	Data consumption, push notification, API call	WebSockets, HTTPS (REST), MQTT

"Southbound" and "northbound" are always relative to the gateway: southbound faces the devices, northbound faces the cloud.

WPAN versus WLAN

Stage A usually uses a **wireless personal area network (WPAN)** or a **wireless local area network (WLAN)**. A WPAN is a short-range network centered on a person's or device's immediate surroundings, typically within about 10 m; its principle is personal-centric connectivity rather than wide coverage. Table 4.2 compares them.

Table 4.2 WPAN versus WLAN

Feature	WPAN	WLAN (Wi-Fi)
IEEE standard	802.15 family (Bluetooth origin, Zigbee/Thread on 802.15.4, UWB on 802.15.4z)	802.11 (Wi-Fi 6, 6E, 7)
Typical range	1 to 10 m (more with mesh or BLE long range)	30 to 100 m
Power needs	Ultra-low; optimized for coin cells	Higher; usually rechargeable or mains power
Data speed	Low to moderate (kbps to a few Mbps)	Very high (hundreds of Mbps to Gbps)
Module cost	Very low	Low to moderate

Feature	WPAN	WLAN (Wi-Fi)
Main goal	Cable replacement, sensor and peripheral links	High-speed internet and network access

Use a WLAN for devices that move large amounts of data, need direct internet access, or are plugged into power. **Use a WPAN** for battery-powered sensors, wearables, and remote controls that are always near a hub such as a phone or gateway.

Global Perspective: The home network as an IoT platform. In 2004 and 2005, the author of this textbook led an IoT start-up that built one of the first home networks for Haier, now one of the world's largest home-appliance manufacturers. The goal then, connecting refrigerators, air conditioners, and washing machines through a short-range home network to a central controller, is the same goal that Matter pursues today. What has changed is cost, standards, and cloud scale, not the basic architecture.

4.3 Why constrained devices need a gateway

Many real-world IoT devices cannot connect to the cloud directly. Examples include microcontrollers with 32 KB of RAM, Zigbee mesh nodes that do not speak IP at all, and industrial sensors with serial-only interfaces such as RS-485. A full TCP/IP stack plus **Transport Layer Security (TLS)** and a certificate chain can need more memory than an entire budget microcontroller has. The gateway exists to absorb that cost: it speaks the devices' native protocols on one side and secure, cloud-compatible protocols, typically **MQTT over TLS**, on the other. It is not a convenience. It is the reason the sensor node can stay cheap and tiny.

Edge-node hardware

Table 4.3 summarizes the subsystems of a typical edge gateway.

Table 4.3 Hardware architecture of an IoT edge node

Subsystem	Components and technologies	Primary function
Compute core	Multi-core Arm or x86 processors (Raspberry Pi, industrial PCs, NVIDIA Jetson)	Runs Linux (Debian, Ubuntu Core, Yocto) and container runtimes (Docker, containerd)
Edge AI acceleration	Neural processing units, AI accelerators (for example Hailo-8, Google Coral Edge TPU), FPGAs	Offloads machine-learning inference for fast anomaly detection and vision
Southbound interfaces	Zigbee, BLE, LoRa radios, RS-485/Modbus, CAN bus, GPIO	Collects telemetry from field sensors, MCUs, and actuators
Northbound interfaces	Cellular (4G LTE, 5G), Wi-Fi 6, Gigabit Ethernet	High-bandwidth uplink to the cloud
Hardware root of trust	Secure element, Trusted Platform Module (TPM 2.0)	Protects private keys and X.509 certificates; supports secure boot

Edge-node software

Inside the gateway, a layered software stack typically includes the operating system; protocol adapters for each southbound interface; a local message broker; a rules or stream-processing engine; a local database for **store-and-forward** buffering (often SQLite or a time-series database); a device-management agent that handles updates and remote configuration; and a cloud connector. Platforms such as **AWS IoT Greengrass** and **Azure IoT Edge** package these functions so that developers deploy them as managed components.

4.4 What the edge does: four objectives

An edge node addresses four core trade-offs of distributed systems (Table 4.4).

Table 4.4 The four edge objectives

Objective	Description	Reasoning
Data filtering	Removing noise before sending data, for example a camera ignoring a swaying tree, or a gateway sending only one-minute averages	**Bandwidth efficiency**: saves money by not sending useless data
Real-time analytics	Running rules or AI models locally to detect anomalies, such as a machine about to fail	**Safety and speed**: decisions in milliseconds, not seconds
Protocol translation	Converting industrial or field protocols (Modbus, OPC UA, Zigbee, BLE GATT) into web protocols (MQTT, HTTPS)	**Interoperability**: old machines can talk to modern cloud apps
Local survivability	Keeping the factory or home running even if the internet goes down, and buffering data until it returns	**Reliability**: critical systems cannot depend on a perfect connection

A fifth benefit follows from these: **deterministic latency**. A cloud round trip commonly takes 50 to 100 ms or more and varies with network conditions. Rules executed on site run in microseconds to milliseconds, every time.

Worked Example 4.1: How much bandwidth does edge filtering save?

A building has 100 sensors. Each sends one 200-byte message per second. Compare the daily data sent to the cloud (a) with no filtering, (b) with the edge sending one summary per sensor per minute, and (c) with the edge sending only anomalies, about 10 events per sensor per day.

(a) Messages per day = 100 × 86,400 s = 8,640,000. Data = 8,640,000 × 200 bytes = 1,728,000,000 bytes ≈ **1.73 GB per day** (about 52 GB per month).

(b) Summaries per day = 100 × 1,440 minutes = 144,000. Data = 144,000 × 200 bytes = 28,800,000 bytes = **28.8 MB per day**, a 60-fold reduction (98.3 percent less).

(c) Events per day = 100 × 10 = 1,000. Data = 1,000 × 200 bytes = **0.2 MB per day**.

Interpretation. Figure 4.2 shows the difference on a logarithmic scale. Over cellular backhaul, where data is billed by the megabyte, edge filtering can decide whether a project is profitable. The trade-off is that raw data discarded at the edge cannot be analyzed later, so many systems keep raw data locally for a few days and send only summaries upstream.

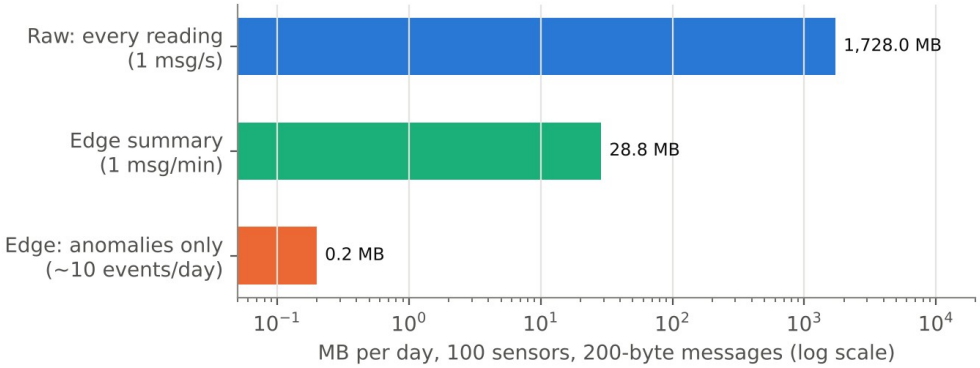


Figure 4.2 Daily data volume for 100 sensors with and without edge filtering (logarithmic scale).

Worked Example 4.2: Why some decisions cannot wait for the cloud

A vision system rejects defective bottles on a conveyor moving at 2 m/s. How far does a bottle travel during (a) an 80 ms cloud round trip, and (b) a 5 ms edge inference?

(a) Distance = 2 m/s × 0.080 s = 0.16 m = **16 cm**.

(b) Distance = 2 m/s × 0.005 s = 0.01 m = **1 cm**.

With cloud processing, the reject mechanism would have to sit at least 16 cm downstream, and any network delay spike would let defective bottles pass. With edge processing, the decision is fast and, just as importantly, consistent. The cloud still has a role: it collects images of rejected bottles from every line and retrains the model.

Edge computing application cases

- **Transportation: autonomous vehicles.** Onboard computers process LiDAR and camera data to avoid collisions. The vehicle uses the cloud only to update maps and report traffic patterns.
- **Commerce: smart retail.** AI cameras detect when a shopper picks up an item. The edge handles checkout logic locally for privacy and speed; the cloud manages inventory across all stores.
- **Healthcare: precision monitoring.** A wearable heart monitor detects arrhythmia with local AI and immediately alerts the user's phone over Bluetooth, while sending periodic summaries to the doctor's cloud portal.

4.5 Northbound communication and edge runtimes

Before an edge node sends information upstream, it optimizes what crosses the network:

- **Protocol translation** bridges field protocols (Modbus, Zigbee, CAN bus, BLE GATT) into structured formats such as JSON over MQTT.
- **Filtering and aggregation** remove noise and send only anomalies or summary statistics, reducing bandwidth and cloud storage cost.
- **Store-and-forward** keeps messages in a local database (for example SQLite) when the internet connection drops and synchronizes automatically when it returns. Chapter 7 tests this behavior in the demo system.
- **Dedicated runtimes** such as AWS IoT Greengrass and Azure IoT Edge manage routing, machine-learning inference, and cloud synchronization.

4.6 MQTT: the messaging protocol of IoT

MQTT (originally Message Queuing Telemetry Transport) is a lightweight publish/subscribe messaging protocol designed for unreliable networks and constrained devices. It is an OASIS standard (versions 3.1.1 and 5.0) and carries much of the world's IoT telemetry in smart homes, remote sensors, and industrial automation (HiveMQ Team, 2026a).

The publish/subscribe model

MQTT has four core elements:

- **Publisher:** a device or gateway that sends data.
- **Broker:** a server that receives every message and distributes it to interested clients.
- **Subscriber:** an app, service, or device that wants specific data.
- **Topic:** a named, hierarchical routing path such as `home/room1/temp`.

The flow is **publisher → broker → subscriber**. A postal analogy helps: the publisher is the mail sender, the broker is the post office, the subscriber is the recipient, and the topic is the mailing address. Unlike a letter, one published message can be delivered to many subscribers at once.

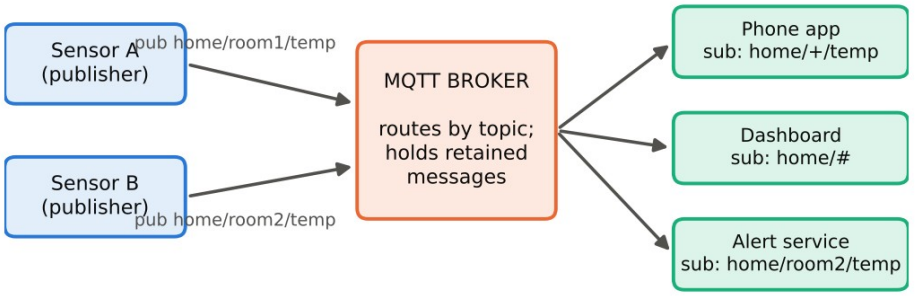


Figure 4.3 MQTT publish/subscribe. Publishers and subscribers know only the broker and the topic names.

The broker **decouples** publishers and subscribers in three ways: in *space* (they do not know each other's addresses), in *time* (they need not be online simultaneously when sessions or retained messages are used), and in *synchronization* (neither blocks waiting for the other). This makes it easy to add a new subscriber, such as an analytics service, without changing any device.

Topics and wildcards

Topics are UTF-8 strings with levels separated by forward slashes, for example `iotcourse/alice/room1/temp`. Subscribers can use two wildcards (HiveMQ Team, 2026b):

- **`+` (single-level wildcard)** matches exactly one level. `iotcourse/alice/+/temp` matches `iotcourse/alice/room1/temp` and `iotcourse/alice/room2/temp`, but not `iotcourse/alice/room1/humidity`.
- **`#` (multi-level wildcard)** matches any number of levels and must be last. `iotcourse/alice/#` matches everything under `iotcourse/alice/`.

Good topic design uses a consistent hierarchy from general to specific (for example `organization/site/area/device/measurement`), avoids leading slashes and spaces, and never puts personal data in topic names.

Quality of service

MQTT offers three **quality of service (QoS)** levels for each message:

Table 4.5 MQTT quality of service levels

QoS	Name	Guarantee	Cost	Typical use
0	At most once	Fire and forget; message may be lost	Lowest	Frequent sensor readings where the next

QoS	Name	Guarantee	Cost	Typical use
				reading replaces a lost one
1	At least once	Delivered, possibly more than once (receiver may see duplicates)	One acknowledgment	Telemetry and commands that must arrive; receiver tolerates duplicates
2	Exactly once	Delivered once and only once	Four-step handshake	Billing events or commands where duplicates cause harm

Retained messages, last will, and keep-alive

- A **retained message** is stored by the broker as the last known value on a topic. A new subscriber immediately receives it, rather than waiting for the next publish. This is ideal for status values such as "valve = open."
- A **last will and testament (LWT)** is a message that the broker publishes on a client's behalf if the client disconnects unexpectedly, for example `device/42/status = offline`.
- **Keep-alive** pings let client and broker detect a dead connection.

MQTT versus HTTP

HTTP follows a **request/response** model: the client must ask for data, and every request carries text headers that can be hundreds of bytes long. MQTT keeps one persistent connection open, uses a fixed header as small as 2 bytes, and lets the broker **push** data as soon as it arrives. For battery-powered devices on intermittent links that send small, frequent messages, MQTT uses less bandwidth and energy. HTTP remains useful for occasional bulk uploads, firmware downloads, and web APIs (CDEBYTE, 2023).

4.7 Bidirectional control and digital twins

Communication between edge and cloud is bidirectional.

Uplink (edge to cloud) carries:

- **telemetry**, raw and processed sensor values;
- **inference alerts**, the outputs of local AI models and event triggers;
- **operational state**, device health, battery level, firmware version, and connectivity metadata.

Downlink (cloud to edge) carries:

- **device shadow (digital twin) updates**, JSON documents that synchronize *desired* and *reported* state;

- **control commands**, such as actuation directives or schedule changes;
- **over-the-air (OTA) updates**, new firmware, container versions, or retrained AI models.

Device shadows

A **device shadow** (AWS) or **device twin** (Azure) is a JSON document stored in the cloud that represents a device's state. It has two main sections: `desired`, written by apps, and `reported`, written by the device. The difference between them, the `delta`, tells the device what to change.

```
{
  "state": {
    "desired":  { "fan_pwm": 60, "relay": "ON" },
    "reported": { "fan_pwm": 0,  "relay": "OFF", "temp": 31.2 }
  }
}
```

An app can set `fan_pwm` to 60 percent even while the device is asleep or offline. When the device next connects, it reads the delta, applies the change, and updates `reported`. Shadows make remote control reliable over unreliable links (Amazon Web Services, n.d.-b). The broader term **digital twin** also describes richer virtual models of machines, factories, or cities that combine live data with simulation.

4.8 Security and identity between edge and cloud

Communication between edge and cloud is strictly encrypted and authenticated:

1. **Data in transit: TLS 1.2 or 1.3.** TLS encrypts all traffic to prevent interception or tampering. With **mutual TLS (mTLS)**, both sides prove their identity, not just the server.
2. **Identity verification: X.509 certificates.** Each device or gateway holds a unique digital certificate and a private key, ideally stored in a secure element or TPM. The server presents its own certificate, which the device checks against a trusted root certificate authority.
3. **Session authorization: device registry and policies.** The cloud platform checks the connecting certificate against its registry of authorized devices and applies a policy that defines which topics the device may publish or subscribe to.

Chapter 6 walks through the mTLS handshake step by step.

4.9 Cloud computing fundamentals

Cloud computing is the on-demand delivery of computing power, storage, databases, applications, and other IT resources over the internet with pay-as-you-go pricing

(Simplilearn, 2020). Instead of buying and managing physical servers, organizations rent services from providers such as Amazon Web Services (AWS), Microsoft Azure, Google Cloud, Alibaba Cloud, Huawei Cloud, and Oracle Cloud.

Cloud infrastructure

Cloud providers operate large **data centers** filled with servers, storage, and networking equipment (Google Cloud Tech, 2020). These resources are **virtualized**, divided into virtual machines, containers, and managed services, and grouped into geographic **regions**, each with several isolated **availability zones**. Users connect over the internet through web consoles and **application programming interfaces (APIs)**.

Service models

Table 4.6 Cloud service models

Model	What the provider manages	What the customer manages	Examples
Infrastructure as a Service (IaaS)	Physical servers, storage, network, virtualization	Operating system, middleware, applications, data	Virtual machines (Amazon EC2, Azure VMs, Akamai/Linode)
Platform as a Service (PaaS)	Everything up to the runtime and middleware	Applications and data	AWS IoT Core, Azure IoT Hub, managed databases
Software as a Service (SaaS)	The entire application	Data and configuration	Google Workspace, Microsoft 365, fleet-management SaaS

Databases for IoT

Cloud databases are usually delivered as **database as a service (DBaaS)**: the provider handles hardware, operating system, patching, backups, and scaling, so the customer focuses on data and application logic. IoT workloads use several types of database:

- **Time-series databases** (for example InfluxDB, TimescaleDB, Amazon Timestream) are optimized for timestamped readings and fast range queries.
- **NoSQL key-value and document databases** (for example Amazon DynamoDB, Azure Cosmos DB, MongoDB) scale horizontally and store flexible JSON records. The demo system stores telemetry in a DynamoDB table with the device ID as the partition key and the timestamp as the sort key.
- **Relational databases** (PostgreSQL, MySQL) store device metadata, customers, and billing.
- **Object storage and data lakes** (Amazon S3, Azure Blob Storage) archive raw data cheaply for later analytics and machine learning.

IoT data has a **lifecycle**: *hot* data from the last hours or days lives in fast databases for dashboards and alerts; *warm* data is summarized for weeks or months of trend analysis; *cold* data is archived for compliance and model training (CodeLucky, 2025).

4.10 The IoT cloud service: four core modules

Every major IoT cloud platform includes four essential modules (Amazon Web Services, n.d.-a):

1. **Message broker (IoT core).** The "front door" of the cloud. It receives data from devices, usually over MQTT, and sends commands back.
2. **Device registry and identity.** A database of every authorized device, linked to its X.509 certificate, so that only known devices can connect.
3. **Device shadow (twin).** The JSON representation of each device's desired and reported state.
4. **Rules engine.** A logic layer that evaluates each message and routes it. For example: "If temperature is above 30 °C, send an alert to the user's phone and save the reading to a database."

Worked platform example: AWS IoT Core

Figure 4.4 summarizes how a reading enters AWS IoT Core and flows to other services.

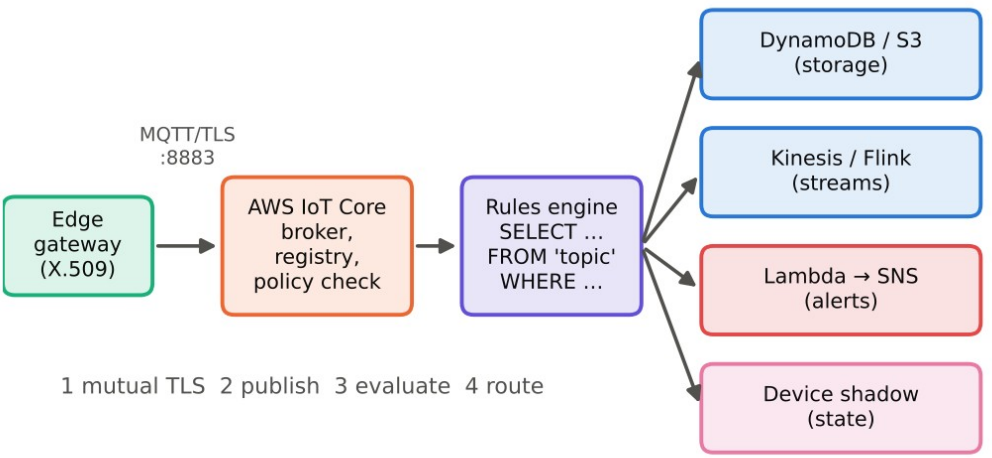


Figure 4.4 Ingestion and routing in AWS IoT Core.

Step 1: Secure mutual authentication. The edge gateway opens a TCP connection to the account's endpoint (in the form `<prefix>-ats.iot.<region>.amazonaws.com`) on **port 8883**. It verifies the server's certificate against the trusted **Amazon Root CA 1**, then

presents its own **X.509 client certificate**, proving possession of the matching private key. AWS IoT Core checks its registry and the **IoT policy** attached to the certificate to decide which topics the gateway may use.

Step 2: Protocol transport. Once authenticated, devices send messages by:

- **MQTT** (the primary protocol) to hierarchical topics such as `factory/line1/sensor/temperature`;
- **HTTPS** POST requests for occasional batch uploads;
- **MQTT over WebSockets** (port 443), often used by browsers and mobile apps;
- for very high-volume streams such as video or vibration, Greengrass **stream manager**, which buffers on disk and exports directly to Amazon Kinesis or Amazon S3.

Step 3: Rules engine evaluation. The **rules engine** listens to incoming messages without any servers to manage, and evaluates them with SQL-like statements, for example:

```
SELECT device_id, temp, humidity, timestamp() AS ingest_time
FROM 'iot/telemetry/+'
WHERE temp > 0
```

Step 4: Routing to downstream services. Rule actions send matching messages to:

- **storage**, such as Amazon DynamoDB for operational time-series data or Amazon S3 for archives and data lakes;
- **stream analytics**, such as Amazon Kinesis or Amazon Managed Service for Apache Flink, for real-time aggregation and anomaly detection;
- **serverless compute**, such as an **AWS Lambda** function triggered on exceptions, which can send an alert through **Amazon SNS** (email, SMS, or push) or publish a command back to a device.

On the edge side, **AWS IoT Greengrass** ingests telemetry from field devices over Modbus, BLE, or OPC UA into a local broker, buffers messages locally if the backhaul fails, and automatically flushes the buffer to AWS IoT Core over the authenticated mTLS channel when the connection returns.

> **Key Concept: Services change; patterns endure.** Cloud providers retire and rename services regularly. Google Cloud IoT Core was retired in August 2023; AWS IoT Analytics stopped accepting new customers in 2024 and reached end of support in December 2025; Amazon SageMaker Edge Manager was discontinued in 2024. The four-module pattern (broker, registry, shadow, rules) and the edge objectives remain the same, so students should learn the pattern and then check current documentation before naming a specific service in a design.

Key protocols between cloud and clients

Table 4.7 Key protocols between devices, cloud, and clients

Protocol	Transport	Model	Typical use
MQTT	TCP (TLS on 8883)	Publish/subscribe	Device telemetry and commands; ideal for intermittent, low-bandwidth links
HTTPS (REST)	TCP (TLS on 443)	Request/response	Bulk uploads, firmware downloads, historical queries from apps
WebSockets	TCP (TLS on 443)	Full-duplex stream	Real-time dashboards; MQTT over WebSockets in browsers and mobile apps
CoAP	UDP	Request/response with observe	Very constrained devices and networks

Why connect IoT devices to the cloud?

- **Global accessibility and control.** The cloud acts as a relay, so a user in one country can securely command a device in another through a phone app.
- **Fleet management and OTA updates.** The cloud can push a security patch or a new AI model to millions of devices.
- **Volume.** Local hardware cannot store the continuous data streams of a large fleet.
- **Intelligence.** Training deep-learning and predictive models needs GPU clusters that do not exist at the edge.
- **Distance.** Peer-to-peer connections between devices spread across the world are fragile and hard to manage; a central broker simplifies them.
- **Maintenance.** Centralized control is the only practical way to monitor, patch, and audit a global fleet.

Building an IoT cloud: three options

How does a company such as a smart irrigation manufacturer build its IoT back end? Three options exist:

1. **Run IoT software on your own servers**: install an open-source MQTT broker (for example Eclipse Mosquitto or EMQX) and a database in a private data center.
2. **Rent infrastructure (IaaS)** from a cloud provider and run the same software on virtual machines. The NxEco smart irrigation system, this book's running application case, runs its IoT server on rented Akamai (Linode) infrastructure in this way (Section 4.12).
3. **Use a managed IoT platform (PaaS)** such as AWS IoT Core or Azure IoT Hub, paying per message and per connected device, and let the provider operate the broker, registry, and scaling.

Option 1 gives the most control and the most work; option 3 gives the fastest start and the least control, with ongoing usage fees and some vendor lock-in.

4.11 IoT cloud providers

AWS, Microsoft Azure, and Google Cloud are the three largest general-purpose cloud providers. Their IoT offerings differ (Table 4.8).

Table 4.8 IoT services of the three largest cloud providers (check current documentation)

Feature	AWS	Microsoft Azure	Google Cloud
Device connectivity service	AWS IoT Core	Azure IoT Hub	No first-party IoT broker since Google Cloud IoT Core was retired (2023); partners' brokers feed Pub/Sub
Edge runtime	AWS IoT Greengrass	Azure IoT Edge; Azure IoT Operations	Google Distributed Cloud; Coral Edge TPU hardware
Typical storage	DynamoDB, S3, Timestream	Cosmos DB, Blob Storage, Azure Data Explorer	Bigtable, Cloud Storage, BigQuery
AI/ML	SageMaker	Azure Machine Learning	Vertex AI, BigQuery ML

Other important platforms include **Alibaba Cloud IoT Platform** and **Huawei Cloud IoT Device Access** (widely used in China and parts of Asia, Africa, and the Middle East), **Siemens Insights Hub** (formerly MindSphere) for industrial customers, and open-source platforms such as **ThingsBoard**, which companies can host themselves.

AWS's IoT business has focused on three high-growth verticals: **industrial IoT** (digital twins for predictive maintenance, for example AWS IoT TwinMaker), the **consumer device back end** that powers many smart-home brands, and **fleet and logistics** (for example AWS IoT FleetWise for collecting vehicle data).

> **Global Perspective: Data sovereignty.** Where data is stored is often a legal question. The EU's GDPR restricts transfers of personal data outside the European Economic Area; China's Personal Information Protection Law and India's Digital Personal Data Protection Act impose their own rules on cross-border transfers. Cloud regions on every inhabited continent let companies keep data in the right jurisdiction, and edge computing can keep sensitive data, such as video of shoppers or patient vital signs, on site, sending only anonymous summaries to the cloud.

4.12 IoT application case: NxEco's edge and cloud split

NxEco divides the work between the controller at the edge and the cloud in a way that illustrates all four edge objectives and all four cloud modules.

Table 4.9 Placement of NxEco functions

Function	Device (controller)	Cloud (NxEco servers)	Application
Store schedules and keep time	Yes (flash memory)	Copy of schedules	Edit in app or portal
Switch valves	Yes		
Run as a timer without internet	Yes (local survivability)		
Rain-sensor interrupt	Yes		
Collect weather and ET data		Yes	
Calculate daily watering budget	Applies the adjustment	Calculates it	Manual override
Watering history and water reports	Logs runs	Stores and summarizes	Trends, weekly email
Device registry and ownership	Serial number, one-time password	Account and controller registry	Register, rename
Remote control, sharing, voice		Relays commands	App, portals, Alexa, Google Home

Edge objectives. The controller provides **local survivability**: if the internet fails, it keeps watering on its stored schedule like a traditional timer, and the rain sensor still works. It performs **protocol translation** in the simplest sense, turning cloud messages into 24 V AC valve signals. **Real-time analytics** and **data filtering** are minimal, because irrigation decisions are made once a day, not in milliseconds.

Cloud modules. NxEco's cloud has the four modules of Section 4.10: a message service that exchanges schedules and status with controllers; a **device registry** keyed by serial number; a stored **state** for each controller (its schedules, settings, and online status, which the app shows with a green radio icon); and a **rules and calculation engine** that converts weather data into each day's adjustment and sends weekly reports.

Three presentation channels. The same cloud serves three audiences: the **NxEco app** for homeowners; the **MySmartDrop** web portal, which manages up to 50 controllers at no charge; and the **SNAP** professional portal, which manages up to 5,000 controllers for landscape companies and property managers (NxEco, n.d.-f). This is a good example of one IoT back end supporting both consumer and business customers.

Inside the MySmartDrop portal. A signed-in portal account shows how the cloud's stored state is presented (NxEco, 2026). The portal has three menu pages:

- **Home**: the garden's name and the five-day weather forecast for its location (Table 2.12), with a temperature chart.

- **Controller**: one row per registered controller with its **online status** (a green or red icon taken from the cloud's state store), its **controller code**, a 12-digit hexadecimal identifier that has the form of a Wi-Fi MAC address, the owner's name for it, its group, whether zone 12 is used as a master valve, and action buttons for **Watering** (manual run), **Watering Plan** (programs A to D), **Delay Spraying** (rain delay), **Setup**, and **Remove**. The account examined had two controllers: "Home Garden", online with a master valve, and an offline unit named "test".
- **Water Record**: watering history for each controller, in total and zone by zone, with a date-range filter and a time-zone selector. The time zone matters because a cloud serving controllers worldwide stores events in a common time base and must convert them to the garden's local time for display.

The portal is therefore a thin presentation layer over the four cloud modules: the registry supplies the controller list, the state store supplies online status and programs, the rules engine applies the weather adjustment, and the database supplies the water records.

Infrastructure. NxEco runs its IoT server on rented general-purpose cloud infrastructure (Akamai, formerly Linode) rather than a managed IoT platform such as AWS IoT Core, the second of the three build options described in Section 4.10. This gives the company control and predictable cost, at the price of operating its own broker, database, and security updates.

Chapter summary

- The **three-tier architecture** splits work among devices, the edge, and the cloud. The pipeline has a proximity network (southbound), a backhaul (northbound), and a presentation link.
- **Gateways** exist because constrained devices cannot afford TCP/IP plus TLS. Edge nodes combine compute, optional AI acceleration, southbound and northbound interfaces, and a hardware root of trust.
- The four **edge objectives** are data filtering, real-time analytics, protocol translation, and local survivability. Filtering can reduce data by orders of magnitude; edge decisions are faster and more deterministic.
- **MQTT** uses publish/subscribe through a broker, hierarchical topics with + and # wildcards, QoS 0, 1, and 2, retained messages, and last-will messages.
- Communication is **bidirectional**: telemetry, alerts, and state go up; shadow updates, commands, and OTA updates come down. **Device shadows** synchronize desired and reported state.
- Edge-to-cloud security uses **TLS/mTLS**, **X.509 certificates**, and a **device registry** with policies.
- **Cloud computing** offers IaaS, PaaS, and SaaS; IoT uses time-series, NoSQL, relational, and object storage.

- Every IoT cloud has a **broker, registry, shadow, and rules engine**. AWS IoT Core authenticates with mTLS on port 8883, evaluates messages with SQL rules, and routes them to storage, streams, and serverless functions.
- Cloud services change; learn the patterns and verify current offerings.

Key terms

availability zone; backhaul; broker; data center; DBaaS; device registry; device shadow; digital twin; edge computing; edge gateway; gNodeB; IaaS; keep-alive; last will and testament; local survivability; MEC; MQTT; mutual TLS; northbound; PaaS; presentation link; protocol translation; proximity network; publish/subscribe; QoS; RedCap; region; retained message; rules engine; SaaS; southbound; store-and-forward; time-series database; TLS; topic; user equipment; wildcard; WLAN; WPAN; X.509 certificate.

Review questions

Multiple choice

1. Why do many constrained IoT devices need an edge gateway to reach the cloud?
    a. Gateways make sensors more accurate
    b. Cloud platforms accept data only from routers of the same vendor
    c. Gateways replace the need for any network
    d. The devices cannot run a full TCP/IP and TLS stack, so the gateway translates and secures their data
2. Which edge objective keeps a factory's control loops running when the internet connection drops?
    a. Data monetization
    b. Local survivability
    c. Protocol licensing
    d. Cloud elasticity
3. 100 sensors each send one 200-byte message per second. About how much data is sent per day without edge filtering?
    a. 17 MB
    b. 170 KB
    c. 1.7 GB
    d. 17 GB
4. What does the AWS IoT Core rules engine do?
    a. Manufactures sensors
    b. Charges batteries remotely
    c. Replaces the MQTT broker

   d. Evaluates incoming messages with SQL-like statements and routes them to services

5. Which task is best performed in the cloud rather than at the edge?
   a. Stopping a machine within milliseconds
   b. Training machine-learning models on historical data from thousands of devices
   c. Filtering sensor noise before transmission
   d. Keeping a door lock working during an outage
6. In MQTT, which subscription receives messages from `farm/field2/soil` and `farm/field7/soil` but not `farm/field2/air`?
   a. `farm/#`
   b. `farm/+/soil`
   c. `farm/+/#`
   d. `+/soil`
7. Which MQTT QoS level guarantees delivery but may deliver duplicates?
   a. QoS 0
   b. QoS 1
   c. QoS 2
   d. QoS 3
8. A new subscriber immediately receives the last known valve status. Which MQTT feature makes this possible?
   a. Last will
   b. Keep-alive
   c. Retained message
   d. Wildcard
9. In a device shadow, the section written by an app to request a change is:
   a. reported
   b. desired
   c. delta
   d. metadata
10. AWS IoT Core accepts MQTT over TLS from devices on port:
   a. 80 b. 1883 c. 8883 d. 3306

Short answer

11. Explain the difference between southbound and northbound communication, with one technology example of each.
12. Describe the four core modules of an IoT cloud service.

13. Compare MQTT and HTTP for a battery-powered sensor that sends a small reading every minute.
14. Explain how store-and-forward works at the edge and why it matters.

Applied problems

15. A delivery fleet has 400 trucks. Each truck's gateway samples GPS and engine data once per second, 120 bytes per sample, and operates 12 hours per day. (a) How much data per day would the fleet send raw? (b) The edge instead sends one 300-byte summary every 30 seconds plus about 20 event messages of 150 bytes per truck per day. How much data per day is that? (c) What is the percentage reduction?
16. For a wearable heart monitor, place at least five functions (sensing, filtering, arrhythmia detection, alerts, long-term storage, dashboards, OTA updates) in a Device | Edge | Cloud table and justify two of the placements using the four edge objectives.
17. **NxEco case.** Using Table 4.9, identify which NxEco functions continue during an internet outage and which stop, and explain which edge objective this demonstrates.

Lab 4: MQTT publish/subscribe

Collaboration 4 and Assignment 2, Part 1. Students act as both a sensor (publisher) and a phone app (subscriber), exchanging real MQTT messages through a public cloud broker. No hardware, installation, or cloud account is required.

> **Safety note.** The public broker is open to everyone on the internet. Use only made-up sensor data, and never publish personal information.

Tools. The HiveMQ MQTT WebSocket client (search "HiveMQ WebSocket client" or open it from the HiveMQ website) in two browser tabs; the public broker `broker.hivemq.com`, WebSocket port **8000**.

Procedure.

1. **Connect two clients.** Open the WebSocket client in two tabs. In each, set Host = `broker.hivemq.com`, Port = `8000`, and a unique Client ID, then click **Connect**.
2. **Tab A, the "phone app" (subscriber).** Add a subscription to `iotcourse/<yourname>/+/temp`. The + wildcard matches any single room.
3. **Tab B, the "sensor" (publisher).** Publish at least five JSON messages to `iotcourse/<yourname>/room1/temp` and `iotcourse/<yourname>/room2/temp`, for example `{"device":"node1","temp":24.8,"unit":"C"}`. Include one reading above 30 °C.
4. **Test wildcards.** Publish once to `iotcourse/<yourname>/room1/humidity`. Does Tab A receive it? Explain why or why not.

5. **Test QoS.** Repeat one publish with **QoS 1**. Note any difference in the client's display.
6. **Test retained messages.** Publish one message with **Retain** checked. Disconnect Tab A, reconnect, and resubscribe. Record what arrives immediately and explain why.
7. **Capture evidence.** Take screenshots of both tabs showing topics and received messages.
8. **Record the test log** (Table 4.10) and explain each result.

Table 4.10 MQTT lab test log

Test	Action	Expected result	Observed result	Explanation
1	Publish room1/temp (QoS 0)	Received by subscriber		
2	Publish room2/temp	Received (matches +)		
3	Publish room1/humidity	Not received		
4	Publish with QoS 1	Received; broker acknowledges		
5	Publish with Retain, reconnect subscriber	Retained message delivered on subscribe		

Assignment 2, Part 2: Edge-or-cloud design report. Choose one scenario: (1) a smart-retail checkout camera, (2) a wearable heart monitor, (3) predictive maintenance for 50 factory machines, (4) smart irrigation for a 200-acre farm, or (5) a delivery-truck fleet. Then:

- build a Device | Edge | Cloud table with at least five functions;
- justify the edge functions using at least two of the four edge objectives;
- perform a **bandwidth check** with and without edge filtering, showing the arithmetic as in Worked Example 4.1;
- choose one cloud platform and name its edge tool, then identify one security or privacy risk and how the design reduces it.

References and further reading

Amazon Web Services. (n.d.-a). *How AWS IoT works*. AWS IoT Core Developer Guide. https://docs.aws.amazon.com/iot/latest/developerguide/aws-iot-how-it-works.html

Amazon Web Services. (n.d.-b). *AWS IoT Device Shadow service*. AWS IoT Core Developer Guide. https://docs.aws.amazon.com/iot/latest/developerguide/iot-device-shadows.html

Amazon Web Services. (n.d.-c). *AWS IoT Analytics end of support*. AWS IoT Analytics User Guide. https://docs.aws.amazon.com/iotanalytics/latest/userguide/iotanalytics-end-of-support.html

Bigelow, S. J. (2021, December 8). *What is edge computing? Everything you need to know*. TechTarget. https://www.techtarget.com/it-infrastructure/definition/What-is-edge-computing-Everything-you-need-to-know

CDEBYTE. (2023). *Why use MQTT instead of HTTP in IoT applications?* [Video]. YouTube. https://www.youtube.com/watch?v=bmBN_pP5rWU

CodeLucky. (2025). *IoT database systems: Solutions for the IoT* [Video]. YouTube. https://www.youtube.com/watch?v=qTwedxwLDvA

CodeLucky. (2026). *AWS IoT Core explained* [Video]. YouTube. https://www.youtube.com/watch?v=yJ-9uQ-p48Q

Google Cloud Tech. (2020). *What is a data center?* [Video]. YouTube. https://www.youtube.com/watch?v=Amow8BJm5Go

HiveMQ Team. (2026a, February 9). *MQTT publish/subscribe architecture (pub/sub): MQTT Essentials, part 2*. HiveMQ. https://www.hivemq.com/blog/mqtt-essentials-part2-publish-subscribe/

HiveMQ Team. (2026b, February 9). *MQTT topics, wildcards, & best practices: MQTT Essentials, part 5*. HiveMQ. https://www.hivemq.com/blog/mqtt-essentials-part-5-mqtt-topics-best-practices/

NxEco. (n.d.-a). *NxEco smart watering* [Website]. https://www.nxeco.com

NxEco. (n.d.-b). *Smart controller specification* (Ver. 10.1) [PDF]. https://www.nxeco.com/Public/public/file/manual/Nxeco_Smart_Control_Spec_Ver10.1.pdf

NxEco. (n.d.-c). *NxEco smart weather-based irrigation controller: App manual* (Ver. 10.2) [PDF]. https://www.nxeco.com/Public/public/file/manual/NxEco_Smart_Manual_Ver10.2.pdf

NxEco. (n.d.-f). *Features* [Web page]. https://www.nxeco.com/features.html

NxEco. (2026). *MySmartDrop smart watering portal* [Web application]. Retrieved October 22, 2026, from https://www.mysmartdrop.com

Projectcubicle. (2025). *Edge computing vs cloud: Why you're probably using the wrong one!* [Video]. YouTube. https://www.youtube.com/watch?v=QrkT91wz8Vo

SaM Solutions. (2025). *10 best IoT platforms for 2025* [Video]. YouTube. https://www.youtube.com/watch?v=xfnsEz4gPsg

Simplilearn. (2020). *Cloud computing in 6 minutes* [Video]. YouTube. https://www.youtube.com/watch?v=M988_fsOSWo

Smart Home 101. (2022). *What is Zigbee and how it works | Zigbee network explained* [Video]. YouTube. https://www.youtube.com/watch?v=pFDkibWy0yg

Tech Tutor Hub. (2024). *IoT architecture: Device layer, communication layer, cloud layer, application layer* [Video]. YouTube. https://www.youtube.com/watch?v=pXu6qq3ATJM

CHAPTER 5

IoT Firmware and Software: From the Microcontroller to the Cloud

Learning Objectives

After studying this chapter, students will be able to:

1. Describe the standard IoT workflow and the four-layer firmware and software architecture.
2. Explain eight characteristics that make IoT software different from ordinary PC software.
3. Define firmware and explain its layered structure: hardware abstraction layer, real-time operating system, application logic, and connectivity stack.
4. Trace the firmware program flow of initialization, main loop, interrupts, and sleep, and explain the role of a watchdog.
5. Identify the ten key characteristics of reliable firmware and apply defensive techniques such as validation, retry, and fail-safe behavior.
6. Describe firmware development tools and procedures, including safe over-the-air updates.
7. Describe edge, gateway, and cloud software environments and the data-centric software stack.
8. Evaluate the business and safety cost of firmware failures.

Where this fits: Week 5. This chapter supports Collaboration 5 *Build, Break, and Harden: Making IoT Firmware Reliable* (Collaboration Report 2). The Week 5 midterm covers Chapters 1 to 4.

Opening case: the car that would not start

A visitor arrived at a friend's home in a brand-new electric SUV. The next morning, the vehicle would not start. The hardware was fine; the battery was charged. The problem was software: a firmware crash, apparently triggered when the new owner performed an unexpected sequence of operations that the developers had not tested. A similar failure appears in consumer products such as smart irrigation controllers, which occasionally freeze when they receive several commands at once from the app, the buttons, and the schedule.

These stories share a cause: firmware that was not built to handle the unexpected. Modern products, from vehicles to thermostats, are defined as much by their code as by their hardware. A major challenge in firmware design is keeping the system stable when many real-time events and commands arrive at the same time. This chapter explains what firmware is, how it is structured, and how engineers build, test, and update it safely.

5.1 The standard workflow of an IoT system

Software runs at every stage of an IoT system. A standard workflow has four stages:

1. **Data acquisition and processing (the edge).** Sensors convert environmental parameters into electrical signals. The MCU digitizes them, filters noise, and prepares only relevant data for transmission.
2. **Connectivity and transport (the network).** A radio link (Bluetooth, Wi-Fi, NB-IoT, or another technology) carries data off the device, and the internet routes encrypted packets to remote servers.
3. **Orchestration and intelligence (the cloud).** Centralized infrastructure manages millions of concurrent device connections, stores data, maintains device shadows, and runs analytics.
4. **Analysis and application (value generation).** Dashboards, apps, alerts, and business systems turn data into decisions.

5.2 The four-layer firmware and software architecture

An IoT software system is best described as a **distributed digital nervous system**: a multi-layered suite of code that lets inanimate objects sense, think, and act in real time. The four-layer architecture from Chapter 1 also describes its software:

- **Perception layer: firmware.** Low-level code on the device's microcontroller manages the hardware, reading sensors and controlling actuators such as motors and valves.
- **Network layer: connectivity software.** This software manages the handshake between the device and the outside world. It runs protocols such as BLE, Zigbee, MQTT, or cellular stacks, packaging data securely and efficiently without draining the battery. IoT engineers must be able to embed these stacks on the MCU of end devices or gateways and connect them to cloud servers.
- **Middleware layer: cloud platform software.** The "brain" of the system ingests streams from millions of devices, stores them, runs analytics, and manages device twins. This software is usually provided by a platform vendor; IoT customers must know how to configure, initialize, and deploy its resources.
- **Application layer: user interface software.** Mobile apps (Android, iOS) and web dashboards translate complex sensor data into simple charts, alerts, and control buttons.

5.3 What makes IoT software different

IoT software is fundamentally different from traditional software because it must operate across heterogeneous devices, unreliable networks, and real-time environments. It transforms a "dumb" piece of hardware into a smart, connected asset that makes autonomous decisions. Unlike traditional software that lives on a single computer, it is spread across three environments: the physical device, the network gateway, and the cloud. Eight characteristics define it (Table 5.1).

Table 5.1 Eight characteristics of IoT software

Characteristic	What it means for software
1. Real-time and event-driven	Reacts to sensor events (temperature change, motion) with minimal and bounded delay; timing is critical for safety and control
2. Tight hardware integration	Interacts directly with sensors, actuators, MCUs, and radios; must handle hardware faults and noise gracefully
3. Heterogeneous connectivity	Supports diverse protocols (BLE, Zigbee, Wi-Fi, LoRaWAN, 5G) and copes with intermittent links and variable bandwidth
4. Distributed architecture	Spans device firmware, edge nodes, and cloud platforms that must coordinate
5. Power-aware and resource-constrained	Minimizes CPU cycles, memory, and radio transmissions so batteries last months or years
6. Security and privacy built in	Provides authentication, encryption, secure boot, secure updates, and data protection from the start, not as an afterthought
7. Remote management and OTA updates	Supports remote configuration, diagnostics, monitoring, and firmware updates for devices that are hard to reach
8. Data-centric and analytics-driven	Filters and preprocesses data at the edge, feeds cloud analytics and machine learning, and drives real-time dashboards

Two more properties apply to the system as a whole: **scalability and resilience**. A cloud back end must survive the "thundering herd," the moment when millions of devices reconnect at once after a power cut or a broker restart. Well-designed device firmware adds a random delay ("jitter") before reconnecting so that the herd arrives as a gentle stream.

5.4 What is firmware?

Firmware is a class of software that provides low-level control of a device's specific hardware. It is the bridge between physical hardware and higher-level software, programmed directly into the device's non-volatile memory (Awati & Lutkevich, 2024). Four properties distinguish it:

- **Persistence.** Firmware is stored in flash memory, EEPROM, or ROM, so it remains when power is removed.

- **Direct control.** It reads and writes MCU registers to toggle pins, read sensors, and manage power.
- **Monolithic nature.** Unlike a PC, which runs an operating system and many separate applications, a typical IoT device runs a single, cohesive firmware **image** that handles everything from booting to network connections.
- **The device's "soul."** Without firmware, the microcontroller is silent silicon. With it, the device senses, decides, and acts.

Firmware runs on two main categories of IoT hardware: **end devices** (sensors, actuators, MCUs, and wireless modules) and **gateways** (Wi-Fi routers, hubs, and other edge-processing devices). Gateways usually run a full operating system such as Linux, with firmware for the boot process and radios, while end devices typically run firmware alone.

Table 5.2 Firmware, operating systems, and applications

Aspect	Firmware	Operating system	Application software
Purpose	Controls specific hardware	Manages resources for many programs	Performs user tasks
Where stored	Flash/ROM on the device	Disk or flash	Disk or flash
How often changed	Rarely; through careful updates	Periodic updates	Frequently
Example	ESP32 sensor-node image; router firmware	Linux on a gateway, Android on a phone	Mobile app, dashboard

5.5 The layered firmware architecture

Although a firmware image is a single file, well-designed firmware is organized internally into layers (Figure 5.1).

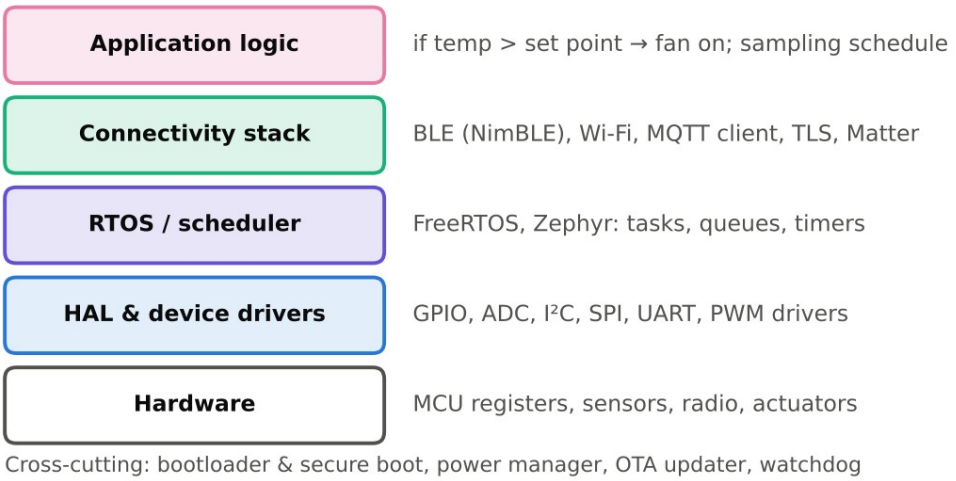


Figure 5.1 Layered firmware architecture.

1. **Hardware abstraction layer (HAL) and drivers.** Standardizes how the code talks to different chips and peripherals (GPIO, ADC, I^2C, SPI, UART, PWM). If the product moves to a different MCU, only this layer should need major changes.
2. **Real-time operating system (RTOS).** Manages concurrent tasks, such as reading sensors, sending radio data, and updating a display, with predictable timing. FreeRTOS and Zephyr are widely used.
3. **Application logic.** The decision-making code specific to the product, for example "if temperature > 50 °C, turn on the fan."
4. **Connectivity stack.** Protocol libraries for Wi-Fi, Bluetooth (including recent versions such as Bluetooth 6.0), Thread, Matter, MQTT, and TLS.

Cross-cutting components include the **bootloader** and **secure boot** (which verify that only authorized firmware runs), a **power manager**, an **OTA updater**, and a **watchdog**.

Major technical challenges

- **Memory footprint.** Many MCUs have a few hundred kilobytes of RAM or less; every byte counts. Developers avoid dynamic memory allocation where possible because fragmentation can cause crashes after days of operation.
- **Intelligent power management.** "Race to sleep" code performs tasks quickly and then drops the processor into a sleep mode that draws only microamps or nanoamps.
- **Security and root of trust.** Cryptographic secure boot prevents execution of unauthorized or malicious firmware; keys must be protected in hardware.

Bare-metal loops and real-time operating systems

Simple firmware uses a **bare-metal super-loop**: a single infinite loop that checks each task in turn. It is easy to understand but hard to extend; if one task takes too long, every other task waits.

An **RTOS** divides the work into **tasks** (threads), each with a **priority**. A **scheduler** always runs the highest-priority task that is ready, preempting lower-priority work. Tasks communicate through **queues**, coordinate with **semaphores** and **mutexes**, and wait on **timers** without wasting CPU time (Susnjara & Smalley, 2025). The firmware of the class demo sensor node, for example, runs three FreeRTOS tasks on the ESP32's two cores (Table 5.3).

Table 5.3 FreeRTOS tasks in the demo sensor node (Li, 2026)

Task	Core	Priority	Stack (bytes)	Role
vTaskBleServer	0	5 (highest)	4,096	BLE advertising and handling GATT writes (actuation commands)
vTaskSensorSample	1	3	3,072	I^2C burst reads and sensor compensation math
vTaskPowerManager	1	2	2,048	Scheduling the RTC wake-up and entering deep sleep

Giving the radio task the highest priority ensures that an incoming "turn off the relay" command is handled promptly even while sensors are being read.

Worked Example 5.1: Budgeting flash memory for safe updates

An ESP32 module has 4 MB (4,096 KB) of flash. The firmware image is about 1,200 KB today and is expected to grow by 25 percent over the product's life. The design needs a bootloader (about 32 KB), a partition table (4 KB), non-volatile storage for settings and certificates (about 24 KB), an OTA data partition (8 KB), and **two** application slots so that an update can be downloaded while the old image keeps running (Section 5.9). Does it fit?

Future image size = 1,200 KB × 1.25 = 1,500 KB per slot.

Two slots = 3,000 KB.

Total = 32 + 4 + 24 + 8 + 3,000 = **3,068 KB**, leaving about 1,028 KB for logs or a file system.

Interpretation. The design fits, but only because the engineers planned for growth. A device with 2 MB of flash could not hold two 1,500 KB slots, and would have to use a riskier single-slot update scheme or a cheaper but less capable update method.

5.6 The firmware program flow

Almost all firmware follows the same program flow (Figure 5.2):

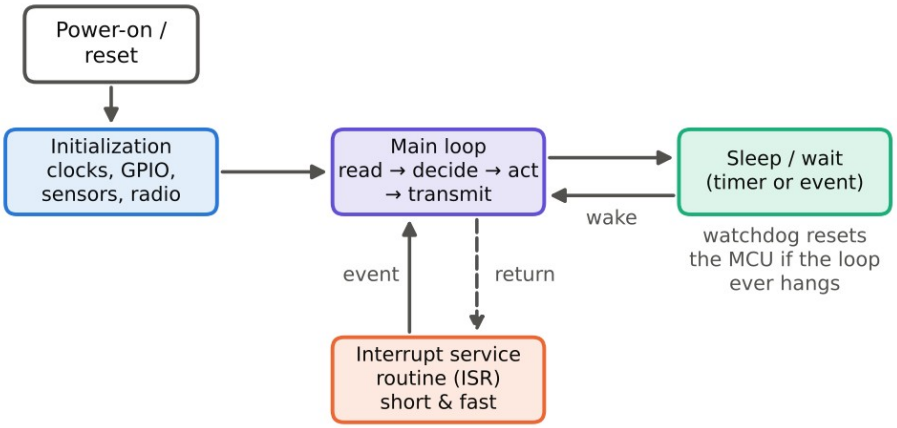


Figure 5.2 Firmware program flow: initialization, main loop, interrupts, and sleep, protected by a watchdog.

1. **Power-on or reset.** The MCU starts executing the bootloader, which verifies and launches the application image.
2. **Initialization (startup routines).** Configure clocks, GPIO directions, the ADC, serial buses, sensors, and the radio; load settings from non-volatile storage.
3. **Main loop (runs forever).** Read inputs from sensors, process data using programmed logic, control outputs (motors, LEDs, displays), and handle communication (UART, SPI, I^2C, and the radio).
4. **Sleep or wait.** When there is nothing to do, enter a low-power mode until a timer or event wakes the MCU.
5. **Interrupts.** When a hardware event occurs (a button press, a sensor's data-ready signal, a received radio packet, or a timer expiry), the MCU pauses the main loop and runs an **interrupt service routine (ISR)**. ISRs must be short and fast: set a flag or copy data to a buffer, then return, and let the main loop or a task do the heavy work.

A **watchdog timer** guards the whole flow. The firmware must "kick" (reset) the watchdog regularly; if the loop ever hangs, the watchdog expires and resets the MCU, restoring service automatically.

```
void setup() {                  // 2. Initialization
  initClocksAndGpio();
  initSensors();
  initRadio();
  enableWatchdog(8000);         // reset if not kicked within 8 s
}
```

```
void loop() {                    // 3. Main loop
  if (dataReadyFlag) {           // set by an ISR (5. Interrupts)
    dataReadyFlag = false;
    Reading r = readSensors();
    decideAndAct(r);
    transmit(r);
  }
  kickWatchdog();
  enterLightSleepUntilEvent(); // 4. Sleep or wait
}
```

5.7 Ten key characteristics of IoT firmware

Reliable firmware is *reliable, efficient, responsive, and predictable*. Table 5.4 lists ten characteristics that together achieve these goals.

Table 5.4 Ten key characteristics of IoT firmware

#	Characteristic	Description
1	Real-time responsiveness	Responds quickly to inputs and interrupts, ensuring timely control of hardware
2	Reliability and stability	Runs continuously for long periods; handles errors and unexpected conditions gracefully
3	Efficiency	Uses minimal CPU, memory, and energy to maximize performance and lifespan
4	Hardware control capability	Interfaces directly with GPIO, ADC, UART, SPI, I^2C; controls sensors and actuators precisely
5	Interrupt handling	Uses ISRs for fast response to time-critical events
6	Deterministic behavior	Produces predictable, repeatable timing, critical for control applications
7	Low-power operation	Uses sleep modes and power management to extend battery life
8	Modularity and maintainability	Organized into clear modules that are easier to update, debug, and extend
9	Security	Protects firmware and data with secure boot, encryption, and access control
10	Upgradability	Supports wired or OTA updates for improvements and fixes after deployment

5.8 Building firmware: tools and procedure

Development tools

Firmware development needs both hardware and software tools.

Development hardware

- **Development boards** carrying the target MCU, such as Arduino boards, STM32 Nucleo or "Blue Pill" boards, ESP32 DevKits, and Nordic nRF52840 DK.
- **Programmers and debuggers** that load firmware into flash and let engineers pause, step through, and inspect running code, using **JTAG** or **SWD** interfaces (for example ST-LINK, J-Link) or simple USB and SPI programmers (for example a USBasp with the Progisp tool for AVR chips).
- **Peripheral devices** such as sensors, actuators, displays, logic analyzers, and power monitors.

Software environment

- **Integrated development environments (IDEs):** Arduino IDE, STM32CubeIDE, Keil MDK, Microchip Studio (formerly Atmel Studio), and Visual Studio Code with PlatformIO or vendor extensions.
- **Compilers and toolchains:** GCC for Arm or Xtensa and the Arm Compiler convert C or C++ source code into machine instructions.
- **Software development kits (SDKs):** libraries and drivers from the chip vendor, such as ESP-IDF, the STM32 HAL and Cube libraries, and the nRF Connect SDK (based on Zephyr).
- **Simulators:** tools such as **Wokwi** simulate MCUs, sensors, and actuators in a browser, letting students and engineers test logic before touching hardware (Wokwi, n.d.).

Firmware development procedure

A disciplined procedure reduces the risk of field failures:

1. **Define requirements:** functions, timing, power budget, security, and update needs.
2. **Select hardware** and confirm the MCU's memory and peripherals fit (Worked Example 5.1).
3. **Set up the environment:** IDE, toolchain, SDK, version control (Git).
4. **Develop bottom-up:** bring up the HAL and drivers, then the RTOS tasks, then application logic and the connectivity stack.
5. **Build and flash** the image to the target board with a programmer.
6. **Debug** with breakpoints, serial logging, and logic analyzers.
7. **Test** at several levels: unit tests for logic, hardware-in-the-loop tests with real sensors, stress tests (many commands at once, power interruptions, radio loss), and field trials.
8. **Release:** assign a version number, sign the image cryptographically, and archive the build.

9. **Maintain:** monitor devices in the field and deliver fixes through staged OTA updates.

5.9 Reliability, testing, and over-the-air updates

Defensive firmware

Field conditions are hostile: wires corrode, connectors loosen, voltages sag, radios lose signal, and users press buttons in unexpected orders. Defensive firmware assumes things will go wrong:

- **Validate every input.** A disconnected DHT22 returns "not a number" (NaN). Firmware must check with `isnan()` and range limits before acting on a value.
- **Retry with limits.** Retry a failed read a small, fixed number of times with a delay, then give up and report an error.
- **Fail safe.** When data is invalid, put actuators in the state that causes the least harm. For a heater, that is OFF; for a freezer alarm, it may be ON.
- **Use timeouts.** Never wait forever for a sensor, network, or user.
- **Protect against brownouts and resets.** Persist important state in non-volatile memory and resume correctly after a reset.
- **Use the watchdog** to recover from hangs.
- **Log errors** with counters and timestamps so problems can be diagnosed remotely.

Worked Example 5.2: Retry timing for a slow sensor

A firmware loop reads a DHT22 every 2 seconds. The DHT22 can be read at most once every 2 seconds (0.5 Hz). The engineer proposes "retry up to 3 times" when a read fails. What is the worst-case delay, and is the design acceptable for a greenhouse fan?

Each retry must wait at least 2 s, so 3 retries add up to 3 × 2 s = 6 s, making the worst-case decision delay about 8 s from the last good reading.

For a greenhouse, where temperature changes over minutes, 8 seconds is acceptable. For a motor-overcurrent protection loop, it would not be; that design would need a faster sensor and an immediate fail-safe action instead of retries. *Retry policies must match the speed of the process being controlled.*

Worked Example 5.3: What does a firmware bug cost?

A company has sold 50,000 smart controllers. A firmware bug causes 4 percent of them to freeze each month, generating support calls. Compare (a) sending technicians to reflash affected units at US$80 per visit with (b) fixing the bug with an OTA update.

(a) Affected units per month = 50,000 × 0.04 = 2,000. Monthly cost of visits = 2,000 × US$80 = **US$160,000**, and the cost recurs until every unit is fixed by hand. Fixing all 50,000 units would cost US$4,000,000.

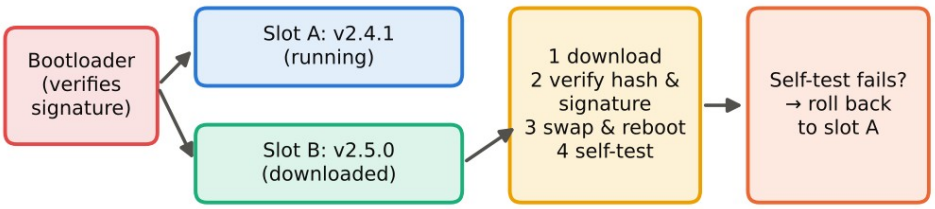
(b) OTA update: the main costs are engineering time to fix and test the bug (perhaps a few weeks of developer time) and the cloud bandwidth to deliver the image (Worked Example 5.4). The cost is likely to be a small fraction of option (a).

Beyond money, frozen controllers may flood a garden or let crops die, damaging **customer trust** and the brand. For safety-critical devices such as vehicles or medical equipment, a firmware failure can cause injury and trigger regulatory recalls. *Upgradability is a business requirement, not just a technical feature.*

Safe over-the-air updates

OTA updates are powerful and dangerous: a faulty update can "brick" thousands of devices at once. Safe OTA design uses several techniques (Prylada, 2024):

- **A/B (dual-bank) partitions.** The new image downloads into the inactive slot while the old image keeps running. After verification, the bootloader switches slots. If the new image fails its self-test, the device **rolls back** to the known-good image (Figure 5.3).
- **Cryptographic signing.** The bootloader accepts only images signed with the manufacturer's private key, so attackers cannot install malicious firmware.
- **Integrity checks.** A hash confirms that the image was not corrupted in transit.
- **Delta updates.** Sending only the differences between versions reduces download size for slow or metered links.
- **Staged rollouts.** Update 1 percent of the fleet (a "canary" group), monitor for problems, then expand to 10 percent and finally 100 percent.
- **Power and battery checks.** Do not start an update when the battery is low or power is unstable.



A/B (dual-bank) OTA keeps a known-good image so a failed update never bricks the device.

Figure 5.3 A/B over-the-air update with automatic rollback.

Worked Example 5.4: How long does an OTA update take, and how much data?

(a) A 1.2 MB firmware image is sent to a sensor over BLE at an effective application throughput of 50 KB/s. Download time = 1,200 KB ÷ 50 KB/s = **24 s**, plus verification

and reboot time. During this time the radio is fully active, so the energy cost is significant for a coin-cell device.

(b) The same image is delivered to a fleet of 10,000 gateways. Total data = 10,000 × 1.2 MB = **12 GB**. With a 1 percent canary stage first, only 120 MB is sent before the team confirms the update is safe.

5.10 Edge and gateway software

IoT networking software runs on three main infrastructure components:

1. **WPAN access firmware** on end devices (for example a BLE or Zigbee stack);
2. **edge gateway software** that bridges WPAN devices to IP networks;
3. **cloud IoT agents** that connect gateways to cloud platforms.

Development environments for edge and gateway software

Table 5.5 Edge and gateway software environments

Category	Common choices
Operating system	Linux (most common: Raspberry Pi OS, Ubuntu, Debian, Yocto-based builds)
IDEs and editors	Visual Studio Code, Eclipse IDE, JetBrains PyCharm
Middleware and edge frameworks	Node-RED (visual flow-based wiring of devices and services), Eclipse Kura, AWS IoT Greengrass, Azure IoT Edge
Programming languages	C/C++ (performance and hardware control), Python (rapid development and AI), Java (enterprise systems), JavaScript/Node.js (lightweight services)
Protocol and test tools	MQTT Explorer, Mosquitto command-line clients, HTTP/REST clients, WebSocket tools

A gateway program is often surprisingly short. The class demo gateway, written in Python, uses the **bleak** library to receive BLE data and an MQTT client library to publish it securely (Chapter 7 shows the full flow):

```
# simplified gateway loop
async def on_notify(_, data: bytearray):
    reading = parse_packet(data)              # binary -> dict
    payload = json.dumps(reading)
    if wan_up():
        mqtt.publish(f"iot/telemetry/{reading['device_id']}", payload,
qos=1)
    else:
        buffer_db.insert(payload)             # store-and-forward
```

5.11 The data-centric edge and cloud software stack

IoT systems generate continuous data, so their software stack is organized around data: efficient collection, processing, storage, analysis, and visualization. Edge software handles local analytics and filtering to reduce latency and bandwidth; cloud software provides scalable storage, advanced analytics, and machine learning (Table 5.6).

Table 5.6 The data-centric IoT software stack

Stage	Example technologies
Communication protocols	MQTT 5.0 for telemetry; gRPC for fast service-to-service calls; DDS for real-time industrial systems
Stream processing	Apache Flink, Apache Spark Structured Streaming, Amazon Kinesis
Storage	Time-series: InfluxDB, TimescaleDB; documents and metadata: MongoDB; object storage and data lakes: Amazon S3 with Apache Iceberg tables
Processing and query	Amazon EMR, AWS Glue, Amazon Athena, Amazon Redshift
Visualization	Grafana, Tableau, Amazon QuickSight
AI training and deployment	PyTorch, Hugging Face models, Amazon SageMaker; edge deployment through Greengrass components or LiteRT (Chapter 8)

5.12 IoT application case: NxEco firmware and software

NxEco's software spans all four layers of the firmware and software architecture.

Controller firmware (perception layer). The firmware reads the dial and buttons, drives the LCD and LEDs, keeps time, stores up to four schedule programs (A to D) with up to four start times per day for each zone, runs zones **sequentially** from lowest to highest so that only one valve is open at a time, applies the daily weather adjustment and rain delay, and saves everything to flash so that a power cut does not erase the schedule. Firmware version 2.4.2 was announced for the HWN12-200 and HWN12-200+ (NxEco, n.d.-a). Sequential zone operation is a deliberate firmware rule: opening several valves at once would drop water pressure and could overload the 24 V AC transformer.

Connectivity software (network layer). The firmware includes the Wi-Fi stack with WPA2, the button-driven provisioning procedure, and the client that registers the controller with the cloud using its serial number and a one-time password shown on the screen. Unregistering produces a new serial password, a simple defense against someone re-registering a second-hand controller to their own account.

Cloud software (middleware layer). Server software stores schedules in the cloud, calculates daily budgets, keeps 30-day and 12-month usage trends, and sends daily and weekly email summaries (NxEco, n.d.-c).

Application software. The NxEco app runs on iOS and Android and is available in English, Simplified Chinese, and Traditional Chinese. The iOS listing showed version 2.2.4, last updated in September 2020 (NxEco, Inc., 2020), while the website referred to Android version 2.2.6 and offered a downloadable Android installation file for version 2.2.5.

The NxEco mobile app as an IoT application

The app is the homeowner's main window on the system. From a software point of view it is a **thin client**: it holds almost no logic of its own, but sends requests to the NxEco cloud, which stores the schedules and relays commands to the controller. This is why the app, the MySmartDrop web portal, and voice assistants show the same programs and the same water records: they are three front ends over one cloud state (Section 4.12). It is also why the controller keeps watering if the app stops working.

The app's main functions map onto the layers of this book:

Table 5.7 NxEco app functions mapped to IoT layers (NxEco, n.d.-c; NxEco, Inc., 2020)

App function	What it does	Layer and chapter
Connect to WiFi	Sends the home network name and password to the controller during button-triggered setup	Network layer, provisioning (Ch. 3)
Register controller	Links the controller's serial number and one-time password to the user's account	Device registry (Ch. 4, 5)
Programs A to D, copy zone schedule	Edits weekday or day-of-month schedules and run times; the cloud pushes them to the controller's flash	Application to device, downlink (Ch. 4, 5)
Smart watering, ET adjustment, fine tune	Turns weather-based adjustment on or off and trims it by a percentage	Cloud rules engine (Ch. 2, 4)
Manual watering, rain delay	Runs a zone now, or delays watering up to 120 hours, or indefinitely	Remote actuation (Ch. 2, 4)
Water reports	Shows 30-day and 12-month trends of watering minutes	Analytics and presentation (Ch. 4, 9)
Valet Key	Gives a gardener limited control without the owner's password	Access control, least privilege (Ch. 6)
Alexa and Google Home	Links the account to a voice assistant through the cloud	Ecosystem integration (Ch. 3, 9)
Online status icon	Shows whether the controller is connected to the cloud	Device state, monitoring (Ch. 4)

What the version history teaches. An app store's version history is a public record of a product's software life cycle. The NxEco iOS history shows a typical pattern (NxEco, Inc., 2020):

- **2016, version 1.7.4:** added support for the United Kingdom, Austria, Argentina, and Switzerland, a reminder that weather services, time zones, and units must be localized for each market.
- **2016 to 2018, versions 2.0.x:** support for second-generation controllers and new models, a copy-zone-schedule function, and ET-sensitive watering. The app had to support **old and new hardware generations at the same time**.
- **2019, versions 2.1.x:** fixes for input boxes and for reading the Wi-Fi network name, notices for water-restricted days, and a maximum rain delay extended to 120 hours **only on controllers with firmware V2.3.3 or later**. App features depend on the firmware version in the field, so the app must detect it and adapt.
- **2020, versions 2.2.1 to 2.2.4:** an "infinite delay" feature, interface improvements, and bug fixes. Version 2.2.4 (September 2020) was still the latest iOS release in October 2026.

Software health indicators. The same listing gives several signals that an IoT buyer or student can check for any product:

Table 5.8 Software health indicators of the NxEco app (NxEco, Inc., 2020)

Indicator	NxEco app (October 2026)	Why it matters
Time since last update	About six years	Phones and operating systems change every year; an unmaintained app may stop working or miss security fixes
Minimum operating system	iOS 8.0 (released 2014)	A very old target suggests old libraries for networking and encryption
Privacy label	Not provided	Users cannot see what data is collected (Ch. 6)
Accessibility declaration	Not provided	Users with disabilities cannot tell whether the app supports them
Ratings	4.8 from 5 ratings	Too few ratings to judge reliability
Android availability	Not found in a U.S. Google Play search; an installation file was offered on the vendor's website	Installing apps from outside the official store bypasses the store's malware checks and automatic updates

The controller's design softens the risk. Because schedules live in the controller's flash memory and in the cloud, the garden keeps watering even if the app fails, and the MySmartDrop web portal (Section 4.12) offers a second way in that does not depend on a phone operating system. This is a good illustration of the **local survivability** objective from Chapter 4 applied to software as well as networks.

Discussion: the software life cycle of a consumer IoT product. The NxEco documentation describes how to see the firmware version but not how firmware updates are

delivered. Compare this with Section 5.9: how would a company deliver a security fix to thousands of installed controllers, how would it avoid "bricking" a controller during an update, and what happens to customers if the mobile app is no longer updated for new phone operating systems? The firmware freeze described in this chapter's opening case, triggered by several commands at once, is exactly the kind of fault that safe OTA updates and stress testing are designed to catch.

Chapter summary

- IoT software spans four stages (edge acquisition, network transport, cloud orchestration, application value) and four layers (firmware, connectivity, cloud middleware, user interface).
- IoT software is real-time, hardware-integrated, heterogeneous, distributed, power-aware, secure by design, remotely managed, and data-centric; at scale it must also be resilient to the "thundering herd."
- **Firmware** is persistent, low-level code in non-volatile memory that directly controls hardware. It is organized into a HAL, an RTOS, application logic, and a connectivity stack, plus a bootloader, secure boot, power manager, OTA updater, and watchdog.
- The program flow is **initialization → main loop → sleep**, with short **ISRs** for events and a **watchdog** for recovery.
- Ten characteristics define good firmware, from real-time responsiveness to upgradability.
- Development uses boards, debuggers, IDEs, toolchains, SDKs, and simulators, and follows a disciplined procedure from requirements to maintenance.
- Defensive firmware validates inputs, retries with limits, fails safe, and logs errors. Safe OTA uses A/B partitions, signing, integrity checks, staged rollouts, and rollback.
- Gateways run Linux-based software (Node-RED, Greengrass, Python); the cloud stack is organized around streaming, storage, analytics, visualization, and AI.

Key terms

A/B partition; bare-metal super-loop; bootloader; brownout; canary rollout; delta update; deterministic behavior; EEPROM; fail-safe; firmware; firmware image; flash memory; FreeRTOS; hardware abstraction layer (HAL); IDE; interrupt service routine (ISR); JTAG; mutex; Node-RED; OTA update; preemption; priority; queue; race to sleep; real-time operating system (RTOS); rollback; scheduler; SDK; secure boot; semaphore; SWD; task; thundering herd; toolchain; watchdog timer; Zephyr.

Review questions

Multiple choice

1. Which layer of the IoT software architecture reads sensors and controls actuators directly?
    a. Application
    b. Middleware
    c. Network
    d. Perception (firmware)
2. Firmware is stored in:
    a. volatile RAM
    b. non-volatile memory such as flash
    c. the cloud only
    d. the processor's cache
3. Which firmware layer standardizes access to GPIO, I^2C, and SPI across different chips?
    a. Application logic
    b. Connectivity stack
    c. Hardware abstraction layer
    d. Bootloader
4. An RTOS scheduler always runs:
    a. the oldest task
    b. the highest-priority ready task
    c. the task with the largest stack
    d. tasks in alphabetical order
5. What should an interrupt service routine do?
    a. Perform long calculations
    b. Wait for network responses
    c. Run briefly, set a flag or copy data, and return
    d. Print large logs
6. What happens if firmware stops "kicking" the watchdog timer?
    a. The radio turns off
    b. The MCU resets
    c. The battery charges
    d. The OTA update starts
7. Which technique lets a device roll back if a new firmware image fails?
    a. Delta updates

b. A/B (dual-bank) partitions
c. QoS 2
d. Data filtering

8. A disconnected DHT22 causes the reading NaN. Which C/C++ function detects it?
   a. `isnan()`
   b. `abs()`
   c. `delay()`
   d. `digitalWrite()`
9. Which tool lets engineers wire IoT devices and services visually on a gateway?
   a. Keil MDK
   b. Node-RED
   c. JTAG
   d. Progisp
10. Updating 1 percent of a fleet first, monitoring, and then expanding is called:
    a. fleet learning b. a canary or staged rollout c. store-and-forward d. a thundering herd

Short answer

11. Explain three ways in which IoT software differs from ordinary PC software.
12. Describe the four layers of firmware architecture and give one example of what belongs in each.
13. Explain why ISRs should be short, and what happens to the heavy work.
14. Explain the business and safety reasons why upgradability is essential for IoT devices.

Applied problems

15. A device has 2 MB (2,048 KB) of flash. It needs a 32 KB bootloader, a 4 KB partition table, 24 KB of settings storage, and an 8 KB OTA data partition. (a) What is the largest application image that fits if the design uses two A/B slots of equal size? (b) If today's image is 850 KB, what percentage growth can the design absorb?
16. A soil probe reads its sensor every 15 minutes. Write pseudocode for a hardened read that validates the reading (valid range 0 to 100 percent), retries up to two times with a 1-second delay, and fails safe (keeps the valve closed and reports an error) if all attempts fail. Explain why "valve closed" is the safe state.
17. **NxEco case.** Why does NxEco's firmware run zones sequentially rather than simultaneously, and which firmware characteristic from Table 5.4 does this rule support?
18. **NxEco app.** Using Tables 5.7 and 5.8, (a) explain why the app is called a thin client and what keeps the garden watering if the app stops working; (b) explain why the

120-hour rain delay worked only on controllers with firmware V2.3.3 or later, and what the app must do to handle this; (c) name two software health indicators you would check before recommending an IoT product to a client.

Lab 5: Build, break, and harden

Collaboration 5 (Collaboration Report 2). Students run IoT firmware in a simulator, deliberately cause a failure, harden the code, and reflect on the business impact.

Procedure.

1. **Build.** In Wokwi, start a new ESP32 project (or open the Lab 2 project) with a DHT22 and an LED. Modify the firmware so that it (1) initializes the sensor and LED in `setup()`, (2) reads the sensor every 2 seconds in the main loop, and (3) turns the LED on when the temperature is above a chosen threshold. Print each reading to the Serial Monitor. Save the public link and a screenshot.
2. **Break.** In the diagram editor, delete the wire between the DHT22's data pin and the ESP32, then run again. Record what the firmware does. A naïve version prints `nan` and may make a wrong decision (for example, comparisons with NaN are always false, so the LED never turns on even if the room is hot).
3. **Harden.** Modify the firmware to detect invalid readings, retry, and fail safe:

```
float readTempWithRetry(int attempts) {
  for (int i = 0; i < attempts; i++) {
    float t = dht.readTemperature();
    if (!isnan(t) && t > -40 && t < 80) return t;   // valid range check
    Serial.printf("Read failed (attempt %d)\n", i + 1);
    delay(2000);                                    // DHT22: 0.5 Hz max
  }
  return NAN;                                       // all attempts failed
}

void loop() {
  float t = readTempWithRetry(3);
  if (isnan(t)) {
    digitalWrite(LED_PIN, LOW);                     // fail safe: actuator
OFF
    Serial.println("ERROR: sensor fault, actuator held OFF");
    errorCount++;
  } else {
    digitalWrite(LED_PIN, t > THRESHOLD ? HIGH : LOW);
    Serial.printf("T=%.1f C\n", t);
  }
  delay(2000);
}
```

4. **Retest.** Run with the wire still disconnected, record the new behavior, then reconnect the wire and confirm normal operation resumes.
5. **Analyze.**

- Map the code to the firmware program flow: initialization, main loop, and (if used) interrupts or sleep.
- Identify which firmware characteristics the fix improves (for example reliability and stability, deterministic behavior, hardware control).
- Explain how testing helps a company prevent or recover from a firmware failure in the field, and what such a failure could cost in money, safety, and customer trust (Worked Example 5.3).

6. **Submit** the report with test records, screenshots, and APA 7th edition citations.

Table 5.9 Lab 5 test record template

Test	Condition	Expected behavior	Observed behavior	Evidence (figure)
Build	Sensor connected, T below threshold	LED off, readings printed		
Build	Sensor connected, T above threshold	LED on		
Break	Data wire removed (original code)	NaN printed; decision wrong or undefined		
Harden	Data wire removed (hardened code)	3 retries, error printed, LED held off		
Recover	Wire reconnected	Normal operation resumes		

References and further reading

Awati, R., & Lutkevich, B. (2024, November 12). *What is firmware? Definition, types and examples*. TechTarget. https://www.techtarget.com/whatis/definition/firmware

DeepSea Developments. (2024). *Firmware vs software: What is the difference?* [Video]. YouTube. https://www.youtube.com/watch?v=LikjS_y4n5Y

Flashback Team. (2023). *Extracting firmware from embedded devices* [Video]. YouTube. https://www.youtube.com/watch?v=nruUuDalNR0

IBM Technology. (2024). *Securing your IoT devices* [Video]. YouTube. https://www.youtube.com/watch?v=7zWVxrjjIpE

Intelligent Product Solutions. (2018). *How to develop faster, better IoT software solutions* [Video]. YouTube. https://www.youtube.com/watch?v=eQ-W9M01yUI

Li, L. (2026). *IoT Education Course demo system* [Interactive web application]. GitHub Pages. https://liangli99.github.io/IoT-Demo/dashboard.html

NxEco, Inc. (2020). *NxEco* (Version 2.2.4) [Mobile application software]. App Store. https://apps.apple.com/app/id972339992

NxEco. (n.d.-a). *NxEco smart watering* [Website]. https://www.nxeco.com

NxEco. (n.d.-b). *Smart controller specification* (Ver. 10.1) [PDF]. https://www.nxeco.com/Public/public/file/manual/Nxeco_Smart_Control_Spec_Ver10.1.pdf

NxEco. (n.d.-c). *NxEco smart weather-based irrigation controller: App manual* (Ver. 10.2) [PDF]. https://www.nxeco.com/Public/public/file/manual/NxEco_Smart_Manual_Ver10.2.pdf

NxEco. (n.d.-d). *Panel manual* (Ver. 10.1) [PDF]. https://www.nxeco.com/Public/public/file/manual/NeEco_Panel_Manual_Ver10.1.pdf

OnLogic. (2023). *Connected devices at the edge using AWS IoT Greengrass* [Video]. YouTube. https://www.youtube.com/watch?v=zmJD_8KKxP0

Prylada. (2024, December 2). *Over-the-air firmware updates in the IoT context*. IoT For All. https://www.iotforall.com/over-the-air-firmware-updates-in-iot

RTOS LOGICS. (2021). *What is the need of an RTOS in an embedded system* [Video]. YouTube. https://www.youtube.com/watch?v=ZJ8rtkMoxFI

Susnjara, S., & Smalley, I. (2025, March 26). *What is a real-time operating system (RTOS)?* IBM. https://www.ibm.com/think/topics/real-time-operating-system

White, E., & White, C. (Hosts). (n.d.). *Embedded* [Audio podcast]. https://embedded.fm/

Wokwi. (n.d.). *ESP32 simulation*. Wokwi Docs. https://docs.wokwi.com/guides/esp32

CHAPTER 6

Securing IoT Systems: Trust, Privacy, Ethics, and Sustainability

Learning Objectives

After studying this chapter, students will be able to:

1. Explain why IoT security is structurally harder than conventional IT security.
2. Identify threats at each layer of an IoT system using the OWASP IoT Top 10 and the STRIDE model.
3. Apply defense in depth and zero-trust principles to an IoT design.
4. Explain the cryptographic building blocks of IoT security and walk through a mutual TLS handshake.
5. Write a least-privilege access policy and assess risks with a likelihood and impact matrix.
6. Evaluate the privacy, ethical, organizational, and environmental impacts of IoT adoption, including major global regulations.

Where this fits: Weeks 3 to 6. Security and privacy appear in every collaboration from the Week 1 teardown to the Week 7 business proposal. This chapter gathers the security, ethical, organizational, and environmental material in one place and supports SM Assignment 1 (question 4), the security risk in Assignment 2, and the security implications required in Assignment 4.

Opening case: the thermometer in the fish tank

Security researchers have described a case in which attackers entered a casino's network through an internet-connected thermometer in a lobby aquarium. The thermometer was meant to report water temperature to the maintenance company. Because it sat on the same network as other systems and was not properly isolated, the attackers used it as a foothold to reach a database and copy data out of the building.

The thermometer itself held nothing valuable. Its value to an attacker was its *position*: a small, forgotten, rarely updated computer inside the network perimeter. Every IoT device is such a computer. A company may protect its servers and laptops carefully and still be exposed through a camera, a smart TV in a meeting room, or a building-management

controller. This chapter explains how to think about IoT risk and how to design systems that remain trustworthy when some of their parts fail.

6.1 Why IoT security is hard

Network security in IoT is structurally harder than in conventional IT for several reasons:

- **A vast attack surface.** A single company may have thousands of devices from dozens of vendors, each with its own firmware, protocols, and cloud services.
- **Physical exposure.** Devices sit in public places, fields, and customers' homes. Attackers can open them, read their memory chips, and connect to debug ports.
- **Constrained resources.** A microcontroller with kilobytes of RAM may struggle to run strong cryptography, so designers are tempted to skip it.
- **Long lifetimes.** Industrial equipment and infrastructure stay in service for 10 to 20 years, far longer than a laptop, while new vulnerabilities are discovered every month.
- **Missing update mechanisms.** Many older or cheaper devices cannot be updated at all, so they remain permanently vulnerable.
- **Complex supply chains.** Firmware includes open-source libraries, vendor SDKs, and third-party modules. A flaw in one component can affect millions of products.
- **Cyber-physical consequences.** IoT devices unlock doors, control heaters, dose medicine, and steer vehicles. A security failure can cause physical harm, not only data loss.

Security goals are often summarized as the **CIA triad**: **confidentiality** (only authorized parties can read data), **integrity** (data and commands are not altered), and **availability** (the system works when needed). For IoT, a fourth goal is just as important: **safety**, meaning that failures and attacks do not injure people or damage property.

6.2 Threats across the IoT stack

Threats by layer

Table 6.1 IoT threats by layer

Layer	Example threats	Example consequences
Device (perception)	Default or hard-coded passwords; open debug ports (JTAG, UART); firmware extraction and reverse engineering; tampering with sensors	Device takeover; stolen keys; false readings
Network	Eavesdropping; replay of captured commands; spoofed devices or gateways; man-in-the-middle; radio jamming	Data disclosure; unauthorized actuation; loss of service

Layer	Example threats	Example consequences
Edge and cloud (middleware)	Over-permissive access policies; leaked credentials; insecure APIs; misconfigured storage	Fleet-wide compromise; mass data breach
Application	Weak user authentication; insecure mobile apps; phishing of administrators	Account takeover; remote control by attackers

Lessons from real incidents

- **Mirai botnet (2016).** Malware scanned the internet for cameras and routers that accepted factory-default usernames and passwords, infected hundreds of thousands of them, and used them to launch massive denial-of-service attacks, including one that disrupted the domain name provider Dyn and many major websites.
- **Connected car (2015).** Security researchers demonstrated that they could remotely control a Jeep Cherokee's steering and brakes through its internet-connected entertainment system. The manufacturer recalled about 1.4 million vehicles to install a software fix.
- **Medical devices (2017).** The U.S. Food and Drug Administration announced a firmware update for about 465,000 implanted pacemakers to address cybersecurity vulnerabilities, requiring patients to visit clinics for the update.
- **Cloud-connected cameras (2021).** Attackers who obtained an administrator credential for a security-camera company accessed live feeds from about 150,000 cameras in hospitals, schools, factories, and prisons. A single cloud credential exposed an entire customer base.

Each incident maps to a weakness in this chapter: default credentials, missing network segmentation, devices that were hard to update, and over-privileged cloud access.

The OWASP IoT Top 10

The Open Worldwide Application Security Project (OWASP) publishes a widely used list of the ten most important IoT security weaknesses:

1. Weak, guessable, or hard-coded passwords
2. Insecure network services
3. Insecure ecosystem interfaces (web, cloud, and mobile interfaces)
4. Lack of a secure update mechanism
5. Use of insecure or outdated components
6. Insufficient privacy protection
7. Insecure data transfer and storage
8. Lack of device management
9. Insecure default settings
10. Lack of physical hardening

Threat modeling with STRIDE

Threat modeling is a structured way to ask "what could go wrong?" before building a system. The **STRIDE** model, developed at Microsoft, groups threats into six categories (Table 6.2).

Table 6.2 STRIDE threats in IoT

STRIDE category	Property violated	IoT example	Typical control
Spoofing	Authentication	A fake sensor sends false temperatures to the gateway	Device certificates, BLE pairing
Tampering	Integrity	An attacker alters a firmware image or a command	Signed firmware, message integrity codes
Repudiation	Accountability	No record of who opened a smart lock	Tamper-evident audit logs
Information disclosure	Confidentiality	Unencrypted health data is sniffed over the air	Encryption in transit and at rest
Denial of service	Availability	Radio jamming or a flood of messages to the broker	Rate limiting, local survivability, redundancy
Elevation of privilege	Authorization	A compromised camera reaches the corporate network	Network segmentation, least privilege

6.3 Defense in depth and zero trust

No single control can stop every attack. **Defense in depth** places several independent layers of protection around what matters, so that an attacker who defeats one layer meets another (Figure 6.1).

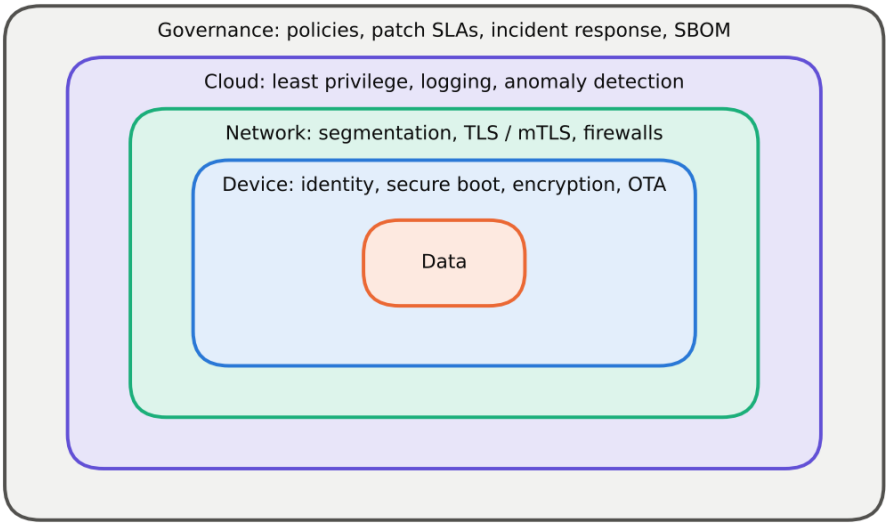


Figure 6.1 Defense in depth for IoT. Each layer assumes the one outside it may fail.

Device layer

- A **unique identity** for every device, provisioned at manufacturing (a certificate and private key, never a shared password).
- A **hardware root of trust** (secure element or TPM) to store keys so they cannot be read out.
- **Secure boot** to run only firmware signed by the manufacturer, and **flash encryption** to protect code and data at rest. The demo sensor node, for example, specifies Secure Boot v2 and AES-XTS flash encryption on the ESP32 (Li, 2026).
- **Disabled debug ports** and removed test credentials in production units.

Network layer

- **Encryption in transit**: BLE link-layer encryption (AES-128 CCM) or Zigbee/Thread network keys locally, and TLS from gateway to cloud.
- **Network segmentation**: IoT devices on their own VLAN or Wi-Fi network, separated by firewalls from business systems, with only necessary traffic allowed.

Cloud layer

- **Least-privilege policies** that let each device do only what it needs (Section 6.5).
- **Logging and anomaly detection**, for example alerts when a device publishes to unusual topics or at unusual rates.
- **Strong authentication** for administrators and apps, with multi-factor authentication and short-lived credentials.

Governance layer

- A **software bill of materials (SBOM)** listing every component in the firmware, so the company knows which products are affected when a library vulnerability is announced.
- A **vulnerability disclosure policy** that tells researchers how to report flaws.
- **Patch commitments**: how long the product will receive security updates and how quickly critical flaws will be fixed.
- An **incident response plan**.

Zero trust is a related principle: *never trust, always verify*. No device or user is trusted merely because it is "inside" the network. Every connection is authenticated, authorized, and encrypted, and access is granted only for the specific resource needed. The fish-tank thermometer would not have been a useful foothold in a zero-trust network.

6.4 Cryptography essentials

Students do not need to implement cryptography, but they must understand what each building block does.

- **Symmetric encryption** uses the same secret key to encrypt and decrypt. It is fast and suits constrained devices. **AES-128** is the standard; BLE uses it in **CCM** mode, which provides both encryption and an integrity check.
- **Asymmetric (public-key) cryptography** uses a key pair: a **public key** that can be shared and a **private key** that must remain secret. It solves the problem of establishing trust and sharing keys with strangers. **RSA-2048** and **elliptic-curve cryptography (ECC)**, such as the P-256 (secp256r1) curve, are common. ECC offers similar security with much smaller keys and less computation, which suits IoT.
- **Hash functions** such as **SHA-256** produce a fixed-length fingerprint of data. Any change to the data changes the hash, which is how firmware integrity is checked.
- **Digital signatures** combine hashing and a private key: the manufacturer signs a firmware image, and devices verify the signature with the manufacturer's public key.
- **Certificates** (**X.509**) bind a public key to an identity, such as a device name, and are signed by a **certificate authority (CA)**. A device trusts a server's certificate if it chains back to a **root CA** that the device already trusts, such as Amazon Root CA 1.

Worked Example 6.1: Why brute force fails against AES-128

An AES-128 key has 128 bits, so there are $2^{128} \approx 3.4 \times 10^{38}$ possible keys. Suppose an attacker can test one trillion (10^{12}) keys per second.

Time to try every key = $3.4 \times 10^{38} \div 10^{12} = 3.4 \times 10^{26}$ seconds.

One year $\approx 3.16 \times 10^{7}$ seconds, so the time is about $3.4 \times 10^{26} \div 3.16 \times 10^{7} \approx$ **1.1×10^{19} years**, roughly a billion times the age of the universe.

Interpretation. Attackers almost never break the cryptographic algorithm itself. They steal keys from unprotected memory, exploit default passwords, trick users, or find bugs in the

code around the cryptography. This is why key storage, secure boot, and software updates matter as much as the choice of algorithm.

6.5 Mutual TLS and least privilege

Mutual TLS, step by step

Most people have seen ordinary **TLS** in a web browser: the browser checks the website's certificate, but the website does not check the browser's. **Mutual TLS (mTLS)** adds a second check: the server also verifies the client's certificate. An analogy: ordinary TLS is a bouncer checking a guest's ID; mutual TLS is a border crossing where *both* sides show a passport and both check it against a trusted authority before anyone is let through. Figure 6.2 shows the handshake between the demo gateway and AWS IoT Core.

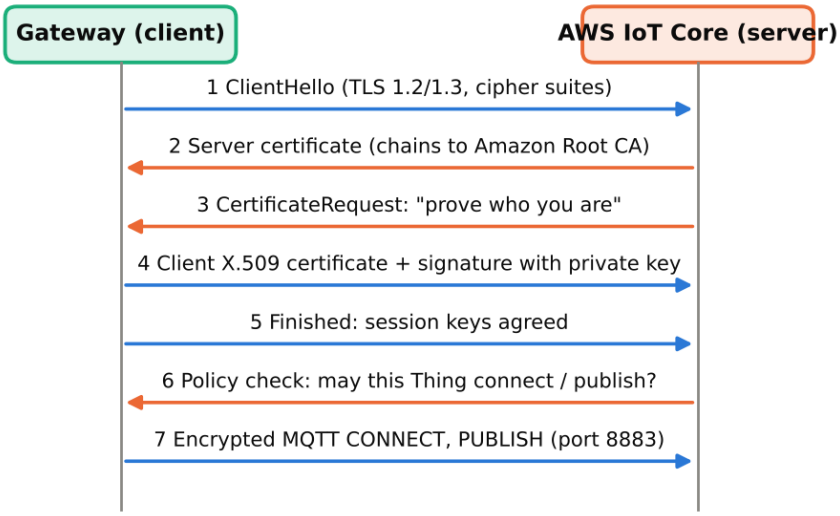


Figure 6.2 The mutual TLS handshake between an edge gateway and AWS IoT Core (simplified).

1. The gateway opens a connection to port 8883 and proposes TLS versions and cipher suites.
2. AWS IoT Core presents its server certificate; the gateway verifies that it chains to **Amazon Root CA 1**.
3. The server requests a client certificate.
4. The gateway presents its **device certificate** and proves it holds the matching **private key** by signing handshake data.
5. Both sides derive session keys; all further traffic is encrypted.
6. AWS IoT Core checks its **registry** and the **policy** attached to the certificate.
7. The gateway sends encrypted MQTT messages.

Three files make up the gateway's entire identity: the device certificate, its private key, and the root CA certificate. Whoever holds the private key *is* the device, so it must be protected, ideally inside a secure element or TPM that never releases it.

Least privilege

The **principle of least privilege** says that every device, user, and service should have only the permissions it needs, and no more. In AWS IoT, permissions are written as JSON policies. The following policy lets a gateway connect only as itself and publish or subscribe only within its project's topic namespace:

```
{
  "Version": "2012-10-17",
  "Statement": [
    { "Effect": "Allow",
      "Action": "iot:Connect",
      "Resource": "arn:aws:iot:*:*:client/$
{iot:Connection.Thing.ThingName}" },
    { "Effect": "Allow",
      "Action": ["iot:Publish", "iot:Receive"],
      "Resource": "arn:aws:iot:*:*:topic/iot/*" },
    { "Effect": "Allow",
      "Action": "iot:Subscribe",
      "Resource": "arn:aws:iot:*:*:topicfilter/iot/*" }
  ]
}
```

If the resource were `"*"` instead, a stolen gateway certificate could publish fake commands to every device in the account, or subscribe to every customer's data. Least privilege limits the **blast radius** of any single compromise.

Worked Example 6.2: The operational load of certificates

A company operates 10,000 gateways, each with a certificate valid for two years. If renewals are spread evenly, how many certificates must be renewed per day? What does this imply?

Renewals per day = 10,000 ÷ (2 × 365) ≈ **13.7 per day**, every day.

No team can do this by hand. Certificate issuance, rotation, and revocation must be automated through the cloud platform's fleet-provisioning and device-management services. Designs should also plan for what happens if a certificate expires on a device that has been offline for months.

Security, end to end: the demo system

The class demo system (Chapter 7) illustrates layered security (Table 6.3).

Table 6.3 Layered security in the IoT Education Course demo system

Layer	Mechanism	Detail
Device	Secure boot and flash	Only signed firmware runs; code and keys

Layer	Mechanism	Detail
	encryption	encrypted at rest
BLE (connected)	Link-layer encryption	LE Secure Connections pairing; AES-128 CCM on the GATT link; rotating private addresses
Northbound transport	Mutual TLS	Bidirectional authentication over MQTT on port 8883
Cryptography	RSA-2048 or ECC P-256, SHA-256	ECC as the lighter alternative
Device identity	X.509 certificates	Device certificate and private key verified against Amazon Root CA 1
Cloud authorization	Least-privilege policy	Scoped to one Thing and the project's topic namespace
Mobile app	Amazon Cognito	Temporary credentials (about 60 minutes); no permanent keys stored in the app

The lesson is that security is not one control but a stack: each layer assumes the one below it can fail. If an attacker compromised only the BLE link, they still could not reach AWS, because the gateway, not the sensor, holds the mTLS credentials. Note also that passive BLE advertising is unencrypted by design. That is an acceptable trade-off for a non-sensitive temperature reading broadcast over a few meters, but it would not be acceptable for a door-lock command, which must use an encrypted, authenticated connection.

6.6 Assessing risk

Security budgets are limited, so organizations prioritize risks. A common method scores each risk for **likelihood** and **impact** on a 1 to 5 scale and multiplies them (Figure 6.3).

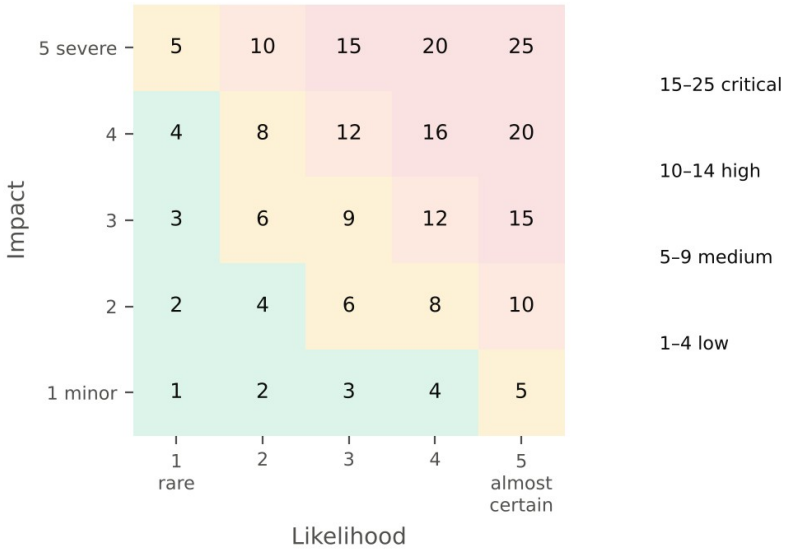


Figure 6.3 A likelihood and impact risk matrix.

Worked Example 6.3: A mini risk register for clinic refrigerators

The clinic from Chapter 3 deploys Zigbee temperature sensors in medicine refrigerators, a gateway, and a cloud dashboard.

Risk	Likelihood	Impact	Score	Rating	Mitigation
Gateway uses the default admin password	4	4	16	Critical	Unique credentials per gateway; disable remote admin
Gateway on the same network as patient records	3	5	15	Critical	Separate VLAN and firewall rules
Sensor battery dies unnoticed	3	4	12	High	Low-battery alerts; "no data" alarm after 15 minutes
Zigbee traffic sniffed in the corridor	2	2	4	Low	Zigbee network encryption is sufficient
Cloud outage hides alarms	2	4	8	Medium	Local buzzer on the gateway (local survivability)

The two critical risks are cheap to fix and should be addressed before go-live. Notice that not every risk is an attack: an unnoticed dead battery threatens the vaccines just as much as a hacker.

6.7 Privacy

IoT devices collect data about people, often continuously and in private spaces. **Personal data** is any information relating to an identified or identifiable person; for IoT, that includes

location, voice, images, health measurements, and patterns of behavior. The privacy challenge is not only what is collected but what can be **inferred**: electricity use reveals when a home is empty; a wearable's heart rate can suggest stress or illness; a car's location history reveals where its owner works, worships, and sleeps.

Privacy by design builds protection into products from the start. Its core practices include:

- **Data minimization**: collect only what the purpose requires (for example, count people in a room with a low-resolution thermal sensor instead of a camera).
- **Purpose limitation**: use data only for the purpose users agreed to.
- **Edge processing**: analyze sensitive data on the device and send only results.
- **Retention limits**: delete raw data after it is no longer needed.
- **Transparency and consent**: explain clearly what is collected, why, and with whom it is shared, and let users choose.
- **Security**: protect data in transit and at rest, because there is no privacy without security.

Privacy law differs by jurisdiction (Table 6.4). Companies selling globally must design for the strictest market.

Table 6.4 Selected privacy laws relevant to IoT

Jurisdiction	Key law	Relevance to IoT
European Union	General Data Protection Regulation (GDPR)	Lawful basis, data minimization, rights of access and erasure, fines up to 4 percent of global turnover
United Kingdom	UK GDPR; PSTI regime (2024)	Data protection, plus mandatory security basics for consumer connectable products
China	Personal Information Protection Law (PIPL)	Consent, sensitive-data rules, restrictions on cross-border transfer
India	Digital Personal Data Protection Act (2023)	Consent-based processing and duties for data fiduciaries
Brazil	Lei Geral de Proteção de Dados (LGPD)	GDPR-like rights and obligations
United States	Sector and state laws (for example HIPAA for health; California's CCPA/CPRA)	Health data rules; consumer rights to know and delete

6.8 Security regulation and standards

Governments are turning good security practice into legal requirements:

- **European Union: Cyber Resilience Act (CRA).** The CRA entered into force in December 2024. It covers hardware and software "products with digital elements" sold in the EU, regardless of where the manufacturer is based. Manufacturers must

design products securely (for example, no default passwords and secure updates), maintain an SBOM, provide security updates for at least five years or the expected product lifetime, and report actively exploited vulnerabilities, with reporting obligations from September 2026 and full application from December 2027.

- **European Union: Radio Equipment Directive.** Since August 2025, internet-connected radio equipment sold in the EU must meet cybersecurity requirements.
- **United Kingdom: PSTI.** Since April 2024, consumer connectable products must not use universal default passwords, must publish a vulnerability disclosure policy, and must state the minimum period of security updates.
- **United States: Cyber Trust Mark.** A voluntary labeling program overseen by the Federal Communications Commission lets consumers identify smart devices that meet baseline security criteria derived from NIST guidance.
- **Standards.** ETSI EN 303 645 (consumer IoT security baseline), NIST IR 8259 (core device capabilities), and IEC 62443 (industrial automation and control systems security) provide detailed requirements that regulators and customers reference.

6.9 Ethical and organizational impacts

Ethical questions

IoT raises ethical questions that technology alone cannot answer:

- **Surveillance and consent.** Workplace sensors that track location or productivity may improve safety but can also enable intrusive monitoring. Are employees informed? Can visitors to a smart building consent to being sensed?
- **Fairness and bias.** AIoT systems that recognize faces or voices may perform worse for some groups if training data is unrepresentative, leading to unequal treatment.
- **Autonomy and accountability.** When an autonomous system makes a harmful decision, who is responsible: the manufacturer, the software developer, the operator, or the owner?
- **Access and the digital divide.** Smart services may be unavailable or unaffordable for rural communities or low-income households, widening existing gaps.
- **Dual use.** The same sensors that monitor elderly people for falls could be used to monitor people without their knowledge.

A practical test is to ask, for every data flow: *Would the people affected expect this, and would they agree if asked?*

Organizational impacts

Adopting IoT changes organizations as well as machines:

- **Skills.** IoT requires embedded, networking, cloud, data, and security skills that few teams have together.

- **IT/OT convergence.** Information technology (IT) teams and operational technology (OT) teams, who run factories and utilities, have different priorities (confidentiality versus uptime and safety) and must learn to work together.
- **Governance.** Someone must own the device inventory, data policies, update schedules, and vendor relationships for the whole life of the deployment.
- **Change management.** Workers may fear that monitoring threatens their jobs or privacy. Successful projects involve them early and share the benefits.
- **New business models.** Selling outcomes or subscriptions instead of products (Chapter 9) changes how sales, finance, and support teams work.

6.10 Environmental impacts and sustainability

IoT has a double environmental effect. It **enables** sustainability by optimizing resource use: smart irrigation saves water, smart thermostats and building management save energy, and connected grids integrate more renewable power. At the same time, it has a **footprint**: devices consume materials and energy, data centers consume electricity and water, and billions of devices eventually become waste.

The United Nations' Global E-waste Monitor 2024 estimated that the world generated about 62 million tonnes of electronic waste in 2022, of which only about 22 percent was documented as properly collected and recycled. IoT adds to this stream with small devices that are often glued shut, contain batteries, and are discarded when a cloud service is discontinued.

Responsible IoT design reduces the footprint:

- **Design for longevity.** Choose components and software that can be supported and updated for ten years or more; publish support periods.
- **Design for repair and recycling.** Use replaceable batteries, screws instead of glue, and labeled materials. Right-to-repair and ecodesign rules in several jurisdictions increasingly require this.
- **Reduce power.** Duty cycling, efficient protocols, and edge filtering lower device and data-center energy use.
- **Harvest energy.** Solar, thermal, vibration, or radio-frequency harvesting can eliminate battery replacement.
- **Plan for end of service.** If a company discontinues a cloud service, devices should keep working locally or be releasable to open alternatives, rather than becoming e-waste.

Worked Example 6.4: Battery waste over a product's life

A city deploys 10,000 parking sensors for 10 years. Design A uses a coin cell that lasts 2 years; Design B uses a larger cell and a more efficient duty cycle that last 10 years. How many battery replacements does each need, ignoring the initial batteries?

Design A: batteries per sensor over 10 years = 10 ÷ 2 = 5, so 4 replacements. Total = 10,000 × 4 = **40,000 replacement batteries** and 40,000 maintenance visits.

Design B: 10 ÷ 10 = 1 battery, so **0 replacements**.

Even if Design B's battery costs more, it eliminates 40,000 batteries of waste and 40,000 technician visits, with their labor cost and vehicle emissions. Power efficiency is an environmental and financial decision at the same time.

6.11 IoT application case: trust and sustainability in NxEco

Least privilege in practice: the Valet Key. Homeowners often employ a gardener or landscape company. NxEco's **Valet Key** lets the owner authorize another user to operate the controller, change schedules and delays, and run manual watering, *without* sharing the owner's account password or Wi-Fi credentials. Access ends when the owner deletes the authorized user (NxEco, n.d.-c). This is the principle of least privilege (Section 6.5) applied to a consumer product, and it avoids the common and risky practice of sharing one password among many people.

Credential hygiene. NxEco's public website lists a demonstration account, with a four-digit password, that anyone can use to try the app on controllers in the company's lab (NxEco, n.d.-f). A shared demo account connected to lab devices is a reasonable marketing tool, but it illustrates two points from this chapter: short numeric passwords are weak against guessing, and any shared credential should be isolated from real customer devices. Students should never reuse such a pattern for production accounts.

Privacy and transparency. Watering schedules and water-use history can reveal when a household is away. The iOS App Store listing did not include the developer's privacy disclosures (NxEco, Inc., 2020), so prospective users could not see what data the app collects. Privacy by design (Section 6.7) would publish what is collected, why, and for how long. The app also did not declare two-factor sign-in, its account accepts a four-digit password, and its last iOS update was in 2020 (Table 5.8). An app that is not updated cannot receive fixes for newly found weaknesses in the networking and encryption libraries it contains, so app maintenance is part of a product's security, not just its convenience.

Security questions to investigate. The public documentation does not describe how controller-to-cloud traffic is encrypted, how controllers authenticate the server, or how firmware updates are signed. A security reviewer applying the OWASP IoT Top 10 would ask for these details before recommending the product to a business.

What a portal review reveals. An authorized, read-only review of the MySmartDrop portal on October 22, 2026, turned up three details that illustrate this chapter (NxEco, 2026). First, requesting a page that does not exist returned a full framework **debug stack trace**, showing file paths and the PHP framework in use. Debug output left on in production is an information-disclosure weakness (the "I" in STRIDE) because it gives an attacker a map of the server. Second, controller actions are addressed by short sequential numbers in the web

address (for example, a controller's watering-plan page ends in a four-digit identifier). That is safe only if the server checks, on every request, that the identifier belongs to the signed-in account; if it did not, changing the number would expose another customer's garden, a flaw known as an **insecure direct object reference**. The reviewer did not test this, and students must never probe systems they are not authorized to test. Third, the remove-controller action asks for confirmation, which protects against accidental clicks but not against forged requests from another website unless the server also uses anti-forgery tokens. Each detail is a question a buyer could ask the vendor.

Sustainability: the environmental value. Landscape irrigation is one of the largest uses of residential water in dry regions. NxEco is certified under the U.S. EPA **WaterSense** program for weather-based irrigation controllers, states savings of up to 40 to 50 percent, and has been eligible for utility rebates such as the Southern Nevada Water Authority's US$100 incentive in Las Vegas (NxEco, n.d.-a). This is IoT's **enabling** effect on the environment. Real data temper the marketing figure: the portal's Water Record for one home controller showed 13,044.65 minutes of actual watering and 2,157.01 minutes saved by weather adjustment, a saving of about 14 percent against the programmed schedule (Worked Example 9.4). Savings depend on climate, on how generous the original schedule was, and on the owner's fine-tune setting. On the **footprint** side, a long-lived, repairable controller with a replaceable outdoor cabinet and continued software support keeps the device out of the e-waste stream; an app that is no longer updated threatens that longevity.

Chapter summary

- IoT security is hard because of vast attack surfaces, physical exposure, constrained resources, long lifetimes, missing updates, complex supply chains, and cyber-physical consequences. Goals are confidentiality, integrity, availability, and safety.
- Threats exist at every layer. Incidents such as Mirai show the cost of default passwords, poor segmentation, and over-privileged cloud access. The **OWASP IoT Top 10** and **STRIDE** structure threat analysis.
- **Defense in depth** layers device, network, cloud, and governance controls; **zero trust** verifies every connection.
- Cryptography provides symmetric encryption (AES), public-key cryptography (RSA, ECC), hashes (SHA-256), signatures, and certificates. Attackers target keys and implementations, not the algorithms.
- **Mutual TLS** authenticates both client and server; **least-privilege** policies limit the blast radius of a compromise.
- Risks are prioritized by **likelihood × impact**.
- **Privacy by design** minimizes, localizes, limits, and explains data collection; laws differ across the EU, UK, China, India, Brazil, and the United States.
- New regulations (EU CRA, RED, UK PSTI, U.S. Cyber Trust Mark) make secure design a legal and market requirement.

- IoT raises ethical questions of surveillance, bias, accountability, and access, and requires organizational change.
- IoT both enables sustainability and adds to e-waste; longevity, repairability, low power, and energy harvesting reduce its footprint.

Key terms

AES; asymmetric cryptography; attack surface; blast radius; certificate authority; CIA triad; Cyber Resilience Act; data minimization; defense in depth; digital signature; e-waste; elliptic-curve cryptography; flash encryption; GDPR; hash function; least privilege; mutual TLS; network segmentation; OWASP IoT Top 10; personal data; privacy by design; PSTI; risk matrix; root of trust; SBOM; secure boot; STRIDE; symmetric encryption; threat modeling; X.509; zero trust.

Review questions

Multiple choice

1. Which weakness did the Mirai botnet primarily exploit?
    a. Broken AES encryption
    b. Default usernames and passwords
    c. Faulty MQTT brokers
    d. Weak Wi-Fi antennas
2. In the STRIDE model, a fake sensor sending false data to a gateway is an example of:
    a. Spoofing
    b. Repudiation
    c. Denial of service
    d. Elevation of privilege
3. Which control best limits the damage if a smart camera on a corporate network is compromised?
    a. Higher camera resolution
    b. Network segmentation
    c. QoS 2
    d. A faster processor
4. In mutual TLS, what is added compared with ordinary TLS?
    a. The server presents a certificate
    b. The client also presents a certificate and proves it holds the private key
    c. Data is sent unencrypted
    d. A password is sent in plain text

5. Which cryptographic tool lets a device verify that a firmware image came from its manufacturer?
    a. A digital signature
    b. A MAC address
    c. A topic wildcard
    d. A retained message
6. Why is ECC often preferred over RSA on constrained devices?
    a. It needs no keys
    b. It provides similar security with smaller keys and less computation
    c. It is not public-key cryptography
    d. It works only on Wi-Fi
7. A risk with likelihood 3 and impact 5 has a score of:
    a. 8
    b. 15
    c. 2
    d. 35
8. Which practice is an example of data minimization?
    a. Storing raw video forever
    b. Counting room occupancy with a low-resolution thermal sensor instead of a camera
    c. Sharing data with all partners by default
    d. Using one password for all devices
9. A software bill of materials (SBOM) helps a company:
    a. reduce radio interference
    b. know which products contain a vulnerable component
    c. increase battery life
    d. route MQTT messages
10. Under the EU Cyber Resilience Act, full obligations apply from:
    a. 2018 b. 2022 c. December 2027 d. 2035

Short answer

11. Explain why IoT security is structurally harder than conventional IT security. Give at least three reasons.
12. Describe one IoT-specific security weakness and explain how device identity, transport encryption, network segmentation, or OTA updates reduces it.
13. Explain the principle of least privilege using the AWS IoT policy example. What could go wrong if the resource were *?

14. Describe one ethical concern and one environmental concern raised by a large IoT deployment, and propose a response to each.

Applied problems

15. Build a five-row risk register (risk, likelihood, impact, score, rating, mitigation) for a home smart lock with a phone app and a cloud service. Identify which risk should be fixed first and why.
16. A company plans 50,000 environmental sensors for 8 years. Option A's battery lasts 2 years; Option B's lasts 8 years and costs US$3 more per sensor. Each battery replacement visit costs US$25. Compare total battery replacement costs (excluding initial batteries) and the number of batteries discarded. Which option would you recommend?
17. **NxEco case.** Explain how NxEco's Valet Key applies the principle of least privilege, and identify one security detail you would ask NxEco to document before recommending the product to a business.

Lab 6: IoT security and privacy review

Supports SM Assignment 1 (question 4), Assignment 2 (security risk), and Assignment 4 (security implication). This lab can be completed individually or in teams.

Objective. Perform a structured security and privacy review of an IoT product or design.

Procedure.

1. **Select a system:** the device from Lab 1, the deployment from Lab 3, or the scenario for Assignment 2 or 4.
2. **Draw a data-flow diagram** showing devices, gateway, cloud, app, and users, with arrows for every data flow. Mark **trust boundaries**, the points where data crosses from one zone of control to another (for example from the BLE link to the gateway, or from the gateway to the internet).
3. **Apply STRIDE** to each element and flow. List at least one plausible threat for at least four STRIDE categories.
4. **Check the OWASP IoT Top 10.** For each item, note whether the system is exposed, protected, or unknown.
5. **Score the risks** with the likelihood and impact matrix (Figure 6.3) and record them in a risk register like Worked Example 6.3.
6. **Propose mitigations** for the two highest risks, naming the defense-in-depth layer each belongs to.
7. **Privacy review.** List every category of personal data collected, whether each is necessary for the purpose, how long it is kept, and whether it could be processed at the edge instead.

8. **Sustainability note.** State the expected device lifetime, battery replacement plan, and end-of-life plan.

Deliverable. A two- to three-page review with the diagram, STRIDE table, risk register, and recommendations, with APA 7th edition citations.

References and further reading

Amazon Web Services. (n.d.). *AWS IoT Core policies*. AWS IoT Core Developer Guide. https://docs.aws.amazon.com/iot/latest/developerguide/iot-policies.html

CyberSentriq. (2025). *IoT security: How to protect your smart devices* [Video]. YouTube. https://www.youtube.com/watch?v=GhGTzD7THXw

European Union. (2024). *Regulation (EU) 2024/2847 of the European Parliament and of the Council on horizontal cybersecurity requirements for products with digital elements (Cyber Resilience Act)*. Official Journal of the European Union.

IBM Technology. (2024). *Securing your IoT devices* [Video]. YouTube. https://www.youtube.com/watch?v=7zWVxrjjIpE

Kirvan, P., Yasar, K., & Shea, S. (2025, August 5). *What is IoT security?* TechTarget. https://www.techtarget.com/iotagenda/definition/IoT-security-Internet-of-Things-security

Li, L. (2026). *IoT Education Course demo system* [Interactive web application]. GitHub Pages. https://liangli99.github.io/IoT-Demo/dashboard.html

NxEco, Inc. (2020). *NxEco* (Version 2.2.4) [Mobile application software]. App Store. https://apps.apple.com/app/id972339992

NxEco. (n.d.-a). *NxEco smart watering* [Website]. https://www.nxeco.com

NxEco. (n.d.-c). *NxEco smart weather-based irrigation controller: App manual* (Ver. 10.2) [PDF]. https://www.nxeco.com/Public/public/file/manual/NxEco_Smart_Manual_Ver10.2.pdf

NxEco. (n.d.-f). *Features* [Web page]. https://www.nxeco.com/features.html

NxEco. (2026). *MySmartDrop smart watering portal* [Web application]. Retrieved October 22, 2026, from https://www.mysmartdrop.com

OWASP Foundation. (2018). *OWASP Internet of Things Top 10*. https://owasp.org/www-project-internet-of-things/

United Nations Institute for Training and Research & International Telecommunication Union. (2024). *Global e-waste monitor 2024*. https://ewastemonitor.info/

CHAPTER 7

IoT System Design Example: Building and Testing an End-to-End Demo

Learning Objectives

After studying this chapter, students will be able to:

1. Describe the five tiers of the IoT Education Course demo system and explain why the gateway tier exists.
2. Justify end-device hardware using the criteria of power, cost, and real-time headroom.
3. Calculate average current and battery life for a deep-sleep duty cycle and explain which parameters dominate.
4. Compare BLE advertising with connected GATT and decode a BLE sensor payload.
5. Explain the gateway's protocol translation, store-and-forward buffering, provisioning, and least-privilege policy.
6. Trace one reading's uplink journey and one command's downlink journey, and analyze a latency budget.
7. Plan and execute system tests, record evidence, and analyze a closed control loop.

Where this fits: Weeks 6 and 7. This chapter is both the design reference for the demo system and the test manual for Collaboration 6 and 7 *Stress-Test the Demo System* (Parts A and B), Assignment 3 (team test report, Part A), and Assignment 4, item 1 (full test report).

Opening case: every layer, working together

Each layer of an IoT system is simple on its own. A temperature sensor on a breadboard can be wired in an afternoon. A Raspberry Pi can run a Python script that publishes MQTT messages in an hour. A cloud rule that saves data to a table takes minutes to configure. Yet when students connect all of these parts for the first time, something almost always fails: the BLE scanner misses packets, the certificate path is wrong, the topic name has a typo, or the phone app shows yesterday's data.

The value of an IoT system, and its engineering challenge, lies in making every layer work together reliably and securely. This chapter studies one complete system, the **IoT Education Course demo system**, from the sensor node to the phone and back. It is the integration chapter of the book: almost every concept from Chapters 1 to 6 appears here in a concrete

form. A browser-based simulation of the system is available at https://liangli99.github.io/IoT-Demo/dashboard.html (Li, 2026). The simulation's values are modeled rather than measured, but they follow the topics, rules, and latency budget described here, so students can treat it as a test bench.

7.1 System overview: five tiers, one pipeline

The demo system has four physical subsystems: battery-powered **sensor nodes**, an **edge gateway**, an **AWS cloud back end**, and a **mobile app** with a web dashboard. Architecturally, it is described with five tiers (Figure 7.1):

1. **Perception layer:** low-power sensor nodes capture environmental and motion data and drive actuators.
2. **Proximity network:** short-range wireless (BLE 5.0) links nodes to a local sink.
3. **Edge gateway:** protocol conversion, local processing, buffering, and secure upstream delivery.
4. **Cloud layer:** AWS IoT Core broker, rules engine, serverless compute, and storage.
5. **Presentation layer:** web dashboard and Android app for live visualization and reverse control.

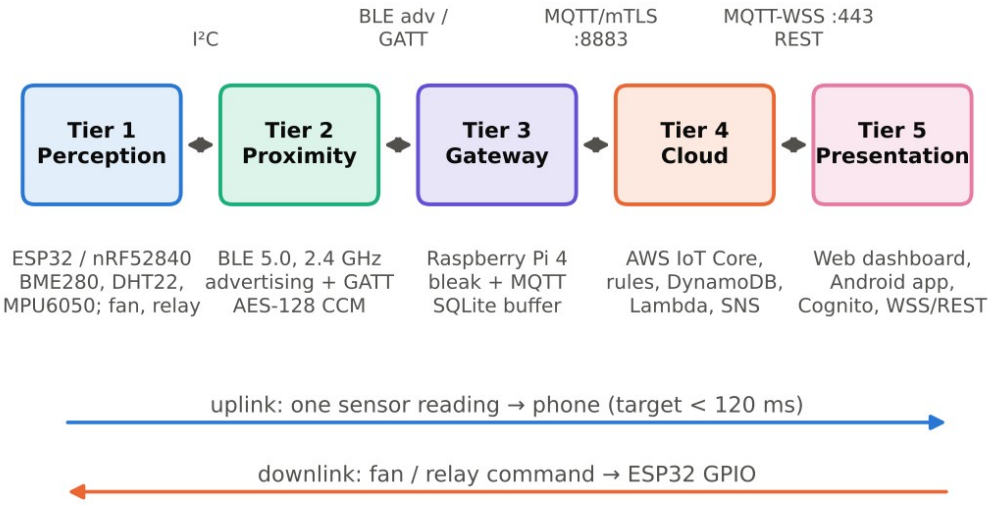


Figure 7.1 The five-tier IoT Education Course demo system. Data flows up (uplink); commands flow down (downlink).

The system includes three sensor nodes (Table 7.1).

Table 7.1 Sensor nodes in the demo system

Node	Role	MCU	Sensors	Power
sensor_node_01	Environmental lab node with actuators	ESP32-WROOM-32	DHT22, MPU6050 (plus fan, relay, LED)	Battery (LiPo)
sensor_node_02	Server-room rack node	ESP32-WROOM-32	BME280	Battery or USB
sensor_node_03	Smart-agriculture node	Nordic nRF52840	Soil/temperature probe	Battery

Key Concept: Why a gateway at all? A full TLS 1.2 or 1.3 stack plus a certificate chain can need more RAM than an entire budget microcontroller has on board, and keeping a Wi-Fi or cellular link alive drains batteries quickly. The gateway absorbs that cost. It is not a convenience; it is the reason the sensor node can stay cheap and tiny. Whenever a design choice in this system seems arbitrary, return to this idea.

7.2 Tier 1: perception-layer hardware

Hardware selection

Table 7.2 lists the recommended parts for a sensor node.

Table 7.2 End-device hardware

Component	Recommended parts	Notes
MCU	ESP32-WROOM-32 or Nordic nRF52840	Low cost; BLE 5.0 capable
Environmental sensor	DHT22 or Bosch BME280	Temperature, humidity, pressure (BME280 over I^2C/SPI)
Motion sensor	TDK MPU6050 or ST LSM6DSV16X	6-axis accelerometer and gyroscope
Power	3.7 V LiPo with TP4056 charger, or CR2032 coin cell	Sized to the duty cycle (Section 7.3)

Engineers apply three selection criteria, **in this order**:

1. **Power efficiency.** The node must survive months on a small battery. Power comes first because battery life cannot be added after the fact.
2. **Unit cost.** Cheap enough to deploy several nodes per team, or thousands per customer.
3. **Real-time headroom.** Enough processing to service a BLE stack and one or two sensors without dropping samples.

These are the same criteria a production embedded-systems team uses. The two candidate MCUs illustrate the trade-off (Table 7.3).

Table 7.3 ESP32 versus nRF52840 for a battery sensor node

Parameter	ESP32-WROOM-32	Nordic nRF52840
Core	Dual-core Xtensa LX6, up to 240 MHz	Single-core Arm Cortex-M4F, 64 MHz
Deep-sleep current	About 10 µA	Below 1 µA (about 0.4 µA)
Radio current while transmitting	Tens of mA (up to about 84 mA in the demo profile)	About 5 mA at 0 dBm
Wireless	Wi-Fi and Bluetooth	Bluetooth LE, Thread, Zigbee
Approximate unit cost (chip)	US$3 to 4	US$7.50 to 10

The ESP32 is cheaper and more powerful, and its Wi-Fi is useful during development, but its radio is power-hungry. It compensates by spending more than 95 percent of its time in deep sleep. The nRF52840 costs more but runs far longer on the same battery. For a classroom demo, cost wins; for a commercial product with a five-year battery requirement, the nRF52840 would likely win.

Pin map of the lab node

Table 7.4 Pin map of sensor_node_01 (design reference)

ESP32 GPIO	Function	Connected to	Protection or pull-up
21	I^2C SDA	BME280 (address 0x76), MPU6050 (0x68)	4.7 kΩ pull-up to 3.3 V
22	I^2C SCL (400 kHz)	BME280, MPU6050	4.7 kΩ pull-up to 3.3 V
4	Single-wire data	DHT22	10 kΩ pull-up to 3.3 V
18	PWM (5 kHz)	5 V cooling fan through a MOSFET	10 kΩ gate pull-down
23	Relay drive (active high)	Solid-state relay	Opto-isolated input
2	Status LED	Blue LED	Series resistor

Notice how Chapter 2's lessons appear in the pin map: the MCU never drives the fan or relay directly; a MOSFET and an opto-isolated relay act as driver circuits.

7.3 Firmware and the deep-sleep duty cycle

The node's firmware follows a repeating cycle:

1. **Wake** from deep sleep on the internal real-time clock (RTC) timer.
2. **Power up the sensors** through a GPIO pin, sample them over I^2C, and format the payload.
3. **Broadcast** the data with a BLE advertisement or a GATT notification.

4. **Return to deep sleep** to minimize average power.

The fraction of time the device is active is its **duty cycle**. Because active current is thousands of times larger than sleep current, the duty cycle controls battery life (Figure 7.2).

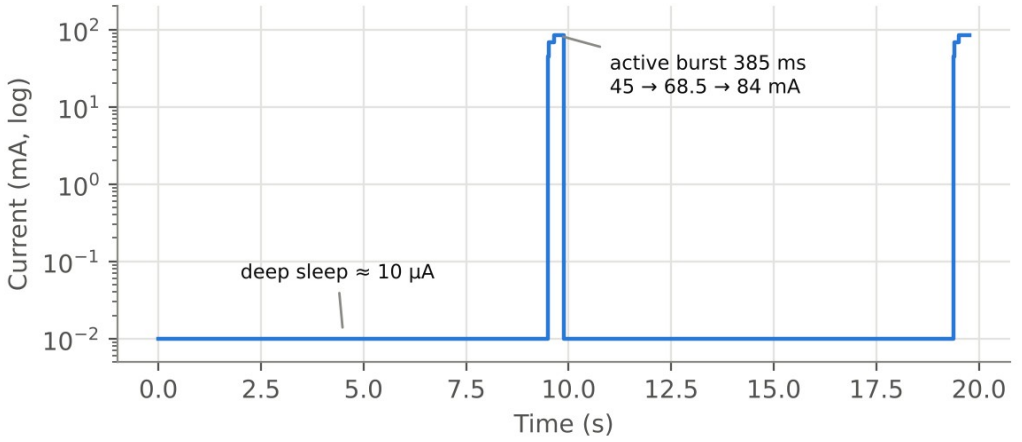


Figure 7.2 Current profile of one sensor node over two cycles (logarithmic scale). A short, intense active burst is followed by a long deep sleep.

Calculating battery life

The **average current** is the time-weighted average of the current in each phase:

$$I_{avg} = \Sigma\ (I_k \times t_k) \div T_{cycle}$$

where I_k and t_k are the current and duration of phase k and T_{cycle} is the total cycle time. Battery life in hours is then

$$Life\ (h) = (C \times D) \div I_{avg}$$

where C is the battery capacity in mAh and D is a **derating factor** (for example 0.85) that accounts for capacity lost to temperature, aging, self-discharge, and the cut-off voltage of the regulator. Simple estimates sometimes omit the derating factor ($D = 1$), which gives an optimistic upper bound.

Worked Example 7.1: A simple two-phase estimate

A node samples every 30 s. Each active burst lasts about 50 ms at 80 mA; deep sleep draws 10 µA. The battery is 2,000 mAh. Estimate battery life without derating.

Duty cycle = 0.05 s ÷ 30 s ≈ 0.00167 (0.17 percent).

Average current = (0.99833 × 0.010 mA) + (0.00167 × 80 mA) ≈ 0.0100 + 0.1333 ≈ **0.143 mA** (143 µA).

Life = 2,000 mAh ÷ 0.143 mA ≈ 13,955 h ≈ **581 days** (about 19 months).

Active current is about 80 mA and sleep current is about 10 µA, a ratio of 8,000 to 1. That ratio is the reason months of battery life are possible at all. Notice also that the active term (0.133 mA) is more than 13 times the sleep term (0.010 mA): **active time, not sleep current, dominates** this design.

Worked Example 7.2: A realistic four-phase profile

Measurements of the demo node's power profile show four phases per cycle: wake and initialization (45 mA for 15 ms), sensor sampling (68.5 mA for 120 ms), BLE broadcast (84 mA for 250 ms), and deep sleep (10 µA for the sleep interval S). Estimate life for S = 30 s with a 2,000 mAh cell and 15 percent derating.

Active charge per cycle = (45 × 0.015) + (68.5 × 0.120) + (84 × 0.250) = 0.675 + 8.220 + 21.000 = **29.895 mA·s**.

Sleep charge per cycle = 0.010 mA × 30 s = 0.300 mA·s.

Cycle time = 30 + 0.385 = 30.385 s.

Average current = (29.895 + 0.300) ÷ 30.385 ≈ **0.994 mA**.

Life = 2,000 × 0.85 ÷ 0.994 ≈ 1,710 h ≈ **71 days**.

Interpretation. This estimate is about one eighth of Worked Example 7.1's, for the same sleep interval. The difference comes almost entirely from the active window (385 ms instead of 50 ms) and from derating. Battery-life estimates are only as good as their measured phase currents and durations, which is why engineers profile real hardware with a power analyzer. Figure 7.3 shows how life grows as the sleep interval increases.

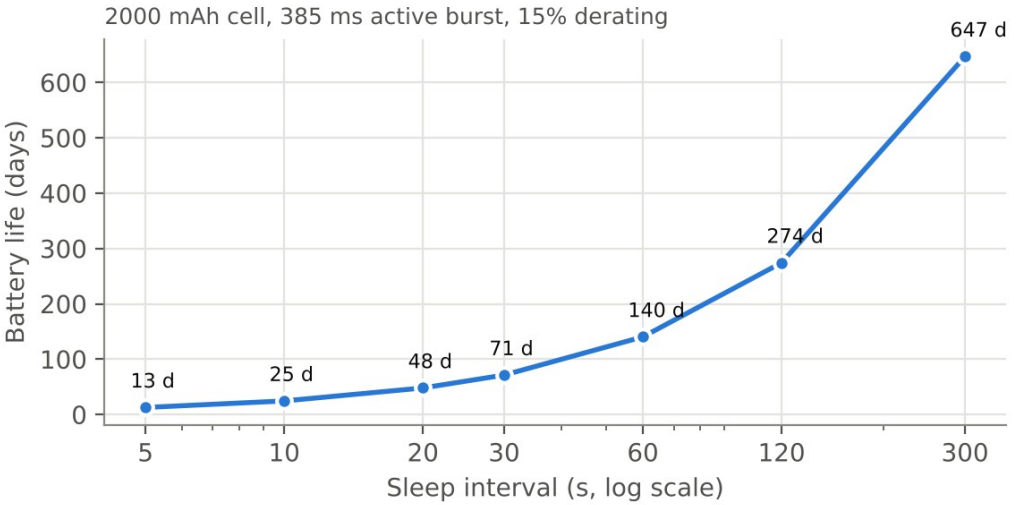


Figure 7.3 Estimated battery life versus sleep interval for the four-phase profile of Worked Example 7.2 with a 2,000 mAh cell.

Two practical cautions apply. First, the sample interval is a real design decision, not an arbitrary constant: in Worked Example 7.1, sampling every 10 s instead of every 30 s cuts battery life by roughly two thirds. Second, a small coin cell such as a CR2032 (about 220 mAh) has high internal resistance and cannot reliably deliver pulses of 80 mA or more; its

voltage sags and the MCU may reset. Coin-cell designs need low-peak-current radios such as the nRF52840, buffer capacitors, or both.

7.4 Tier 2: the proximity network

Choosing the local wireless protocol

Table 7.5 Local wireless protocol options for the demo

Protocol	Standard	Band	Data rate	Battery life	Topology
BLE 5.0	Bluetooth (802.15.1 origin)	2.4 GHz	1 to 2 Mbps	Months to years	Star, mesh
Wi-Fi 4/5	802.11 b/g/n/ac	2.4 / 5 GHz	54 to 433 Mbps	Days to weeks	Star (access point)
Zigbee 3.0	802.15.4	2.4 GHz	250 kbps	Years	Mesh
LoRaWAN	LoRa chirp spread spectrum	Sub-GHz	0.3 to 50 kbps	5 to 10 years	Star of stars

The rule of thumb from Chapter 3 applies: *range and speed cost power; pick the slowest, shortest-range protocol that still meets the application's latency need, and do not reach for Wi-Fi just because it is familiar.* The demo uses **BLE 5.0** with both advertising and GATT, which maximizes sensor-node battery life and delegates all long-range IP work to the gateway.

The best protocol is always contextual. If this were a smart irrigation valve on a 40-acre (16-hectare) farm instead of a desk demo, range would dominate and **LoRaWAN** would win.

Two ways to talk southbound: advertising and GATT

Table 7.6 BLE advertising versus connected GATT

Property	Passive advertising (broadcast)	Connected GATT
Analogy	A radio broadcast: anyone can listen, no one replies	A phone call: two-way and reliable
Reliability	No acknowledgment; packets can be missed	Acknowledged link-layer delivery
Power	Lowest	Higher; the link must be maintained
Security	Unencrypted by default	Pairing and AES-128 CCM encryption
Direction	One-way (node to any scanner)	Two-way (reads, writes, notifications)
Demo use	Periodic sensor telemetry	Actuator commands (fan, relay, LED)

Telemetry uses advertising because a missed temperature reading is replaced by the next one. Actuation uses connected GATT because a dropped "turn off the relay" command is unacceptable.

BLE parameters and the GATT database

The demo node advertises on the three primary advertising channels (37, 38, and 39) about once per second at +3 dBm, under the name `ESP32-NODE-01`, announcing the standard **Environmental Sensing** service. When the gateway connects, it uses the faster LE 2M physical layer, a 15 ms connection interval, and a 247-byte maximum transmission unit (MTU). Table 7.7 shows the GATT database.

Table 7.7 GATT database of the demo node (design reference)

Service (UUID)	Characteristic (UUID)	Properties	Encoding
Environmental Sensing (0x181A)	Temperature (0x2A6E)	Read, Notify	int16, units of 0.01 °C
	Humidity (0x2A6F)	Read, Notify	uint16, units of 0.01 % RH
	Pressure (0x2A6D)	Read	uint32, units of 0.1 Pa
Battery (0x180F)	Battery Level (0x2A19)	Read	uint8, percent
Device Information (0x180A)	Model Number (0x2A24), Firmware Revision (0x2A26)	Read	Text
Custom Actuation (0xFF00)	Cooling Fan PWM (0xFF01)	Read, Write	uint8, 0 to 100 %
	Relay Switch (0xFF02)	Read, Write	0x00 = OFF, 0x01 = ON
	LED State (0xFF03)	Read, Write	3 bytes (R, G, B)

Standard 16-bit UUIDs such as 0x181A are short forms of 128-bit UUIDs built on the Bluetooth base UUID (`0000xxxx-0000-1000-8000-00805F9B34FB`). Custom services in production should use randomly generated 128-bit UUIDs to avoid collisions.

Worked Example 7.3: Decoding BLE sensor bytes

(a) The Temperature characteristic (int16, 0.01 °C) returns the bytes `74 09`. BLE uses **little-endian** byte order (least significant byte first). What is the temperature?

Value = 0x0974 = (9 × 256) + 116 = 2,420. Temperature = 2,420 × 0.01 = **24.20 °C**.

(b) For compact advertising, the node also defines a 12-byte packed structure: `node_id` (uint8), `temp_c_x10` (int16), `hum_pct_x10` (uint16), `pres_hpa` (uint16), `vib_mg` (int16), `battery_pct` (uint8), and `crc16` (uint16). Check the length: 1 + 2 + 2 + 2 + 2 + 1 + 2 = **12 bytes**. Decode the little-endian packet `01 F5 00 46 02 F5 03 0C 00 5E` followed by a 2-byte CRC.

Field	Bytes	Value	Engineering value
node_id	01	1	sensor_node_01
temp_c_x10	F5 00	0x00F5 = 245	24.5 °C
hum_pct_x10	46 02	0x0246 = 582	58.2 % RH
pres_hpa	F5 03	0x03F5 = 1,013	1,013 hPa
vib_mg	0C 00	0x000C = 12	12 mg (0.012 g)
battery_pct	5E	94	94 %

The demo's live advertising frame (Proximity page) uses a related 15-byte layout: the flags, then a service-data field for 0x181A carrying temperature, humidity, pressure, and battery voltage as little-endian 16-bit values, for example `02 01 06 0B 16 1A 18 74 09 02 14 96 27 8B 01`. The gateway verifies the CRC before trusting the data. Compared with sending the same values as JSON text (about 90 bytes), the packed structure is about seven times smaller, which saves radio time and battery energy.

7.5 Tier 3: the edge gateway

Gateway hardware and software

Table 7.8 Gateway hardware and software

Item	Option A (default)	Option B (industrial)
Hardware	Raspberry Pi 4 Model B (4 GB): Broadcom BCM2711, quad-core Cortex-A72 at 1.5 GHz	Advantech UNO-220 industrial gateway
Operating system	Raspberry Pi OS Lite 64-bit or Ubuntu Server 22.04 LTS (headless)	Same
Software	Python 3 (asyncio) with **bleak** for BLE and an MQTT client (AWS IoT Device SDK or paho-mqtt); SQLite buffer	Same
Southbound	Onboard BLE 5.0 adapter (`hci0`, BlueZ stack)	Same
Northbound	Dual-band Wi-Fi 802.11ac or Gigabit Ethernet to AWS IoT Core	Same

Think of the gateway as a translator between two people who speak different languages at different volumes: it listens quietly in BLE, then speaks securely in MQTT over TLS so that the cloud can hear it from anywhere.

The gateway pipeline

1. **Scan and connect.** The bleak client scans for advertising nodes and connects to those that need GATT sessions.

2. **Receive.** Advertisements and GATT notifications arrive as raw bytes.
3. **Unpack and validate.** The gateway unpacks binary fields (Python's `struct.unpack` with little-endian formats), checks the CRC, and rejects implausible values.
4. **Translate.** It builds a normalized JSON message:

```
{
  "device_id": "sensor_node_01",
  "timestamp": 1775736000,
  "temp": 24.5, "humidity": 58.2, "pressure": 1013,
  "vibration": 0.012, "battery": 94, "rssi": -62,
  "actuator_state": { "relay": "OFF", "fan_pwm": 0, "led_mode": "SOLID" }
}
```

5. **Publish.** If the wide-area link is up, it publishes to `iot/telemetry/<device_id>` with QoS 1.
6. **Store-and-forward.** If the link is down, it inserts the message into a local **SQLite** database in write-ahead-logging (WAL) mode. When the link returns, it **drains** the buffer to AWS IoT Core in timestamp order, then resumes live publishing.
7. **Relay commands.** It subscribes to `iot/commands/<device_id>` and translates each command into a GATT write, then publishes an acknowledgment.

Worked Example 7.4: Sizing the store-and-forward buffer

Three nodes each report every 10 s. The SQLite buffer can hold 250,000 records. (a) What is the arrival rate? (b) How long an outage can the buffer absorb? (c) If the gateway drains 250 records per second on reconnection while new records keep arriving, how long does it take to empty a full buffer?

(a) Rate = 3 nodes ÷ 10 s = **0.3 records/s** (18 per minute).

(b) Capacity = 250,000 ÷ 0.3 ≈ 833,000 s ≈ **231 hours** (about 9.6 days).

(c) Net drain rate = 250 − 0.3 ≈ 249.7 records/s. Time = 250,000 ÷ 249.7 ≈ **1,001 s** (about 17 minutes).

Interpretation. Buffer capacity depends on the arrival rate. The same 250,000 records would last only about 3 hours for 100 nodes reporting every 5 seconds (20 records/s). Designers must state the assumed rate whenever they quote an outage duration, and testers should check that quoted figures are consistent with the measured rate.

7.6 Northbound security and provisioning

The gateway connects to AWS IoT Core with **MQTT over mutual TLS** on port **8883**, using QoS 1 (at-least-once delivery) and 2048-bit RSA or ECC (secp256r1) keys. Chapter 6 explains the four-step mTLS handshake.

Provisioning the gateway

1. Create an **AWS IoT Thing** (for example `Gateway_Pi4_Demo`) in the AWS IoT console.

2. Generate and download the **device certificate**, **private key**, and **Amazon Root CA 1** certificate.
3. Attach a **least-privilege IoT policy** scoped to the Thing's own client ID and the project's topic namespace (Chapter 6, Section 6.5).
4. Install the three files on the gateway with restrictive file permissions, ideally storing the private key in a TPM or secure element.

Gateway code: from reading to MQTT publish

```
from awsiot import mqtt_connection_builder
from awscrt import mqtt
import json

mqtt_conn = mqtt_connection_builder.mtls_from_path(
    endpoint="a1xxxxxxxxxxx-ats.iot.us-east-1.amazonaws.com",
    cert_filepath="Gateway_Pi4.cert.pem",        # (1) identity
    pri_key_filepath="Gateway_Pi4.private.key",  # (1) identity
    ca_filepath="AmazonRootCA1.pem",             # (1) trust anchor
    client_id="Gateway_Pi4_Demo")
mqtt_conn.connect().result()

payload = {"device_id": "sensor_node_01", "temp": 24.5,
           "humidity": 58.2, "timestamp": 1775736000}
mqtt_conn.publish(
    topic="iot/telemetry/sensor_node_01",        # (2) topic convention
    payload=json.dumps(payload),
    qos=mqtt.QoS.AT_LEAST_ONCE)                  # (3) delivery guarantee
```

1. The certificate, private key, and root CA are the gateway's entire mTLS identity.
2. A consistent topic convention (`iot/telemetry/<node>`) keeps each device's traffic separate and matches the policy.
3. QoS 1 trades a little bandwidth for a delivery guarantee. With QoS 0, a dropped packet would silently lose that reading forever.

7.7 Tier 4: the AWS cloud back end

Rules, storage, and alerts

A single IoT rule, `TelemetryRoutingRule`, evaluates every telemetry message:

```
SELECT device_id, temp, humidity, pressure, vibration, battery,
       timestamp() AS ingest_time
FROM 'iot/telemetry/+'
WHERE temp > 0
```

The rule has two actions (Table 7.9).

Table 7.9 Rule actions in the demo cloud

Target	Service	Configuration
1: Storage	Amazon DynamoDB	Table `SensorTelemetry`; partition key `device_id` (string); sort key `timestamp` (number)
2: Alerting	AWS Lambda → Amazon SNS	Function `HighTempAlertFn` checks `temp > 30.0 °C` and publishes an alert to the `IoTAlerts` topic (email, webhook, queue, or push)

Choosing `device_id` as the partition key and `timestamp` as the sort key lets the app query "all readings for node 1 in the last 24 hours" efficiently, because DynamoDB stores each device's readings together, sorted by time.

The Lambda function's logic is short:

```
def lambda_handler(event, context):
    temp = event.get("temp", 0)
    if temp <= 30.0:
        return {"statusCode": 200, "body": "Below threshold"}
    sns.publish(TopicArn=os.environ["SNS_TOPIC_ARN"],
                Subject=f"[IoT Alert] High Temp: {event.get('device_id')}",
                Message=f"Temperature {temp} C. Inspect node and activate
cooling.")
    return {"statusCode": 200, "body": "SNS alert dispatched"}
```

Device shadow

Each node also has a **device shadow** (Chapter 4). The app writes the desired fan speed or relay state; the gateway applies the change when the node is reachable and updates the reported state. The shadow is what makes reverse control reliable over an unreliable BLE and internet path.

Two paths to the phone

Table 7.10 Push and pull paths from cloud to app

Path	Flow	Purpose
Real-time push	IoT Core → MQTT over WebSocket Secure (port 443) to the app, or an IoT rule → AWS AppSync (GraphQL subscriptions)	"What is happening now": low latency, small payloads, no polling
Historical pull	App → Amazon API Gateway (`GET /telemetry/{node_id}`) → Lambda → DynamoDB query → JSON	"What happened yesterday": larger queries that need not be instant

Almost every real-time dashboard uses this split. A stock-trading app, for example, streams live prices but loads historical charts on request.

Worked Example 7.5: How fast does the table grow?

Three nodes each write one item every 10 s. Each item is about 200 bytes. How many items and how much data accumulate per day and per year?

Items per day = 3 × (86,400 ÷ 10) = **25,920**.

Data per day = 25,920 × 200 bytes ≈ **5.2 MB**; per year ≈ 1.9 GB.

At this scale storage is trivial. For 10,000 nodes, the same design would write 86.4 million items per day. The cost would then be dominated by write operations, which is why large fleets aggregate at the edge, archive old data to S3, and use time-to-live settings to delete expired items automatically.

7.8 Tier 5: the presentation layer

The app has three screens, each with one job:

1. **Device discovery:** shows each node's online or offline status, queried from DynamoDB through API Gateway.
2. **Live dashboard:** gauges and charts of real-time data received over MQTT/WebSocket, authenticated with **Amazon Cognito**.
3. **Actuation and control:** sends commands through AWS IoT Core to the target BLE device: relay on or off, fan PWM, LED mode, and the node's sleep interval.

The reference Android implementation uses Kotlin with Jetpack Compose and a model-view-intent (MVI) architecture; the web dashboard uses a canvas-based chart engine. Screens are a styling reference; each team designs its own.

Several design details deserve attention:

- **No permanent secrets in the app.** Cognito exchanges the user's identity for temporary AWS credentials that expire after about 60 minutes, and an IAM policy lets the app subscribe to telemetry and publish only to command topics.
- **Reconnection with backoff.** After a lost connection, the app retries after 1 s, then 2, 4, 8, 16, and up to 32 s, with random jitter, and then fetches missed data through the REST path.
- **Optimistic control with rollback.** When the user toggles the relay, the switch moves immediately; if no acknowledgment arrives within 3.5 s, the app rolls the switch back and shows an alert.
- **Automatic climate control with hysteresis.** In *Auto* mode, cooling switches on at 26.0 °C and off at 24.0 °C (Chapter 2). Separately, the cloud raises an alarm above 30.0 °C.

A command message looks like this:

```
{
  "command_id": "c-7f3a",
  "device_id": "sensor_node_01",
```

```
  "timestamp": 1775736042,
  "action": "SET_ACTUATOR",
  "params": { "relay": "ON", "fan_pwm": 60, "led_mode": "PULSING",
"sleep_s": 10 },
  "source": "android_app",
  "qos": 1
}
```

7.9 Putting it together: journeys and latency

One reading's uplink journey

1. The ESP32 wakes, samples the DHT22 over its data pin, and broadcasts a BLE advertisement (unencrypted by design; Chapter 6).
2. The gateway's bleak scanner receives it, unpacks the bytes, validates the CRC, and builds JSON.
3. The gateway publishes to `iot/telemetry/sensor_node_01` over MQTT with mTLS on port 8883.
4. AWS IoT Core authenticates, authorizes, and passes the message to the rules engine, which writes it to DynamoDB and, if the temperature exceeds 30 °C, invokes the alert Lambda.
5. IoT Core pushes the message to the subscribed app over MQTT on WebSocket Secure (port 443), and the gauge updates.

One command's downlink journey

Figure 7.4 shows a fan command traveling from the app to the motor and the acknowledgment returning.

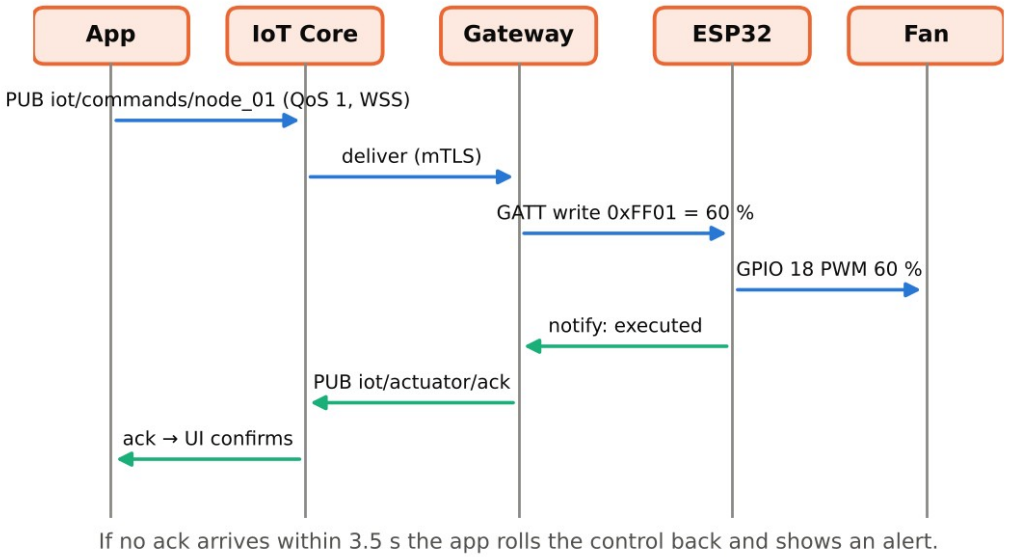


Figure 7.4 Downlink sequence for a fan command, with acknowledgment.

For each arrow in both journeys, students should be able to name the **protocol** and the **security mechanism** in play. One hop, BLE advertising, has no encryption by design; the trade-off is acceptable for a non-sensitive temperature over a few meters, but not for a command.

The latency budget

A **latency budget** allocates the end-to-end delay target among the hops. The design target for the demo is an uplink of under 120 ms (Table 7.11 and Figure 7.5).

Table 7.11 Uplink latency budget of the demo design

Hop	Budget (ms)
1. Perception → proximity (I^2C read and BLE transmission)	12 to 18
2. Proximity → edge gateway (BlueZ and bleak processing)	8 to 15
3. Gateway → AWS IoT Core (mTLS, port 8883)	22 to 38
4. Rules engine → Lambda / DynamoDB	15 to 25
5. IoT Core → mobile app (MQTT over WSS, port 443)	18 to 24

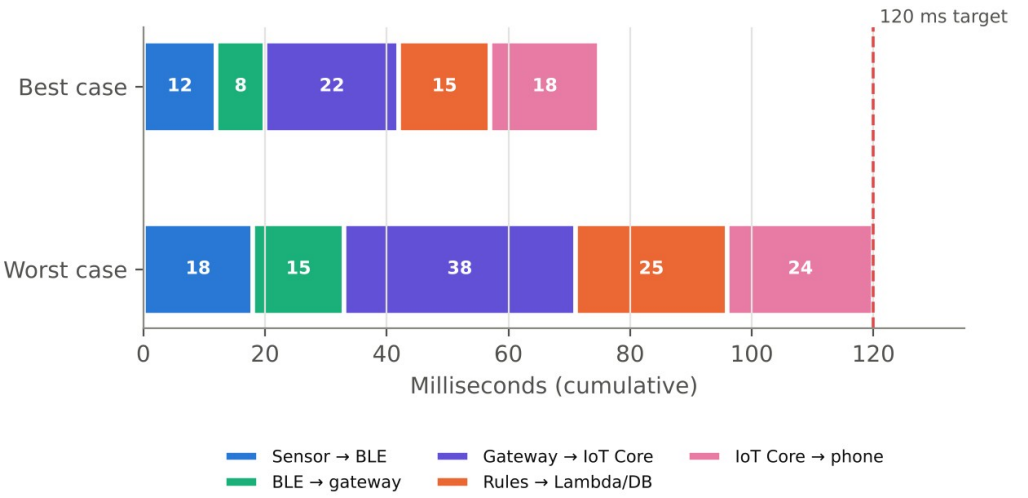


Figure 7.5 Cumulative uplink latency: best and worst case per hop against the 120 ms target.

Design review question. Add the best-case and worst-case values in Table 7.11. Does the worst case still meet the 120 ms target, and how much margin is left? Which hop has the largest share and the most variability, and why? (Hint: which hop crosses the public internet?) A good engineer checks whether a design's numbers are internally consistent before trusting them.

7.10 System bring-up in four phases

Real systems are brought up one tier at a time, with a clear check at each phase, so that a team that is stuck can say exactly which phase failed rather than "nothing works" (Table 7.12).

Table 7.12 Four-phase system bring-up

Phase	Focus	Action	Verification
1	Hardware and BLE	Power on the ESP32 node	Status LED shows advertising; node visible in a BLE scanner app such as nRF Connect
2	Gateway ingestion	Start the gateway Python service	Console logs show incoming BLE data being parsed
3	AWS cloud	Open the AWS IoT MQTT test client and subscribe to `iot/#`	JSON telemetry appears; DynamoDB rows append with timestamps
4	Mobile app	Launch the app	Live chart plots; relay or LED toggle responds within the acknowledgment timeout

Experienced engineers test the full chain the night before any demonstration: BLE pairing problems and Wi-Fi networks with captive portals are the two most common last-minute failures.

7.11 Testing an IoT system

Principles of test evidence

A test is only as useful as its record. Every test in Lab 7 records:

- **Test ID and purpose:** what property is being checked.
- **Conditions:** date, time zone, browser or hardware version, configuration (for example simulation speed), and who ran the test.
- **Expected result:** taken from the design (the slides, this chapter, or a specification) *before* running the test.
- **Observed result, with numbers:** not "it worked" but "the fan reached 60 percent PWM 1.4 s after the click."
- **Pass or fail**, and a **figure reference** to a screenshot or log excerpt.

Good measurement practice includes repeating measurements and reporting the **mean**, **minimum**, and **maximum**; noting whether timestamps are in **UTC** or local time (logs often mix them); and resetting the system to a known baseline before each test.

Worked Example 7.6: Summarizing latency measurements

A tester runs an end-to-end latency probe five times and records totals of 142, 151, 138, 160, and 149 ms.

Mean = (142 + 151 + 138 + 160 + 149) ÷ 5 = 740 ÷ 5 = **148 ms**. Minimum = **138 ms**. Maximum = **160 ms**. Range = 22 ms.

If the per-hop breakdown shows the gateway-to-cloud hop averaging 45 ms, that hop holds the largest share (about 30 percent) and is the first candidate for optimization, for example by choosing a closer cloud region or keeping the TLS session open.

Recording defects

When observed behavior differs from expected behavior, testers record a **defect**:

Table 7.13 Defect log fields

Field	Content
ID	D-01, D-02, …
Location	Which tier, page, panel, or log
Steps to reproduce	Exact actions, starting from a known baseline
Expected vs. observed	What should happen, and what did happen, with numbers

Field	Content
Severity	High (wrong safety or security behavior, data loss), medium (wrong values or misleading display), low (cosmetic)
Real-world impact	What would happen if a real business relied on it
Suggested fix	A concrete engineering change

Analyzing a closed control loop

When a cooling actuator is engaged, whether the temperature actually falls below a threshold depends on three things:

- **Set points and hysteresis:** where the controller switches on and off (for example on at 26 °C, off at 24 °C, alarm at 30 °C).
- **Actuator capacity:** how much heat the fan or compressor can remove per minute compared with the heat being added. If the heat load exceeds the cooling capacity, the temperature keeps rising even with the fan at 100 percent.
- **Dynamics:** delays between the command, the actuator's response, and the sensor's response (Chapter 2's latency constraint).

Plotting temperature against time after the cooling command, and marking the 30 °C and 24 °C lines, shows whether the loop succeeds, how long it takes, and whether it oscillates.

7.12 From demo to real deployment

The demo handles three nodes on a desk. Three discussion questions reveal what changes in a real deployment:

1. **Scale.** What breaks first with 10,000 nodes? Manual certificate creation, per-device rules, a single gateway's BLE capacity (the demo allows only three simultaneous connections), DynamoDB write costs, and alert floods all need redesign: fleet provisioning, many gateways, edge aggregation, and alert deduplication.
2. **Data type.** What changes if the sensor sends a video frame instead of a temperature? BLE cannot carry video; the node would need Wi-Fi or 5G, mains power, edge AI to send only events, and the cloud path would use streaming services rather than MQTT messages.
3. **Network failure.** If AWS were unreachable for an hour, where would buffering and local logic belong? At the gateway: store-and-forward for data, and local rules (for example the hysteresis controller) so that cooling continues without the cloud.

7.13 IoT application case: NxEco compared with the demo system

Comparing a commercial product with the class demo system shows how design choices follow from constraints.

Table 7.14 The demo system and NxEco compared

Design question	Demo system (Chapter 7)	NxEco controller
Power source of the device	Battery (LiPo or coin cell)	Mains (24 V AC adapter)
Local radio	BLE 5.0	2.4 GHz Wi-Fi or Ethernet
Separate gateway?	Yes (Raspberry Pi)	No; the home router is enough
Why	Battery node cannot afford Wi-Fi and TLS	Mains power makes Wi-Fi and TLS affordable
Cloud	Managed IoT platform (AWS IoT Core)	Company-operated servers on rented infrastructure
Main sensing	Physical sensors (temperature, humidity, vibration)	Weather and ET data from the cloud, optional rain sensor
Main actuators	Fan (PWM), relay, LED	Up to 36 irrigation valves (24 V AC) and a master valve
Decision timing	Seconds (overheat alarm, hysteresis)	Once per day (watering budget)
Offline behavior	Gateway store-and-forward	Controller runs stored schedule as a timer
Users	One app	App, two web portals, voice assistants, delegated access

The most important lesson is the **gateway question** from Section 7.1. The demo system needs a gateway because its sensor nodes run on batteries and cannot afford a Wi-Fi radio and TLS stack. The NxEco controller is plugged into the wall and sits near a router, so the expensive tier disappears. If NxEco added battery-powered soil-moisture probes in each zone, the controller itself would have to become a gateway: it would gain a BLE, Zigbee, or LoRa radio facing the probes (southbound) while keeping Wi-Fi facing the cloud (northbound), exactly the pattern of Chapter 4.

Worked Example 7.7: How much water is one zone using?

An NxEco zone waters a lawn with spray heads delivering a combined 10 liters per minute. In summer, the weather-adjusted schedule runs the zone 12 minutes per day (Worked Example 2.6); a fixed timer would run it 20 minutes every day.

Smart schedule: 12 min × 10 L/min = **120 L per day**.

Fixed timer: 20 min × 10 L/min = **200 L per day**.

Saving = 80 L per day, or 40 percent. Over a 120-day summer, that is 80 × 120 = **9,600 liters** for one zone. A 12-zone property would save on the order of 100,000 liters a season, which is why water utilities offer rebates for WaterSense controllers. The calculation also shows why a flow sensor would be valuable: without one, the controller cannot confirm that 10 L/min is really flowing, or detect a broken head that doubles it.

Chapter summary

- The demo system has **five tiers**: perception (ESP32/nRF52840 nodes), proximity (BLE 5.0), edge gateway (Raspberry Pi 4), cloud (AWS IoT Core, DynamoDB, Lambda, SNS), and presentation (web dashboard and Android app). The gateway exists because sensors cannot afford TLS.
- Hardware is chosen by **power, cost, and real-time headroom**, in that order.
- **Battery life** = capacity × derating ÷ average current; average current is the time-weighted sum of phase currents. Active time usually dominates.
- **BLE advertising** suits telemetry; **connected GATT** suits commands. Payloads are compact, little-endian, and CRC-checked.
- The gateway **translates** binary BLE data into JSON over **MQTT/mTLS**, **buffers** during outages, and **relays** commands.
- The cloud **rule** stores every reading in DynamoDB and triggers a **Lambda/SNS** alert above 30 °C; **push** and **pull** paths serve the app.
- The app uses **Cognito** temporary credentials, backoff reconnection, optimistic control with rollback, and hysteresis.
- **Latency budgets**, **four-phase bring-up**, disciplined **test records**, and **defect logs** turn a demo into an engineered system.

Key terms

acknowledgment timeout; actuator capacity; average current; bring-up; derating factor; defect log; duty cycle; Environmental Sensing service; exponential backoff; latency budget; little-endian; optimistic update; packed structure; partition key; power profile; provisioning; push/pull paths; sort key; store-and-forward; test evidence.

Review questions

Multiple choice

1. Why does the demo system include an edge gateway?
    a. To make sensors more accurate
    b. Because a sensor node cannot afford a full TLS stack and long-range radio
    c. To replace the cloud
    d. To charge the batteries
2. The first criterion for choosing the sensor-node MCU is:
    a. clock speed
    b. power efficiency
    c. screen size
    d. Wi-Fi support

3. A node draws 80 mA for 0.1 s and 0.01 mA for 9.9 s in every 10 s cycle. Its average current is about:
    a. 0.81 mA
    b. 8.0 mA
    c. 0.08 mA
    d. 80 mA
4. Which BLE mode should carry a “turn off the relay” command?
    a. Passive advertising
    b. Connected GATT
    c. Beacon broadcast
    d. Classic audio
5. The bytes `74 09` from an int16 characteristic in units of 0.01 °C represent:
    a. 29.80 °C
    b. 24.20 °C
    c. 7.49 °C
    d. 0.97 °C
6. During a WAN outage, the gateway:
    a. discards readings
    b. stores readings in a local SQLite buffer and forwards them later
    c. sends readings by BLE to the cloud
    d. turns off all nodes
7. The DynamoDB table uses `device_id` as partition key and `timestamp` as sort key so that:
    a. each device’s readings are stored together in time order for efficient range queries
    b. data is encrypted
    c. alerts are sent faster
    d. the table never grows
8. Which service gives the mobile app temporary AWS credentials?
    a. Amazon SNS
    b. Amazon Cognito
    c. AWS Lambda
    d. Amazon S3
9. In the app’s Auto mode, cooling turns off at:
    a. 30 °C
    b. 26 °C
    c. 24 °C
    d. 16 °C

10. In four-phase bring-up, which phase verifies that DynamoDB rows append with timestamps?
    a. Phase 1 b. Phase 2 c. Phase 3 d. Phase 4

Short answer

11. Explain why active time usually dominates battery life in a duty-cycled sensor node, using Worked Example 7.1.
12. Compare the push and pull paths from cloud to app and give one use for each.
13. For each arrow in the uplink journey, name the protocol and the security mechanism. Which hop has no encryption, and why is that acceptable?
14. Explain how set points, hysteresis, and actuator capacity together determine whether a cooling loop succeeds.

Applied problems

15. Using the four-phase profile of Worked Example 7.2 (active charge 29.895 mA·s per cycle, sleep current 0.01 mA, active window 0.385 s), estimate battery life for a 2,600 mAh 18650 cell at a 120 s sleep interval with 15 percent derating.
16. A gateway serves 40 nodes, each reporting every 5 s, and has a 250,000-record buffer. (a) How long an outage can it absorb? (b) Propose two design changes that would let it survive a 24-hour outage, and state the trade-off of each.
17. **NxEco case.** NxEco plans battery-powered soil-moisture probes in each zone. Describe how the controller's role in the architecture would change and which radio technologies it might add.

Lab 7A: Stress-test the demo system, Part A (Week 6)

Collaboration 6 and Assignment 3 (team test report, Part A). Teams run three guided tours and four fault-injection tests (T1 to T7), trace the data flow tier by tier, measure key values, and record everything in a team test log.

Demo system. https://liangli99.github.io/IoT-Demo/dashboard.html (desktop Chrome or Edge recommended; no login, hardware, or AWS account required). The dashboard has two control panels: **Pipeline Condition & Fault Injection** (Temp Spike, Toggle Relay, Toggle Gateway WAN, 50-Pkt Burst Ingest, Reset Normal Baseline) and **Automated Guided Demos & Tours** (three tours that advance with a floating HUD card for each step; press Esc to stop). The phone in Tier 5 has its own buttons: *Test Overheat Alert (>30°C)* and *Actuate ESP32 Fan (Downlink)*.

Team roles. Assign four roles and rotate the Operator after every two tests:

- **Test Operator:** drives the demo.
- **Recorder:** copies the text of every HUD card and fills in the test log.

- **Timekeeper:** times each step with a clock and the on-screen time.
- **Analyst:** checks the numbers against this chapter and the slides.

Before starting. Click *Reset Normal Baseline*, set speed to 1x, and record the date, browser, and time zone. Note whether each logged value is in UTC or local time.

Stage 1: trace the pipeline with the guided tours. For each tour, record what each HUD card says, which tier is highlighted, and what changes on the dashboard. Take at least one screenshot per test and number it as a figure.

- **T1 Full Architecture Tour (13 steps).** For each tier, record the hardware, protocol, and security mechanism. During steps 7 to 12, record the anomaly sequence: trigger temperature, Lambda function, phone display, actuator engaged by the closed loop, and how the alert clears. After the tour, read the **INGESTED** counter, wait exactly 60 s, and read it again to compute the ingestion rate in packets per minute. Repeat at 5x speed.
- **T2 Uplink: Sensor to Cell App (5 steps).** For each step, record the tier, device, protocol or port, data format, and MQTT topic. Check that the Tier 1 sensor-buffer temperature matches the phone. On the *Latency & Diagnostics* tab, run the end-to-end latency probe five times; record each hop and the total, compute mean, minimum, and maximum (Worked Example 7.6), and identify the hop with the largest share.
- **T3 Downlink: Cell App to Fan Actuation (5 steps).** Record the Tier 1 actuator panel (GPIO 18) first: state, PWM, and RPM. Run the tour (or press *Actuate ESP32 Fan* on the phone) and record, for each step, the MQTT topic, transport and port, GATT characteristic UUID, GPIO pin, PWM duty, RPM, and motor current. Time how long the fan panel takes to change after the click. On the *Reverse Actuation & Control* tab, copy the PUB → GW-ACK → ESP32-EXEC log lines, record the topic, JSON payload, QoS, and UUID, compute the PUB-to-EXEC time, and compare it with the round-trip time the dashboard reports.

Stage 2: inject faults and measure the response.

- **T4 Overheat alert and closed-loop cooling.** Reset, then press *Test Overheat Alert* (or *Temp Spike*). Record the peak temperature, phone banner, dashboard alert, relay/compressor state (GPIO 23), and number of alerts in 30 s. On the *AWS IoT Cloud & Rules* tab, identify the SQL rule and its two actions. Press *Actuate ESP32 Fan* and record the temperature every 10 s for 60 s. Then clear the alert, reset, and record which tiers return to normal.
- **T5 Gateway WAN outage (store-and-forward).** Record INGESTED and Edge Buffer. Click *Toggle Gateway WAN* and record WAN status, Edge Buffer, and INGESTED at 10, 20, and 30 s. Reconnect and record the flush message (packets replayed). Compare that number with what the T1 rate predicts for 30 s.
- **T6 50-packet burst (ingestion and storage).** Record INGESTED, click *50-Pkt Burst Ingest*, and record INGESTED again. On the cloud tab, examine the DynamoDB

`SensorTelemetry` table and record its partition key, sort key, and the timestamps of the burst rows.

- **T7 Battery life.** Open *Perception Layer → Power Profiling & Battery Longevity Calculator*. Record the projected life for (a) the 1,200 mAh LiPo at 10 s and 60 s sleep and (b) the CR2032 at 60 s sleep. **Verify one result by hand** using Section 7.3: average current = Σ(phase current × phase time) ÷ cycle time; life (h) = capacity × 0.85 ÷ average current.

Finally, on the *Fleet Telemetry & Charts* tab, click *Export Telemetry Dataset (CSV)* and keep the file for the report appendix and Part B.

Table 7.15 Team test log template

Test	Key values to record	Expected (design)	Observed (numbers)	Pass/ Fail	Evidence
T1	Hardware, protocol, security per tier; anomaly sequence; INGESTED over 60 s at 1x and 5x				Fig.
T2	Per-step tier, protocol/port, format, topic; sensor vs. phone temperature; 5 probes (mean/min/max, dominant hop)				Fig.
T3	Fan panel before/after; topic, UUID, GPIO, current; click-to-fan time; PUB−EXEC vs. reported RTT				Fig.
T4	Peak temperature; alerts; alerts in 30 s; SQL rule and actions; temperature every 10 s for 60 s; state after reset				Fig.
T5	INGESTED and Edge Buffer at 0/10/20/30 s; replayed vs. expected packets				Fig.
T6	INGESTED before/after; partition and sort keys; burst timestamps				Fig.
T7	Calculator results; hand calculation				Fig.

Assignment 3 report contents. (1) Test environment and roles; (2) results table for T1 to T7 with expected, observed, pass/fail, and figure references; (3) quantitative analysis with calculations: ingestion rate at 1x and 5x, latency statistics and dominant hop, expected versus replayed packets in the WAN outage, and battery life with hand verification; (4) four to five pages plus appendix (test log, screenshots, CSV), APA 7th edition, citing the demo as Li (2026).

Lab 7B: Verify, break, and redesign, Part B (Week 7)

Collaboration 7 and Assignment 4, item 1. Part B verifies what the dashboard claims in Part A, stresses the control loop, and redesigns the system for one real-world pressure.

1. **Cross-check the tours against the live panels.** A good tester does not trust a single screen. Compare:
   - the MQTT topic and GATT characteristic UUID on the downlink tour cards with the PUB and GW-ACK lines in the actuation log;
   - the reported round-trip time with the PUB-to-EXEC time measured in T3 and with the time the fan actually took to respond;
   - the Edge Buffer counter during the T5 outage with the number of packets flushed on reconnect;
   - the actuator used for cooling in the Full Architecture Tour (relay, GPIO 23) with the actuator the phone drives (fan, GPIO 18);
   - the state of every tier after *Reset Normal Baseline*;
   - the calculator's battery-life output in T7 with the hand calculation.

   Record **at least two defects** in a defect log (Table 7.13).
2. **Stress the closed control loop.** Reset, press *Test Overheat Alert*, and immediately press *Actuate ESP32 Fan*. Record temperature, phone status, and fan RPM every 5 s for 90 s, and count the alerts. Plot temperature against time from the log or a fresh CSV export, with lines at 30 °C and 24 °C. Does cooling bring the temperature below the alarm threshold and below the hysteresis off-threshold? Explain the result as a control-loop problem (Section 7.11).
3. **Redesign for one pressure.** Choose **scale** (for example 10,000 nodes), **battery life** (for example a five-year coin-cell target), or **network failure** (for example a 24-hour outage). State which tiers change, the trade-offs, and one security implication, supported by measured data from Parts A and B.

References and further reading

Amazon Web Services. (n.d.). *AWS IoT Device Shadow service*. AWS IoT Core Developer Guide. https://docs.aws.amazon.com/iot/latest/developerguide/iot-device-shadows.html

Borgini, J. (2025, July 14). *10 top design principles of IoT*. TechTarget. https://www.techtarget.com/iotagenda/tip/Top-8-IoT-design-principles-for-successful-product-creation

CodeLucky. (2026). *AWS IoT Core explained* [Video]. YouTube. https://www.youtube.com/watch?v=yJ-9uQ-p48Q

IBM Technology. (2024). *Securing your IoT devices* [Video]. YouTube. https://www.youtube.com/watch?v=7zWVxrjjIpE

Li, L. (2026). *IoT Education Course demo system* [Interactive web application]. GitHub Pages. https://liangli99.github.io/IoT-Demo/dashboard.html

NxEco. (n.d.-a). *NxEco smart watering* [Website]. https://www.nxeco.com

NxEco. (n.d.-b). *Smart controller specification* (Ver. 10.1) [PDF]. https://www.nxeco.com/Public/public/file/manual/Nxeco_Smart_Control_Spec_Ver10.1.pdf

OnLogic. (2023). *Connected devices at the edge using AWS IoT Greengrass* [Video]. YouTube. https://www.youtube.com/watch?v=zmJD_8KKxP0

CHAPTER 8

The Artificial Intelligence of Things (AIoT)

Learning Objectives

After studying this chapter, students will be able to:

1. Explain how AI and IoT combine into AIoT and how systems move from "smart" (fixed rules) to "intelligent" (learning and predicting).
2. Compare the places where AI can run: in the chip, at the edge device, at the gateway, and in the cloud.
3. Describe the TinyML workflow and calculate the effect of model quantization on memory.
4. Explain why cloud AI still handles most enterprise IoT workloads, and describe hybrid patterns such as federated learning.
5. Match core AI technologies (computer vision, predictive analytics, natural language processing, reinforcement learning, anomaly detection) to IoT applications and business value.
6. Evaluate AIoT models using precision and recall, and identify the main implementation challenges.
7. Recommend where AI should run for a business scenario.

Where this fits: Week 7, Video 1. This chapter supports the "where AI runs" requirement of the Assignment 4 business proposal and the AIoT enhancement in the Week 1 teardown.

Opening case: listening to machines

Escalators, elevators, pumps, and wind turbines all vibrate. A healthy machine has a characteristic vibration signature; a bearing that is beginning to wear adds small, high-frequency components long before anyone can hear or feel them. For decades, maintenance teams replaced parts on fixed schedules, which wasted good parts, or repaired them after failure, which caused expensive downtime and sometimes danger.

Today, a vibration sensor with a small processor can run a machine-learning model that has learned what "normal" sounds like for that specific machine. When the pattern drifts, the device sends an alert, perhaps three weeks before failure. Meanwhile, the cloud collects data from thousands of similar machines worldwide and trains better models that are pushed

back to every sensor. This is the **Artificial Intelligence of Things (AIoT)**: IoT provides the senses, AI provides the judgment, and together they solve problems before people notice them.

8.1 IoT plus AI equals AIoT

If IoT is the **nervous system** that senses the world, artificial intelligence is the **brain** that decides what to do about it (AI Smartbytes, 2025; Kirvan et al., 2025).

- **IoT contributes** connectivity, data collection, and real-time monitoring.
- **AI contributes** edge computing intelligence, predictive analytics, and real-time decision making.
- **AIoT integration** creates complete proactive ecosystems capable of autonomous decisions.

In the mid-2020s, leading technology companies moved from talking about "connected devices" to "autonomous agents": systems that perceive, decide, and act with limited human supervision. AIoT is the physical foundation of that shift, sometimes called **physical AI**.

8.2 From smart to intelligent

Traditional IoT is **linear**: sense, transmit, store, analyze, and then (often days later) a person acts. AI shortens this to **sense, analyze, act** in real time (Chapter 1, Figure 1.3).

Table 8.1 Smart versus intelligent IoT

Dimension	Smart IoT	Intelligent IoT (AIoT)
Logic	Predefined rules written by engineers	Models learned from data
Behavior	Executes pre-programmed actions; reactive	Analyzes, predicts, and adapts; proactive
Example	"If temperature > 30 °C, turn on the fan"	"Temperature is rising faster than usual for this time of day and occupancy; pre-cool now"
Improvement over time	Only when engineers change the rules	Continuously, as models are retrained on new data
Limitation	Static; cannot handle situations nobody anticipated	Needs good data; harder to explain and verify

AI does not replace rules entirely. Safety-critical limits, such as "never exceed 120 °C," remain hard-coded rules, while AI optimizes behavior within those limits.

8.3 AIoT system integration

A production AIoT architecture integrates every layer, from sensor silicon to cloud AI, into a coherent system with three kinds of connection:

- **Data contracts** define exactly what each layer sends: field names, units, sampling rates, and quality flags. A model trained on temperature in °C will fail silently if a new sensor reports °F.
- **Feedback loops** carry decisions down to actuators and carry outcomes back up for learning. Did the predicted failure happen? Was the alert useful?
- **Operational controls** manage models like software: versioning, testing, staged rollout, monitoring, and rollback. This discipline is called **MLOps** (machine-learning operations).

8.4 Where AI lives

AI can run at four places in an IoT system (Figure 8.1). Moving left brings lower latency, better privacy, less bandwidth, and offline operation; moving right brings more computing power, more data, and fleet-wide learning.

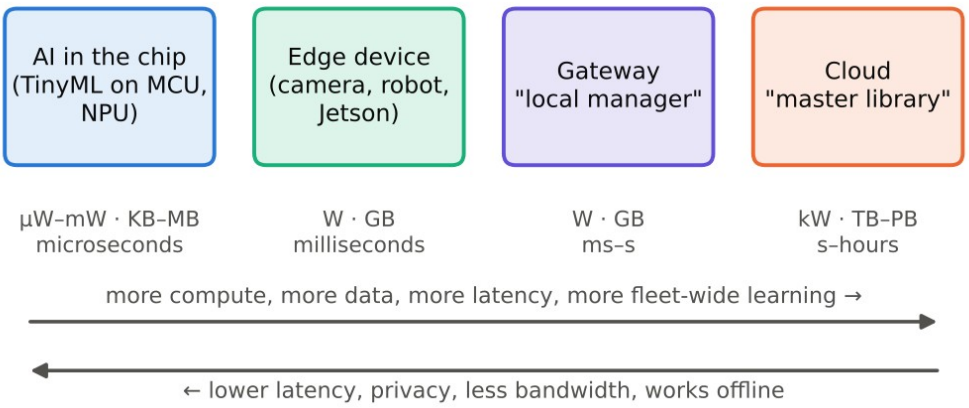


Figure 8.1 Where AI can run in an IoT system, from microwatt microcontrollers to kilowatt cloud clusters.

AI in the chip

AI is increasingly embedded directly in silicon:

- **Microcontrollers with machine-learning support.** Boards such as the Arduino Nano 33 BLE Sense and ESP32-S3-based modules run small neural networks with libraries such as **LiteRT for Microcontrollers** (formerly TensorFlow Lite for Microcontrollers) (Google for Developers, 2026).

- **Neural processing units (NPUs).** Dedicated silicon for the matrix mathematics of neural networks, such as Apple's Neural Engine in phones and Google's Edge TPU in Coral modules. Many new microcontrollers now include small NPUs.
- **Systems-on-chip (SoCs).** Modules such as NVIDIA Jetson for robotics and vision, and Qualcomm platforms for mobile and industrial devices, combine CPUs, GPUs, and AI accelerators.

These chips have severely limited memory and compute compared with cloud GPUs, so models must be compressed before deployment. This is a genuine engineering and business trade-off, and one of the fastest-growing areas of the AI industry.

Edge AI and TinyML

Edge AI runs inference on or near the device. **TinyML** is the subset that runs on microcontrollers with kilobytes of memory and milliwatts of power. Its key advantages are:

- **Latency:** no waiting for a cloud round trip.
- **Bandwidth:** only results, not raw data, are transmitted.
- **Privacy:** raw audio, video, or health data never leaves the device.
- **Availability:** works offline.

Typical TinyML tasks include keyword spotting ("Hey device"), gesture recognition from accelerometers, vibration anomaly detection, and simple image classification such as person detection.

The gateway as "local manager"

A gateway acts as the **local manager** for a cluster of devices. It aggregates data from many sensors and runs larger models than a microcontroller could, for example correlating vibration, temperature, and current from all machines on a production line. It needs more power than a sensor but far less than a data center, and it transmits only vital insights to the cloud.

Cloud AI as "master library"

The cloud is the **master library**. It does not handle millisecond decisions; it handles long-term learning. It takes data from millions of devices to train and sharpen models, which are then pushed back to the edge. In a cloud-AI solution, the device remains relatively "thin": it collects and transmits data, and the intelligence lives in the cloud.

```
IoT device (sensors/actuators)
    |  data transmission: MQTT, HTTPS, CoAP
Gateway / edge node (optional preprocessing)
    |  network: Wi-Fi, 4G/5G, LoRaWAN, Ethernet
Cloud IoT platform (AWS IoT Core, Azure IoT Hub, partner brokers)
    |
AI/ML services (training, inference, analytics)
```

```
    |
Business applications (dashboards, alerts, automation)
```

The key distinction from edge AI: in cloud AI, *the device is a data source; the cloud is the brain.*

Table 8.2 Comparing AI locations

Location	Typical hardware	Power	Memory	Response time	Best for
In the chip (TinyML)	MCU, small NPU	µW to mW	KB to MB	Microseconds to ms	Wake words, gestures, simple anomalies
Edge device	Smart camera, robot, Jetson module	Watts	GB	Milliseconds	Vision, robotics, real-time control
Gateway	Raspberry Pi, industrial PC with accelerator	Watts	GB	ms to seconds	Multi-sensor correlation, site-level optimization
Cloud	GPU and CPU clusters	Kilowatts	TB to PB	Seconds to hours	Training, fleet learning, long-term analytics

8.5 The TinyML workflow

Deploying AI to a device follows a repeatable workflow (Figure 8.2):

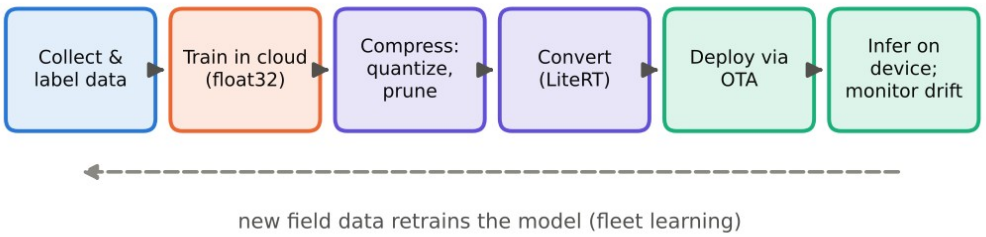


Figure 8.2 The TinyML workflow. Field data flows back to retrain the model.

1. **Collect and label data** from real devices in real conditions: normal operation and, ideally, examples of faults.
2. **Train in the cloud** with full-precision (32-bit floating-point) arithmetic on GPUs.
3. **Compress** the model:
   - **Quantization** stores weights as 8-bit integers instead of 32-bit floats, cutting memory by about four times with a small loss of accuracy.

   - **Pruning** removes connections that contribute little.
   - **Knowledge distillation** trains a small "student" model to imitate a large "teacher" model.
4. **Convert** the model to a format the device runtime understands (for example LiteRT for Microcontrollers).
5. **Deploy** it to the fleet through secure OTA updates (Chapter 5).
6. **Infer on the device and monitor**: track accuracy and **drift** (when real-world data slowly changes so the model becomes less accurate) and collect new data to retrain.

Worked Example 8.1: Will the model fit on the microcontroller?

A vibration classifier has 250,000 parameters (weights). The target ESP32 has 520 KB of SRAM, of which about 200 KB is available after the RTOS, BLE stack, and buffers. Will the model fit (a) in 32-bit floating point and (b) quantized to 8-bit integers?

(a) Float32 uses 4 bytes per parameter: 250,000 × 4 = 1,000,000 bytes ≈ **977 KB**. It does not fit in RAM (and would also consume a large part of flash).

(b) Int8 uses 1 byte per parameter: 250,000 × 1 = 250,000 bytes ≈ **244 KB**. Weights can be stored in flash, so this fits in flash, but the working memory for activations (the intermediate results) must still fit in the 200 KB of RAM. If it does not, the engineers must reduce the model's layer sizes or choose an MCU with more memory or an NPU.

Interpretation. Quantization is often the difference between "impossible" and "possible" on a microcontroller. The business trade-off is a small accuracy loss and engineering effort versus a cheaper device with lower latency and better privacy.

Worked Example 8.2: Edge AI and bandwidth

A store has a 1080p camera streaming compressed video at about 4 Mbps. (a) How much data per day would it send to the cloud? (b) An edge AI model instead sends a 1 KB event message each time it detects a shelf running empty, about 500 times per day. Compare.

(a) 4 Mbps × 86,400 s = 345,600 Mb = 43,200 MB ≈ **43.2 GB per day** per camera.

(b) 500 × 1 KB = **0.5 MB per day**.

The edge approach sends roughly 86,000 times less data, keeps video of shoppers on site (a privacy benefit under laws such as GDPR), and works even if the internet connection is slow. The cloud still receives enough information to manage inventory across all stores.

8.6 Why cloud AI still dominates

Despite the attention on edge intelligence, cloud AI still handles the majority of IoT AI workloads in enterprises. The business reasons are substantial:

1. **Computational scale.** Training sophisticated models requires GPU clusters that cannot exist on a device. Cloud infrastructure scales elastically and is rented by the hour.
2. **Fleet learning.** A model trained on data from thousands of sensors is far more accurate than anything a single edge device learns alone.

3. **Model management.** Updating, retraining, and versioning models in the cloud is simpler and less risky than pushing new firmware to thousands of devices.
4. **Rich analytics.** The cloud keeps historical data for years, enabling trend analysis, audits, and compliance reporting.
5. **Lower device cost.** Keeping intelligence in the cloud keeps hardware simpler and cheaper, which is essential for million-device deployments.

In practice, most systems are **hybrid**: models are **trained in the cloud and run at the edge**, with the cloud monitoring performance and retraining periodically.

Federated learning

Federated learning trains a shared model across many devices without collecting their raw data. Each device trains on its own data and sends only model updates (gradients or weights) to a server, which combines them. Phones use this technique to improve keyboard predictions without uploading what users type. For IoT, it allows hospitals, factories, or homes to benefit from shared learning while keeping sensitive data local. The updates themselves can leak information or be poisoned by malicious participants, so protecting them is an active area of security research.

8.7 Cloud AI and IoT platforms

Cloud providers offer integrated IoT and AI platforms: ingestion services receive live device data, feed it into machine-learning pipelines for real-time inference or training, and surface results in enterprise systems (Table 8.3).

Table 8.3 Selected cloud AI and IoT platforms (verify current offerings)

Provider	IoT ingestion	AI/ML services
Amazon Web Services	AWS IoT Core, AWS IoT Greengrass	Amazon SageMaker; Amazon Rekognition (vision); Amazon Bedrock (generative AI)
Microsoft Azure	Azure IoT Hub, Azure IoT Operations	Azure Machine Learning; Azure AI services
Google Cloud	Partner MQTT brokers into Pub/Sub (Google Cloud IoT Core was retired in 2023)	Vertex AI; BigQuery ML
Siemens	Insights Hub (formerly MindSphere)	Industrial AI analytics
Alibaba Cloud / Huawei Cloud	IoT Platform / IoT Device Access	Platform for AI (PAI) / ModelArts

8.8 Core AI technologies in IoT

Table 8.4 Core AI technologies in IoT

Technology	IoT application	Business value
Computer vision	Quality inspection on production lines; occupancy counting; license-plate recognition	Inspects every item, 24 hours a day, consistently
Predictive analytics	Vibration monitoring of motors, elevators, jet engines, and turbines	Predicts failures weeks in advance, avoiding unplanned downtime
Anomaly detection	Detecting unusual energy use, leaks, intrusions, or sensor faults	Catches problems nobody wrote a rule for
Natural language processing	Voice-controlled industrial equipment; smart-home assistants	Hands-free operation in sterile or dangerous environments
Reinforcement learning	Warehouse robots, drones, HVAC optimization	Learns efficient strategies in changing environments
Generative AI at the edge	Cameras that summarize events in natural language; assistants on gateways	Turns raw sensor data into human-readable reports and decisions

Evaluating an AIoT model: precision and recall

An AI alert is useful only if it is right often enough. Two measures matter:

Precision = true alerts ÷ all alerts raised Recall = true alerts ÷ all real events

High **precision** means few false alarms; high **recall** means few missed events. Improving one often worsens the other, so the right balance depends on the business cost of each kind of error.

Worked Example 8.3: Is the predictive-maintenance model worth it?

A factory runs 50 machines. Last year, 12 machines suffered unplanned bearing failures, each causing 8 hours of downtime at US$5,000 per hour, plus US$4,000 in emergency repair. A pilot AIoT model raised 15 alerts: 10 were real impending failures (true alerts) and 5 were false alarms. Two failures occurred without any alert. Each alert triggers a planned inspection costing US$600; a planned repair costs US$1,500 and avoids downtime.

Model quality. Real events = 10 detected + 2 missed = 12. Precision = 10 ÷ 15 = **67 percent**. Recall = 10 ÷ 12 = **83 percent**.

Cost without AI = 12 failures × (8 h × US$5,000 + US$4,000) = 12 × US$44,000 = **US$528,000**.

Cost with AI:

- Inspections: 15 × US$600 = US$9,000
- Planned repairs for detected failures: 10 × US$1,500 = US$15,000
- Missed failures: 2 × US$44,000 = US$88,000

- Total = **US$112,000**

Annual benefit ≈ US$528,000 − US$112,000 = **US$416,000**, before the cost of sensors, software, and staff. Even with one false alarm for every two real alerts, the model pays for itself because a missed failure costs about 73 times as much as a false alarm. In a different business, where false alarms are expensive (for example evacuating a building), the model would need much higher precision.

8.9 Challenges of implementing AIoT

AI is only as good as the data it receives. If an IoT sensor is poorly calibrated or noisy, the AI will make confident but wrong decisions. Several challenges recur:

1. **Data quality.** Missing values, drifting sensors, wrong units, and mislabeled examples corrupt models. Data pipelines need validation and quality flags.
2. **Data heterogeneity.** Sensors from different brands use different formats and sampling rates, making it hard for one model to understand the whole system. Data contracts and standards such as Matter help.
3. **Energy constraints.** Running AI is computationally expensive. Balancing a sensor's battery life against the "smartness" of its model is a constant engineering battle.
4. **Model compression trade-offs.** Quantization and pruning save memory and energy but can reduce accuracy, especially on rare events.
5. **Model drift.** Equipment ages, seasons change, and user behavior shifts; models must be monitored and retrained.
6. **Federated learning security.** Protecting model updates from leakage and poisoning is a priority.
7. **Explainability and safety.** Operators must understand why a system acted, especially in healthcare, transport, and industry. Hard safety limits should remain in deterministic code.
8. **Ethics.** AIoT can enable surveillance or biased decisions (Chapter 6). Purpose limitation and human oversight are essential.

8.10 Applying AIoT to the demo system

The IoT Education Course demo system (Chapter 7) uses fixed rules: an alert above 30 °C and hysteresis cooling between 26 °C and 24 °C. Several AIoT enhancements are possible, each with a natural location:

Table 8.5 AIoT enhancements for the demo system

Enhancement	Where AI runs	Why there
Vibration anomaly detection on the MPU6050	On the node (TinyML)	Raw vibration is high-rate; sending only anomaly scores saves battery and

Enhancement	Where AI runs	Why there
		bandwidth
Adaptive sleep interval (sample faster when temperature changes quickly)	On the node or gateway	Saves energy during stable periods while capturing fast events
Predictive pre-cooling based on time of day, occupancy, and weather	Gateway (inference), cloud (training)	Combines several data sources; needs local action even during outages
Fleet-wide model of normal server-room behavior	Cloud	Requires data from many sites and long history
Natural-language daily summary for managers	Cloud (generative AI)	Uses historical data and large models

8.11 IoT application case: making NxEco intelligent

NxEco is a **smart** system in the sense of Section 8.2: the cloud applies rules that combine weather forecasts, monthly ET, plant type, and rain information to adjust watering. Its app also draws on a database of plant types and community tips (NxEco, Inc., 2020). It is not yet **intelligent**: it does not learn from the results of its own watering. Several AIoT enhancements are possible, each with a natural location.

Table 8.6 AIoT enhancements for NxEco

AIoT enhancement	Data needed	Where AI runs	Benefit
Learn each zone's real water need from soil-moisture feedback	Wireless soil-moisture probes per zone	Cloud training; controller applies results	Closes the open loop identified in Section 2.11
Leak and broken-head detection	Flow sensor on the main line	Controller (edge) for fast shut-off; cloud for fleet patterns	Stops water loss within minutes
Hyper-local weather correction	Fleet of controllers plus rain sensors across a city	Cloud (fleet learning)	Better rain skips than a zip-code forecast
Predictive scheduling at the cheapest hours	Electricity and water tariffs, forecasts	Cloud	Lower cost and better use of water restrictions
Natural-language assistant for landscapers	Schedules, history, alerts from many sites	Cloud (generative AI)	Faster management of hundreds of controllers in SNAP

The most valuable first step would probably be **leak detection**, because it combines a cheap sensor with a clear business value and a decision that must be made at the edge (Section 8.4): a controller that sees flow when no zone is open should close the master valve immediately, without waiting for the cloud. Fleet learning could then improve the model

across thousands of installed controllers, the same pattern that Chapter 8 describes for autonomous vehicles.

Chapter summary

- **AIoT** combines IoT's senses with AI's judgment, moving systems from **smart** (rules) to **intelligent** (learning and prediction) and from reactive to proactive.
- Production AIoT needs **data contracts**, **feedback loops**, and **MLOps**.
- AI can run **in the chip** (TinyML, NPUs), on **edge devices**, on the **gateway** ("local manager"), or in the **cloud** ("master library"). Left means faster, more private, and offline-capable; right means more powerful and fleet-wide.
- The **TinyML workflow** collects data, trains in the cloud, compresses (quantization cuts memory about fourfold), converts, deploys by OTA, and monitors drift.
- **Cloud AI still dominates** enterprise workloads because of compute scale, fleet learning, model management, analytics, and lower device cost; most real systems are hybrid. **Federated learning** shares learning without sharing raw data.
- Computer vision, predictive analytics, anomaly detection, NLP, reinforcement learning, and edge generative AI each create distinct business value. **Precision** and **recall** measure whether alerts are worth acting on.
- Challenges include data quality, heterogeneity, energy, compression, drift, security, explainability, and ethics.

Key terms

AIoT; anomaly detection; cloud AI; computer vision; data contract; drift; edge AI; federated learning; fleet learning; inference; knowledge distillation; LiteRT; MLOps; NPU; physical AI; precision; predictive analytics; pruning; quantization; recall; reinforcement learning; TinyML; training.

Review questions

Multiple choice

1. In the AIoT analogy, IoT is the nervous system and AI is the:
    a. skin
    b. brain
    c. muscles
    d. skeleton
2. AI shortens the traditional IoT sequence to:
    a. sense, store, wait
    b. sense, analyze, act

c. transmit, store, analyze
d. act, sense, store

3. Which location is described as the “master library” that retrains models with data from millions of devices?
   a. The sensor chip
   b. The gateway
   c. The cloud
   d. The phone
4. Quantizing a model from 32-bit floats to 8-bit integers reduces its weight memory by about:
   a. half
   b. four times
   c. eight times
   d. 32 times
5. Which is a key advantage of edge AI over cloud AI?
   a. Unlimited computing power
   b. Lower latency and better privacy
   c. Easier fleet-wide training
   d. Unlimited storage
6. Why does cloud AI still handle most enterprise IoT workloads?
   a. Devices cannot transmit data
   b. Training needs large compute, and fleet learning improves accuracy
   c. Edge AI is illegal
   d. Clouds have no latency
7. Federated learning allows:
   a. devices to share raw data freely
   b. training a shared model without collecting raw data centrally
   c. models to run only in the cloud
   d. sensors to skip calibration
8. A model raises 20 alerts, 15 of which are real; there were 25 real events. Its recall is:
   a. 75 percent
   b. 60 percent
   c. 80 percent
   d. 15 percent
9. When real-world data changes so a model becomes less accurate, this is called:
   a. quantization
   b. pruning
   c. drift

d. distillation

10. Which AI technology is most associated with predicting machine failures from vibration?
    a. Natural language processing b. Predictive analytics c. Reinforcement learning d. Generative AI

Short answer

11. Explain the difference between smart IoT and intelligent IoT with an example from agriculture.
12. Describe the role of the gateway as a "local manager" in AIoT.
13. Give three business reasons why cloud AI still dominates, and one situation in which edge AI is clearly better.
14. Explain why data quality is the most important challenge for AIoT.

Applied problems

15. A keyword-spotting model has 80,000 parameters. (a) What is its weight memory in float32 and int8? (b) Which would fit in 64 KB of free flash?
16. An AIoT leak detector in an office building raised 40 alerts last year; 8 were real leaks and 32 were false alarms; 2 real leaks were missed. Each false alarm costs US$150 in staff time; each missed leak costs US$20,000 in damage. (a) Compute precision and recall. (b) Compute the annual cost of errors. (c) Would you tune the model for higher precision or higher recall? Explain.
17. **NxEco case.** Of the AIoT enhancements in Table 8.6, which one must make its decision at the edge rather than in the cloud, and why?

Lab 8: Design an AIoT enhancement

Supports Assignment 4 (business proposal: where AI runs, and why).

Part 1: Anomaly detection on real telemetry (spreadsheet). Use the telemetry CSV exported from the demo system in Lab 7A.

1. Open the CSV in Excel or Google Sheets and sort by device and timestamp.
2. For one node's temperature column, compute a **rolling mean** and **rolling standard deviation** over the previous 10 readings (for example `=AVERAGE(C2:C11)` and `=STDEV(C2:C11)` copied down).
3. Compute a **z-score** for each reading: z = (x − mean) ÷ standard deviation, using the statistics of the previous window.
4. Flag readings with $|z| > 3$ as anomalies.
5. Inject a temperature spike in the demo (as in T4), export a new CSV, and check whether the spike is flagged and how many readings after the spike are flagged.

6. Discuss: this simple statistical model is a baseline. Where could it run (node, gateway, or cloud)? What would a machine-learning model add, and what data would it need?

Part 2: AIoT design worksheet. For the business scenario of Assignment 4:

1. Describe the decision that AI should improve and its business value.
2. List the data needed, its source, sampling rate, and quality risks.
3. Choose the AI technology (Table 8.4).
4. Decide where training and inference run (Table 8.2), and justify with latency, bandwidth, privacy, cost, and offline needs.
5. Estimate one number: model size after quantization, bandwidth saved, or the cost of false alarms versus missed events.
6. Identify one ethical or security risk and its mitigation.

References and further reading

AI Smartbytes. (2025). *The Internet of Things and AI: A powerful combination* [Video]. YouTube. https://www.youtube.com/watch?v=gNPQ2TPre1o

Bottacci, F. (2023). *ENEL: Largest production deployment of AI and IoT [AIoT] applications* [Video]. YouTube. https://www.youtube.com/watch?v=uYgwBojDE7k

Google for Developers. (2026, May 28). *LiteRT for microcontrollers*. Google AI Edge. https://developers.google.com/edge/litert/microcontrollers/overview

Kirvan, P., Yasar, K., & Gillis, A. S. (2025, August 1). *What is the artificial intelligence of things (AIoT)?* TechTarget. https://www.techtarget.com/iotagenda/definition/Artificial-Intelligence-of-Things-AIoT

NxEco, Inc. (2020). *NxEco* (Version 2.2.4) [Mobile application software]. App Store. https://apps.apple.com/app/id972339992

NxEco. (n.d.-a). *NxEco smart watering* [Website]. https://www.nxeco.com

NxEco. (n.d.-c). *NxEco smart weather-based irrigation controller: App manual* (Ver. 10.2) [PDF]. https://www.nxeco.com/Public/public/file/manual/NxEco_Smart_Manual_Ver10.2.pdf

POND IoT. (2025). *What are the bigger IoT trends of 2025* [Video]. YouTube. https://www.youtube.com/watch?v=_44OjwYpw-Q

CHAPTER 9

IoT Business Models, Business Cases, and Roadmaps

Learning Objectives

After studying this chapter, students will be able to:

1. Describe the main IoT business domains and business models, including product sales, subscriptions, data monetization, managed services, and product-as-a-service.
2. Explain how value and margins are distributed across the IoT technology stack.
3. Analyze the strategic obstacles and trends shaping IoT businesses.
4. Compare two contrasting IoT business strategies through the Xiaomi and Enel X cases.
5. Build a research-supported IoT business case with costs, benefits, payback, return on investment, and net present value.
6. Develop a phased implementation roadmap with stage gates and key performance indicators.
7. Communicate an IoT solution persuasively to an executive panel, using measured evidence.

Where this fits: Week 7, Video 2, and Week 8. This chapter supports Assignment 4 items 2 and 3 (business proposal and team presentation) and SM Assignment 2, the final IoT solution presentation and reflection.

Opening case: selling hours, not engines

In the 1960s, the British engine maker Rolls-Royce began offering airlines a different deal: instead of buying spare parts and repairs whenever an engine needed them, an airline could pay a fixed amount for every hour the engine flew. The arrangement, later known as "Power by the Hour" and developed into the TotalCare service, changed the manufacturer's incentives. Rolls-Royce now earned more when engines stayed in the air, so it invested in making them more reliable and easier to maintain.

Modern sensors made this model far more powerful. Today's aircraft engines stream health data during flight, and analytics predict when a component will need attention. The manufacturer schedules maintenance before failures occur, and the airline pays for **uptime** rather than hardware. This shift from selling products to selling outcomes, called

servitization, is one of the most important ways IoT creates business value. This chapter explains how companies make money from IoT, and how students can build and present a business case of their own.

9.1 IoT business domains

IoT businesses operate in four broad domains:

1. **Connected devices.** Developing and selling sensors, actuators, wearables, appliances, vehicles, and machines that communicate over networks.
2. **Data collection and analytics.** Gathering device data and applying analytics and AI to extract actionable insights.
3. **Automation and control.** Using IoT to optimize processes, reduce human intervention, and improve efficiency, for example in factories, buildings, and farms.
4. **Platforms and services.** Providing cloud platforms and software for device management, data storage, analytics, and application development.

Many successful companies combine several domains: they sell devices, run a platform, and charge for analytics.

9.2 IoT business models

A **business model** describes how a company creates, delivers, and captures value. IoT supports several models, often combined (Borgini, 2025a; Elizalde, n.d.).

Table 9.1 IoT business models

Business model	How revenue is earned	Strengths	Weaknesses	Examples
Product sales	One-time sale of connected hardware	Simple; immediate cash	Low margins; no recurring revenue; cloud costs continue after sale	Smart plugs, thermostats
Subscription (SaaS)	Recurring fees for software, analytics, device management, or security	Predictable recurring revenue; high margins	Customers must see ongoing value; churn risk	Camera cloud storage, fleet-tracking software
Data monetizatio n	Selling insights or anonymized, aggregated data, or using data to improve other products	Uses an asset the company already has	Privacy, consent, and legal risk; trust	Traffic data from connected cars; retail footfall analytics
Managed services	End-to-end solutions including installation, monitoring, maintenance, and support	Deep customer relationships; high switching costs	Labor-intensive; harder to scale	Building-management services, managed industrial monitoring

Business model	How revenue is earned	Strengths	Weaknesses	Examples
Product-as-a-service / outcome-based	Customer pays for usage or results (hours, lux, uptime, energy saved) rather than ownership	Aligns incentives; turns customer CapEx into OpEx	Provider carries asset and performance risk; needs reliable data	Engine "power by the hour"; light-as-a-service; compressed air as a service
Hardware as a hook (ecosystem)	Low-margin hardware drives adoption; profit comes from services, content, and ecosystem	Rapid scale; strong platform lock-in	Requires capital and a large user base	Xiaomi's smart-home ecosystem (Section 9.7)

Benefits that business models monetize

Whatever the model, IoT creates value through a small set of benefits:

- enhanced operational efficiency and reduced costs;
- better decisions through real-time data;
- new revenue streams from products and services;
- higher customer satisfaction through personalized experiences;
- greater sustainability through optimized resource use.

A business case must show which of these benefits apply, who receives them, and how the company captures a share.

9.3 Value across the IoT technology stack

The companies that build, connect, and manage IoT systems earn very different margins depending on their layer of the stack (Table 9.2 and Figure 9.1).

Table 9.2 Indicative margins across the IoT stack (vary widely by company)

Segment	Business model	Indicative gross margin
Hardware	One-time sale of sensors, chips, modules, and devices	Low (about 10 to 15 percent for many device makers)
Connectivity	Monthly data plans (cellular, satellite, LoRaWAN)	Moderate (about 20 to 30 percent)
Platform (PaaS)	Subscription and usage fees for device management and IoT cloud services	High (about 40 to 60 percent)
Analytics and applications (SaaS)	Fees for AI-driven insights, dashboards, and vertical applications	Very high (often 70 percent or more)

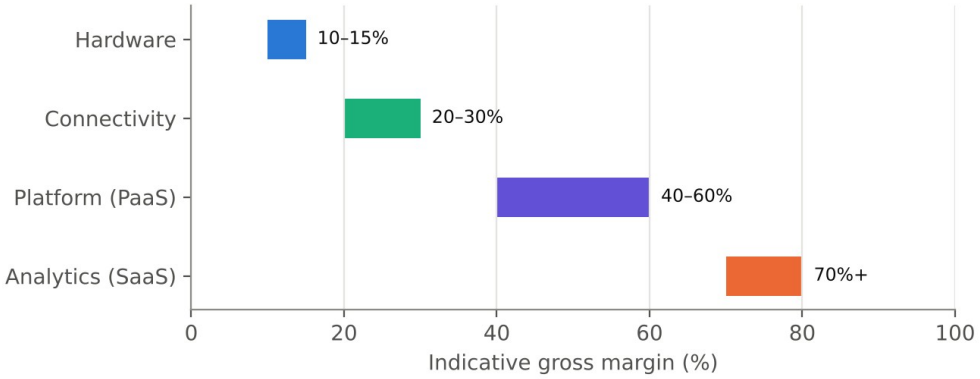


Figure 9.1 Indicative gross margins across the IoT technology stack.

The pattern explains why so many hardware companies try to add subscriptions: hardware is necessary but competitive, while software and analytics capture more of the value. It also explains why customers resist subscriptions that do not deliver clear ongoing value.

Scale of the opportunity

Estimates of the IoT market vary enormously depending on what is counted, so they should be read as orders of magnitude. About 21 billion IoT devices were connected by the end of 2025, with close to 40 billion forecast by 2030 (Sinha, 2025). The research firm IDC forecast that IoT devices would generate close to 80 zettabytes of data per year by 2025. McKinsey has estimated that IoT could enable between US$5.5 trillion and US$12.6 trillion of economic value globally by 2030, most of it in factories, cities, health, and logistics rather than consumer gadgets. Geographically, China and the Asia-Pacific region lead in device volume, while the United States and Europe capture a large share of software and analytics revenue.

IoT across industries

IoT's indirect influence, making existing industries more efficient, is larger than the direct sales of the technology itself:

- **Manufacturing:** predictive maintenance, asset tracking, quality inspection, and process optimization.
- **Healthcare:** remote patient monitoring, connected medical devices, and hospital asset tracking.
- **Transportation and logistics:** fleet management, route optimization, cold-chain monitoring, and real-time tracking.
- **Smart homes and buildings:** energy management, security, comfort, and space utilization.
- **Agriculture:** precision farming, soil monitoring, livestock tracking, and automated irrigation.

- **Energy and utilities:** smart meters, grid sensors, distributed batteries, and EV charging.

9.4 Strategic obstacles

While the business is growing, several friction points slow adoption:

- **Cybersecurity cost and liability.** Securing fleets, monitoring threats, insuring against incidents, and meeting new regulations such as the EU Cyber Resilience Act (Chapter 6) add cost throughout a product's life.
- **E-waste and repair regulations.** Governments increasingly require longer support periods, repairability, and recycling, so companies must design hardware that lasts many years.
- **Interoperability.** Customers fear being locked into one vendor's ecosystem. Standards such as Matter reduce this concern in homes; industrial standards such as OPC UA serve factories.
- **Unclear return on investment.** Many organizations run pilots that never scale ("pilot purgatory") because benefits were not measured or the business case was weak.
- **Skills and organizational change.** IoT projects need embedded, cloud, data, and security skills, and close cooperation between IT and operations (Chapter 6).

9.5 Trends shaping IoT business

Three trends stand out in 2026 (POND IoT, 2024):

1. **The AIoT revolution: generative AI at the edge.** Instead of sending raw data to the cloud, devices increasingly reason locally. A camera can summarize an event in plain language and decide whether to alert someone, creating new opportunities to license edge intelligence.
2. **Servitization: product-as-a-service.** Companies sell uptime and outcomes instead of equipment. In "light-as-a-service" contracts, for example, a lighting company owns and maintains the fixtures at a customer's site and charges for the light delivered, replacing components before they fail. The customer's spending shifts from capital expenditure (CapEx) to operating expenditure (OpEx).
3. **Matter consolidation.** In the consumer market, Matter is reducing the fragmentation between ecosystems. As devices from different brands work together locally, competition moves from closed hardware ecosystems toward cross-brand services and automations.

Other trends include 5G RedCap and satellite connectivity for remote assets, and growing demand for verifiable sustainability data.

9.6 Case 1: Xiaomi, the smart-home ecosystem

From phone maker to ecosystem platform

Xiaomi, founded in Beijing in 2010, began as a smartphone company and became one of the world's largest consumer IoT platforms. Its business is unusual because it does not make every "thing" itself. Instead, it provides a common platform, design standards, supply-chain access, and capital to hundreds of partner companies, whose products all connect through the same app and cloud.

By mid-2026, Xiaomi reported more than 1.1 billion connected IoT devices on its AIoT platform, excluding smartphones and tablets, more than 24 million users with five or more connected devices, and over 120 million monthly active users of its Xiaomi Home app. IoT and lifestyle products contributed roughly a quarter to a third of company revenue in recent quarters (company results as reported in news coverage). Xiaomi is also one of the largest deployers of Matter-certified and cellular IoT devices, and it ranks among the leaders in wearables, tablets, and smart TVs in many countries.

The "Human × Car × Home" strategy

With the launch of its SU7 electric vehicle in 2024 and later models, Xiaomi extended its ecosystem strategy to cars:

Table 9.3 Xiaomi's "Human × Car × Home" strategy

Dimension	The ecosystem experience
Human	Phones and wearables track health, location, and intent in real time
Car	The electric vehicle acts as a mobile "smart room" that, for example, pre-cools the home as the owner drives back
Home	Hundreds of millions of connected devices coordinated as one system

The business model: hardware as a hook

Xiaomi's monetization has three tiers:

1. **Low-margin hardware.** It sells good-quality sensors, lights, appliances, and gadgets at near-cost prices to achieve ubiquity in homes. In 2018, its founder publicly pledged that the company's overall net profit margin on hardware would not exceed 5 percent. The "price-to-performance" strategy offers products that look and feel premium at much lower prices.
2. **Services and subscriptions.** Once hardware is in the home, Xiaomi earns from internet services: cloud storage for cameras, health insights for wearables, premium features, advertising, and financial services.

3. **Data-driven retail.** Knowing which devices a household owns lets the company recommend accessories, consumables, and partner services through its app and stores.

The investment and incubator model

Xiaomi does not manufacture most of its ecosystem products. It identifies promising hardware start-ups, such as Roborock (robot vacuums) and Huami (now Zepp Health, wearables), and provides capital, supply-chain access, industrial design standards, and distribution. The result is a "department store of the future" with thousands of products, without the overhead of owning every factory.

The software moat: one app for everything

The real signature of the ecosystem is the unified software experience. The Xiaomi Home app (formerly Mi Home) manages Xiaomi devices and partner brands in one place; products bought years apart work under the same app; and a rule engine brings simple automation to non-technical users ("if the door lock opens after 6 p.m., turn on the hallway lights and start the kettle"). Xiaomi's cloud is large by number of users but modest by direct revenue, because it is not sold as a general-purpose cloud. Like the clouds that Apple and Samsung run for their own devices, it is an **ecosystem-enabling** cloud rather than a competitor to AWS, Azure, or Google Cloud.

Future services built on this platform include home security as a service, energy optimization that adjusts appliances to electricity prices, and elderly-care monitoring with non-camera sensors such as radar that detect falls or changes in daily routines.

In one sentence: Xiaomi uses high-quality, low-cost hardware to place its software and services in every corner of daily life, a high-volume strategy that has brought smart living to middle-class households around the world.

9.7 Case 2: Enel X, the grid orchestrator

From utility to digital platform

Enel, the Italian energy group, operates electricity networks and generation in Europe and the Americas. Through its **Enel X** business, it has moved beyond selling electrons to managing the flow of *data* across an increasingly decentralized grid. Enel was an early pioneer of smart meters, rolling out remotely read meters to tens of millions of customers in Italy from the early 2000s. Its strategic challenge today is balancing a grid that is moving away from coal and gas toward variable solar and wind power.

Demand response and virtual power plants

Enel X's signature service turns passive energy consumers into active market participants:

- **The concept.** Enel X connects thousands of industrial and commercial sites (batteries, EV chargers, heating and cooling systems, and industrial processes) into a **virtual power plant (VPP)**.
- **The mechanism.** When the grid is under stress, for example on a cloudy, windless evening, the grid operator would traditionally start a gas-fired plant. Instead, the VPP platform signals enrolled sites to reduce consumption or discharge batteries for a short period.
- **The business model.** Enel X pays customers for this flexibility and earns revenue from grid operators and capacity markets. Saving energy at the right moment becomes a tradable commodity.

In 2025, Enel reported managing nearly 5 GW of demand response in North America and more than 9 GW of flexible capacity globally, with VPP operations in 14 countries (Enel North America, 2025). That is comparable to the output of several large power stations, delivered without building one.

Additional services

- **Smart EV charging.** Connected chargers act as IoT nodes that communicate with the grid to charge vehicles during the cheapest, cleanest periods, and **vehicle-to-grid (V2G)** pilots let parked EVs sell power back during peaks.
- **Energy as a service (EaaS).** For businesses and cities, Enel X can install solar panels, batteries, and efficient lighting with little or no upfront cost, manage them through its IoT platform, and share in the guaranteed energy savings, turning energy from a variable cost into a predictable managed service.
- **Digital twins and ESG data.** Sensors and digital twins monitor transformers, turbines, and customer assets to extend equipment life, and the same data supports customers' sustainability reporting.

Comparing the two cases

Table 9.4 Xiaomi and Enel X compared

Dimension	Xiaomi (smart home)	Enel X (energy)
Primary customer	Consumers	Businesses, cities, grid operators
Primary goal	Convenience and lifestyle	Efficiency, reliability, decarbonization
Value proposition	Affordable smart living; time saved; security	Lower energy cost; new revenue from flexibility; carbon reduction
Revenue driver	High-volume hardware plus internet	Grid-scale orchestration, flexibility

Dimension	Xiaomi (smart home)	Enel X (energy)
	services	payments, managed energy services
Role of IoT data	Personalization, automation, ecosystem lock-in	Real-time balancing, verification of performance, asset health
Key risk	Privacy, low hardware margins, competition	Regulation, market rules, performance guarantees

A smart-home business focuses on the **human interface**; an energy IoT business focuses on **system resilience**. Both depend on the same layers studied in this book: devices, connectivity, edge, cloud, and applications.

9.8 Building an IoT business case

A **business case** is a structured argument that an investment is worthwhile. For IoT, a strong business case answers seven questions:

1. **What is the business need?** The problem, who has it, and what it costs today, with evidence.
2. **What is the solution?** The technology for each tier (device, connectivity, edge, cloud, application), justified with explicit criteria (Chapters 2 to 7).
3. **Where does intelligence run?** Device, edge, or cloud, and why (Chapter 8).
4. **What is the business model?** How the company or customer captures value (Section 9.2).
5. **What does it cost, and what does it return?** Capital and operating costs, quantified benefits, and financial measures.
6. **What are the risks?** Technical, security, privacy, organizational, and environmental risks with mitigations (Chapter 6).
7. **How will it be implemented?** A phased roadmap with stage gates and KPIs (Section 9.9).

Financial measures

- **Capital expenditure (CapEx):** one-time costs such as devices, gateways, installation, and integration.
- **Operating expenditure (OpEx):** recurring costs such as connectivity, cloud services, support, security monitoring, and battery replacement.
- **Payback period:** the time for cumulative net benefits to repay the initial investment: Payback = CapEx ÷ monthly net benefit.
- **Return on investment (ROI)** over a period: ROI = (total net benefit − CapEx) ÷ CapEx.
- **Net present value (NPV):** the sum of future net cash flows discounted to today's value, minus the investment. A positive NPV means the project creates value after

accounting for the time value of money: NPV = −CapEx + Σ $CF_t \div (1 + r)^t$, summed over periods t = 1 to n, where CF_t is net cash flow in period *t* and *r* is the discount rate per period.

Worked Example 9.1: A cold-chain monitoring business case

A supermarket chain has 40 stores with 3 freezer cabinets each (120 freezers). It considers IoT temperature monitoring with alerts and automated food-safety logs.

Costs.

Item	Calculation	Amount (US$)
Sensor kits	120 × 180	21,600
Gateways	40 × 250	10,000
Installation and integration	Quoted	10,400
CapEx total		**42,000**
Connectivity	40 gateways × 8 per month	320 per month
Cloud platform and software	120 sensors × 2.75 per month	330 per month
OpEx total		**650 per month**

Monthly benefits.

Benefit	Assumption	Amount (US$/month)
Avoided stock losses	2 freezer failures per month across the chain, 1,200 of stock each; alerts prevent 80 percent	1,920
Energy savings	Early detection of open doors and failing compressors: 5 per freezer	600
Labor savings	Automated temperature logs replace manual checks	580
Total benefit		**3,100**

Financial results.

Monthly net benefit = 3,100 − 650 = **US$2,450**.

Payback = 42,000 ÷ 2,450 ≈ **17.1 months** (Figure 9.2).

Three-year ROI = (2,450 × 36 − 42,000) ÷ 42,000 = (88,200 − 42,000) ÷ 42,000 = **110 percent**.

NPV over 36 months at an 8 percent annual discount rate (0.667 percent per month) ≈ **US$36,200**.

Interpretation. The project pays back in under 18 months and creates value even after discounting. The largest benefit, avoided stock losses, rests on two assumptions (failure frequency and the 80 percent prevention rate) that a pilot should measure. A sensitivity check is essential: if only 50 percent of losses were prevented, the monthly benefit would fall

by US$720 and payback would stretch to about 24 months. Non-financial benefits, such as food safety and regulatory compliance, strengthen the case further.

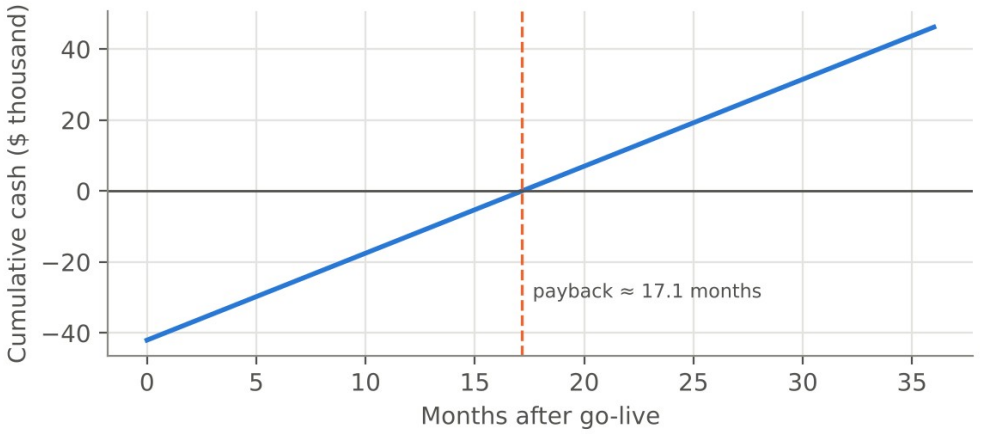


Figure 9.2 Cumulative cash flow and payback for the cold-chain business case.

Worked Example 9.2: Unit economics of a subscription device

A start-up sells a smart water-leak sensor. The device costs US$45 to make and sells for US$49 (US$4 hardware margin). Customers pay a US$6 monthly subscription for alerts and insurance-discount reports, with an 80 percent gross margin on the subscription. On average, 2 percent of subscribers cancel each month. Acquiring a customer (marketing and sales) costs US$60. Is the model healthy?

Expected customer lifetime ≈ 1 ÷ monthly churn = 1 ÷ 0.02 = **50 months**.

Lifetime subscription gross profit = 50 × (US$6 × 0.80) = 50 × US$4.80 = US$240.

Customer lifetime value (LTV) ≈ US$240 + US$4 = **US$244**.

LTV ÷ customer acquisition cost (CAC) = 244 ÷ 60 ≈ **4.1**.

A common rule of thumb treats an LTV-to-CAC ratio above 3 as healthy. The model works only because of the subscription; hardware alone earns US$4 per customer, less than the cost of acquiring them. If churn rose to 5 percent per month, lifetime would fall to 20 months, LTV to about US$100, and the ratio to about 1.7, which would not be sustainable.

9.9 The implementation roadmap

A **roadmap** turns a business case into a plan. IoT roadmaps are usually phased, with a **stage gate** at the end of each phase where leaders decide, based on evidence, whether to continue, adjust, or stop (Figure 9.3).

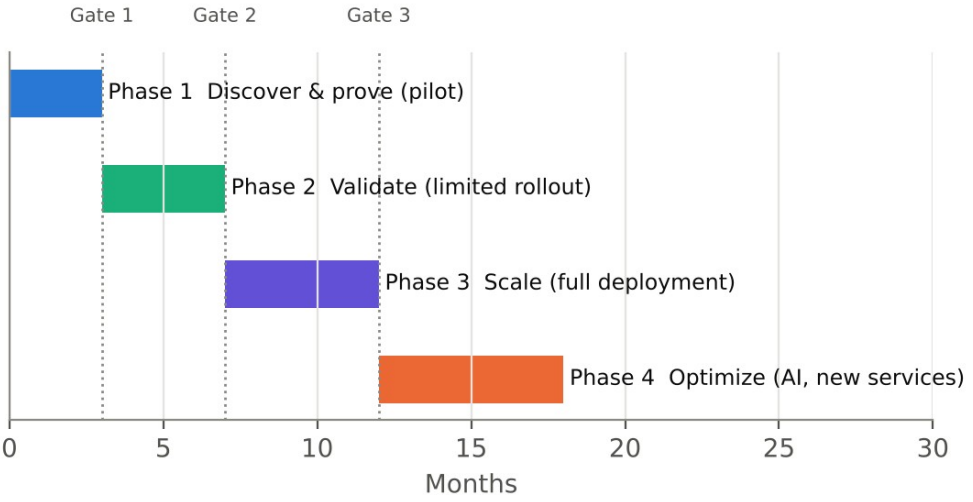


Figure 9.3 A phased IoT implementation roadmap with stage gates.

Table 9.5 Implementation roadmap phases

Phase	Purpose	Typical activities	Gate criteria (examples)
1. Discover and prove (pilot)	Prove technical feasibility and measure real benefits	Install in one site; validate sensors, connectivity, alerts; measure baseline and improvement	Data completeness above 98 percent; alerts verified; measured benefit supports the business case
2. Validate (limited rollout)	Prove operations and economics at moderate scale	Roll out to 10 to 20 percent of sites; train staff; integrate with business systems; security review	Unit costs within budget; staff adoption; no critical security findings
3. Scale (full deployment)	Deliver benefits across the organization	Automated provisioning; fleet management; support processes	KPIs met across sites; support load sustainable
4. Optimize	Extract more value	Add AI models, new services, and data products; renegotiate costs	ROI above target; new revenue or savings identified

Key performance indicators (KPIs) make progress measurable. Examples include device uptime, data completeness, alert precision and recall (Chapter 8), mean time to respond to alerts, energy or water saved, downtime avoided, cost per device per month, and customer satisfaction.

Roadmaps should also plan for **change management** (who must work differently, and how they will be trained and involved), **security and privacy reviews** before each phase, and **end-of-life** decisions for devices (Chapter 6).

9.10 Communicating the solution

A business case succeeds only if decision makers understand and believe it. In the IoT Education Course, the final deliverable is a 10-minute presentation to a classmate panel acting as business leaders, followed by questions.

Evidence-based communication

Decision makers trust numbers they can check. Strong presentations turn measured test results (Chapter 7) into a clear technical argument:

- state the business problem in one sentence;
- show the solution architecture on one slide, tier by tier;
- present key measured values (ingestion rate, latency per hop, buffer behavior, battery life) in tables or charts, not paragraphs;
- explain what the measurements mean for the business ("the gateway buffered 30 seconds of outage without data loss, so stores keep their food-safety records during internet failures");
- be honest about limits and defects found, and show the redesign that addresses them;
- end with the business model, financial summary, roadmap, and a clear request.

A recommended slide structure (10 to 12 slides)

1. Title and team (names and roles).
2. The business problem and why it matters (with a number).
3. Team collaboration setup: how the team organized and ran the tests.
4. System architecture: the five tiers and the technology chosen for each.
5. Critical steps and coefficients: key test steps and measured values.
6. Major records of one test (one of T2 to T7) with a screenshot or table.
7. Conclusions from the tests: what works, limits, defects found.
8. Redesign for one real-world pressure.
9. Business application: the target business, business model, and where AI runs.
10. Financial summary: costs, benefits, payback, ROI.
11. Roadmap with phases and KPIs.
12. References (APA 7th edition).

Slide guidance: one idea per slide; numbers in tables or charts; readable fonts (at least 18 points); a source on every slide that uses outside data.

Answering panel questions

Good panel questions test the evidence or the decision, for example: "How do you know the battery will last that long?", "What happens to the stores if the cloud is down for a day?", "What is the biggest assumption in your payback figure?", or "Who owns the data, and how is customer privacy protected?" Effective answers are short, cite evidence, acknowledge uncertainty honestly, and explain how the pilot phase will reduce it.

9.11 IoT application case: the NxEco business model

NxEco's business model combines several patterns from Section 9.2:

- **Product sales without a subscription.** Customers buy the controller, extender, and cabinet; weather updates, smart adjustments, the app, and the MySmartDrop portal carry no monthly fee (NxEco, n.d.-f). The cloud cost must therefore be covered by the hardware margin, which is why efficient cloud design matters. The same is true of **app maintenance**: without recurring revenue, there is little money to update the app for each new phone operating system, and the NxEco iOS app has not been updated since 2020 (Section 5.12). A one-time sale funds a device that may run for ten years, but not ten years of software.
- **Business-to-business reach.** The SNAP portal for up to 5,000 controllers and the Valet Key target landscape contractors and property managers, a channel in which one customer controls hundreds of installations.
- **Utility partnerships.** EPA WaterSense certification qualifies the controller for water-utility rebates, effectively letting public water agencies subsidize the purchase. The app also referred to "Water Bucks" programs in selected cities.
- **Ecosystem compatibility.** Support for Amazon Alexa and Google Home places the product inside the large smart-home ecosystems discussed in the Xiaomi case.

Worked Example 9.3: A homeowner's payback (illustrative)

The figures below are assumptions for teaching, not NxEco prices. A homeowner in a dry city pays for an installed smart controller costing US$220, receives a US$100 utility rebate, and spends about US$90 per month on outdoor water during a 6-month irrigation season. The controller cuts outdoor water use by 30 percent.

Net cost = 220 − 100 = **US$120**.

Seasonal saving = 90 × 6 × 0.30 = **US$162** per year.

Payback = 120 ÷ 162 ≈ **0.74 years**, or about 9 months, within the first watering season.

Without the rebate, payback would be 220 ÷ 162 ≈ 1.4 years. The rebate halves the payback period, which shows why utility partnerships are a powerful sales channel for water-saving IoT products.

Worked Example 9.4: Valuing real savings from the MySmartDrop portal

The Water Record page of a home controller with seven active zones reported the following totals (NxEco, 2026):

Table 9.6 Watering minutes by zone from the MySmartDrop Water Record (NxEco, 2026)

Zone	Name	Actual watering (min)	Saved (min)	Saving rate
1	Rose	3,939.77	592.54	13.1%
2	Green Hill	1,724.30	248.99	12.6%
3	Street	1,436.07	201.47	12.3%
4	Under Balcony	1,014.00	173.45	14.6%
5	Desert	1,373.50	240.74	14.9%
6	French	1,507.50	250.05	14.2%
7	Smile	2,049.51	449.77	18.0%
	Total	**13,044.65**	**2,157.01**	**14.2%**

Saving rate = saved ÷ (actual + saved). For the controller: 2,157.01 ÷ (13,044.65 + 2,157.01) = 2,157.01 ÷ 15,201.66 ≈ **14.2 percent**.

The portal reports minutes, not water. To value the saving, assume an average zone flow of 10 liters per minute (2.64 gallons per minute) and a water price of US$8 per 1,000 gallons (assumptions for teaching).

Water saved = 2,157.01 min × 2.64 gal/min ≈ **5,700 gallons** (about 21,600 liters).

Money saved = 5,700 ÷ 1,000 × US$8 ≈ **US$46** over the period recorded.

Interpretation. The measured saving is real but much smaller than the "up to 40 to 50 percent" in marketing material, and a business case built on the marketing figure would overstate the benefit about three times. Two lessons follow. First, IoT data should replace assumptions in a business case as soon as they are available (Section 9.8). Second, the controller measures **time**, not **volume**; adding a flow meter would let the portal report gallons and dollars directly and detect leaks, a feature customers might pay for.

Strategic questions. Students can evaluate NxEco with the tools of this chapter: Would a premium subscription (for example flow-based leak alerts or AI scheduling) be accepted by customers who chose the product partly because it has no fee? Could anonymized, aggregated watering data be valuable to water utilities planning for drought, and what consent would that require (Chapter 6)? And how should a small company compete with larger smart-irrigation brands and with smart-home ecosystems that may add irrigation features of their own?

Chapter summary

- IoT businesses operate in **connected devices, data and analytics, automation and control, and platforms and services**.

- Business models include **product sales, subscriptions, data monetization, managed services, product-as-a-service**, and **hardware as a hook**; most companies combine several.
- Margins rise from **hardware** (low) through **connectivity** and **platforms** to **analytics** (high), which drives the shift to recurring services.
- Obstacles include security cost, regulation, e-waste rules, interoperability, unclear ROI, and skills. Trends include **edge generative AI**, **servitization**, and **Matter** consolidation.
- **Xiaomi** uses low-margin hardware, an incubator model, and a unified app to build a consumer ecosystem; **Enel X** orchestrates distributed energy assets into virtual power plants and energy-as-a-service. One focuses on the human interface, the other on system resilience.
- A business case answers seven questions and quantifies **CapEx, OpEx, payback, ROI, and NPV**, with sensitivity checks; subscription businesses also track **LTV and CAC**.
- A **phased roadmap** with stage gates and KPIs reduces risk.
- Persuasive presentations turn **measured evidence** into a business argument and answer questions honestly.

Key terms

business case; business model; CAC; capital expenditure (CapEx); churn; data monetization; demand response; energy as a service; hardware as a hook; KPI; LTV; managed services; net present value (NPV); operating expenditure (OpEx); payback period; pilot purgatory; product-as-a-service; return on investment (ROI); roadmap; sensitivity analysis; servitization; stage gate; subscription; vehicle-to-grid (V2G); virtual power plant (VPP).

Review questions

Multiple choice

1. Paying for engine flying hours instead of buying parts is an example of:
    a. data monetization
    b. servitization (product-as-a-service)
    c. hardware as a hook
    d. product sales
2. Which layer of the IoT stack typically has the highest gross margins?
    a. Hardware
    b. Connectivity
    c. Platform
    d. Analytics and applications (SaaS)

3. Xiaomi's "hardware as a hook" strategy earns most of its profit from:
    a. high hardware margins
    b. internet services and the ecosystem
    c. selling cloud services to enterprises
    d. electricity trading
4. A virtual power plant creates value by:
    a. building a new gas plant
    b. coordinating many distributed assets to reduce demand or supply power when the grid is stressed
    c. selling smart TVs
    d. storing data in the cloud
5. A project costs US$60,000 and produces a net benefit of US$2,500 per month. Its payback period is:
    a. 12 months
    b. 24 months
    c. 30 months
    d. 36 months
6. A positive NPV means that a project:
    a. has no risk
    b. creates value after accounting for the time value of money
    c. pays back in one month
    d. has no operating costs
7. Monthly churn of 4 percent implies an expected customer lifetime of about:
    a. 4 months
    b. 25 months
    c. 40 months
    d. 96 months
8. A stage gate is:
    a. a firewall rule
    b. a decision point at the end of a phase where leaders use evidence to continue, adjust, or stop
    c. a type of gateway hardware
    d. an MQTT topic
9. Which trend is reducing fragmentation among smart-home ecosystems?
    a. Matter
    b. RS-485
    c. QoS 2
    d. TinyML

10. Which slide practice is recommended for an executive presentation?
    a. Dense paragraphs of text b. One idea per slide, with numbers in tables or charts c. No sources d. 40 slides for 10 minutes

Short answer

11. Compare the subscription and product-as-a-service business models, with one IoT example of each.
12. Explain why many IoT hardware companies try to add recurring services.
13. Compare Xiaomi's and Enel X's IoT strategies in terms of customer, value proposition, and revenue driver.
14. Describe the four phases of an IoT implementation roadmap and give one gate criterion for the pilot phase.

Applied problems

15. A hotel group considers smart room-energy controls for 600 rooms. CapEx is US$90 per room plus US$18,000 integration. OpEx is US$0.60 per room per month. Expected energy savings are US$4.50 per room per month. (a) Compute CapEx, monthly net benefit, and payback. (b) Compute three-year ROI. (c) If savings were only US$3.00 per room per month, what would the payback be?
16. For the business scenario in your Assignment 4, write the seven-question outline of a business case (one or two sentences per question) and identify the single assumption that most affects the financial result.
17. **NxEco case.** Repeat Worked Example 9.3 with an outdoor water bill of US$60 per month and a 25 percent saving. What is the payback with and without the US$100 rebate?
18. **NxEco portal data.** Using Table 9.6, (a) which zone uses the largest share of the controller's watering, and what share? (b) Which zone has the highest saving rate? (c) If the average flow were 15 liters per minute instead of 10, how many liters would the controller have saved?

Lab 9: Business proposal and final presentation

Assignment 4 items 2 and 3, and SM Assignment 2 (Week 8).

Part 1: Business proposal (Assignment 4, item 2). Choose a real business scenario and explain how to apply the demo IoT sensor system to it. Use this outline:

1. **Business need:** the problem, who has it, and its current cost, with sources.
2. **Recommended business model** (product sales, subscription, data monetization, managed service, or product-as-a-service) and why.
3. **Technology for each tier:** device, proximity network, gateway, cloud, application, justified with explicit criteria and measured data from Labs 7A and 7B.

4. **Where AI runs** (device, edge, or cloud) and why (Lab 8).
5. **Expected value:** a financial estimate (CapEx, OpEx, payback, ROI) with a sensitivity check (Worked Example 9.1).
6. **One strategy from the Xiaomi or Enel X case** that the business could adopt, and how.
7. **Risks:** at least one security or privacy risk and one organizational or environmental risk, with mitigations (Lab 6).
8. **Roadmap:** phases, stage gates, and KPIs.

Items 1 and 2 of Assignment 4 together are five to six pages plus appendix, in APA 7th edition format with at least five credible sources.

Part 2: Team presentation (Assignment 4, item 3, presented in Week 8). Prepare about 10 to 12 slides for a 10-minute presentation, following the structure in Section 9.10. The presentation must cover the team's collaboration setup and each student's role; critical test steps and coefficients; the major records of one test (one of T2 to T7); conclusions from the records; and potential business applications.

Part 3: Executive panel (Week 8 collaboration). While classmates present, act as a business-leadership panel. For at least three presenters, complete the feedback form below and share it as the instructor directs. Ask at least one question during the session.

Table 9.7 Executive panel feedback form

Feedback item	Notes
1. Strongest piece of evidence presented	
2. Weakest or least convincing evidence, and what would strengthen it	
3. A risk or question the presenter did not address	
4. Panel decision: approve, approve with changes, or revise and resubmit, with one reason	

Part 4: Revision and reflection (SM Assignment 2). After class, revise the slides using the panel feedback and submit the upgraded PowerPoint. In the GAP discussion, answer two reflection questions aloud: (1) Which feedback changed your presentation, and why? (2) What would you do differently in a real IoT deployment?

References and further reading

Borgini, J. (2025a, January 6). *12 top IoT business models for 2025*. TechTarget. https://www.techtarget.com/iotagenda/tip/Top-9-IoT-business-models

Borgini, J. (2025b, July 14). *10 top design principles of IoT*. TechTarget. https://www.techtarget.com/iotagenda/tip/Top-8-IoT-design-principles-for-successful-product-creation

Bottacci, F. (2023). *ENEL: Largest production deployment of AI and IoT [AIoT] applications* [Video]. YouTube. https://www.youtube.com/watch?v=uYgwBojDE7k

Elizalde, D. (n.d.). *7 IoT business models that are transforming industries*. https://danielelizalde.com/monetize-your-iot-product/

Enel North America. (2025, September 17). *Enel named a top virtual power plant aggregator* [Press release]. https://www.enelnorthamerica.com/about-us/newsroom/search-press/press/2025/09/virtual-power-plant-2025

Enel X. (n.d.). *Powering a smarter grid: How Enel X's virtual power plant works*. https://www.enelx.com/au/en/resources/powering-a-smarter-grid-how-enel-x-s-vpp-works

IoT83. (2025). *Critical challenges in IoT-driven business transformation* [Video]. YouTube. https://www.youtube.com/watch?v=ttIISel0anM

Next Vault. (2025). *The rise of Xiaomi: The next big tech giant after Apple and Tesla?* [Video]. YouTube. https://www.youtube.com/watch?v=rTYESm8y16U

Nir, M. (2024). *Emerging IoT business models* [Video]. YouTube. https://www.youtube.com/watch?v=V86i7Hq-b6Q

NxEco. (n.d.-a). *NxEco smart watering* [Website]. https://www.nxeco.com

NxEco. (n.d.-f). *Features* [Web page]. https://www.nxeco.com/features.html

NxEco. (2026). *MySmartDrop smart watering portal* [Web application]. Retrieved October 22, 2026, from https://www.mysmartdrop.com

POND IoT. (2024). *IoT trends in 2026* [Video]. YouTube. https://www.youtube.com/watch?v=UIDJsHjHbUw

Sinha, S. (2025, October 28). *Number of connected IoT devices growing 14% to 21.1 billion globally in 2025*. IoT Analytics. https://iot-analytics.com/number-connected-iot-devices/

APPENDIX A

Answers to Review Questions

Short-answer responses below are model answers; students' answers may differ in wording and examples but should contain the same key ideas. Calculations show the main steps.

Chapter 1

Multiple choice: 1 a; 2 c; 3 d; 4 d; 5 b; 6 b; 7 b; 8 c; 9 b; 10 d.

11. A sensor converts a physical quantity into an electrical signal or data (for example a smart thermostat's temperature sensor); an actuator converts an electrical command into physical action (for example a smart lock's motorized bolt).

12. Purpose is what the system is for; it determines how elements and interconnections are arranged, yet it appears in no single part and is revealed only by behavior. A smart irrigation controller's purpose (right water at the right time) cannot be seen by looking at its valve or sensor.

13. Risk: fine-grained electricity data reveals when a household is home, asleep, or on vacation. Mitigation: data minimization (report at lower resolution), encryption, and clear consent and retention rules.

14. Hysteresis uses separate on and off thresholds. Small fluctuations around one threshold no longer cross the other, so the actuator does not switch rapidly back and forth.

15. (a) Messages per day = 86,400 ÷ 30 = 2,880; bytes = 2,880 × 64 = 184,320 bytes ≈ 184 KB. (b) 2,000 × 184,320 = 368,640,000 bytes ≈ 369 MB per day. (c) Images per truck = 40 × 300 KB = 12,000 KB = 12 MB; new total ≈ 12.18 MB per truck, about **66 times** the original volume.

16. Open answer. Example: sensors (heart rate, accelerometer) and vibration motor in the perception layer; BLE radio and the phone acting as a gateway in the network layer; vendor cloud in the middleware layer; app and weekly health report in the application layer. Purpose: help the wearer understand and improve fitness and health. Boundary: the tracker, phone app, and vendor cloud (the user's body and daily activity are the environment). Interface: the BLE link between tracker and phone.

17. Valve output and rain sensor: perception. Wi-Fi radio (and the home router): network. Weather engine: middleware (cloud). MySmartDrop portal: application. Example interface:

the 2.4 GHz Wi-Fi link between the controller and the router, or the API between the NxEco cloud and the app.

Chapter 2

Multiple choice: 1 b; 2 c; 3 b; 4 c; 5 d; 6 c; 7 b; 8 b; 9 b; 10 c.

11. Resolution is the smallest change a system can register (for example 0.08 °C per ADC count); accuracy is how close the reading is to the true value (for example ±1 °C). A sensor can show tiny changes yet be offset by a whole degree.

12. Power (a relay coil draws about 70 to 90 mA, more than a GPIO pin can supply); latency (a valve takes seconds to open); wear (a mechanical relay is rated for a limited number of operations).

13. End devices must be low power, small, and cheap; MCUs integrate processor, memory, and peripherals on one chip and sleep at microamps. Gateways must run Linux, TLS, many connections, and sometimes AI, which needs a microprocessor's speed and memory.

14. Chattering is rapid on/off switching when a value hovers near a single threshold because of noise. Hysteresis adds a band between on and off thresholds so the actuator switches only after a meaningful change.

15. (a) At 50 kPa the sensor outputs 0.5 + 0.04 × 50 = 2.5 V; after the divider, 1.25 V. (b) Count = 1.25 ÷ 3.3 × 4,095 ≈ **1,551**. (c) The ADC sees 0.25 to 2.25 V for 0 to 100 kPa (a 2.0 V span, 50 kPa per volt). One count = 0.806 mV, so 0.000806 × 50 ≈ **0.040 kPa** per count.

16. Open answer. Example: ON at 5.0 °C, OFF at 3.0 °C. The 2 °C band exceeds sensor noise and gives the room time to cool, limiting compressor starts; check by observing cycle length (at least 10 minutes per cycle keeps starts at or below 6 per hour). Constraint: compressor inrush current and wear; handled by a contactor or SSR rated for the motor load and a minimum off-time in firmware.

17. (a) 30 × 4.2 ÷ 7.0 = 18 minutes. (b) Examples: a shaded or sandy zone with different needs, or a broken pipe; a soil-moisture probe (or a flow sensor for the broken pipe) would detect it.

Chapter 3

Multiple choice: 1 c; 2 d; 3 b; 4 b; 5 c; 6 d; 7 b; 8 b; 9 b; 10 c.

11. The application creates the message (layer 7); it is formatted and encrypted (6); belongs to a session (5); TCP or UDP adds ports and sequencing (4); IP adds addresses (3); the link layer frames it with MAC addresses and a checksum (2); the radio sends bits (1). The

receiver reverses the process. Layers 1 and 2 matter most for battery devices because every transmitted bit and every retransmission costs energy, and headers above add bits.

12. Laptop networking assumes ample power for always-on radios, megabytes of memory for full TCP/IP and TLS stacks, and stable connections. IoT devices have milliwatt budgets, kilobytes of RAM, and intermittent links, so they need lightweight protocols, sleep-friendly access methods, and gateways.

13. Advertising is a one-way, connectionless broadcast that any scanner can receive; ideal for periodic telemetry or beacons. A GATT connection is a two-way, acknowledged, optionally encrypted link organized into services and characteristics; ideal for commands and configuration.

14. Example: default passwords allow mass takeover (Mirai). Device identity (unique per-device credentials or certificates provisioned at manufacturing) removes the shared secret attackers exploit.

15. RSSI(8 m) = −50 − 10 × 3 × $\log_{10}(8)$ = −50 − 27.1 = **−77.1 dBm**. Maximum range: −90 = −50 − 30 $\log_{10}(d)$, so $\log_{10}(d)$ = 1.333 and d ≈ **21.5 m**.

16. Recommend **LoRaWAN** in a star-of-stars topology with one or two gateways at the farm buildings. Range (kilometers), power (years on batteries), data rate (10 bytes per 30 minutes is tiny), and cost (cheap end nodes, no subscription with a private gateway) all fit. Trade-off: very low data rate and duty-cycle limits, higher latency, and the need to install and maintain gateways; NB-IoT is an alternative if cellular coverage is good and a subscription is acceptable.

17. NxEco is mains powered, mounted near an existing router, sends little data, and adds no gateway cost, so Wi-Fi's power draw is acceptable. Battery nodes must minimize power, so they use BLE and rely on a gateway for long-range, encrypted IP communication.

Chapter 4

Multiple choice: 1 d; 2 b; 3 c; 4 d; 5 b; 6 b; 7 b; 8 c; 9 b; 10 c.

11. Southbound faces the devices (for example BLE or Modbus from sensors to the gateway); northbound faces the cloud (for example MQTT over TLS on Ethernet or LTE from the gateway to AWS IoT Core).

12. Message broker (receives and routes device messages); device registry (authorized devices and their certificates); device shadow (desired and reported state); rules engine (evaluates messages and routes them to storage, analytics, or functions).

13. MQTT keeps one persistent connection, has a header as small as 2 bytes, and pushes data through a broker; HTTP opens request/response exchanges with large text headers. For a

small reading every minute, MQTT uses less bandwidth and energy and supports QoS and last-will messages.

14. When the WAN fails, the gateway stores messages in a local database and forwards them in order when the link returns. It prevents data loss and keeps local functions running during outages (local survivability).

15. (a) Samples per truck = 12 × 3,600 = 43,200; bytes = 43,200 × 120 = 5,184,000; fleet = 400 × 5.184 MB ≈ **2,074 MB (about 2.07 GB)** per day. (b) Summaries = 43,200 ÷ 30 = 1,440 × 300 bytes = 432,000 bytes; events = 20 × 150 = 3,000 bytes; per truck 435,000 bytes; fleet ≈ **174 MB** per day. (c) Reduction = 1 − 174 ÷ 2,074 ≈ **91.6 percent**.

16. Example: Device: sensing, filtering, arrhythmia detection (real-time analytics and survivability: works without a phone or network). Edge (phone): immediate alerts to the user, buffering (local survivability). Cloud: long-term storage, doctor dashboards, model retraining, OTA updates. Justifications should cite at least two edge objectives.

17. Continue: stored schedules, valve switching, time keeping, rain-sensor interrupt, manual control. Stop: new weather adjustments, remote control, cloud reports and sharing. This demonstrates local survivability.

Chapter 5

Multiple choice: 1 d; 2 b; 3 c; 4 b; 5 c; 6 b; 7 b; 8 a; 9 b; 10 b.

11. Any three of: real-time and event-driven; tight hardware integration; heterogeneous, intermittent connectivity; distributed across device, edge, and cloud; power-constrained; must be secure by design; remotely managed and updated; data-centric.

12. HAL and drivers (I^2C driver for a BME280); RTOS (FreeRTOS tasks and queues); application logic (hysteresis fan control); connectivity stack (NimBLE BLE stack, MQTT client).

13. While an ISR runs, other interrupts and the main program are delayed, which can break timing and lose events. ISRs set a flag or copy data to a buffer; the main loop or an RTOS task does the processing later.

14. Business: fixing bugs or vulnerabilities remotely avoids costly visits and recalls and allows new features. Safety: unpatchable devices stay vulnerable and may malfunction dangerously; regulations increasingly require security updates.

15. (a) Overhead = 32 + 4 + 24 + 8 = 68 KB; remaining 2,048 − 68 = 1,980 KB; each slot ≤ **990 KB**. (b) Growth = 990 ÷ 850 − 1 ≈ **16.5 percent**.

16. Pseudocode: for attempt in 1..3: read value; if value is valid (not NaN and 0 ≤ value ≤ 100) return value; else wait 1 s. If all attempts fail: close valve, report error, increment error

counter. “Valve closed” is safe because a sensor fault should not cause uncontrolled watering, flooding, or water waste; a missed irrigation cycle is less harmful and is detected by the error report.

17. Opening several valves at once would drop water pressure and could overload the 24 V AC supply. Running zones one at a time supports reliability and stability (and deterministic, predictable behavior).

18. (a) The app holds little logic; it sends requests to the cloud, which stores schedules and relays commands, so app, portal, and voice assistants share one cloud state. The controller stores its programs in flash and keeps watering on schedule without the app (local survivability). (b) The longer delay is executed by the controller, so older firmware cannot honor it; the app must read the controller’s firmware version and offer the feature only where it is supported (or prompt a firmware update). (c) Any two of: time since last update, minimum operating system, privacy label, accessibility declaration, number of ratings, availability in the official app store.

Chapter 6

Multiple choice: 1 b; 2 a; 3 b; 4 b; 5 a; 6 b; 7 b; 8 b; 9 b; 10 c.

11. Any three: vast attack surface; physical exposure; constrained resources for cryptography; long lifetimes; missing update mechanisms; complex supply chains; cyber-physical safety consequences.

12. Example: devices on a flat network can be used as footholds (the fish-tank thermometer). Network segmentation places IoT devices on a separate VLAN with firewall rules so a compromised device cannot reach business systems.

13. The policy allows the gateway to connect only with its own client ID and to publish or subscribe only under its topic namespace. With *, a stolen certificate could connect as any client and read or write every topic in the account, compromising the entire fleet.

14. Example ethical concern: workplace occupancy sensors used to monitor individual productivity; response: aggregate data, transparency, and worker consultation. Environmental concern: battery and device e-waste; response: long-life batteries or energy harvesting, repairable design, and take-back programs.

15. Open answer. A typical first priority is weak app authentication or default credentials (high likelihood and high impact: an attacker could unlock the door), mitigated with strong authentication and multi-factor login.

16. Option A: 8 ÷ 2 = 4 batteries per sensor, so 3 replacements; 3 × 50,000 = 150,000 visits × US$25 = **US$3,750,000** (plus battery cost); 200,000 batteries used in total. Option B: no

replacements; extra cost 50,000 × US$3 = **US$150,000**; 50,000 batteries in total. Recommend **B**: far lower cost and 150,000 fewer batteries discarded.

17. The gardener receives only operating rights for the owner's controllers, without the owner's password or Wi-Fi credentials, and access can be revoked by deleting the user. Details to request: encryption of controller-to-cloud traffic, server authentication, or signed firmware updates.

Chapter 7

Multiple choice: 1 b; 2 b; 3 a; 4 b; 5 b; 6 b; 7 a; 8 b; 9 c; 10 c.

11. In Worked Example 7.1, sleep contributes about 0.010 mA but the brief 80 mA burst contributes about 0.133 mA, over 13 times more, because active current is 8,000 times the sleep current. Shortening the active window or sampling less often helps more than reducing sleep current further.

12. Push (IoT Core to app over MQTT/WebSocket) delivers live values with low latency; use for gauges and alerts. Pull (app to API Gateway, Lambda, and DynamoDB) retrieves history on demand; use for 24-hour charts.

13. Sensor to gateway: BLE advertising, no encryption (acceptable for non-sensitive temperature over a few meters; commands use encrypted GATT). Gateway to IoT Core: MQTT over mutual TLS on 8883 with X.509 certificates and a least-privilege policy. IoT Core to app: MQTT over WebSocket Secure on 443 with Cognito temporary credentials and SigV4 signing.

14. Set points decide when cooling starts and the alarm triggers; hysteresis decides when cooling stops and prevents chattering; actuator capacity decides whether cooling can remove heat faster than it is added. If capacity is too small, temperature stays above the thresholds even with the actuator fully on.

15. Average current = (29.895 + 0.01 × 120) ÷ 120.385 = 31.095 ÷ 120.385 ≈ 0.258 mA. Life = 2,600 × 0.85 ÷ 0.258 ≈ 8,556 h ≈ **357 days**.

16. (a) Rate = 40 ÷ 5 = 8 records/s; capacity = 250,000 ÷ 8 = 31,250 s ≈ **8.7 hours**. (b) Option 1: enlarge the buffer to at least 8 × 86,400 = 691,200 records (trade-off: more storage and longer drain time on reconnection). Option 2: aggregate at the edge during outages, for example one summary per node per minute (40 ÷ 60 ≈ 0.67 records/s, about 104 hours of capacity); trade-off: loss of raw detail during the outage.

17. The controller would become a gateway: a low-power southbound radio (BLE, Zigbee, or LoRa) to the probes plus the existing Wi-Fi or Ethernet northbound link to the cloud, with local buffering and translation.

Chapter 8

Multiple choice: 1 b; 2 b; 3 c; 4 b; 5 b; 6 b; 7 b; 8 b; 9 c; 10 b.

11. Smart: an irrigation controller waters when soil moisture falls below a fixed set point. Intelligent: it predicts each field's water need from soil data, weather forecasts, and crop models, and schedules irrigation in advance at the cheapest hours.

12. The gateway aggregates data from many devices, runs larger models than sensors can (for example correlating several machines), acts locally during outages, and sends only insights to the cloud.

13. Computational scale for training, fleet learning across many devices, simpler model management, rich historical analytics, and lower device cost. Edge AI is clearly better when decisions are needed in milliseconds, connectivity is unreliable, or raw data is sensitive (for example in-store video).

14. Models learn from data; noisy, biased, mislabeled, or inconsistent data produces confident but wrong decisions that are hard to detect, no matter how sophisticated the algorithm.

15. (a) Float32: 80,000 × 4 = 320,000 bytes (about 313 KB); int8: 80,000 bytes (about 78 KB). (b) **Neither** fits in 64 KB; the model must be pruned or redesigned (for example to roughly 60,000 parameters or fewer after quantization).

16. (a) Precision = 8 ÷ 40 = **20 percent**; recall = 8 ÷ 10 = **80 percent**. (b) Cost = 32 × US$150 + 2 × US$20,000 = US$4,800 + US$40,000 = **US$44,800**. (c) A missed leak costs about 133 times a false alarm, so recall should remain high (or be increased); precision should be improved only if it does not reduce recall, for example with better features or a second confirming sensor.

17. Leak and broken-head detection: when flow appears with no zone open, the controller must close the master valve immediately, without waiting for a cloud round trip or depending on the internet.

Chapter 9

Multiple choice: 1 b; 2 d; 3 b; 4 b; 5 b; 6 b; 7 b; 8 b; 9 a; 10 b.

11. Subscription: the customer owns the device and pays recurring fees for software or services (for example camera cloud storage). Product-as-a-service: the provider owns and maintains the asset and the customer pays for usage or outcomes (for example light-as-a-service or engine flying hours).

12. Hardware margins are low and one-time, while cloud and support costs continue after sale; recurring services earn higher margins, predictable revenue, and closer customer relationships.

13. Xiaomi targets consumers with affordable convenience and earns from high-volume hardware plus internet services; Enel X targets businesses and grid operators with efficiency, reliability, and decarbonization, and earns from grid flexibility payments and managed energy services.

14. Discover and prove (pilot); validate (limited rollout); scale (full deployment); optimize (AI and new services). Pilot gate example: measured benefit supports the business case with data completeness above 98 percent.

15. (a) CapEx = 600 × US$90 + US$18,000 = **US$72,000**; net monthly benefit = 600 × (US$4.50 − US$0.60) = **US$2,340**; payback = 72,000 ÷ 2,340 ≈ **30.8 months**. (b) ROI = (2,340 × 36 − 72,000) ÷ 72,000 = (84,240 − 72,000) ÷ 72,000 ≈ **17 percent**. (c) Net = 600 × (3.00 − 0.60) = US$1,440; payback = 72,000 ÷ 1,440 = **50 months**.

16. Open answer; should address all seven questions in Section 9.8 and identify a dominant assumption (often the size or frequency of the avoided loss).

17. Seasonal saving = 60 × 6 × 0.25 = US$90 per year. With rebate: 120 ÷ 90 ≈ 1.3 years. Without rebate: 220 ÷ 90 ≈ 2.4 years.

18. (a) Zone 1 (Rose): 3,939.77 ÷ 13,044.65 ≈ **30 percent** of all watering. (b) Zone 7 (Smile): 449.77 ÷ 2,499.28 ≈ **18.0 percent**. (c) 2,157.01 min × 15 L/min ≈ **32,355 liters** (about 8,550 gallons).

APPENDIX B

Formula Sheet and Demo System Reference

B.1 Formula sheet

Table B.1 Formula sheet

Topic	Formula	Chapter
ADC resolution	$LSB = V_{ref} \div 2^n$	2
ADC count to voltage	$V = count \times V_{ref} \div (2^n - 1)$	2
Nyquist criterion	$f_s > 2 \times f_{max}$	2
Ohm's law (coil current)	$I = V \div R$	2
Power in dBm	$P_{dBm} = 10 \times \log_{10}(P \div 1\,mW)$; $P = 10^x$ mW with $x = P_{dBm} \div 10$	3
Log-distance path loss	$RSSI(d) = RSSI(d_0) - 10 \times n \times \log_{10}(d \div d_0)$	3
Transmission efficiency	η = payload ÷ (payload + overhead)	3
Data volume	messages per day × bytes per message × number of devices	1, 4
Average current	$I_{avg} = \Sigma\,(I_k \times t_k) \div T_{cycle}$	7
Battery life	$Life\ (h) = C \times D \div I_{avg}$	7
Buffer capacity	records ÷ arrival rate (records per second)	7
Mean of *n* measurements	$mean = \Sigma\, x_i \div n$	7
Model weight memory	parameters × bytes per parameter (4 for float32, 1 for int8)	8
Precision and recall	precision = true alerts ÷ all alerts; recall = true alerts ÷ all real events	8
Risk score	likelihood × impact	6
Payback period	CapEx ÷ monthly net benefit	9
Return on investment	(total net benefit − CapEx) ÷ CapEx	9
Net present value	$-CapEx + \Sigma\, CF_t \div (1 + r)^t$	9
Customer lifetime	≈ 1 ÷ monthly churn	9

B.2 IoT Education Course demo system reference card

The values below are the design reference for the demo system described in Chapter 7 (Li, 2026). The browser simulation models these values; students should verify what it actually shows during Labs 7A and 7B.

Table B.2 Demo system reference card

Item	Value
Sensor nodes	sensor_node_01 (ESP32, DHT22 + MPU6050, fan, relay, LED); sensor_node_02 (ESP32, BME280); sensor_node_03 (nRF52840, probe)
Actuators (node 01)	Fan: PWM on GPIO 18 (MOSFET driver); relay: GPIO 23 (opto-isolated SSR); status LED: GPIO 2
BLE	BLE 5.0, 2.4 GHz; advertising channels 37/38/39; Environmental Sensing service 0x181A; custom actuation service 0xFF00 (fan 0xFF01, relay 0xFF02, LED 0xFF03)
Gateway	Raspberry Pi 4 (4 GB), Raspberry Pi OS 64-bit, Python (bleak + MQTT client), SQLite store-and-forward
Northbound	MQTT over mutual TLS, port 8883, X.509 (RSA-2048 or ECC P-256), Amazon Root CA 1, QoS 1
Topics	Telemetry `iot/telemetry/<device_id>`; commands `iot/commands/<device_id>`; acknowledgments `iot/actuator/ack`; shadow `$aws/things/<thing>/shadow/...`
Rule	`SELECT device_id, temp, humidity, pressure, vibration, battery, timestamp() AS ingest_time FROM 'iot/telemetry/+' WHERE temp > 0`
Storage	DynamoDB `SensorTelemetry`: partition key `device_id` (string), sort key `timestamp` (number)
Alerting	Lambda `HighTempAlertFn` when temp > 30.0 °C → SNS topic `IoTAlerts`
App control	Auto cooling ON at ≥ 26.0 °C, OFF at ≤ 24.0 °C; acknowledgment timeout 3.5 s; Cognito temporary credentials (about 60 min); MQTT over WSS on 443
Latency budget (uplink)	12–18 / 8–15 / 22–38 / 15–25 / 18–24 ms per hop; target < 120 ms
Power profile (node)	Wake 45 mA × 15 ms; sample 68.5 mA × 120 ms; BLE 84 mA × 250 ms; deep sleep 10 µA

B.3 Bill of materials (three sensor nodes and one gateway)

Table B.3 Demo bill of materials (indicative prices)

#	Item	Part	Qty	Unit (US$)	Extended (US$)
1	Perception MCU	Espressif ESP32-WROOM-32D	3	5.80	17.40
2	Environmental sensor	Bosch BME280	3	4.50	13.50
3	6-axis IMU	TDK MPU6050	3	3.20	9.60
4	Secondary sensor	Aosong DHT22 / AM2302	1	3.80	3.80
5	LiPo cell	PKCELL LP503450 (1,200 mAh)	3	6.20	18.60
6	Charger and protection	TP4056 USB-C module	3	1.10	3.30
7	Gateway host	Raspberry Pi 4 Model B (4 GB)	1	55.00	55.00
8	microSD card	SanDisk High Endurance 32 GB	1	11.50	11.50
9	Power supply	Raspberry Pi official USB-C supply	1	8.00	8.00
10	Relay module	Songle SRD-05VDC-SL-C module	1	2.50	2.50
11	Breadboards	MB-102	3	4.20	12.60
12	Passive component kit	Resistors, capacitors, MOSFETs	1	4.50	4.50
13	Jumper wires	DuPont set	1	3.60	3.60
14	Cloud services	AWS (within free-tier limits for a class demo)	—	0.00	0.00
	Total				**163.90**

APPENDIX C

Glossary

6LoWPAN. A standard that compresses IPv6 headers so IPv6 can run over low-power wireless networks such as IEEE 802.15.4. (Ch. 3)

A/B partition. Two firmware slots that allow a new image to be installed while the old one remains available for rollback. (Ch. 5)

Access point (AP). A device that connects wireless clients to a wired network. (Ch. 3)

Actuator. A device that converts an electrical command into physical action, such as motion, heat, or switching. (Ch. 2)

ADC (analog-to-digital converter). A circuit that samples an analog voltage and converts it to a binary number. (Ch. 2)

Advertising (BLE). Short, connectionless broadcasts on BLE channels 37, 38, and 39 that announce a device and can carry small amounts of data. (Ch. 3, 7)

AIoT (Artificial Intelligence of Things). The combination of IoT and AI that lets systems learn, predict, and act autonomously. (Ch. 1, 8)

Application layer. The top layer of the four-layer IoT architecture, where apps, dashboards, and business decisions deliver value. (Ch. 1)

Backhaul. The wide-area link from a gateway to the cloud. (Ch. 4)

BLE (Bluetooth Low Energy). A 2.4 GHz short-range wireless technology optimized for low-power devices. (Ch. 3)

Broker (MQTT). The server that receives published messages and delivers them to subscribers by topic. (Ch. 4)

Business case. A structured argument, including costs, benefits, and risks, that an investment is worthwhile. (Ch. 9)

CapEx (capital expenditure). One-time investment costs such as devices and installation. (Ch. 9)

Certificate authority (CA). A trusted entity that signs digital certificates. (Ch. 6)

Control loop. The repeated sequence of sensing, deciding, and acting that keeps a system at a set point. (Ch. 2)

CRC (cyclic redundancy check). A checksum used to detect transmission errors. (Ch. 3)

Data minimization. Collecting only the data needed for a stated purpose. (Ch. 6)

dBm. Power in decibels relative to one milliwatt. (Ch. 3)

Defense in depth. Multiple independent layers of security controls. (Ch. 6)

Derating factor. A multiplier below 1 that accounts for battery capacity that cannot be used in practice. (Ch. 7)

Device shadow (device twin). A cloud JSON document holding a device's desired and reported state. (Ch. 4)

Digital twin. A virtual model of a physical asset kept up to date with live data. (Ch. 4, 9)

Downlink. Commands and updates flowing from the cloud or app toward devices. (Ch. 1)

Drift. The gradual loss of accuracy of a sensor or model as conditions change. (Ch. 2, 8)

Duty cycle. The fraction of time a device is active. (Ch. 7)

Edge computing. Processing data near where it is produced, on devices or gateways. (Ch. 4)

Encapsulation. Adding a header (and trailer) at each layer as data moves down the protocol stack. (Ch. 3)

Evapotranspiration (ET). Water lost from soil by evaporation plus from plants by transpiration, usually in millimeters or inches per day; used to set irrigation amounts. (Ch. 2)

Federated learning. Training a shared model across devices without collecting their raw data. (Ch. 8)

Firmware. Low-level software stored in non-volatile memory that controls a device's hardware. (Ch. 5)

Gateway. A device that connects local IoT devices to wider networks, often translating protocols and adding security. (Ch. 1, 4)

GATT (Generic Attribute Profile). The BLE framework that organizes data into services and characteristics. (Ch. 3, 7)

GPIO. General-purpose input/output pins on a microcontroller. (Ch. 2)

HAL (hardware abstraction layer). Firmware layer that hides chip-specific details behind standard interfaces. (Ch. 5)

Hysteresis. Using different thresholds for switching on and off to prevent rapid toggling. (Ch. 2)

I²C / SPI / UART. Common serial buses for connecting sensors and modules to microcontrollers. (Ch. 2)

IIoT. The Industrial Internet of Things. (Ch. 1)

ISM band. Unlicensed radio spectrum such as 2.4 GHz and regional sub-GHz bands. (Ch. 3)

ISR (interrupt service routine). A short function that runs immediately when a hardware event occurs. (Ch. 5)

KPI. Key performance indicator. (Ch. 9)

Latency budget. An allocation of an end-to-end delay target among the hops of a system. (Ch. 7)

Least privilege. Granting only the permissions required for a task. (Ch. 6)

LoRaWAN. A low-power wide-area network using LoRa modulation in sub-GHz bands. (Ch. 3)

MAC address. A hardware address used at the data link layer. (Ch. 3)

Master valve. An irrigation valve on the main supply line that opens only while a zone is watering, limiting leaks. (Ch. 2)

Matter. A smart-home application standard that runs over Thread, Wi-Fi, and Ethernet for cross-brand interoperability. (Ch. 3, 9)

MCU (microcontroller unit). A single-chip computer with processor, memory, and peripherals. (Ch. 2)

Mesh topology. A network in which nodes relay messages for each other. (Ch. 3)

Middleware layer. The layer that processes, stores, and manages data at the edge and in the cloud. (Ch. 1)

MQTT. A lightweight publish/subscribe messaging protocol widely used in IoT. (Ch. 4)

Mutual TLS (mTLS). TLS in which both client and server authenticate with certificates. (Ch. 4, 6)

NB-IoT / LTE-M. Cellular standards designed for low-power IoT devices. (Ch. 3)

Northbound / southbound. Communication from the gateway toward the cloud / toward devices. (Ch. 4)

NPV (net present value). The discounted value of future cash flows minus the investment. (Ch. 9)

OpEx (operating expenditure). Recurring operating costs such as connectivity and cloud fees. (Ch. 9)

OSI model. A seven-layer reference model for network communication. (Ch. 3)

OTA (over-the-air) update. Remote delivery of new firmware or software to devices. (Ch. 5)

Payback period. The time needed for net benefits to repay an investment. (Ch. 9)

Perception layer. The bottom layer of the four-layer architecture, containing sensors, actuators, and MCUs. (Ch. 1)

Precision / recall. Measures of how many alerts are correct and how many real events are detected. (Ch. 8)

Privacy by design. Building privacy protections into a product from the start. (Ch. 6)

Product-as-a-service. A business model in which customers pay for use or outcomes rather than ownership. (Ch. 9)

PWM (pulse-width modulation). Switching an output on and off rapidly to control average power, for example motor speed. (Ch. 2)

QoS (quality of service, MQTT). Delivery guarantee levels 0, 1, and 2. (Ch. 4)

Quantization (ADC). Rounding sampled values to discrete levels. (Ch. 2)

Quantization (AI). Storing model weights with fewer bits, for example 8-bit integers. (Ch. 8)

Relay. A switch that lets a small control signal switch a separate high-power circuit. (Ch. 2)

Retained message. An MQTT message the broker stores and sends to new subscribers. (Ch. 4)

RFID / NFC. Radio identification technologies; NFC is a very short-range (about 4 cm) variant used for payments and pairing. (Ch. 3)

RISC. Reduced Instruction Set Computer, a processor design philosophy favoring simple, fast instructions. (Ch. 2)

ROI (return on investment). Net gain divided by investment. (Ch. 9)

RSSI. Received signal strength indicator, usually in dBm. (Ch. 3)

RTOS (real-time operating system). A small operating system that schedules tasks with predictable timing. (Ch. 5)

Rules engine. A cloud component that evaluates messages and routes them to actions. (Ch. 4)

SBOM (software bill of materials). A list of all software components in a product. (Ch. 6)

Secure boot. A process that runs only firmware with a valid signature. (Ch. 5, 6)

Sensor. A device that detects a physical quantity and converts it into an electrical signal or data. (Ch. 2)

Servitization. The shift from selling products to selling services and outcomes. (Ch. 9)

Set point. The target value a control loop maintains. (Ch. 2)

Star topology. A network in which every device connects directly to a central hub or gateway. (Ch. 3)

Store-and-forward. Buffering data locally during outages and sending it later. (Ch. 4, 7)

STRIDE. A threat-modeling framework: spoofing, tampering, repudiation, information disclosure, denial of service, elevation of privilege. (Ch. 6)

Systems thinking. Analyzing a whole system through its elements, interconnections, purpose, boundary, and interfaces. (Ch. 1)

TCP / UDP. Transport protocols; TCP is reliable and ordered, UDP is lightweight and connectionless. (Ch. 3)

Thread. An IPv6-based low-power mesh network protocol. (Ch. 3)

TinyML. Machine learning on microcontrollers. (Ch. 8)

TLS (Transport Layer Security). A protocol that encrypts and authenticates network connections. (Ch. 4, 6)

Topic (MQTT). A hierarchical name used to route messages, such as `home/room1/temp`. (Ch. 4)

Uplink. Data flowing from devices toward the cloud and applications. (Ch. 1)

Virtual power plant (VPP). A coordinated group of distributed energy assets that acts like a power station. (Ch. 9)

Virtual sensing. Estimating a physical quantity from other data, such as weather reports, instead of measuring it with a local sensor. (Ch. 2)

Watchdog timer. A hardware timer that resets the MCU if firmware stops responding. (Ch. 5)

WaterSense. A U.S. EPA certification program for water-efficient products, including weather-based irrigation controllers. (Ch. 6)

WPAN / WLAN. Wireless personal area network / wireless local area network. (Ch. 4)

X.509 certificate. A standard digital certificate binding a public key to an identity. (Ch. 4, 6)

Zero trust. A security principle of never trusting by location and always verifying each connection. (Ch. 6)

Zigbee. An IEEE 802.15.4-based mesh networking protocol for home and building automation. (Ch. 3)

Index

Page numbers in bold show where a term is defined or explained in most detail. Definitions of key terms are also collected in the glossary, Appendix C.

Symbols and numbers

6LoWPAN, 46, **51**

A

access point, 45, 47–48, 52
accuracy, 13, 19, 22, **24–25**, 157–158, 161
acknowledgment timeout, 142, 196
actuator, 1, **4**, **19–21**, **26–27**, 66, 88, 128
actuator capacity, **144**
ADC, *see* analog-to-digital converter (ADC)
advertising (BLE), 45, **53–54**, 92, 116, 127, 172, 196
AES, 58, 112–113, 116, 133
AIoT, 1, 119, 153, 171
analog-to-digital converter (ADC), 19, **22–24**, **28–29**, 91, 195
anomaly detection, 68, 77, 112, 153, 156, 160–161
application layer, 5, 46, 54, 88
architecture, 1, **4–5**, 19, 65, 87, 139, 155, 179
Arm, 19, **28–31**, 68, 95, 130
ASK, 48
attack surface, 7, 109
availability zone, **75**
average current, 127, **131–132**, 195

B

backhaul, 47, 55, 65, **67**, 70, 77
bare-metal super-loop, **92**
blast radius, **115**
BLE, *see* Bluetooth Low Energy (BLE)
Bluetooth Low Energy (BLE), 2, 44, **53**, 67, 88, 111, 127, 155, 196
bootloader, **91–93**, 97
boundary, 1, **8**
bring-up, 142
broker, 6, 66, **71–73**, 89, 111, 128, 156
brownout, 96
business case, 167, **175–177**
business model, 9, 120, **167–169**

C

CAC, 177
CapEx, *see* capital expenditure (CapEx)
capital expenditure (CapEx), 169, 171, **175–176**, 195
certificate authority, 74, **113**
chattering, **33**
churn, 168, 177, 195
CIA triad, **109**
CISC, **31**
cloud AI, 153, 155, 157–159
CMSIS, **30**
CoAP, 21, 46, **52**, 78, 156
computer vision, 153, 160
control loop, 8, 19, **31–34**, 127
CRC, 50–51, 134–136, 140
CSMA/CA, 50
Cyber Resilience Act, 11, 118, 171

D

data center, 2, 6, **75**, 78, 120, 156
data contract, **155**, 161
data link layer, 44, 47, **50**
data minimization, **11**, **118**
data monetization, 167
DBaaS, 75
dBm, 44, **49–50**, 130, 134, 195
decapsulation, **47**
defect log, 143
defense in depth, 108, **111–112**
delta update, **97**
demand response, 174
derating factor, **131**
device registry, 74, 76, **80**, 100
device shadow, 65, **73–74**, **76**, 88, **138**
digital signature, **113**
digital twin, **73–74**, 79, 174
downlink, **6**, **73**, 100, 127–128, 140–141
drift, 24, 153, **158**, 161
driver circuit, 9, 27, 32, 130
duty cycle, 120, 127, **129–131**

E

edge AI, 144, **157–158**
edge computing, 5, 67, 70, 79, 154
edge gateway, 6, 66, 98, 114, **128**
EEPROM, 89
element, 1, **7–8**, 20, 68, 112, 137
elliptic-curve cryptography, **113**
emergent behavior, 9
encapsulation, **46**
energy as a service, **174**
environment, **8**, 20, **37**, 56, 87, 122, 160
Environmental Sensing service, 54, 134, 196
evapotranspiration (ET), 12, **35**
e-waste, 120, 122, 171

F

fail-safe, 87, 96
federated learning, 153, **159**, 161
feedback loop, **8**, **155**
firmware, 1, **4**, **9**, 28, 47, 73, 87, **89–90**, 109
firmware image, 90, 92, 97, 113

first mile, 4, **44–45**
flash encryption, **112**
flash memory, 37, 80, 89, 92, 101
fleet learning, **12**, **158**, 162
flyback diode, **27**
FreeRTOS, 29, 91–92
FSK, 48

G

gateway, 1, **4**, 20, 45, **65–67**, 87, **89–90**
GATT, 54, 69, 92, 116, 127
GDPR, 11, 79, 118, 158
gNodeB, **67**
GPIO, **27–29**, 68, 91, 130, 196

H

HAL, *see* hardware abstraction layer (HAL)
hardware abstraction layer (HAL), 87, **91**, 95
hardware as a hook, 172
hash function, **113**
hysteresis, **8**, 19, 27, **33**, 139, 144–145, 161

I

IaaS, 75, 78
I^2C, **22**, 91, 129
IDE, 95, 98
IIoT, 3
inference, 68, 70–71, 73, 156, 159, 162
instruction set architecture (ISA), **30**
interconnection, 1, **7–8**, 45
interface, 1, **8–9**, 25, 68, 88, 110, 175
Internet of Things, **1–3**
interrupt service routine (ISR), **93–94**
IPv4, 44, 46, **51**
IPv6, 44, 46, **51**, 54, 56
ISA, *see* instruction set architecture (ISA)
ISM band, 53
ISR, *see* interrupt service routine (ISR)

J

JTAG, **95**, 109

K

keep-alive, **73**
knowledge distillation, **158**

L

LAN, 52
last will and testament, **73**
latency budget, 127–128, **141**, 196
least privilege, 13, 100, 111, 114–115, 121
linearity, 19, 24
LiteRT, 99, 155, 158
little-endian, **134–136**
local survivability, 65, 69, **80**, **101**, 117
LoRaWAN, 4, 49, **53**, **55**, 89, **133**, 156, 169
LPWAN, 53
LTE-M, 6, 49, **53**, **55–56**
LTV, 177

M

M2M, 2
MAC address, 47, 81
managed services, 75, 167
master valve, **37**, 81, 145, 162
Matter, 51, **54**, 68, 91, 161, 171–172
MCU, *see* microcontroller unit (MCU)
MEC, 67
mesh topology, 44, **57–58**
microcontroller (MCU), 1, **9**, 19, 68, 87, 109, 129, 155
microcontroller unit (MCU), 4, 19, **27–30**, 49, 88, 129, 157
middleware layer, 5, 88, 99
MLOps, **155**
modulation, 27, 44, **48**, 55
MQTT, 6, 21, 44, 65, **71–73**, **76–78**
mutex, 92
mutual TLS, **74**, 108, **114**, 136, 196

N

NB-IoT, 49, **53**, **55–56**, 66
net present value (NPV), 167, **175–176**, 195
network layer, 5, 46, 88, **112**
network segmentation, **58**, **110–112**
NFC, 44, 53–55
Node-RED, 98
northbound, 6, **67–68**, 116, 135, 196
NPU, 157–158
NPV, *see* net present value (NPV)
Nyquist criterion, **23**, 195

O

operating expenditure (OpEx), 169, 171, **175–176**
OpEx, *see* operating expenditure (OpEx)
opto-isolator, **27**
OSI model, 47
OTA update, 96–97
OWASP IoT Top 10, 108, 110, 121

P

PaaS, 75, 78, 169
packed structure, 51, 134–135
packet, **46–47**, **51**, 88, 127
partition key, 75, 138, 196
payback period, **175**, 180, 195

perception layer, 5, 10, **20**, 88, 99, **128**
personal data, 72, 79, **117–118**
physical AI, **154**
physical layer, **47**, 134
pilot purgatory, 171
power profile, 132, 196
precision, 19, **24–25**, 70, 153, **160–161**, 170, 195
predictive analytics, 153–154, 160
presentation link, 65, **67**
priority, **92**, 161
privacy by design, **118**, 121
product-as-a-service, 167, 171
protocol translation, 65–66, **71**, **80**, 127
provisioning, 13, **59**, 99, 115, 127, 178
proximity network, 6, 65, **67**, **128**, 133
pruning, **158**, 161
PSK, 48
PSTI, 11, 118–119
publish/subscribe, 65, 71–72, 78
purpose, 1, **7–9**, 28, 54, 79, 90, 118, 138
PWM, **26–28**, 91, 130, 196

Q

QAM, 48
QoS, 72–73, 136–137, 196
quantization, 19, **22–23**, 153, **157–158**, 161
queue, **92**, 138

R

race to sleep, 91
range, 5, 19, 44, **55–57**, 67, 96, 128
real-time operating system (RTOS), 5, 30–31, **91–92**, 95, 158
recall, 97, 153, **160**, 178, 195
RedCap, 53, 66, 171
region, 1, 49, **75–76**, 79, 122, 143, 170
reinforcement learning, 153, 160
relay, 4, 26, 54, 74, 92, 129, 196
resolution, 19, **23–25**, 118, 195
response time, 19, 24, 66
retained message, 65, **72–73**
return on investment (ROI), 167, 171, **175–176**, 178–179, 195
RFID, 2–3, 44, 53–55
RISC, 19, 28, **30–31**
RISC-V, 28, 31
risk matrix, 117
roadmap, 167, 175, **177–179**
ROI, *see* return on investment (ROI)
rollback, 97, 139, 155
root of trust, 91, 112
RSSI, 49–50, 195
RTOS, *see* real-time operating system (RTOS)
rules engine, **76–77**, 81, 100, 128, 140–141

S

SaaS, 75, 168–169
sampling, 19, **22–23**, 25, 132, 155, 161
SBOM, 113, 119
scheduler, **92**
SDK, 95, 135
secure boot, 68, 89, **91**, 94, **112**, 114–115
segment, **46–47**, **51**, 57–58, 169
semaphore, **92**
sensitivity, 19, 24, 176
sensor, 1, **3–4**, **19–22**, 44, 65, 88
servitization, 168, 171
servo motor, 26
set point, 1, 6, **32**, **34**, 144
smart sensor, **22**, 24, 32
solenoid, 9, 26, 34, 37
sort key, 75, 138, 196
southbound, 6, **67–69**, 133, 135, 145
SPI, **22**, 91, 129
SRAM, 28, 30, 158
stage gate, 167, 175, **177–178**
star topology, **58**
store-and-forward, **69**, **71**, 98, 127, **136**, 144–145, 196
STRIDE, 108, **111**, 121
subscription, 13, 49, 120, 138, 167
SWD, **95**
symmetric encryption, **113**
systems thinking, 1, **7**

T

task, 32, **90–93**, 95, 156
TCP, 44, 46–47, 51, 68, 76, 78
TDMA, 50
teardown, 1, **9**, 108, 153
test evidence, 143
Thread, 30, 44, **53–54**, 67, 91, 112, 130
threat modeling, **111**
thundering herd, 89
time-series database, 69, **75**
TinyML, 153, **156–157**, 161
TLS, 46, 68, 91, 108, **114**, **116**, 129, 196
toolchain, 95
topic, 47, 65, **71–73**, 112, 127, 195
training, 76, 99, 119, 156
transducer, **20**, **22**
transmission efficiency, 44, **50**, 195
TrustZone, **31**

U

UART, 28, 91, 93–94, 109
UDP, 44, 46, 51–52, 78
uplink, **6**, 65, **73**, 127, 196
user equipment, **66**

V

virtual power plant (VPP), 3, 9, **174**
virtual sensing, **35–36**
VPP, *see* virtual power plant (VPP)

W

watchdog timer, **93**
wildcard, 65, 72
WLAN, 52, 67–68
WPA3, **52**
WPAN, 44, 67–68, 98

X

X.509, 68, 74, 76–77, **113**, 116, 196
X.509 certificate, 76

Z

Zephyr, 29, 91, 95
zero trust, **111**, **113**
Zigbee, 4, 29, 44, **53–54**, **56–57**, 66, 88, 112
Z-Wave, 44, 49, **53–54**